The Backend Engineer's Library

Concurrency and Parallelism

Volume 4

Contents

Chapter 1 — Models of Concurrency: A Map of the Territory

What this chapter covers. This volume spends nine subsequent chapters dissecting mutexes and futexes, memory models and happens-before edges, lock-free queues, event loops, actors, channels, and structured concurrency. Before any of that, we need a map — a single frame that tells you which of those mechanisms is even relevant to a problem in front of you, and which hazards you have signed up for by choosing it. That is this chapter's job. We start with the distinction that organizes everything else: concurrency is not parallelism. We then survey the models available to you, each with an honest account of what it costs, preview the four families of hazard that the rest of the volume exists to defeat, and develop the small body of quantitative law — Amdahl, Gustafson, the Universal Scalability Law, and Little's Law — that lets you reason about scaling with arithmetic rather than folklore. The chapter is deliberately broad and forward-references aggressively; the goal is not mastery of any one model but the ability to *place* a concurrency problem correctly, because choosing the wrong model is a mistake no amount of careful locking later will repair.

The distributed-systems lens is unusually load-bearing here. A distributed system is concurrency writ large: the same four hazards reappear across machines, with weaker guarantees and no shared memory to fall back on. Nearly every idea in this volume has a twin in Volume 6, and I will point at the twins as we go.

Learning goals — after this chapter you should be able to:

- State the concurrency/parallelism distinction precisely, and explain why backend services are overwhelmingly concurrency-bound rather than compute-parallel.
- Place the major concurrency models — shared memory, lock-free, actors, CSP, event loops, coroutines, data parallelism, STM — on a single map, and name the trade-off each makes.
- Distinguish a **data race** from a **race condition** with precision, and explain why a data-race-free program can still be badly broken.

- State Amdahl's Law and Gustafson's Law correctly, explain what question each actually answers, and stop misapplying one to the other's question.
- Use the Universal Scalability Law to explain why throughput can *decrease* as you add threads, and apply it to thread-pool and connection-pool sizing.
- Apply Little's Law ($L = \lambda W$) to size a pool from measured arrival rate and service time.
- Choose a model deliberately for I/O-bound, CPU-bound, and coordination-heavy workloads.

Concurrency is not parallelism

The single most useful distinction in this volume was given its canonical formulation by Rob Pike in his 2012 talk *Concurrency Is Not Parallelism*:

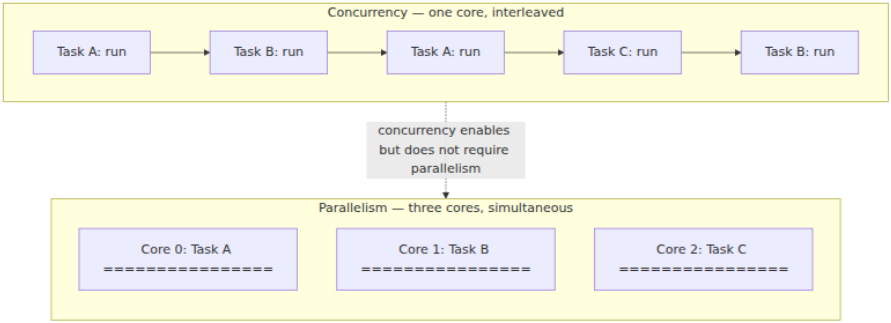
> Concurrency is about *dealing with* lots of things at once. Parallelism is about *doing* lots of things at once.

These are not two words for one idea, and they are not points on a spectrum. They are answers to different questions.

Concurrency is a property of a program's structure. It is a way of decomposing a problem into independently executing tasks whose lifetimes overlap. A concurrent program is one that is *composed* of activities that could, in principle, proceed independently — a request handler, a background flusher, a health-check loop, a metrics reporter. Concurrency is about composition: how you carve a problem into parts that can make progress without waiting for each other, and how those parts communicate. It is a design-time property, and it is meaningful even on a machine with exactly one core.

Parallelism is a property of a program's execution. It is the simultaneous execution of multiple computations on multiple physical execution units. Parallelism is about throughput on real hardware: two cores retiring instructions in the same cycle, eight lanes of a SIMD register being multiplied at once, a thousand GPU threads in lockstep. It requires hardware that can do more than one thing at an instant, and it is measured in wall-clock speedup.

The relationship between them is asymmetric, and getting the direction right matters. Concurrency *enables* parallelism: if you have decomposed your program into independent tasks, a runtime can schedule them onto multiple cores. But concurrency does not *require* parallelism — a concurrent program runs perfectly well on one core by interleaving its tasks — and parallelism does not require the kind of concurrency we mostly care about in this volume. A vectorized dot product is parallel with no concurrent structure at all: there are no independently-progressing tasks, just one operation applied across many data elements at once.



The practical payoff of the distinction is that it tells you what a given change can possibly buy you. Adding cores to a program with no concurrent structure buys nothing. Adding concurrent structure to a program that is already saturating its cores buys nothing but overhead. And — the case that dominates backend engineering — adding concurrent structure to a program that spends its life *waiting* buys you nearly everything, on any number of cores.

Why backend services are concurrency-bound, not compute-bound

Consider what a typical request handler in a backend service actually does. It parses a request, issues a query to a database, waits. It calls two internal services over gRPC, waits. It reads a value from a cache, waits. It serializes a response and writes it to a socket, waits. Between those waits it performs a few microseconds of actual computation: field validation, a permission check, some JSON marshaling.

The ratio is brutal. A local NVMe read is on the order of 100 microseconds; a database query across a data-center network is single-digit milliseconds; a call to a third-party API over the public Internet can

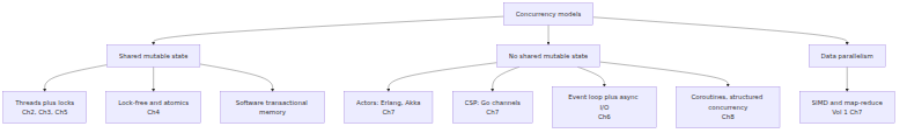
be hundreds of milliseconds. Against those numbers, the handler's own CPU work — call it 50 microseconds — is a rounding error. Volume 2, Chapter 7 develops the latency hierarchy in detail; the summary is that the gap between "compute" and "wait" in a backend service spans four to six orders of magnitude.

This has a direct consequence for how you should think about concurrency. If a request spends 99% of its wall-clock time blocked on I/O, then a single core can, in principle, service roughly 100 concurrent requests before the CPU is the constraint. The scarce resource is not compute — it is the *ability to have many outstanding waits at once*. That is a concurrency problem, not a parallelism problem. It is why an event loop on one thread (Chapter 6) can outperform a thread-per-request server on eight cores for I/O-heavy workloads, and why "we added more cores and nothing got faster" is such a common and predictable disappointment.

The corollary matters too: the minority of backend work that genuinely is compute-bound — image transcoding, compression, cryptography, analytics aggregation, model inference — is a *parallelism* problem, and should be handled with the parallelism tools (a bounded worker pool sized to cores, data-parallel decomposition, SIMD; see Volume 1, Chapter 7) rather than with the concurrency tools. Mixing the two up in one pool is a classic source of latency disasters: a handful of CPU-bound tasks will occupy every worker and starve hundreds of cheap I/O-bound ones behind them.

A taxonomy of models

There is no single concurrency model, and the choice among them is the most consequential architectural decision in this volume. Here is the map. Each entry gets an honest statement of what it buys and what it costs; each gets a full chapter later.



Shared memory with locks

Threads share an address space and mutate shared data structures under mutual exclusion. This is the default model in C, C++, Java, C#, and Rust, and the one every backend engineer must understand whether or not they choose it, because it underlies the runtimes of the models that claim to have escaped it.

Buys: Directness. Shared state is simply there — no copying, no serialization, no message plumbing. It maps cleanly onto how the hardware actually works (Volume 1, Chapter 4), so it is efficient when contention is low. Every language has it, every engineer has some fluency in it.

Costs: Every hazard in this volume. Data races if you forget to synchronize; deadlock if you take locks in inconsistent orders; convoy effects and priority inversion under contention; and a composability problem that is more serious than it first appears — two individually thread-safe operations composed together are generally *not* thread-safe, so correctness does not compose the way abstractions are supposed to. Chapters 2, 3, and 5 are largely about paying this bill.

Lock-free and atomics

Coordination via atomic read-modify-write instructions — compare-and-swap and friends — rather than mutual exclusion, with algorithms designed so that some thread always makes progress.

Buys: Immunity to the pathologies of blocking. No thread's stall, preemption, or page fault can block all others, which makes tail latency far more predictable and makes concurrency usable in contexts where blocking is forbidden (signal handlers, real-time paths). Under very high contention on a small hot structure, it can also be substantially faster.

Costs: Difficulty that is hard to overstate. Correctness depends on memory-model fluency (Chapter 3), the ABA problem lies in wait, and safe memory reclamation in non-GC languages is a research-grade problem in its own right. Note carefully that lock-free does **not** mean "faster" — it is a *progress* guarantee, not a performance one. Chapter 4 is the deep dive, and its practical advice is to use battle-tested library implementations rather than writing your own.

Message passing: actors and CSP

Eliminate shared mutable state entirely. Concurrent entities own their state privately and communicate by sending messages. Two major variants:

- **Actors** (Erlang/OTP, Akka): each actor has an identity and a mailbox; you send to an actor. Communication is asynchronous and typically location-transparent, which is what makes the model scale across machines as naturally as across cores.
- **CSP** (Hoare's Communicating Sequential Processes; Go's goroutines and channels): processes are anonymous and you send to a *channel*. Classical CSP rendezvous is synchronous — sender and receiver meet — though Go offers buffered channels as well.

Buys: The data-race hazard is eliminated by construction, since there is no shared mutable state to race on. Failure isolation is dramatically better — Erlang's "let it crash" plus supervision trees is the strongest fault-tolerance story in mainstream concurrent programming. And the model extends across the network essentially unchanged.

Costs: Copying overhead, message-plumbing boilerplate, and the fact that eliminating data races does not eliminate *race conditions* — you can still have ordering bugs, and you gain some new failure modes: unbounded mailbox growth, deadlock via cyclic waits on channels, and the difficulty of reasoning about a system whose control flow is scattered across dozens of mailboxes. Chapter 7.

Event-driven: the event loop

A single thread (or a small number) runs a loop that multiplexes over many non-blocking file descriptors with `epoll` / `kqueue` / `io_uring`, dispatching callbacks as I/O becomes ready. Node.js, nginx, and Redis are the canonical examples.

Buys: Enormous I/O concurrency at very low per-connection cost — a few kilobytes of heap per connection instead of a thread stack — and, because the handler thread is single-threaded, no data races on application state at all. That last property is why Redis can offer atomic operations without any locking.

Costs: The cardinal rule is that you must never block the loop; one slow CPU-bound handler stalls every connection, converting a throughput

problem into a total outage. Callback-based control flow is awkward (hence async/await, Chapter 8), and a single loop uses exactly one core, so you need multiple processes or loops to use the machine. Chapter 6.

Coroutines and structured concurrency

Lightweight, cooperatively-scheduled tasks that can suspend and resume — goroutines, Kotlin coroutines, Python `asyncio` tasks, Java virtual threads — combined with the discipline of *structured concurrency*: every task has a bounded lifetime tied to a lexical scope, so no task can outlive the scope that spawned it.

Buys: Sequential-looking code with event-loop efficiency, and a genuine advance in reliability: structured concurrency makes task leaks and lost errors structurally impossible rather than merely discouraged, and gives cancellation a coherent story. Chapter 8.

Costs: Runtime complexity, coloring problems in some languages (async functions being callable only from async contexts), and debugging tools that lag behind those for threads.

Data parallelism

Apply the same operation across many data elements simultaneously: SIMD instructions, GPU kernels, map-reduce, vectorized columnar query execution.

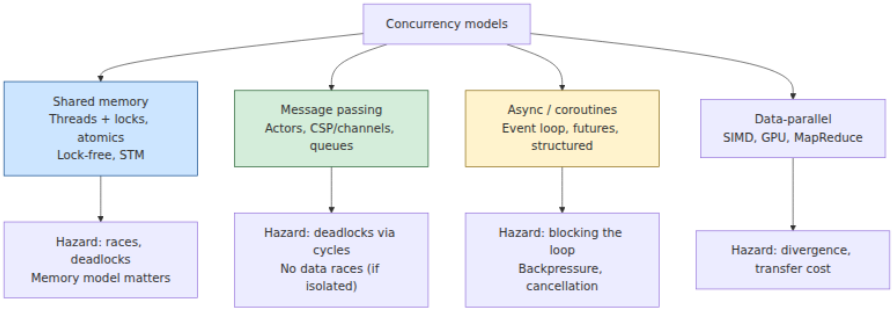
Buys: The best speedups available, when the problem fits. *Costs:* The problem must actually fit — regular, independent, uniform work over bulk data. Most backend request-handling does not. See Volume 1, Chapter 7.

Software transactional memory, and why it did not take over

STM lets you mark a block as atomic and have the runtime handle it — optimistically execute, detect conflicts, roll back and retry. It is genuinely elegant: it *composes*, which is precisely what locks do not do, and Haskell's STM remains its best realization.

It never displaced locks, for reasons worth understanding because they recur elsewhere. The bookkeeping overhead of tracking every read and write is high. Transactions containing side effects that cannot be rolled back — I/O, in particular — are a fundamental problem, and Haskell's solution (use the type system to forbid I/O inside transactions) is unavailable in most languages. Under high contention, repeated rollback

and retry wastes enormous work. Hardware support (Intel TSX) arrived, was repeatedly found to have errata, and was eventually disabled on most parts. The lesson generalizes: optimistic concurrency control is excellent when conflicts are rare and rollback is cheap, and poor otherwise — the same calculus that governs optimistic versus pessimistic locking in databases (Volume 5, Chapter 6).



The four families of hazard

Every concurrency bug you will meet belongs to one of four families. Naming them precisely is worth the effort, because the fixes differ.

Data races versus race conditions

These are constantly conflated, including in otherwise careful writing, and the conflation causes real harm. They are different things.

A **data race** is a precisely-defined, mechanical property: two threads access the same memory location concurrently, at least one access is a write, and the accesses are not ordered by synchronization. That is it. It is a property of an *execution* with respect to a memory model (Chapter 3), it is machine-checkable — this is exactly what ThreadSanitizer detects (Chapter 10) — and in C and C++ it is undefined behavior, meaning the compiler is entitled to do anything at all.

A **race condition** is a semantic property: the correctness of the program depends on the relative timing or interleaving of operations, and some interleavings produce wrong answers. It is a bug in your *logic*, not in your memory access discipline.

The crucial point is that neither implies the other, and the direction people miss is this one:

> **A program can be entirely free of data races and still be riddled with race conditions.**

Consider a check-then-act on a perfectly thread-safe map:

```
// map is a ConcurrentHashMap. Every individual operation is atomic and
// thread-safe. There is no data race anywhere in this code.
if (!map.containsKey(key)) {     // thread A and thread B both observe
absent
    map.put(key, computeValue()); // both compute, both put; one
silently wins
}
```

Every operation here is atomic. ThreadSanitizer will report nothing. And the code is still wrong: two threads can both pass the check, both compute, and one result is silently discarded — which matters a great deal if `computeValue()` is expensive, or allocates a resource, or increments a counter. The fix is not more synchronization primitives but a genuinely atomic compound operation: `map.computeIfAbsent(key, k -> computeValue())`.

This is why "we ran the race detector and it was clean" is a much weaker statement than teams usually take it to be. Race detectors find data races. They do not find race conditions.

Atomicity violations

A generalization of the above: an operation that *must* be indivisible is implemented as several steps, and another thread interleaves between them. The classic is `count++`, which is three operations — load, add, store — and loses updates under concurrency. But the more damaging instances are at the application level: read a balance, validate it, write a new balance; check a quota, then consume it; read-modify-write of a config structure. Each is individually synchronized and collectively broken. Chapter 2 covers the primitives; Chapter 9 covers the patterns that avoid needing them.

Ordering and visibility

The subtlest family, and the subject of Chapter 3. A write performed by one thread may not become visible to another for an unbounded time, and the *order* in which writes become visible need not match the order in which they were issued. This is not a hardware defect; it is the deliberate

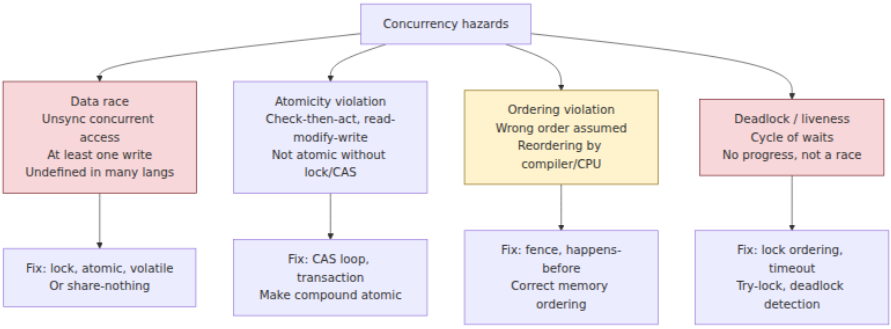
consequence of compiler optimization, store buffers, and out-of-order execution (Volume 1, Chapters 2 and 4). Code that is obviously correct when read sequentially can observe states that appear impossible. Reasoning about this requires the formal machinery of happens-before, and it is the reason Chapter 3 is the hardest chapter in this volume.

Liveness failures: deadlock, livelock, starvation

Not wrong answers but *no* answers.

- **Deadlock:** a cycle of threads each holding a resource the next needs. Nobody proceeds, ever.
- **Livelock:** threads are actively executing and repeatedly reacting to each other, but no useful work completes — the two-people-in-a-corridor problem.
- **Starvation:** some thread is perpetually denied a resource it needs while others make progress; usually an unfairness problem rather than a cycle.

Chapter 5 treats these, including Coffman's four necessary conditions for deadlock and the strategies that break each one.



The quantitative laws

Concurrency discussions collapse into folklore without arithmetic. Four results do most of the practical work.

Amdahl's Law: the ceiling imposed by serial work

Amdahl's Law (Gene Amdahl, 1967) answers this question: *for a problem of fixed size, how much faster does it get as I add processors?*

If a fraction **p** of the work is parallelizable and **(1 − p)** is inherently serial, then with **N** processors the speedup is:

```
              1
S(N) = ──────────────────
       (1 − p) + p / N
```

Take the limit as N → ∞ and the parallel term vanishes:

```
S(∞) = 1 / (1 − p)
```

This is a brutal result. If 5% of your work is serial, your maximum possible speedup is 20× — no matter how many cores you buy. At 10% serial, the ceiling is 10×. Doubling from 64 to 128 cores at p = 0.95 moves speedup from about 15.4× to about 16.6×: you doubled the hardware for an 8% gain.

The serial fraction in a real service is rarely a single identifiable section. It is the sum of many small things: lock acquisition, the single-threaded portion of request parsing, allocation under a shared allocator lock, logging to one file descriptor, a shared counter's cache line. This is why profiling for contention (Volume 2, Chapter 11) is the highest-leverage scaling work available.

Gustafson's Law: the answer to a different question

Amdahl is frequently quoted as proving parallelism is futile. That reading misapplies it, and Gustafson's Law (John Gustafson and Edwin Barsis, 1988) explains why.

Amdahl fixes the *problem size* and asks how much time shrinks. Gustafson observes that in practice people do not use bigger machines to solve the same problem faster — they use them to solve *bigger problems in the same time*. If you fix the execution time and let the problem scale with N, and α is the serial fraction of the *parallel* execution, the scaled speedup is:

```
S(N) = N − α(N − 1)
```

This is linear in N, not asymptotically bounded. The two laws do not contradict each other; they answer different questions, and each is correct for its own. **Amdahl governs latency**: how much faster can this

one request get? **Gustafson governs throughput**: how much more work can I do per unit time? For backend services the Gustafson framing is usually the relevant one — you rarely need one request served eight times faster, you need to serve eight times as many requests — which is exactly why horizontal scaling works as well as it does despite Amdahl's ceiling.

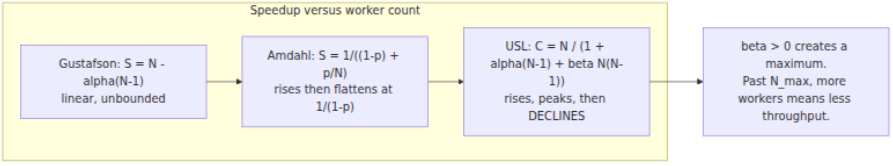
The Universal Scalability Law: why more can be worse

Both Amdahl and Gustafson are optimistic in the same way: they assume that adding capacity is at worst neutral. Reality is harsher. Neil Gunther's **Universal Scalability Law** adds a second penalty term:

$$C(N) = \frac{N}{1 + \alpha(N - 1) + \beta \cdot N(N - 1)}$$

where **α** is the *contention* coefficient (serialization — queueing for a shared resource, the Amdahl term) and **β** is the *coherency* coefficient (crosstalk — the cost of keeping N workers' views of shared state consistent).

The β term is the important one, and it is quadratic. Contention flattens the curve; coherency *bends it back down*. With $\beta > 0$, C(N) has a maximum, after which **adding workers reduces throughput**. That is not a theoretical curiosity — it is the single most commonly observed scaling behavior in production backend systems, and it has a concrete physical cause: cache-line ping-pong between cores (Volume 1, Chapters 4 and 8), lock convoys, and the $O(N^2)$ communication implied by N workers coordinating pairwise.



The operational lesson is direct: **there is an optimal pool size, and it is not "as many as possible."** Thread pools, database connection pools, and worker fleets all exhibit this. The canonical demonstration is connection pooling — teams routinely discover that a pool of 300 database connections delivers *lower* throughput and far worse tail

latency than a pool of 20, because the database is a shared resource whose contention and coherency costs grow with concurrent clients. You find the peak by measurement, not by intuition, and the USL gives you a model to fit measurements to.

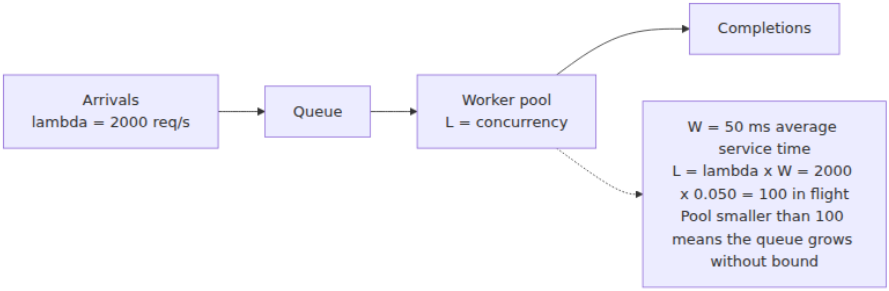
Little's Law: the sizing identity

The most practically useful queueing result, and one of the most general — Little's Law (John Little, 1961) holds for any stable queueing system regardless of arrival distribution, service distribution, or scheduling discipline:

```
L = λ · W
```

where **L** is the average number of items in the system, **λ** the average arrival rate, and **W** the average time an item spends in the system. Stated for a service: *the average number of requests in flight equals the arrival rate times the average latency.*

Its power is that you usually know two of the three and want the third.

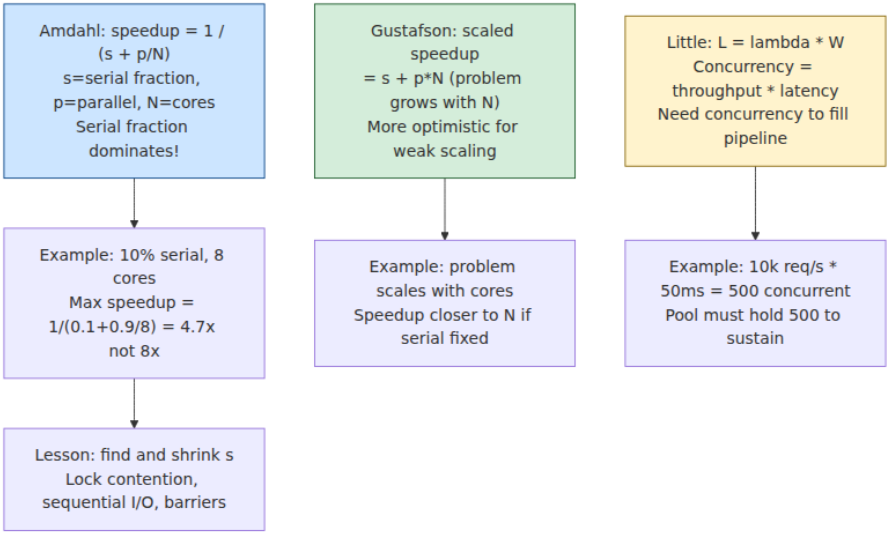


Worked example: your service receives 2,000 requests per second and each takes 50 ms end to end. Then L = 2000 × 0.050 = **100 requests in flight on average**. If your thread pool has 100 workers, you are exactly at capacity with no headroom for variance — a bad place to be. If it has 40, the queue grows without bound and latency diverges. If it has 5,000, you have provisioned 50× the memory and context-switching overhead you need, and you are likely past the USL peak.

Run it the other direction to derive a latency budget: if you have 200 workers and want to hold p50 latency at 20 ms, your sustainable arrival rate is λ = L/W = 200/0.020 = 10,000 req/s.

Two cautions. First, the law applies to a *stable* system — one where arrival rate does not exceed service capacity. Applying it to an overloaded system produces nonsense, because W is diverging. Second, it describes averages, not tails; the tail behavior that actually determines your SLO requires the queueing-theory machinery in Volume 11.

Used together, the laws form a coherent method: **Little's Law sizes the pool, and the USL tells you whether that size is past the point where more workers hurt.** If Little's Law says you need 300 concurrent workers but the USL peak for your workload is at 80, you do not have a pool-sizing problem — you have an architecture problem, and the answer is to reduce W (make requests faster) or to shard the contended resource, not to add workers.



Choosing a model

The decision is mostly determined by the workload, and the errors are predictable.

Workload	Fitting model	Why
High-volume I/O-bound request serving	Event loop, coroutines, virtual threads	Waits dominate; per-task cost must be tiny

Workload	Fitting model	Why
CPU-bound work (transcode, compress, infer)	Bounded pool sized to cores; data parallelism	Compute dominates; more tasks than cores only adds overhead
Stateful coordination (sessions, game state, devices)	Actors	State partitions naturally per entity; serial mailbox removes races
Pipeline / fan-out-fan-in dataflow	CSP channels	Composition and backpressure are first-class
Shared in-process cache or index	Shared memory, sharded locks or lock-free	Genuinely shared data; copying is unaffordable
Bulk numeric transformation	SIMD, GPU, map-reduce	Regular, independent, uniform work

Three failure modes account for most real-world grief:

1. **Blocking calls inside an event loop or coroutine runtime.** One synchronous JDBC call or `time.sleep` in an async handler stalls everything sharing that loop. This is the most common async bug in production, and it is why runtimes provide separate blocking-work pools.
2. **Mixing CPU-bound and I/O-bound work in one pool.** A few expensive tasks occupy every worker and starve hundreds of cheap ones. Separate the pools; give each a bulkhead (Chapter 9).
3. **Sizing pools by intuition.** "More threads means more throughput" is false past the USL peak, and the failure is silent: throughput degrades gradually while latency tails explode.

The distributed-systems lens

Here is the claim that should reframe the rest of this volume: **a distributed system is a concurrent system whose threads run on separate machines, communicate only by messages, and can fail independently.** Every hazard in this chapter reappears at that scale, with weaker guarantees and worse tools.

The correspondences are close to exact:

In-process concurrency	Distributed equivalent	Where
Mutual exclusion (mutex)	Distributed lock, leader election, consensus	Vol 6 Ch6, Ch8
Happens-before (Chapter 3)	Lamport clocks, vector clocks, causal consistency	Vol 6 Ch2
Memory consistency models	Linearizability, sequential and eventual consistency	Vol 6 Ch3, Ch4
Cache coherence between cores	Replication and invalidation between nodes	Vol 6 Ch4
Compare-and-swap (Chapter 4)	Conditional writes, ETags, MVCC, optimistic transactions	Vol 5 Ch6
Actor mailbox	Service endpoint, message queue	Vol 6 Ch9
Deadlock	Distributed deadlock, cyclic RPC waits	Vol 5 Ch6
Lock striping / sharding	Partitioning and sharding	Vol 6 Ch5
Thread starvation	Metastable failure, retry storms	Vol 11

That happens-before appears in both columns is not an analogy: it is *literally the same relation*. Lamport's 1978 paper "Time, Clocks, and the Ordering of Events in a Distributed System" defined happens-before for distributed systems, and the Java Memory Model adopted the same formalism for shared memory two decades later. Learning it once in Chapter 3 pays twice.

The differences are what make distribution harder, and they are worth stating precisely. Within a process, a "thread failure" that leaves shared state half-updated generally takes the whole process down with it — you fail together. Across machines, **partial failure** is the normal case: one node dies while others continue, holding a lock nobody will release, or worse, is merely *slow* and indistinguishable from dead. Message delivery is unreliable and unordered where memory access is not. There is no shared clock. These are why a distributed mutex needs fencing tokens and lease expiry where an in-process mutex needs neither (Chapter 2 closes on exactly this point), and why consensus is a research field while `pthread_mutex_lock` is a library call.

Finally, the USL explains a phenomenon at both scales with one model. Adding threads past the peak reduces throughput because of coherency traffic between cores; adding *nodes* past the peak reduces throughput because of coordination traffic between machines. It is the same β term. This is the quantitative reason that shared-nothing architectures dominate at scale: driving β toward zero by eliminating coordination is the only way to make the curve keep rising, and every technique you will meet in Volume 6 for doing so — partitioning, eventual consistency, CRDTs, caching — is an attack on that one coefficient.

Key takeaways

- **Concurrency is structure; parallelism is execution.** Concurrency is about composing independently-progressing tasks and is meaningful on one core; parallelism is simultaneous execution on multiple units. Concurrency enables parallelism but does not require it.
- **Backend services are overwhelmingly concurrency-bound.** Requests spend the vast majority of their wall-clock time waiting on I/O, so the scarce resource is outstanding waits, not compute. Adding cores to a waiting program buys nothing.
- **A data race and a race condition are different bugs.** A data race is unsynchronized concurrent access with at least one write — mechanical, detectable by ThreadSanitizer, undefined behavior in C/C++. A race condition is a logic-level timing dependency. Data-race-free code can still be thoroughly broken, so a clean race-detector run proves much less than teams assume.
- **The four hazard families** are races (data and logical), atomicity violations, ordering and visibility failures, and liveness failures (deadlock, livelock, starvation).
- **Amdahl and Gustafson answer different questions.** Amdahl fixes problem size and bounds latency speedup at $1/(1 - p)$; Gustafson scales the problem with the machine and yields linear throughput growth. Backend scaling is usually a Gustafson question.
- **The USL is the law that matches production reality.** Its quadratic coherency term β means throughput peaks and then *declines* — there is an optimal pool size and it is not "as many as possible."

- **Little's Law (L = λW) sizes pools from measurements.** 2,000 req/s at 50 ms means 100 requests in flight. Use it with the USL: Little's Law tells you the size you need, the USL tells you whether that size is achievable or whether you must fix the architecture instead.
- **Choose the model from the workload**, and avoid the three classic errors: blocking inside an event loop, mixing CPU-bound and I/O-bound work in one pool, and sizing pools by intuition.
- **A distributed system is concurrency writ large.** The same hazards recur across machines with partial failure, unreliable messaging, and no shared clock. Happens-before is literally the same relation in both settings.

Further reading

- Pike, R., *Concurrency Is Not Parallelism* (Heroku Waza, 2012) — the talk that fixed the distinction in the profession's vocabulary. https://go.dev/blog/waza-talk
- Amdahl, G., "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," *AFIPS Conference Proceedings*, 1967 — the original statement of the ceiling.
- Gustafson, J., "Reevaluating Amdahl's Law," *Communications of the ACM* 31(5), 1988 — the scaled-speedup counterpoint. https://dl.acm.org/doi/10.1145/42411.42415
- Gunther, N., *Guerrilla Capacity Planning* (Springer, 2007) — the Universal Scalability Law, its derivation, and how to fit α and β to measured data.
- Little, J. D. C., "A Proof for the Queuing Formula L = λW," *Operations Research* 9(3), 1961.
- Herlihy, M. and Shavit, N., *The Art of Multiprocessor Programming*, 2nd ed. (Morgan Kaufmann, 2020) — the standard rigorous text on progress conditions and concurrent data structures.
- Goetz, B. et al., *Java Concurrency in Practice* (Addison-Wesley, 2006) — still the best practical treatment of the shared-memory model and its hazards.
- Hoare, C. A. R., "Communicating Sequential Processes," *Communications of the ACM* 21(8), 1978 — the foundation of the CSP model. https://dl.acm.org/doi/10.1145/359576.359585

- Armstrong, J., *Making Reliable Distributed Systems in the Presence of Software Errors* (PhD thesis, KTH, 2003) — the actor model and "let it crash" as realized in Erlang/OTP.
- Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *CACM* 21(7), 1978 — happens-before, and the bridge between this volume and Volume 6.
- Shavit, N. and Touitou, D., "Software Transactional Memory," *PODC*, 1995; and Harris, T. et al., "Composable Memory Transactions," *PPoPP*, 2005 — STM and the composability argument.
- Volume 1, Chapter 4 — Caches and Cache Coherence — the hardware source of the USL's β term.
- Volume 2, Chapter 7 — The Latency Hierarchy — the numbers behind "backend work is I/O-bound."
- Volume 11 — Queueing theory and tail latency, where Little's Law is developed properly.

Chapter 2 — Threads, Mutual Exclusion, and Locks

What this chapter covers. Chapter 1 placed shared-memory-with-locks on the map and noted that you must understand it whether or not you choose it, because it underlies the runtimes of every model that claims to have escaped it. This chapter is that understanding, done rigorously. We begin with the critical-section problem and the three properties any correct solution must have. We then open up a mutex and look at what is actually inside one on Linux — an atomic operation on a word in user space for the uncontended case, and a `futex` syscall to park the thread when contended — because that fast-path/slow-path split explains almost every performance behavior locks exhibit. From there we survey the full family of locking primitives and say honestly when each is appropriate, including the several that are more often wrong than right. We cover condition variables and the mandatory `while` loop that everyone eventually learns the hard way, semaphores, barriers, and latches. Finally we develop the cost model — what contention actually costs in cache-coherence traffic and context switches — and the granularity trade-off that leads to lock striping, which is the in-process ancestor of sharding.

This chapter is about *mechanism*. Chapter 3 supplies the formal memory-model reasoning that explains why these primitives give the visibility guarantees they do; Chapter 5 covers what happens when locking goes wrong in the liveness dimension. Read this one first, but do not consider the topic closed until you have read Chapter 3.

Learning goals — after this chapter you should be able to:

- State the three requirements of a correct mutual-exclusion solution and recognize violations.
- Explain the futex fast-path/slow-path design and predict from it why an uncontended lock costs tens of cycles while a contended one can cost microseconds.
- Choose correctly among spinlocks, adaptive mutexes, reader-writer locks, seqlocks, and RCU, and explain why reader-writer locks so often disappoint.
- Write correct condition-variable code, and explain precisely why the `while` loop is mandatory.
- Distinguish a semaphore from a mutex on the basis of ownership, and know when each fits.
- Reason quantitatively about lock contention in terms of cache-line ping-pong and context switches, and apply lock striping to reduce it.
- Explain why a distributed lock is a fundamentally harder object than an in-process mutex, and what fencing tokens are for.

The critical-section problem

The problem is stated the same way it was in 1965, when Dijkstra formalized it. Multiple threads each have a section of code — the *critical section* — that accesses shared data. We need a protocol governing entry and exit such that three properties hold:

1. **Mutual exclusion.** At most one thread is in the critical section at a time. This is the safety property, and it is the one everyone remembers.
2. **Progress.** If no thread is in the critical section and some threads want to enter, one of them must be able to. The selection cannot be postponed indefinitely, and threads *not* attempting entry must not participate in the decision. This rules out protocols that deadlock when a thread outside the critical section stalls.

3. **Bounded waiting.** There is a bound on the number of times other threads can enter the critical section after a thread has requested entry and before that request is granted. This is the fairness property that prevents starvation (Chapter 5).

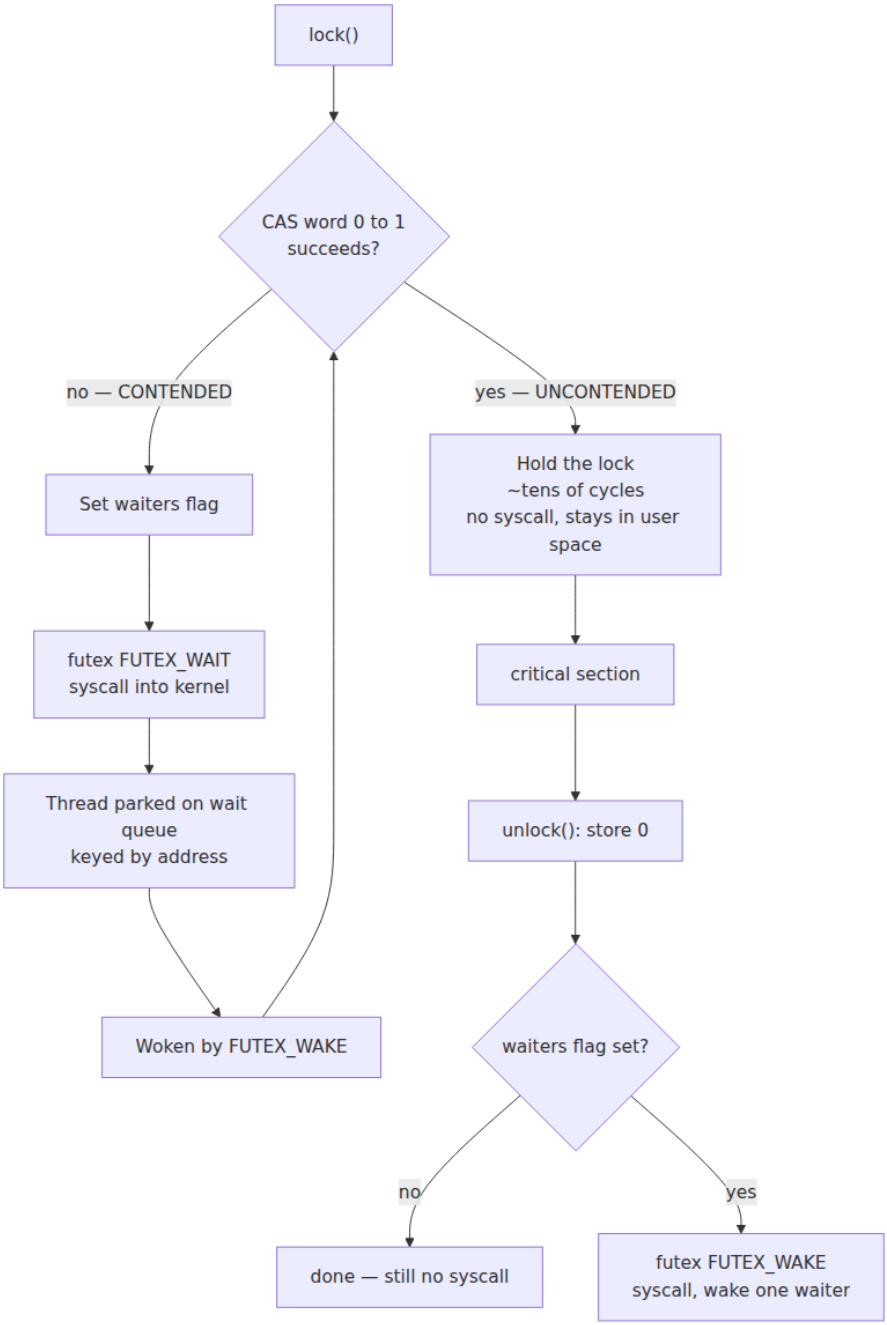
Most bugs in hand-rolled synchronization violate 2 or 3 while satisfying 1, which is why they pass casual testing — the program is *correct*, it just occasionally stops making progress or starves a thread under load. That is also why you should not hand-roll synchronization: Dekker's and Peterson's algorithms are worth studying for insight, but they are wrong on modern hardware unless decorated with memory barriers (Chapter 3), and they are strictly worse than the primitives your platform provides.

What is actually inside a mutex

A mutex is not a kernel object that you call into on every acquisition. If it were, every lock would cost a syscall — hundreds of nanoseconds minimum (Volume 2, Chapter 5) — and shared-memory concurrency would be unusable. The design that makes locks cheap is the **fast path / slow path** split, and on Linux the slow path is built on `futex` (fast userspace mutex).

The mutex is a word in ordinary user memory. Acquisition works like this:

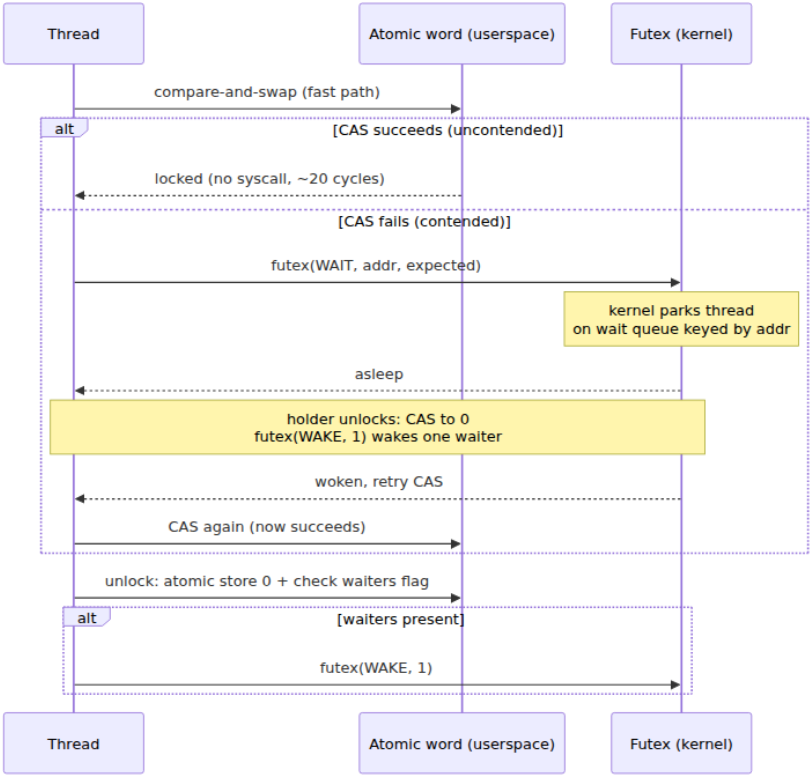
- **Fast path (uncontended).** Atomically compare-and-swap the word from 0 (unlocked) to 1 (locked). If the CAS succeeds, you hold the lock. That is the entire operation: **one atomic instruction, no syscall, no context switch.** It costs on the order of tens of cycles — roughly the cost of an L1 or L2 access plus the atomic's overhead — because the cache line holding the word is already in your core's cache in exclusive state.
- **Slow path (contended).** The CAS fails, meaning someone else holds the lock. Rather than spin forever, the thread marks the word to indicate waiters exist and calls `futex(FUTEX_WAIT, addr, expected)`. The kernel atomically re-checks that the word still holds `expected` — closing the race where the lock was released between the failed CAS and the syscall — and if so, puts the thread to sleep on a wait queue keyed by the address. On release, the holder sees the waiters flag and calls `futex(FUTEX_WAKE, addr, 1)` to wake one waiter.


lock()
CAS word 0 to 1 succeeds?
no — CONTENDED
yes — UNCONTENDED
Set waiters flag
futex FUTEX_WAIT syscall into kernel
Thread parked on wait queue keyed by address
Woken by FUTEX_WAKE
Hold the lock ~tens of cycles no syscall, stays in user space
critical section
unlock(): store 0
waiters flag set?
no
yes
done — still no syscall
futex FUTEX_WAKE syscall, wake one waiter

The consequences of this design are worth stating explicitly, because they drive practical guidance throughout this volume:

- **Uncontended locks are nearly free.** The common advice to "avoid locks because they are slow" is wrong as stated. An uncontended mutex acquisition is a single atomic operation. If your locks are uncontended, they are not your problem.
- **Contended locks are expensive by a factor of hundreds to thousands.** The contended path costs two syscalls plus a context switch out and back — call it several microseconds against tens of nanoseconds. This is why contention, not locking per se, is the thing to measure and attack.
- **The cost is bimodal, not gradual.** As contention rises you do not see a smooth degradation; you see a cliff as acquisitions shift from the fast path to the slow path. This is one reason latency distributions under load develop such ugly tails.

The same architecture appears under other names elsewhere: Windows uses `SRWLOCK` and `WaitOnAddress`, and the JVM's `synchronized` uses a related escalation from thin (CAS-based) to inflated (OS-monitor) locks. The details differ; the fast-path/slow-path principle does not.



The family of locks

Spinlocks

A spinlock busy-waits — repeatedly testing the word in a loop — rather than parking the thread. The trade-off is simple to state and easy to get wrong: **spin when the expected hold time is shorter than the cost of a context switch; park otherwise.** A context switch costs on the order of a microsecond once you count the direct cost plus the cache and TLB pollution that follows (Volume 2, Chapters 1 and 2), so spinning wins only for critical sections measured in tens or hundreds of nanoseconds.

Two rules matter in practice. First, a naive spinlock that hammers an atomic CAS in a tight loop generates continuous cache-line invalidation traffic and can slow down the very thread holding the lock; the standard fix is *test-and-test-and-set* — spin on an ordinary read, and only attempt

the atomic when the read suggests the lock is free — plus a CPU pause hint (`PAUSE` on x86, `YIELD` on ARM) in the loop body.

Second, and more important for backend engineers: **never spin on an oversubscribed core.** If more runnable threads exist than cores, the thread you are spinning to wait for may itself be descheduled, and you will burn your entire time slice waiting for a thread that cannot run until you stop. This is why spinlocks are appropriate in kernels and in carefully-pinned latency-critical code, and are almost always the wrong choice in an application running in a container with a CPU quota — where the scheduler can and will deschedule your lock holder at the worst moment (Volume 2, Chapter 2).

Adaptive mutexes

The pragmatic synthesis: spin briefly, then park. Most implementations spin for a bounded number of iterations on the theory that short critical sections will release quickly, and fall back to the futex path if that fails. Some adapt the spin count based on observed history or on whether the lock holder is currently running on another core. This is the default behavior of `pthread_mutex_t` on glibc under contention and of the JVM's monitors, and it is the right default for application code.

Recursive (reentrant) locks

A recursive lock may be acquired repeatedly by the thread that already holds it, maintaining a count and releasing only when the count reaches zero. Java's `synchronized` and `ReentrantLock` are recursive; `pthread_mutex_t` is not unless you set `PTHREAD_MUTEX_RECURSIVE`.

Recursion is occasionally necessary, but it is more often a signal that you have lost track of your locking discipline. If a function must work both when called with the lock held and when called without, you generally have two functions fighting to be one; the cleaner structure is a public method that acquires the lock and a private `_locked` variant that assumes it, with the invariant documented. Recursive locks also interact badly with condition variables — waiting on a condition releases the lock only once, not the full recursion count, which deadlocks — and they make it much harder to reason about the invariants that hold at lock-acquisition boundaries.

Reader-writer locks, and why they disappoint

An RW lock permits either many concurrent readers or one exclusive writer. The intuition is obvious: reads do not conflict with each other, so let them proceed in parallel. For read-heavy workloads this should be a large win.

It frequently is not, and the reason is worth understanding because it generalizes. To admit multiple readers, the lock must track *how many* readers are active — which means every reader must atomically modify a shared counter on acquisition and again on release. That counter lives in one cache line. Every reader on every core therefore performs a read-modify-write on the same cache line, forcing it to bounce between cores in exclusive state (Volume 1, Chapter 4). The readers do not conflict logically, but their *bookkeeping* conflicts physically on every single operation.

The result is that under high read concurrency with short critical sections, a reader-writer lock can be **slower than a plain mutex**, because a plain mutex touches the same one cache line but does strictly less work per operation. RW locks pay off when critical sections are long enough that genuine reader parallelism outweighs the coherence traffic on the counter — think milliseconds of work under the lock, not nanoseconds.

There is a second problem: **writer starvation**. A steady stream of readers can keep the lock perpetually in read mode, and a waiting writer never gets in. Implementations respond with policy knobs — reader-preferring (maximum read throughput, writers can starve), writer-preferring (new readers block once a writer is waiting, which risks reader starvation and convoying), or fair FIFO queueing (no starvation, but loses much of the concurrency benefit). Know which policy your implementation uses; the default varies by platform and the failure modes are quite different.

Practical guidance: reach for an RW lock only when you have *measured* a read-dominated workload with substantial time under the lock, and measure again afterward. For the common case of a hot in-memory map, sharded plain mutexes or a concurrent map implementation will usually beat an RW lock (Chapter 9).

Seqlocks

A sequence lock optimizes for the case where reads vastly outnumber writes and readers must never block writers. A counter is incremented before and after each write, making it odd during a write and even at rest. A reader snapshots the counter, reads the data, then re-reads the counter: if it changed or was odd, the read was torn and the reader retries.

The properties are unusual. Readers take no locks and perform no writes at all, so there is no coherence traffic from reading — this is the key advantage over an RW lock. Writers are never blocked by readers. The costs: readers may retry indefinitely under write pressure, the protocol requires careful memory barriers to be correct (Chapter 3), and the data being read must tolerate being observed in a torn state and discarded — so it must not contain pointers the reader will dereference. The Linux kernel uses seqlocks for exactly the right sort of thing: reading the system time, which is written rarely and read constantly.

RCU, conceptually

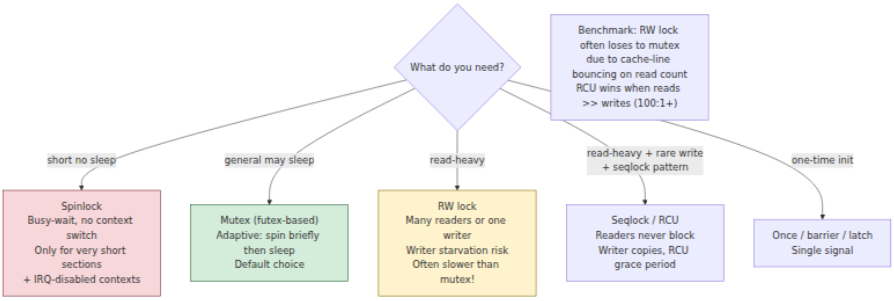
Read-Copy-Update is the Linux kernel's most aggressive answer to the read-mostly problem, and it is worth understanding conceptually even if you never write kernel code, because its idea recurs in user space.

Readers execute with essentially **zero synchronization overhead** — no atomics, no barriers on strongly-ordered architectures, just an ordinary pointer dereference bracketed by markers that delimit a read-side critical section. A writer never mutates data in place. Instead it copies the structure, modifies the copy, and atomically publishes a new pointer. Readers that started before the update continue to see the old version, which remains valid; readers that start after see the new one.

The hard part is reclamation: when can the old version be freed? Only once every reader that might still hold a reference has finished. RCU answers this with the notion of a *grace period* — a wait until every CPU has passed through a quiescent state, guaranteeing no pre-existing reader remains. That deferred-reclamation problem is exactly the one Chapter 4 confronts for lock-free structures, where it is solved with hazard pointers and epoch-based reclamation. Note that garbage-collected languages get this for free, which is a large part of why lock-free and RCU-like patterns are so much easier in Java and Go than in C.

Comparison

Primitive	Reader cost	Writer cost	Blocks readers?	Best fit
Mutex	Full exclusion	Full exclusion	Yes	General purpose; short critical sections
Spinlock	Busy-wait	Busy-wait	Yes	Very short sections, pinned threads, no oversubscription
RW lock	Atomic on shared counter	Exclusive	Yes	Long read sections, measured read-dominance
Seqlock	Two counter reads, may retry	Increment, write, increment	No	Read-mostly small data, e.g. timekeeping
RCU	Essentially free	Copy, publish, wait for grace period	No	Extremely read-dominated; needs reclamation scheme



Condition variables and monitors

A lock provides mutual exclusion. It does not provide a way to *wait for a condition* — "wait until the queue is non-empty," "wait until the connection pool has a free slot." That is what condition variables are for, and together with a mutex they form what Hoare and Brinch Hansen called a **monitor**.

The protocol has three operations. `wait(cv, mutex)` atomically releases the mutex and blocks the calling thread; when it returns, the mutex has been re-acquired. `signal` / `notify` wakes one waiter; `broadcast` / `notifyAll` wakes all of them. The atomicity of release-and-block in `wait` is essential: if the release and the block were separate steps, a signal arriving between them would be lost forever — the **lost wakeup** problem.

The `while` loop is mandatory

Every condition-variable wait must be inside a loop that re-tests the predicate. Not an `if`. A `while`.

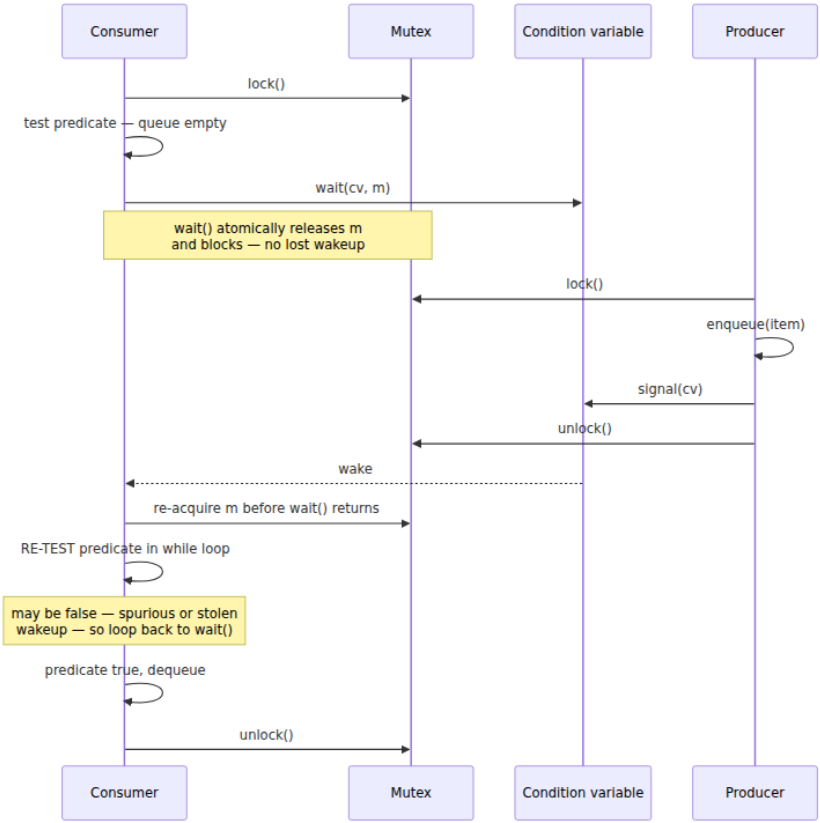
```
pthread_mutex_lock(&m);
while (queue_is_empty(&q)) {        // WHILE, never IF
    pthread_cond_wait(&cv, &m);     // atomically unlocks m and blocks;
}                                   // re-locks m before returning
item = queue_pop(&q);
pthread_mutex_unlock(&m);
```

There are three independent reasons, and each alone is sufficient:

1. **Spurious wakeups.** POSIX explicitly permits `pthread_cond_wait` to return without any corresponding signal. This is not a defect — allowing it makes the implementation simpler and faster on some platforms, and the specification makes the allowance precisely so that correct code must already be robust to it. Java says the same of `Object.wait`.
2. **Stolen wakeups.** Between the signal and your thread actually running, a third thread may acquire the mutex and consume the item you were woken for. You wake, you hold the lock, and the condition is false again.
3. **`broadcast` semantics.** If the code ever uses `notifyAll` — or if multiple distinct conditions share one condition variable — most woken threads will find their predicate false and must go back to sleep.

The `while` loop makes all three harmless with one line, which is why the rule is absolute rather than situational. The equivalent Java is identical in structure:

```
synchronized (lock) {
    while (queue.isEmpty()) {   // WHILE
        lock.wait();
    }
    item = queue.poll();
}
```



A further subtlety: whether you signal while holding the mutex or after releasing it is a performance question, not a correctness one. Signaling while holding can cause the woken thread to immediately block again on the mutex you still hold — the "hurry up and wait" pattern. Modern implementations largely optimize this away with wait-queue transfer, so signal wherever it makes the code clearer, and measure if it matters.

Semaphores, barriers, and latches

A **counting semaphore** holds a non-negative count with two operations: `acquire` (P/wait) decrements, blocking while the count is zero, and `release` (V/post) increments and wakes a waiter. It is a permit dispenser, and it is the natural primitive for bounding access to a pool of N interchangeable resources — connections, buffers, in-flight requests.

The distinction from a mutex is **ownership**, and it is not pedantry. A mutex has an owner: the thread that locked it is the thread that must unlock it, which lets the implementation support recursion, priority inheritance, and error checking. A semaphore has no owner — one thread may acquire and a completely different thread release. That makes a binary semaphore *not* a drop-in mutex, and it makes semaphores the right tool for signaling between threads, which is what people usually build them for.

The most useful backend application is the bulkhead: a semaphore with N permits caps concurrent calls to a downstream dependency, so a slow dependency consumes at most N of your threads instead of all of them. Chapter 9 develops this pattern.

Barriers synchronize a group: every participant blocks at the barrier until all have arrived, then all proceed. This fits phase-structured parallel computation — iterate, synchronize, iterate — and is common in data-parallel work and rare in request handling.

Latches are the one-shot cousin: a counter that counts down to zero and then releases everyone permanently. Java's `CountDownLatch` is the canonical form, and the usual backend use is startup coordination — hold the readiness probe until N subsystems have initialized.

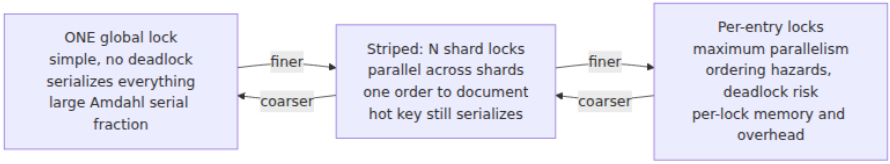
Granularity, striping, and the cost of contention

The granularity trade-off

Coarse-grained locking uses one lock for a large body of state. It is simple, easy to reason about, and nearly impossible to deadlock with a single lock — but it serializes everything, and by Amdahl's Law (Chapter 1) that serial fraction caps your scaling.

Fine-grained locking uses many locks over smaller regions. It permits genuine parallelism, but it multiplies the number of locks a given

operation must hold, which invites deadlock through inconsistent acquisition order (Chapter 5) and adds per-lock overhead that can exceed the parallelism gained.



Lock striping

The standard resolution for hash-based structures is **striping**: partition the data into N independent shards, each with its own lock, and route each key to a shard by its hash. Operations on different shards proceed in parallel; only same-shard operations contend. This was the design of Java's `ConcurrentHashMap` before Java 8 (which moved to per-bin CAS plus synchronized bins, a finer-grained variant of the same idea).

```
final class StripedMap<K, V> {
    private static final int STRIPES = 64;   // power of two
    private final Object[] locks = new Object[STRIPES];
    @SuppressWarnings("unchecked")
    private final Map<K, V>[] shards = new HashMap[STRIPES];

    StripedMap() {
        for (int i = 0; i < STRIPES; i++) {
            locks[i]  = new Object();
            shards[i] = new HashMap<>();
        }
    }

    private int stripeFor(Object key) {
        // spread the hash so that poor hashCode implementations do not
        // collapse onto one stripe
        int h = key.hashCode();
        h ^= (h >>> 16);
        return h & (STRIPES - 1);
    }

    V get(K key) {
        int i = stripeFor(key);
        synchronized (locks[i]) { return shards[i].get(key); }
    }

    V put(K key, V value) {
        int i = stripeFor(key);
        synchronized (locks[i]) { return shards[i].put(key, value); }
    }
}
```

Two caveats travel with striping. First, operations that must span all shards — `size()`, `clear()`, iteration — either take every lock (expensive, and an ordering hazard) or return an approximate answer. Concurrent collections generally choose approximation, which is why `ConcurrentHashMap.size()` is documented as an estimate. Second, striping only helps if keys distribute evenly; a hot key concentrates all its traffic on one stripe and you are back to a single lock. That is exactly the hot-partition problem in distributed sharding (Volume 6, Chapter 5) — the same failure at a different scale.

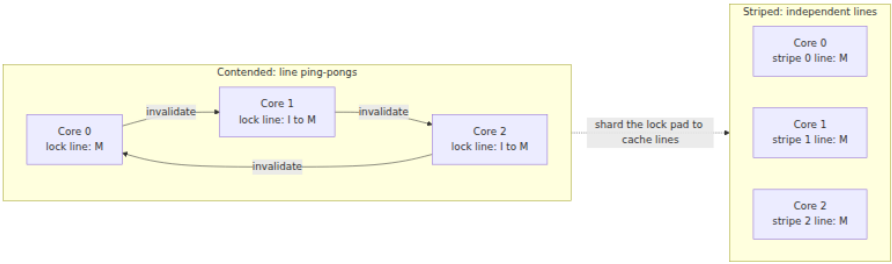
What contention actually costs

The cost model has two components, and only one is the syscall.

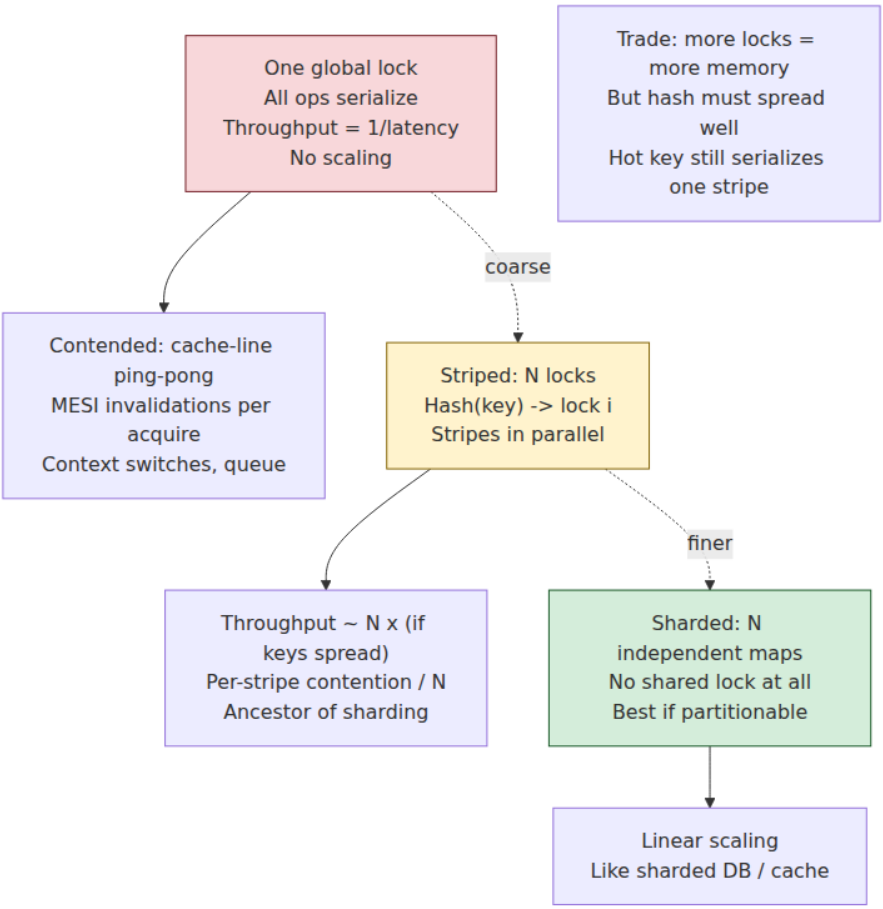
Cache-line ping-pong. A lock word touched by threads on different cores must migrate between their caches. Under the MESI protocol (Volume 1, Chapter 4), each acquisition requires the line in Exclusive/Modified state, invalidating every other copy. On a multi-socket machine, crossing the interconnect costs on the order of 100+ nanoseconds *per transfer*, and the line transfers on every acquisition and release. This cost is paid even when the lock is technically uncontended in the sense that no one blocked — merely being touched by multiple cores is enough.

This is also where **false sharing** bites (Volume 1, Chapter 8): if two independent locks land in the same 64-byte cache line, threads contending for entirely unrelated locks will invalidate each other's line, producing contention that does not exist in the source code. The fix is padding, and in striped designs it is essential — an array of 64 lock objects that share cache lines will perform far worse than the same array padded to a line each.

Context switches. Once a thread parks, you pay the direct switch cost plus the indirect cost of a cold cache and TLB on resumption. This is what turns the contended path from "somewhat slower" into "hundreds of times slower."



The Universal Scalability Law from Chapter 1 now has a concrete physical referent: **α is the serialization from mutual exclusion, and β is this cache-coherence traffic.** That is why adding threads past a point reduces throughput, and why the fix is to eliminate sharing rather than to lock it more cleverly.



Practical guidance

Rules that survive contact with production:

- **Hold locks briefly.** The duration of the critical section is the direct determinant of contention. Compute outside the lock, mutate inside it.
- **Never perform I/O under a lock.** A network call under a mutex converts a millisecond of latency into a millisecond of blocked threads, and it is the fastest route to a lock convoy.
- **Never call unknown code under a lock.** Invoking a user-supplied callback, a listener, or an overridable method while holding a lock

means you have no idea what locks that code will take — it is an open invitation to deadlock (Chapter 5). Java's documented practice of calling listeners outside the lock exists for this reason.

- **Establish and document a lock ordering.** If a code path can hold two locks, there must be a global order and it must be written down. This is the single most effective deadlock prevention.
- **Prefer not sharing at all.** Immutability, thread confinement, and per-thread state eliminate the problem instead of managing it. An immutable object needs no synchronization to be safely read by any number of threads (with the safe-publication caveat of Chapter 3).
- **Measure contention, do not guess at it.** `perf lock`, JFR lock profiling, Go's mutex profiler, and `/proc/lock_stat` will tell you which locks actually block and for how long (Volume 2, Chapter 11). Intuition about which lock is hot is reliably wrong.

Priority inversion deserves a note. A low-priority thread holding a lock can block a high-priority thread, and if a medium-priority thread then preempts the low-priority holder, the high-priority thread waits behind work strictly less important than itself. The Mars Pathfinder lander famously suffered repeated resets from exactly this in 1997. The standard fix is *priority inheritance*: the lock holder temporarily inherits the priority of the highest-priority waiter. This matters in real-time systems; in a normal backend service on CFS it rarely does, but the analogous phenomenon — a low-importance background job holding a lock that request handlers need — is very real, and the fix is the same in spirit: do not let low-priority work hold resources that latency-critical work depends on.

The distributed-systems lens

A mutex provides mutual exclusion among threads sharing memory in one process. The distributed equivalent — mutual exclusion among processes on different machines, implemented with etcd, ZooKeeper, or Redis — looks superficially like the same object and is fundamentally harder. The reasons are exactly the differences catalogued at the end of Chapter 1.

Partial failure. If a thread holding an in-process mutex dies, the process generally dies with it, so the question of an orphaned lock rarely arises. If a *node* holding a distributed lock dies, everyone else is left waiting on a

lock that will never be released. The standard answer is a **lease**: the lock expires automatically after a TTL, and the holder must renew it. This trades a liveness failure for a safety hazard, which brings us to the central problem.

You cannot distinguish a dead node from a slow one. A lock holder that is merely paused — GC pause, page fault storm, a descheduled container, a network blip — may believe it still holds the lock long after its lease expired and another node acquired it. Now two nodes believe they hold mutual exclusion, and if both write to shared storage, the invariant the lock existed to protect is violated.

The fix is a **fencing token**: the lock service issues a monotonically increasing number with each grant, the holder attaches it to every write, and the storage system rejects any write bearing a token lower than the highest it has seen. A stale holder's writes are then refused even if it never learns it lost the lock. Note that this requires the *storage system* to participate — the lock service alone cannot provide the guarantee.

This is the substance of Martin Kleppmann's 2016 critique of the Redlock algorithm. His argument is that Redlock's safety depends on bounded clock drift and bounded pauses — timing assumptions that an asynchronous system cannot guarantee — and that without fencing tokens no lock service can provide mutual exclusion for a resource it does not itself control. Antirez responded defending Redlock's assumptions, and the exchange is worth reading in full; the practical takeaway is uncontroversial regardless of where you land: **if correctness depends on a distributed lock, you need fencing at the resource, and you should prefer a system with a real consensus protocol (Volume 6, Chapter 6) over one built on timeouts.** Chapter 8 of Volume 6 treats this properly.

The second correspondence is scaling. Lock contention is the classic vertical-scaling wall — the α and β terms that cap a single process's throughput. The answer within a process is to partition the contended structure into stripes. The answer across a fleet is to partition the contended data into shards. **These are the same move**, they fail in the same way (hot keys concentrate load on one partition), and they are mitigated the same way (better key distribution, splitting hot partitions, replicating read-heavy ones). If you understand why `ConcurrentHashMap` stripes its locks, you already understand why your database is sharded.

Key takeaways

- Correct mutual exclusion requires **mutual exclusion, progress, and bounded waiting**. Most hand-rolled synchronization satisfies the first and quietly violates the others.
- A mutex is a word in user memory with a **fast path** (one atomic CAS, no syscall, tens of cycles) and a **slow path** (`futex` wait, syscall plus context switch, microseconds). Locking is not slow; *contention* is slow, and the transition between the two is a cliff rather than a slope.
- **Spin only when the hold time is shorter than a context switch, and never when oversubscribed** — which in a CPU-quota'd container means almost never.
- **Reader-writer locks frequently disappoint**, because every reader writes the shared reader counter and that cache line ping-pongs. They pay off only for genuinely long read sections. Watch for writer starvation and know your implementation's policy.
- **Seqlocks and RCU** give readers near-zero overhead by never having them write; the price is retry-on-torn-read for seqlocks and a deferred-reclamation scheme for RCU.
- **Condition-variable waits must sit inside a `while` loop** — spurious wakeups, stolen wakeups, and broadcast semantics each independently require it.
- A **semaphore has no owner**, which distinguishes it from a mutex and makes it the right tool for permits and bulkheads rather than for exclusion.
- **Contention costs cache-line ping-pong plus context switches.** These are the physical referents of the USL's α and β. Lock striping reduces both; padding to cache lines prevents false sharing from re-creating them.
- **Never hold a lock across I/O or a call into unknown code**, always document a global lock order, and prefer immutability and confinement to locking at all.
- A **distributed lock is a different object** from a mutex: partial failure forces leases, and leases plus unbounded pauses force **fencing tokens** enforced at the resource. Lock striping and database sharding are the same idea at different scales.

Further reading

- Dijkstra, E. W., "Solution of a Problem in Concurrent Programming Control," *CACM* 8(9), 1965 — the origin of the critical-section problem.
- Franke, H., Russell, R., and Kirkwood, M., "Fuss, Futexes and Furwocks: Fast Userlevel Locking in Linux," *Ottawa Linux Symposium*, 2002 — the futex design paper.
- Drepper, U., *Futexes Are Tricky* (2011) — the correct-usage reference for building locks on futexes. https://www.akkadia.org/drepper/futex.pdf
- `man 2 futex`, `man 7 pthreads`, `man 3 pthread_cond_wait` — the normative specifications, including the explicit allowance for spurious wakeups.
- Hoare, C. A. R., "Monitors: An Operating System Structuring Concept," *CACM* 17(10), 1974 — the origin of the monitor and condition variables.
- Goetz, B. et al., *Java Concurrency in Practice* (Addison-Wesley, 2006), Chapters 11 and 13 — contention, lock striping, and the `ConcurrentHashMap` design.
- Herlihy, M. and Shavit, N., *The Art of Multiprocessor Programming*, 2nd ed. (Morgan Kaufmann, 2020) — rigorous treatment of spinlock variants and their scaling behavior.
- McKenney, P., *Is Parallel Programming Hard, And, If So, What Can You Do About It?* — the definitive free treatment of RCU by its principal author. https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html
- Kleppmann, M., "How to do distributed locking" (2016) — the fencing-token argument and the Redlock critique. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
- Sanfilippo, S., "Is Redlock safe?" (2016) — the response; read both. http://antirez.com/news/101
- Volume 1, Chapter 4 — Caches and Cache Coherence — the mechanics of the ping-pong described here.
- Volume 2, Chapter 5 — System Calls — why avoiding the syscall on the fast path matters so much.

- Volume 6, Chapter 8 — Distributed Coordination — leases, fencing, and lock services done properly.

Chapter 3 — Memory Models and Happens-Before

What this chapter covers. This is the hardest chapter in the volume, and the most important. Chapter 2 showed you the primitives; this chapter explains why they work, and — more urgently — why code that omits them can observe states that appear logically impossible. The central fact is that **the order you wrote is not the order that executes, and the order that executes is not the order other threads observe.** Compilers reorder, processors reorder, and stores become visible to other cores at different times. None of this is a defect; all of it is deliberate, and all of it is invisible to a single-threaded program by construction. It becomes visible the moment a second thread looks.

We start with why reordering happens at both the compiler and hardware levels, then work the canonical store-buffer litmus test that demonstrates an outcome most engineers initially insist is impossible. We define sequential consistency as the model people wrongly assume they have, compare the hardware models they actually have (x86-TSO versus ARM/POWER), and then move to the practical core: the *language* memory models — Java's, C++'s, and Go's — which are the contracts you actually program against. The unifying concept throughout is **happens-before**, a partial order over memory operations that is the single most valuable formalism in concurrent programming, and one you will meet again, unchanged, in distributed systems.

Learning goals — after this chapter you should be able to:

- Explain the sources of reordering — compiler optimization, store buffers, out-of-order execution, and coherence delays — and why each is invisible to single-threaded code.
- Work the store-buffer litmus test and explain why `r1 == 0 && r2 == 0` is legal on x86.
- Define sequential consistency, and state precisely how x86-TSO and ARM/POWER depart from it.

- Use happens-before correctly: name the edges the Java Memory Model provides and reason about visibility from them.
- State the DRF-SC guarantee precisely, and explain what it does and does not promise.
- Choose C++ memory orders deliberately, and explain what acquire/release actually guarantees.
- Recognize broken double-checked locking and fix it, and apply safe-publication patterns.
- Connect memory consistency to distributed consistency, and happens-before to Lamport clocks.

The problem: your source order is a fiction

Consider this, with `x` and `y` initially zero and no synchronization of any kind:

```
// Thread 1              // Thread 2
x = 1;                   y = 1;
r1 = y;                  r2 = x;
```

Enumerate the interleavings. If Thread 1 runs entirely first, `r1 == 0` and `r2 == 1`. If Thread 2 runs first, `r1 == 1` and `r2 == 0`. If they interleave, both stores precede both loads, so `r1 == 1 && r2 == 1`. Every interleaving yields at least one register holding 1.

The outcome `r1 == 0 && r2 == 0` is nonetheless observable on real hardware, including x86. Run this in a loop on a multi-core machine and you will see it. No interleaving of the program's statements explains it, which means the executed program is not the program you wrote.

Two independent mechanisms produce this.

Compiler reordering

A compiler optimizes under the **as-if rule**: it may perform any transformation that preserves the observable behavior of a *single-threaded* program. Since `x = 1` and `r1 = y` touch different locations and neither depends on the other, swapping them is invisible single-threaded and therefore permitted. Register allocation is more aggressive still: a compiler may hoist a load out of a loop entirely, so that

```
while (!done) { /* spin */ }     // done is a plain non-atomic global
```

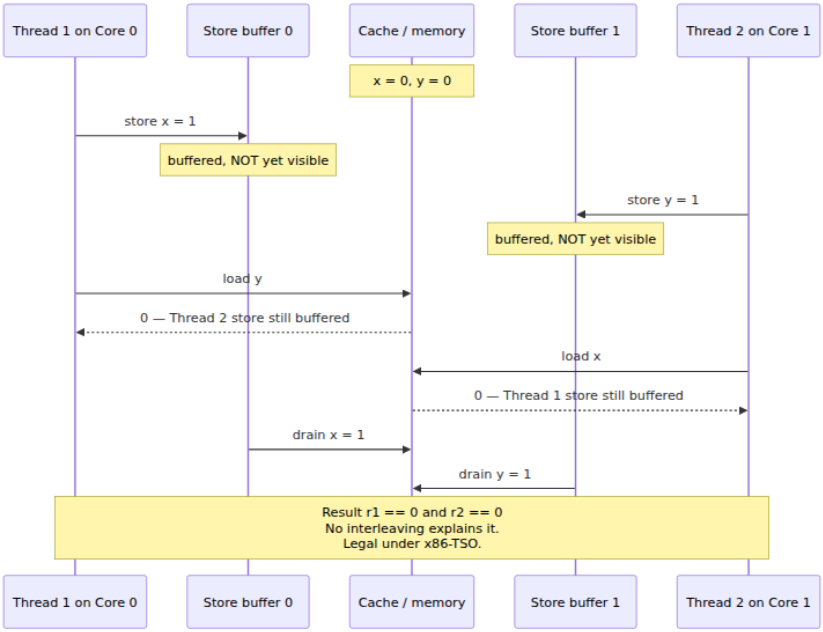
becomes an infinite loop, because the compiler proves nothing in the loop body writes `done` and caches it in a register. This is not a bug — it is a correct optimization under the as-if rule, and it is why `done` must be `volatile` (Java), `std::atomic` (C++), or accessed through `sync/atomic` (Go). Note that C's `volatile` is *not* a synchronization primitive: it suppresses compiler caching but provides no ordering or atomicity guarantees against other threads. Java's `volatile` is a completely different keyword that does provide them.

Hardware reordering

Even with reordering suppressed in the compiler, the processor reorders. The dominant mechanism for our litmus test is the **store buffer**. A store to memory is slow — potentially hundreds of cycles if the line must be fetched in exclusive state — so rather than stall, the core writes into a small FIFO queue and continues executing. The store drains to cache later. Meanwhile, subsequent *loads* execute immediately, and a load may complete before an earlier store in the same thread has become visible to anyone else.

That is exactly our litmus test. Both cores buffer their store and proceed to their load; each load misses the other's still-buffered store; both read 0.

Additional mechanisms compound this: out-of-order execution issues instructions as operands become available rather than in program order; speculative execution runs both sides of a branch; invalidation queues delay the processing of coherence messages so a core may read stale data briefly. Cache coherence (Volume 1, Chapter 4) guarantees that all cores eventually agree on the value of each location, and that writes to a *single* location are seen in a consistent order. It says nothing about the relative order of writes to *different* locations, which is precisely what the litmus test exposes.



Sequential consistency: the model you assume you have

Leslie Lamport defined **sequential consistency** in 1979:

> ... the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.

Two requirements: there exists *some* single global total order of all memory operations, and each thread's operations appear in that order consistently with its own program order. This is the model everyone reasons with intuitively — memory as a single shared object that threads take turns touching — and under it, `r1 == 0 && r2 == 0` is impossible.

No mainstream processor provides it, because enforcing it would require draining the store buffer before every load and largely abandoning out-of-order execution. The cost is far too high, so hardware provides something

weaker and gives you instructions to recover the ordering you actually need, where you actually need it.

Hardware memory models

x86-TSO

x86 and x86-64 implement **Total Store Order**, which is comparatively strong. Under TSO:

- Loads are **not** reordered with other loads.
- Stores are **not** reordered with other stores.
- Loads are **not** reordered with earlier stores *to the same location*.
- **Stores may be reordered with later loads to different locations.** This one reordering is permitted, and it is precisely the store-buffer effect.

So x86 permits exactly one of the four possible reorderings, which is why the litmus test above is the *only* simple way to observe reordering on x86 — and why a great deal of subtly incorrect code appears to work fine when developed and tested exclusively on x86.

ARM, POWER, and RISC-V

These are **weakly ordered**. Essentially any pair of independent memory operations may be reordered: load-load, load-store, store-load, and store-store. ARM additionally does not guarantee multi-copy atomicity in all its variants, meaning two observer cores can disagree about the order in which two stores became visible.

The practical consequence is now a mainstream portability problem rather than an exotic one. AWS Graviton, Ampere, and Apple Silicon are ordinary deployment and development targets. **Code that has been correct in production on x86 for years can fail on ARM**, and it fails rarely, under load, in ways that look like data corruption rather than like a memory-ordering bug. The reordering was always permitted by the language's memory model; x86 simply happened not to exercise it.

The rule follows: **reason from the language memory model, never from the hardware you happen to be testing on.** If your code is correct under the JMM or the C++ model, it is correct everywhere the

compiler targets. If it is merely correct on x86, you have a latent bug with a hardware-refresh trigger.

The message-passing litmus test: where x86 code breaks on ARM

The store-buffer test is the classic, but it is not the one that breaks production code, because few programs depend on that particular pattern. The one that does is **message passing** — the publication idiom every codebase contains somewhere:

```
int  data  = 0;      // plain
int  ready = 0;      // plain — this is the bug

// Thread 1 (producer)        // Thread 2 (consumer)
data  = 42;    // (a)          while (ready == 0) { }   // (c)
ready = 1;     // (b)          r = data;                // (d)
```

The intent is obvious: the consumer spins until the flag is set, then reads the payload. The question is whether `r == 0` is possible at (d).

On **x86-TSO**, no — at the hardware level. Stores are not reordered with stores, so (a) becomes visible before (b); loads are not reordered with loads, so (c) precedes (d). The hardware happens to give you exactly the two orderings this idiom needs. (The *compiler* can still break it by hoisting the load of `ready` out of the loop, which is why this code is wrong even on x86 — but with optimization defeated, the hardware cooperates.)

On **ARM or POWER**, yes, and by two independent routes. Store-store reordering lets (b) become visible before (a), so the consumer sees `ready == 1` while `data` is still 0. Independently, load-load reordering lets (d) execute before (c) completes, so the consumer reads `data` speculatively before it has confirmed the flag. Either alone produces `r == 0`.

This is the concrete shape of "it worked for three years and then we moved to Graviton." The idiom is everywhere — lazy initialization, a worker publishing a completed result, a config pointer swapped in at runtime — and on x86 the hardware silently covered for the missing annotations. The fix is to make the ordering explicit rather than inherited from the ISA: `ready` becomes `volatile` in Java, `std::atomic<int>` with release/acquire in C++, or `atomic.Store` / `atomic.Load` in Go. On x86 those annotations compile to the same plain `MOV` instructions the broken

code already emitted — you pay nothing — while on ARM they emit `STLR` / `LDAR` and the code becomes correct.

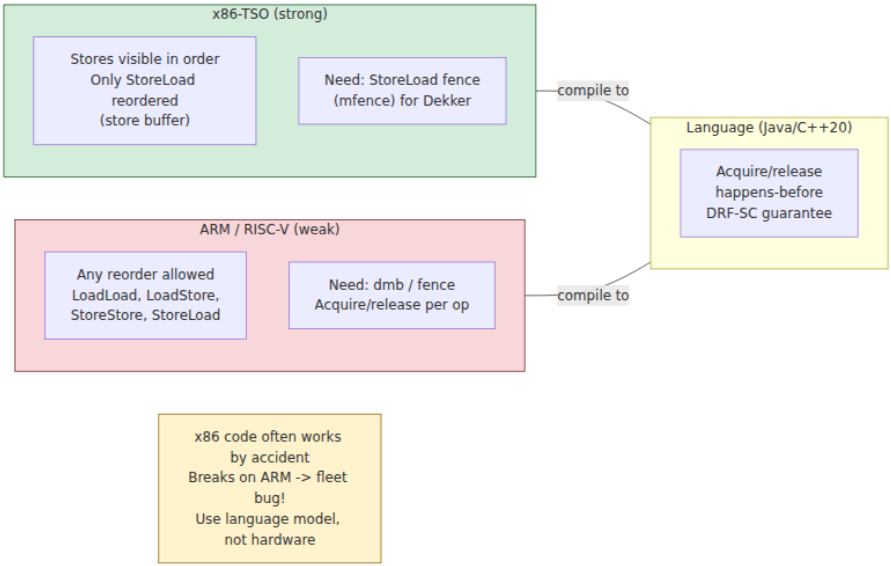
A third pattern, **independent reads of independent writes** (IRIW), is worth knowing exists: four threads, two writing different locations and two reading both in opposite orders. On a multi-copy-atomic machine all observers agree on the order the two writes became visible; on some ARM variants they need not, so two readers can disagree about which write happened first. This is the deepest departure from intuition in commodity hardware, and it is the reason `seq_cst` costs more than release/acquire: `seq_cst` restores a single global order that the machine does not natively provide.

Fences

Hardware provides barrier instructions to recover ordering: on x86 `MFENCE` (full), `LFENCE`, and `SFENCE`, though on x86 a locked read-modify-write instruction acts as a full barrier and is what compilers usually emit; on ARM `DMB`, `DSB`, and the load-acquire/store-release forms `LDAR` / `STLR`. The important categories are:

- **Acquire**: no memory operation after the acquire may be reordered before it. Used on lock acquisition and on reads that publish-check a flag.
- **Release**: no memory operation before the release may be reordered after it. Used on lock release and on the write that publishes a flag.
- **Full barrier**: nothing crosses in either direction.

Acquire and release are the natural pair: a release-store followed by an acquire-load of the same location creates an ordering edge between the two threads, which is the hardware realization of happens-before.



Happens-before: the central formalism

You should almost never reason about store buffers directly. Language memory models give you a better tool: a partial order called **happens-before**, written `→`. The rule that matters is:

> If action A happens-before action B, then the effects of A are visible to B. If neither A → B nor B → A, they are *concurrent*, and if they touch the same location with at least one write, that is a **data race**.

This is the same relation Lamport defined for distributed systems in 1978, adopted for shared memory by JSR-133 two decades later. It is a partial order: transitive, so edges chain, but not total — plenty of pairs are simply unordered, and that is the point.

The edges the Java Memory Model gives you

JSR-133, which fixed the broken original JMM in Java 5, specifies these happens-before edges:

1. **Program order.** Within a single thread, each action happens-before every action later in program order. (Note: this constrains *observed*

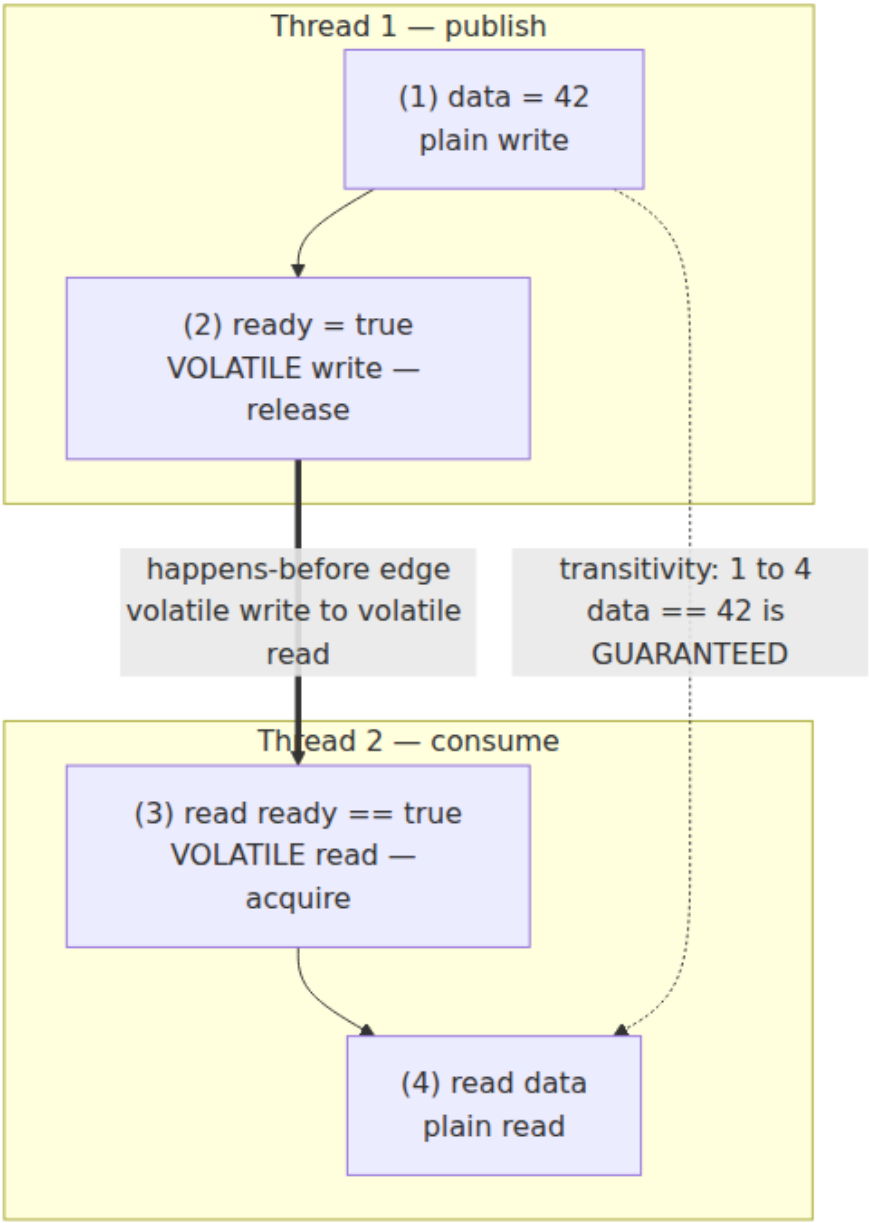
semantics, not the actual execution order — the compiler and CPU may still reorder as long as the thread cannot tell.)

2. **Monitor lock.** An unlock of a monitor happens-before every subsequent lock of that same monitor.
3. **Volatile.** A write to a `volatile` field happens-before every subsequent read of that field.
4. **Thread start.** A call to `Thread.start()` happens-before any action in the started thread.
5. **Thread join.** All actions in a thread happen-before any other thread returns from `join()` on it.
6. **Interruption.** A thread calling `interrupt()` happens-before the interrupted thread detecting it.
7. **Finalizer.** The end of a constructor happens-before the start of that object's finalizer.

Plus transitivity, which is what makes the relation useful: if A → B via program order and B → C via a volatile write/read, then A → C. This is the mechanism by which a volatile flag publishes *everything written before it*, not merely the flag itself:

```
class Publisher {
    private int data;                 // plain field, NOT volatile
    private volatile boolean ready;   // volatile

    void publish() {
        data = 42;         // (1)
        ready = true;      // (2) volatile write — release
    }

    void consume() {
        if (ready) {       // (3) volatile read — acquire
            assert data == 42;  // (4) GUARANTEED to see 42
        }
    }
}
```

The chain is (1) → (2) by program order, (2) → (3) by the volatile rule, (3) → (4) by program order. Transitivity gives (1) → (4), so the read of `data` must observe 42 even though `data` is not itself volatile. Remove `volatile` from `ready` and the chain breaks: there is no edge, the accesses to `data` are concurrent, and the assert can fail — on ARM, routinely.


Thread 1 — publish
(1) data = 42
plain write
(2) ready = true
VOLATILE write —
release
happens-before edge
volatile write to volatile
read
transitivity: 1 to 4
data == 42 is
GUARANTEED
Thread 2 — consume
(3) read ready == true
VOLATILE read —
acquire
(4) read data
plain read

The DRF-SC guarantee

Here is the theorem that makes all of this tractable, and it deserves to be stated precisely:

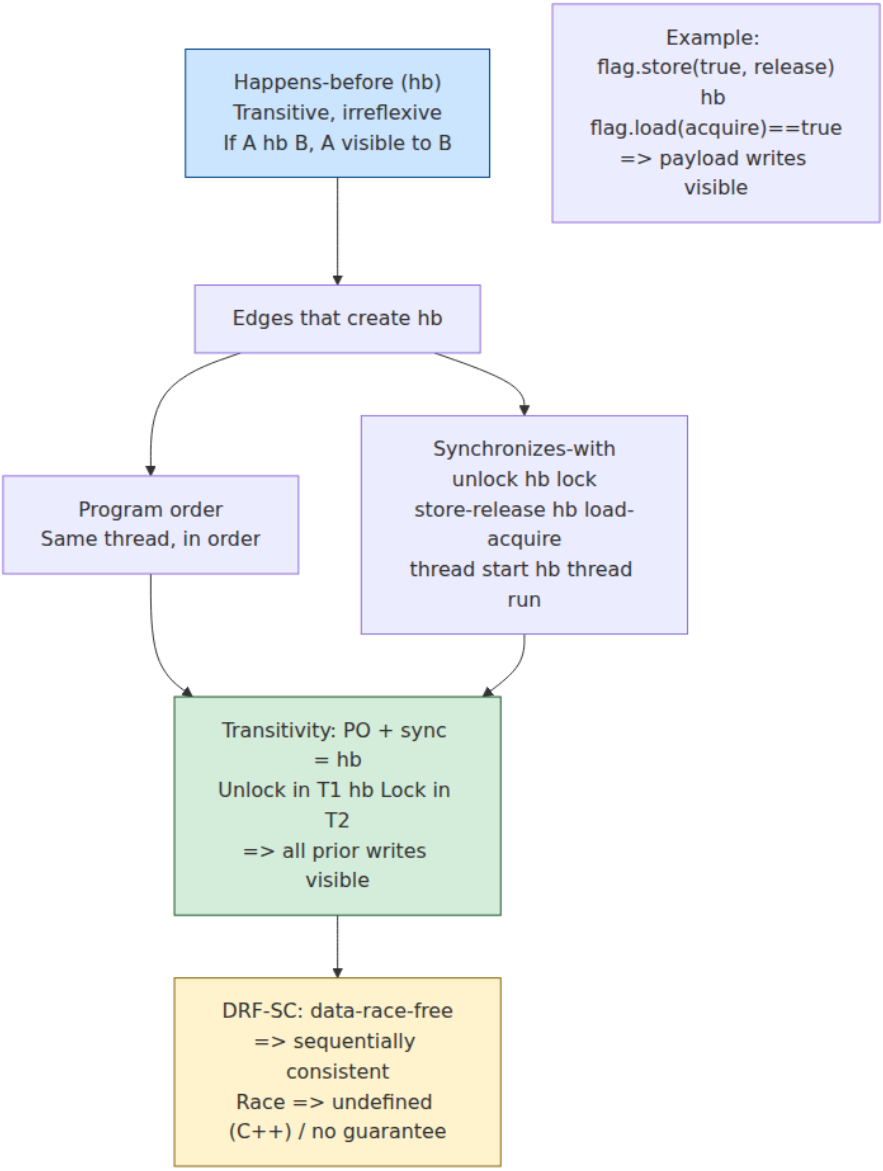
> **If a program is data-race-free — every pair of conflicting accesses is ordered by happens-before — then every execution of that program is sequentially consistent.**

This is the **DRF-SC** guarantee, and it is the central bargain of modern memory models. It says: synchronize properly, and you may go back to reasoning with the simple interleaving model from the start of this chapter. All the store-buffer and reordering complexity is confined to programs that have data races.

Read the guarantee carefully for what it does *not* say. It says nothing about programs that *do* have races, and the consequences there differ sharply by language:

- **C and C++**: a data race is **undefined behavior**. Not "an unpredictable value" — undefined. The compiler may assume races do not occur and optimize accordingly, and the resulting program may do anything at all.
- **Java**: races are not undefined behavior, because the JVM must remain memory-safe for untrusted code. Instead the JMM bounds the damage: you may observe stale or unexpected values, but there are no "out-of-thin-air" values — a read must return a value some write actually wrote. The outcome is weird but not unbounded.
- **Go**: since the 2022 memory model revision, races on most types have implementation-defined but bounded behavior, though races on multiword values such as interfaces and slices can corrupt memory outright.

And note again the point from Chapter 1: DRF-SC gives you sequential consistency, which eliminates *data races* — it does nothing about *race conditions*. A properly synchronized check-then-act is still a bug.


Happens-before (hb)
Transitive, irreflexive
If A hb B, A visible to B
Example:
flag.store(true, release)
hb
flag.load(acquire)==true
=> payload writes
visible
Edges that create hb
Program order
Same thread, in order
Synchronizes-with
unlock hb lock
store-release hb load-
acquire
thread start hb thread
run
Transitivity: PO + sync
= hb
Unlock in T1 hb Lock in
T2
=> all prior writes
visible
DRF-SC: data-race-free
=> sequentially
consistent
Race => undefined
(C++) / no guarantee

Language memory models in practice

C++11 and C11: explicit memory orders

C++11 introduced the first rigorous memory model for the language, with `std::atomic` and explicit `memory_order` arguments. The orders, from weakest:

Order	Guarantee
`relaxed`	Atomicity only. No ordering with respect to other locations.
`consume`	Ordering along data-dependency chains only. Discouraged — see below.
`acquire`	On a load: no later access is reordered before it. Pairs with `release`.
`release`	On a store: no earlier access is reordered after it. Pairs with `acquire`.
`acq_rel`	Both, for read-modify-write operations.
`seq_cst`	Acquire/release plus a single global total order over all `seq_cst` operations. Default.

`memory_order_consume` deserves its own note: it was intended to expose the fact that most hardware respects data dependencies for free, making it cheaper than `acquire` on weak architectures. No compiler implements it as specified — they all promote it to `acquire` — and its specification has been under revision for years. Treat it as effectively deprecated and do not use it.

The release/acquire pair is the workhorse:

```
#include <atomic>
#include <cassert>

int data = 0;                                   // plain
std::atomic<bool> ready{false};

void producer() {
    data = 42;                                              // plain write
    ready.store(true, std::memory_order_release);       // release
}

void consumer() {
    while (!ready.load(std::memory_order_acquire)) {     // acquire
        /* spin — a real implementation should back off */
    }
    assert(data == 42);   // guaranteed: release/acquire synchronizes-
with
}
```

The release store guarantees `data = 42` is not reordered after it; the acquire load guarantees the read of `data` is not reordered before it; and the pairing establishes a *synchronizes-with* edge, which contributes to happens-before. On x86 both compile to plain `MOV` instructions — the hardware already provides these guarantees — so the annotations cost nothing there while remaining necessary for correctness on ARM. That asymmetry is exactly why testing on x86 does not validate your memory ordering.

`seq_cst` is the default because it is the easiest to reason about; it is also the most expensive, typically requiring a full barrier or a locked instruction on stores. Use it unless you have measured a reason not to.

Go

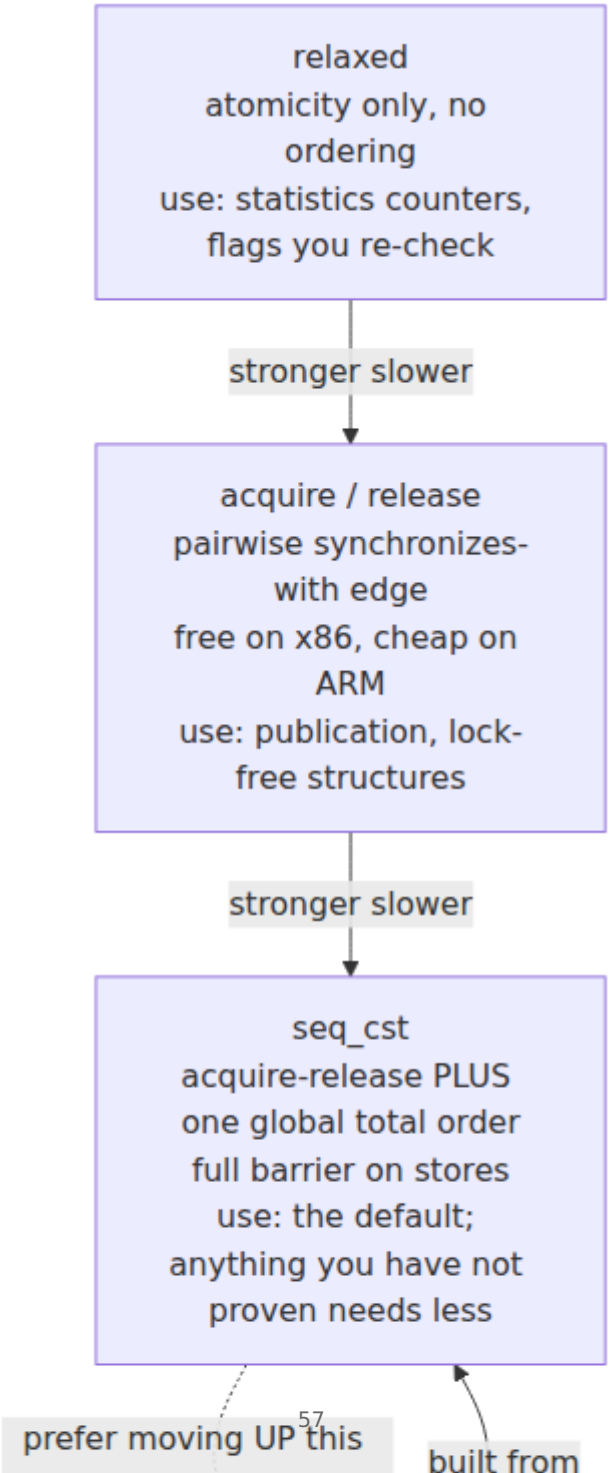
Go's memory model is deliberately minimal, and its guiding advice is worth quoting: *if you must read the rest of this document to understand the behavior of your program, you are being too clever.* The happens-before edges are:

- A send on a channel happens-before the corresponding receive completes.
- A receive from an unbuffered channel happens-before the send completes. (This is the rendezvous property, and it surprises people.)

- The *k*th receive on a channel of capacity *C* happens-before the *(k+C)*th send completes — the rule that makes a buffered channel usable as a semaphore.
- Unlock of a `sync.Mutex` happens-before any subsequent lock.
- `once.Do(f)` — the return of `f()` happens-before any `Do` call returns.
- Goroutine creation happens-before the goroutine starts; a goroutine's exit is *not* ordered with anything by itself.

The 2022 revision of the memory model made an important clarification: the `sync/atomic` operations are now explicitly specified as **sequentially consistent**, aligning Go with the C++ `seq_cst` semantics. Before that, the ordering guarantees of Go's atomics were left informal, and code relying on them was relying on implementation behavior. The revision also formally documented that Go programs with data races have bounded, non-undefined behavior for word-sized types, while warning that races on multiword values can corrupt memory.

The ordering ladder


relaxed
atomicity only, no ordering
use: statistics counters, flags you re-check
stronger slower
acquire / release
pairwise synchronizes-with edge
free on x86, cheap on ARM
use: publication, lock-free structures
stronger slower
seq_cst
acquire-release PLUS one global total order
full barrier on stores
use: the default; anything you have not proven needs less
prefer moving UP this
built from

Publication and safe initialization

Getting an object safely from the thread that constructs it to threads that use it is the most common place these rules bite.

Double-checked locking

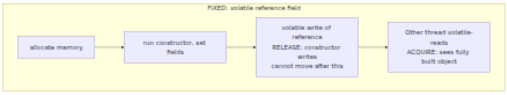
The classic broken idiom, intended to avoid locking on every access to a lazily-initialized singleton:

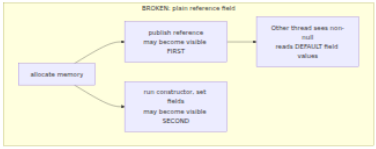
```
// BROKEN — do not use
class Broken {
    private static Resource instance;   // not volatile

    static Resource get() {
        if (instance == null) {                 // (1) unsynchronized
read
            synchronized (Broken.class) {
                if (instance == null) {
                    instance = new Resource();  // (2) construct and
publish
                }
            }
        }
        return instance;                        // (3) may return a
half-built object
    }
}
```

The bug is at (2). `new Resource()` is not atomic — it allocates memory, runs the constructor, and assigns the reference. The compiler and hardware are permitted to make the reference assignment visible *before* the constructor's writes are visible, since within the constructing thread nothing can tell the difference. Another thread executing (1) can therefore observe a non-null `instance` and return at (3) an object whose fields are still default-valued. The failure is rare, timing-dependent, and appears as an inexplicable null or zero field.

Declaring the field `volatile` fixes it under JSR-133: the volatile write at (2) is a release that prevents the constructor's writes from being reordered after it, and the volatile read at (1) is an acquire. Before Java 5 the JMM did not provide this and the idiom was simply unfixable.

```
// Correct, if you insist on DCL
class Correct {
    private static volatile Resource instance;   // volatile is
REQUIRED

    static Resource get() {
        Resource r = instance;          // one volatile read on the
fast path
        if (r == null) {
            synchronized (Correct.class) {
                r = instance;
                if (r == null) {
                    r = new Resource();
                    instance = r;       // volatile write — release
                }
            }
        }
        return r;
    }
}
```





But note that in Java the better answer is usually to avoid DCL entirely. The **initialization-on- demand holder** idiom gets laziness and thread safety from the class-initialization rules, which the JVM already guarantees, with no volatile and no synchronization on the fast path:

```
class Better {
    private Better() {}
    private static class Holder {                 // not initialized
until first use
        static final Resource INSTANCE = new Resource();
    }
    static Resource get() { return Holder.INSTANCE; }   // JVM
guarantees safe publication
}
```

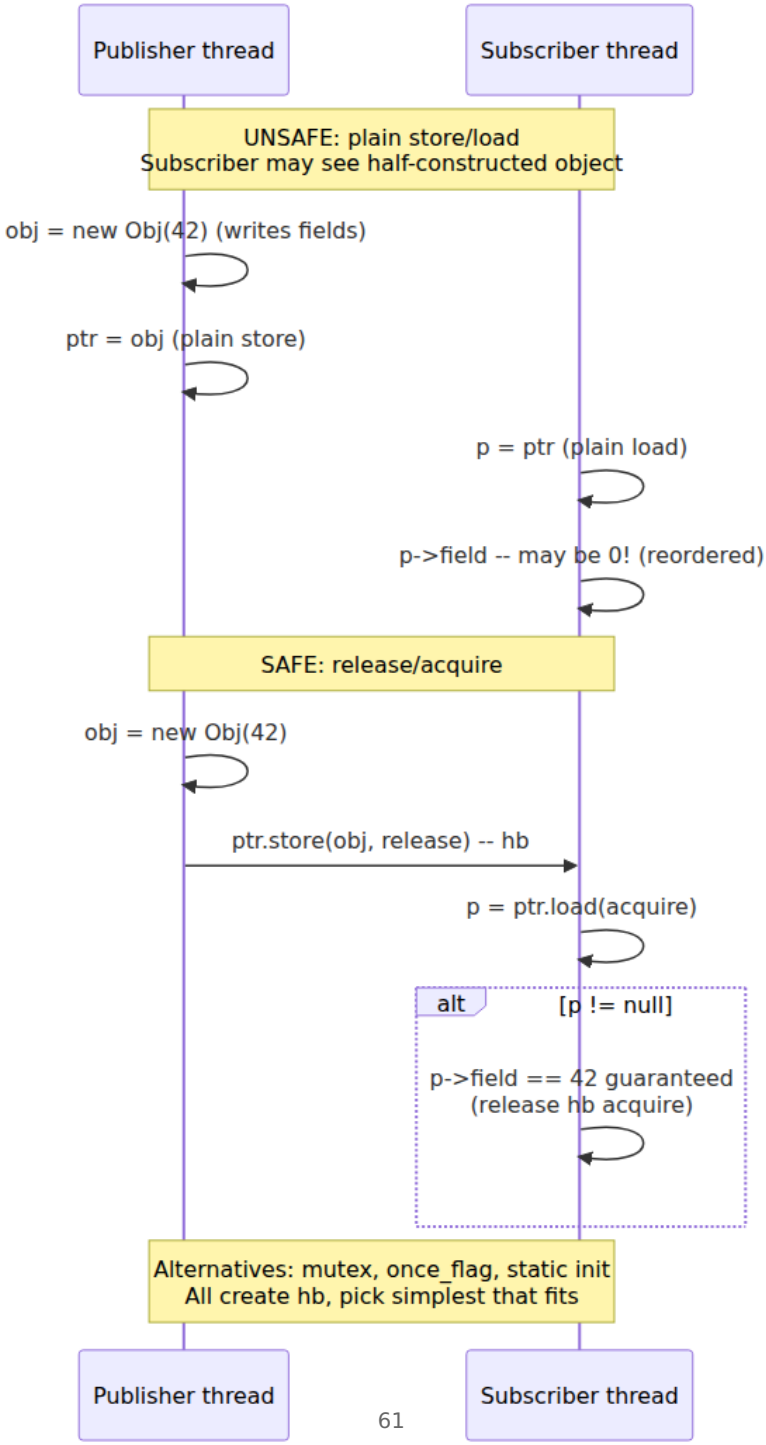
Safe publication and final fields

The general problem is **safe publication**: making an object visible to other threads such that they see it fully constructed. The safe mechanisms in Java are:

- Initialize it from a static initializer (the class-initialization guarantee above).
- Store the reference into a `volatile` field or `AtomicReference`.
- Store it into a field guarded by a lock, and read it under the same lock.
- Store it into a properly-constructed concurrent collection.

There is one more, and it is the best: **immutability**. JSR-133 gives `final` fields a special guarantee — if an object's fields are all `final` and the constructor does not let `this` escape before completing, then any thread that obtains a reference to the object sees the correctly- initialized final fields **even if the reference was published unsafely**. This is why immutable objects can be shared freely with no synchronization at all, and why "make it immutable" is the best available answer to most visibility problems.

The escape caveat is important and easy to violate: registering a listener, starting a thread, or passing `this` to anything from inside a constructor publishes a partially-constructed object and forfeits the guarantee.


Publisher thread
Subscriber thread
UNSAFE: plain store/load
Subscriber may see half-constructed object
obj = new Obj(42) (writes fields)
ptr = obj (plain store)
p = ptr (plain load)
p->field -- may be 0! (reordered)
SAFE: release/acquire
obj = new Obj(42)
ptr.store(obj, release) -- hb
p = ptr.load(acquire)
alt
[p != null]
p->field == 42 guaranteed
(release hb acquire)
Alternatives: mutex, once_flag, static init
All create hb, pick simplest that fits
Publisher thread
Subscriber thread

Practical rules

- **Prefer high-level primitives.** A mutex, a channel, or a concurrent collection gives you the happens-before edges you need, and its author has already reasoned about the memory model. Descending to raw atomics should be a measured decision, not a default.
- **If you write lock-free code, acquire/release is the right default.** `relaxed` is for cases where you genuinely need only atomicity — an approximate statistics counter — and every use of it should carry a comment justifying why no ordering is required.
- **Never reason from x86 behavior.** Correctness is defined by the language model. Test on ARM.
- **Immutability is the cheapest correct answer.** No synchronization, no visibility question, and in Java the final-field guarantee makes even unsafe publication safe.
- **Use the tools.** ThreadSanitizer, the Go race detector, and Java's jcstress exist because human review does not reliably find these bugs (Chapter 10). Run them in CI, not once.

Why these bugs are so hard to find

It is worth being explicit about why memory-ordering bugs have such a bad reputation, because it shapes how you must hunt them.

They are **probabilistic and load-dependent**. The reordering window is a few instructions wide, so the bad interleaving requires two threads to arrive within nanoseconds of each other. That happens rarely at low load and constantly at high load, which is why these bugs reach production: the test suite never reproduces them and the incident does.

They are **erased by observation**. Adding a log statement, attaching a debugger, or compiling at `-O0` inserts work and barriers that close the window. A bug that vanishes under instrumentation is a strong signal you are looking at a memory-ordering or race problem rather than a logic error.

They **present as impossible states**. The symptom is not a crash at the racy line; it is a downstream `NullPointerException` on a field that is assigned in a constructor, or a counter with a value no code path could produce. Engineers reasonably conclude the hardware is faulty or the JVM has a bug. It is neither.

They are **defeated by tools, not by review**. ThreadSanitizer instruments every memory access and maintains a happens-before graph, so it finds races on the interleavings that *did* execute even when they did not produce a visible failure — which is why a TSan run over a normal test suite routinely surfaces bugs that have hidden for years. jcstress goes further for the JVM, running carefully-constructed litmus tests billions of times and recording the observed outcome distribution, which is the only practical way to check claims about the JMM. Use them, and re-read the caution from Chapter 1: they find data races, not race conditions.

The distributed-systems lens

Memory consistency models are the shared-memory ancestor of distributed consistency models. This is not an analogy — it is the same design space one layer up, with the same trade-offs and, in several cases, the same formalism.

Shared memory	Distributed systems
Sequential consistency (Lamport 1979)	Sequential consistency; linearizability (Herlihy & Wing 1990)
Cache coherence per location	Per-object linearizability, single-key consistency
Weak ordering, ARM/POWER	Eventual consistency, causal consistency
Memory barriers	Quorum reads/writes, read-your-writes sessions
Happens-before (JSR-133)	Happens-before (Lamport 1978), vector clocks
Store buffer delaying visibility	Asynchronous replication lag
DRF-SC bargain	"Use transactions and you may reason serially"

The happens-before row is literal identity. Lamport's 1978 paper defined `→` for distributed systems — an event happens-before another if it precedes it in the same process, or if it is a message send whose receive is the other, plus transitivity — and that is structurally the same definition JSR-133 uses with "message send/receive" replaced by "volatile write/read" or "unlock/lock." Vector clocks are the mechanism for *tracking* this

relation across machines; a mutex or volatile field is the mechanism for *establishing* it within one. Learn the relation once and it serves in both places (Volume 6, Chapter 2).

The transferable intuitions are the ones worth internalizing:

Stronger consistency costs performance; weaker consistency costs sanity. `seq_cst` requires barriers that stall the pipeline, exactly as linearizable distributed reads require quorum round-trips or leader leases. Relaxed atomics are fast and treacherous, exactly as eventually-consistent replicas are fast and treacherous. In both settings the engineering discipline is the same: default to the strong model, and weaken only where you have measured a need and can state the invariant that survives.

The store buffer is replication lag. A write sitting in a store buffer, invisible to other cores, is the same phenomenon as a write committed on a primary and not yet applied to a read replica. "I wrote it and then read it back and it was not there" is the same complaint in both settings, and read-your-writes session consistency is the distributed analogue of the rule that a thread always observes its own writes in program order.

The DRF-SC bargain reappears as the transaction bargain. Both say: adopt this discipline, and you may reason with the simple sequential model; violate it, and you are exposed to the full complexity of the underlying system. Serializable transactions are the database's DRF-SC.

Where the analogy breaks is instructive. Shared memory has no partial failure — cores do not independently crash and leave memory half-updated — and no message loss. Distributed systems have both, which is why they need consensus, fencing tokens (Chapter 2), and failure detectors, and why Volume 6 is considerably longer than this chapter.

Key takeaways

- **Source order is not execution order is not observation order.** Compilers reorder under the as-if rule; hardware reorders via store buffers and out-of-order execution. Both are invisible to single-threaded code and both become visible the moment another thread observes.
- The **store-buffer litmus test** produces `r1 == 0 && r2 == 0`, an outcome no interleaving explains, on real x86 hardware.

- **Sequential consistency** is the model you intuitively assume and no mainstream processor provides. **x86-TSO** permits only store-load reordering; **ARM/POWER are weakly ordered** and permit essentially everything. Code correct-on-x86 is not code correct-on-ARM, and ARM servers are now mainstream.
- **Happens-before is the tool to reason with**, not store buffers. Learn the JMM's edges — program order, monitor unlock/lock, volatile write/read, thread start/join — and use transitivity, which is what lets a volatile flag publish everything written before it.
- **DRF-SC**: a data-race-free program executes sequentially consistently. This is the bargain that makes concurrency tractable. It says nothing about racy programs — undefined behavior in C/C++, bounded weirdness in Java — and nothing about race conditions.
- **C++ memory orders** run relaxed → acquire/release → seq_cst; `consume` is effectively deprecated and should not be used. Acquire/release is the right default for lock-free code and is free on x86.
- **Go's model** is intentionally minimal and channel-centric; the 2022 revision specified `sync/atomic` as sequentially consistent.
- **Double-checked locking without `volatile` is broken** because reference publication can be reordered before constructor writes. Prefer the holder idiom in Java.
- **Immutability is the best answer to visibility.** Java's final-field guarantee makes fully-final objects safe to publish even through a data race — provided `this` never escapes the constructor.
- **Memory consistency and distributed consistency are the same design space.** Happens-before is literally Lamport's relation; the store buffer is replication lag; DRF-SC is the transaction bargain.

Further reading

- Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *CACM* 21(7), 1978 — the origin of happens-before. https://dl.acm.org/doi/10.1145/359545.359563
- Lamport, L., "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," *IEEE Trans. Computers* C-28(9), 1979 — the definition of sequential consistency.

- Manson, J., Pugh, W., and Adve, S., "The Java Memory Model," *POPL*, 2005 — the JSR-133 model. https://dl.acm.org/doi/10.1145/1040305.1040336
- *JSR-133: Java Memory Model and Thread Specification Revision*, and Pugh's FAQ — https://www.cs.umd.edu/~pugh/java/memoryModel/jsr-133-faq.html, still the clearest informal explanation of the Java model, including double-checked locking and final-field semantics.
- Adve, S. and Boehm, H.-J., "Memory Models: A Case for Rethinking Parallel Languages and Hardware," *CACM* 53(8), 2010 — why DRF-SC became the industry consensus.
- Boehm, H.-J. and Adve, S., "Foundations of the C++ Concurrency Memory Model," *PLDI*, 2008 — the basis of the C++11 model.
- Sewell, P. et al., "x86-TSO: A Rigorous and Usable Programmer's Model for x86 Multiprocessors," *CACM* 53(7), 2010 — the formalization of x86's actual guarantees.
- *The Go Memory Model* — https://go.dev/ref/mem — including the 2022 revision specifying `sync/atomic` as sequentially consistent.
- cppreference, *std::memory_order* — https://en.cppreference.com/w/cpp/atomic/memory_order — the practical reference, with the litmus examples.
- Preshing, J., *Preshing on Programming* — https://preshing.com/ — the best informal writing on memory ordering, acquire/release, and lock-free correctness.
- Volume 1, Chapters 2 and 4 — Out-of-Order Execution, and Caches and Cache Coherence — the hardware underneath this chapter.
- Volume 6, Chapters 2, 3, and 4 — logical clocks, consistency models, and replication — where every idea here reappears across machines.

Chapter 4 — Atomics, CAS, and Lock-Free Data Structures

What this chapter covers. Chapter 2 built concurrency on mutual exclusion: threads take turns, and a thread that cannot take its turn blocks. This chapter covers the alternative — algorithms that coordinate through atomic read-modify-write instructions and are designed so that

no thread's stall can prevent others from progressing. We define the progress hierarchy precisely, because "lock-free" is a technical term that is constantly misused as a synonym for "fast." We cover the hardware primitives — atomic RMW operations, compare-and-swap, and the LL/SC pair on ARM and POWER — and the CAS retry loop that is the fundamental idiom. We then confront the two problems that make lock-free programming genuinely hard rather than merely fiddly: the **ABA problem**, and **safe memory reclamation**, which is the real reason lock-free structures are so much harder in C++ than in Java. We work through the classic structures — the Treiber stack, the Michael-Scott queue, and the bounded ring buffer as realized in the LMAX Disruptor — and close with the most practically valuable lesson in the chapter, which is about counters.

A warning belongs up front. Lock-free programming demands complete fluency with Chapter 3; if happens-before and acquire/release are not solid, this material will read as plausible and you will write subtly broken code. The honest recommendation for nearly all backend work is to *use* the library implementations described here and not to write your own.

Learning goals — after this chapter you should be able to:

- State the progress hierarchy — blocking, obstruction-free, lock-free, wait-free — and explain why lock-free is a progress guarantee rather than a performance one.
- Write a correct CAS retry loop and explain each memory-order choice in it.
- Explain the ABA problem with a concrete failure trace, and name the standard mitigations.
- Explain why memory reclamation is the hard part of lock-free structures, and describe hazard pointers and epoch-based reclamation.
- Describe the Treiber stack and Michael-Scott queue and their limitations.
- Explain why a shared atomic counter is a scalability disaster and what to do instead.
- Decide, with reasons, when lock-free is worth it and when a mutex is the better engineering call.

The progress hierarchy

These terms have precise definitions from the theory of concurrent objects, and the precision matters because the marketing usage is wrong.

- **Blocking.** A thread's delay can delay others indefinitely. Any mutex-based algorithm is blocking: if the lock holder is preempted, descheduled, or page-faults, every waiter is stuck until it runs again. Note that this is a statement about *possible* delay, not about typical behavior.
- **Obstruction-free.** A thread makes progress if it runs in isolation for long enough — that is, if contention ceases. This is the weakest non-blocking guarantee; it permits livelock, since two threads may repeatedly abort each other forever.
- **Lock-free.** *Some* thread makes progress in a bounded number of system-wide steps. The system as a whole always advances, but any *individual* thread may starve indefinitely, repeatedly losing its CAS to more fortunate threads.
- **Wait-free.** *Every* thread completes its operation in a bounded number of its own steps. The strongest guarantee, and it eliminates starvation entirely. It is also the hardest to achieve; wait-free versions of common structures exist but are usually slower in the common case, which is why they are rare outside real-time systems.

The essential misunderstanding to clear up:

> **Lock-free does not mean fast. It means no thread's stall can block all others.**

A lock-free algorithm can be *slower* than a mutex-based one under most conditions. The CAS retry loop does redundant work under contention — every losing thread throws away its computation and starts over — while a mutex queues waiters and does each unit of work exactly once. What lock-free buys is a *guarantee about the tail*: a thread that is preempted while holding a mutex stalls everyone for a scheduling quantum, and that shows up as a multi-millisecond spike at p99.9. In a lock-free structure, a preempted thread inconveniences only itself.

That property is decisive in three situations: latency-critical paths where tail behavior dominates the SLO; contexts where blocking is *forbidden* rather than merely undesirable — a signal handler cannot take a mutex,

because it may have interrupted the thread that holds it, deadlocking instantly; and real-time systems with hard deadlines. Outside those, the case for lock-free is a performance case that must be made with measurements.

The hardware primitives

All of this rests on atomic read-modify-write instructions: operations that read a location, compute a new value, and write it back **indivisibly**, with no window for another core to interleave.

The common ones are atomic exchange (swap), test-and-set, fetch-and-add, and fetch-and-{or,and,xor}. `fetch_add` is worth singling out because it is *unconditional* — it always succeeds in one operation, with no retry loop — which makes it substantially cheaper than CAS under contention and is the reason a `fetch_add` counter beats a CAS-loop counter.

Compare-and-swap

The universal primitive is **compare-and-swap**: atomically, *if* the location holds `expected`, store `desired` and report success; otherwise report failure and (in the C++ form) write back what was actually found.

```
bool compare_exchange_strong(T& expected, T desired);
```

CAS is theoretically special: Herlihy's 1991 consensus-number result shows CAS has consensus number ∞, meaning it can implement a wait-free version of *any* concurrent object for any number of threads, whereas atomic read/write alone has consensus number 1 and fetch-and-add has 2. Every lock-free structure in practice is built on CAS, and this is why.

On x86 it is `LOCK CMPXCHG`. On ARM and POWER it is built from a **load-linked/store-conditional** pair: `LDREX` / `STREX` or `LDAXR` / `STLXR` monitor an address, and the store succeeds only if nothing wrote it since the load. LL/SC is strictly more expressive than CAS — it detects *any* intervening write, not merely a changed value, which as we will see makes it immune to ABA — but it can fail spuriously due to cache-line events, context switches, or interrupts, so it must always sit in a retry loop. This is why C++ offers both `compare_exchange_strong` (retries internally to hide spurious failure) and `compare_exchange_weak` (may fail spuriously,

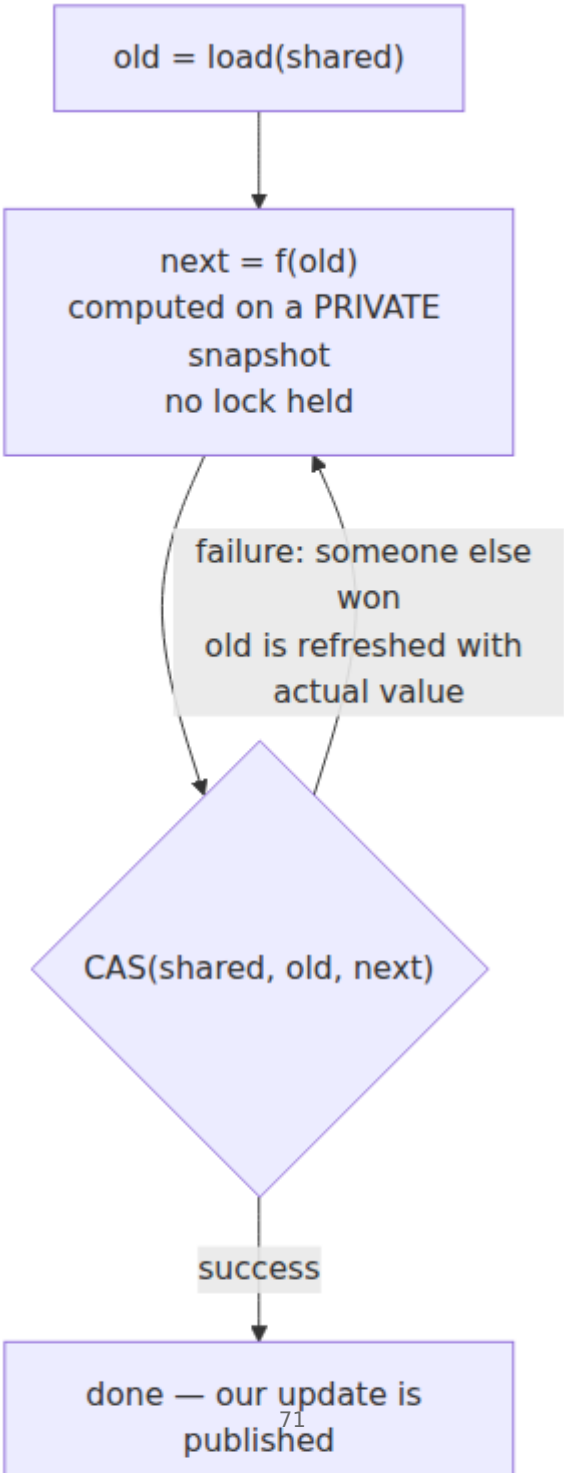
cheaper, correct only inside a loop). **Use `_weak` inside a retry loop and `_strong` when you are not already looping.**

The CAS retry loop

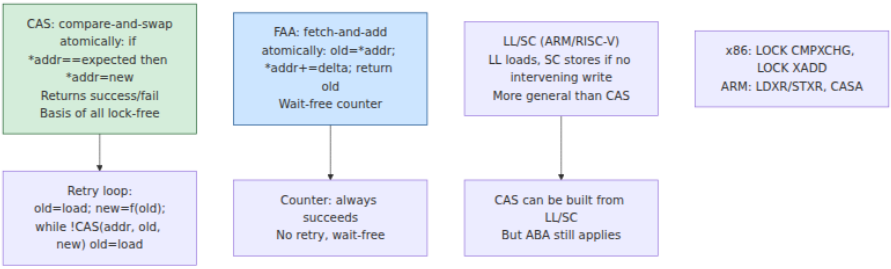
The fundamental idiom of lock-free programming: read the current value, compute the new one, attempt to swap it in, and retry from the top if someone beat you.

```
// Atomically apply an arbitrary function to a shared value.
template <typename T, typename F>
void atomic_update(std::atomic<T>& val, F fn) {
    T old = val.load(std::memory_order_relaxed);   // relaxed: the CAS
validates it
    T next;
    do {
        next = fn(old);                            // compute on a
private snapshot
    } while (!val.compare_exchange_weak(
                old, next,
                std::memory_order_release,         // on success:
publish our writes
                std::memory_order_relaxed));       // on failure: just
reload, no ordering
    // On failure compare_exchange_weak writes the observed value into
`old`,
    // so the loop re-computes from fresh data automatically.
}
```

Three details are load-bearing. The initial load can be `relaxed` because the CAS itself validates the value — if it changed, we retry. The success order is `release` so that writes we performed before publishing become visible to whoever later acquires this value. And `compare_exchange_weak` updates `old` in place on failure, which is what makes the loop re-read for free.


old = load(shared)
next = f(old)
computed on a PRIVATE
snapshot
no lock held
failure: someone else
won
old is refreshed with
actual value
CAS(shared, old, next)
success
done — our update is
published

Notice the shape of the cost. Under low contention this is one atomic operation, cheaper than a mutex. Under high contention, N threads each compute and N−1 discard the work — throughput collapses toward the serialized case while burning CPU on all cores. This is the USL's β term from Chapter 1 in its purest form, and it is why lock-free is not a general-purpose speedup.



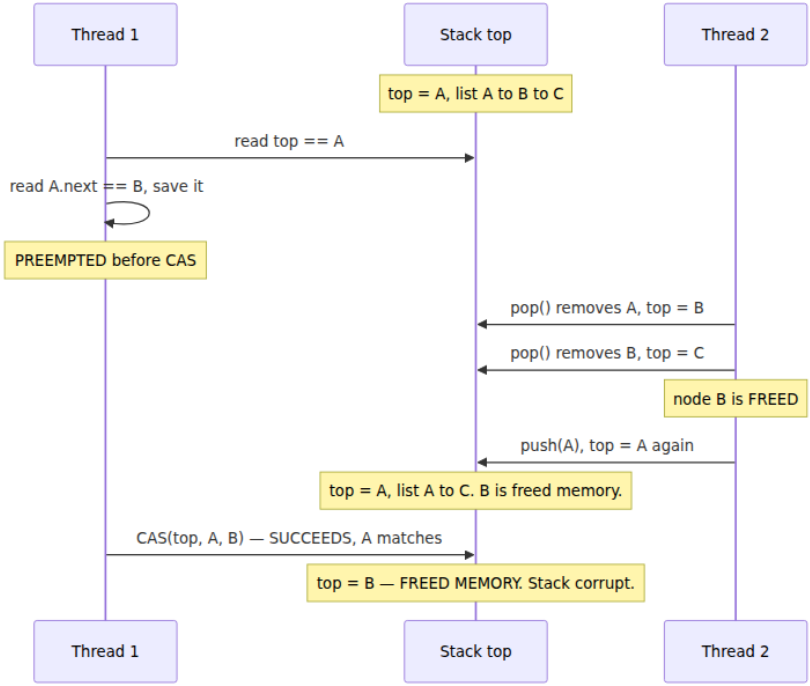
The ABA problem

The subtle hazard that makes naive CAS insufficient. CAS answers the question "*is this location's value still X?*" — but the question you actually needed answered is "*is this location unchanged?*" Those differ when a value can return to a previous state.

Consider a Treiber stack holding A → B → C, with `top` pointing at A.

1. Thread 1 begins `pop()`. It reads `top == A`, reads `A.next == B`, and is preempted just before its CAS.
2. Thread 2 runs to completion: it pops A, pops B, then pushes A back. The stack is now A → C, and node B has been **freed**.
3. Thread 1 resumes and executes `CAS(top, A, B)`. The comparison succeeds — `top` really is A again — so `top` is set to **B**, a node that has been freed and may hold arbitrary reused memory.

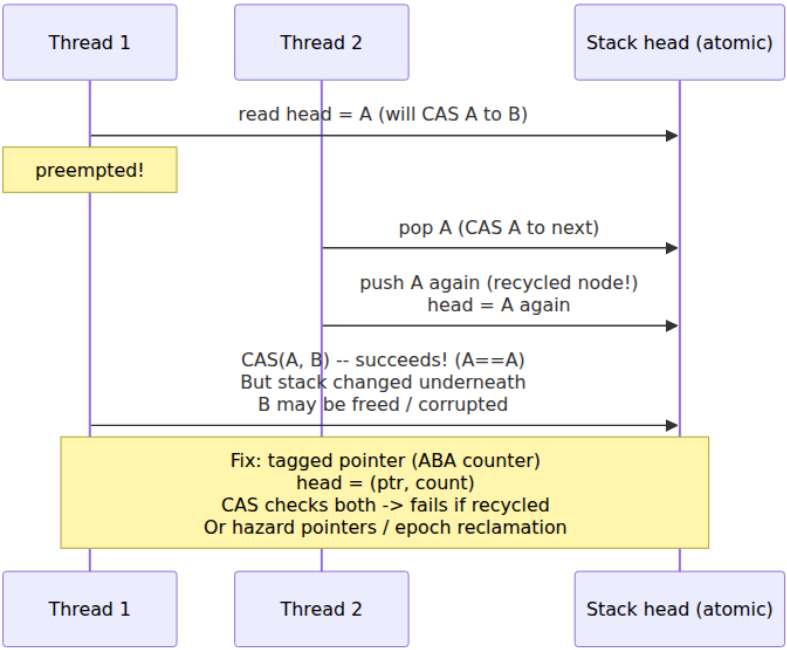
The stack is now corrupt, pointing at freed memory. No individual operation was incorrect; the CAS did exactly what it promised. The flaw is that the pointer value A was reused, and CAS cannot distinguish "unchanged" from "changed back."



Mitigations, in rough order of practicality:

- **Tagged / versioned pointers.** Pack a monotonically increasing counter alongside the pointer and CAS both together, so any modification bumps the version and a stale CAS fails. Requires a double-width CAS (`CMPXCHG16B` on x86-64) or stealing unused pointer bits — on x86-64 the top 16 bits are unused by current implementations, which is a portability bet rather than a guarantee. This is the most common fix.
- **LL/SC.** On ARM and POWER, store-conditional fails on *any* intervening write, so ABA cannot occur. This is a genuine architectural advantage, and it means an algorithm can be ABA-safe on ARM and ABA-broken on x86 — the reverse of the usual portability direction from Chapter 3.
- **Deferred reclamation.** Hazard pointers or epoch-based reclamation, below. If a node is never freed while a reader might hold it, it cannot be reused, and ABA on that pointer disappears.

- **Garbage collection.** In Java or Go the collector will not reclaim a node any thread still references, so the "freed and reused" step cannot happen. ABA can still occur logically if you reuse *values*, but the memory-corruption form is eliminated.



Memory reclamation: the genuinely hard part

Here is the problem that separates lock-free programming from merely difficult programming, and it is the one most treatments underweight.

In a lock-free structure, a thread can hold a pointer to a node it read from the structure. Another thread may concurrently remove that node. **When is it safe to free it?** There is no lock to tell you no readers remain, and no bound on how long a reader might be descheduled while holding the pointer. Free too early and you have a use-after-free — the most exploitable class of memory bug. Never free and you have a leak.

This is the same question RCU answers with grace periods (Chapter 2), and the two standard solutions are:

Hazard pointers (Maged Michael, 2004). Each thread publishes, in a per-thread slot, the pointers it is currently dereferencing — its "hazards." Before freeing a node, a thread scans all published hazard pointers; if any thread has published this node, the free is deferred to a retire list and retried later. The properties are good: bounded memory (each thread protects a fixed number of nodes) and wait-free reads. The costs are a store and a memory barrier on every read to publish the hazard, and an O(threads) scan on every reclamation attempt. C++26 standardizes them as `std::hazard_pointer`; before that, folly and libcds are the practical sources.

Epoch-based reclamation. A global epoch counter advances periodically. Each thread announces the epoch it entered when starting an operation. A node retired in epoch *e* may be freed once every thread has been observed in an epoch later than *e*, which proves no thread can still hold a pre-retirement reference. Reads are much cheaper than with hazard pointers — typically one relaxed store to announce the epoch, no per-pointer bookkeeping — but memory usage is unbounded in the presence of a stalled thread: a single thread that stops in an epoch and never advances prevents all reclamation, and memory grows without limit. Crossbeam in Rust and much of the Java ecosystem's internals use this approach.

Scheme	Read cost	Memory bound	Failure mode
Hazard pointers	Store + barrier per pointer	Bounded	Slower reads; O(N) reclaim scan
Epoch-based	One relaxed store per operation	**Unbounded**	A stalled thread blocks all reclamation
RCU	Near zero	Bounded by grace-period latency	Long grace periods under load
GC (Java, Go)	Zero	Managed by collector	GC pauses; no manual control

The practical conclusion is the one that should shape your engineering: **in a garbage-collected language, lock-free programming is dramatically easier**, because the hardest problem is solved for you by the runtime. This is a substantial and underappreciated argument for Java and Go in concurrency-heavy systems, and it explains why

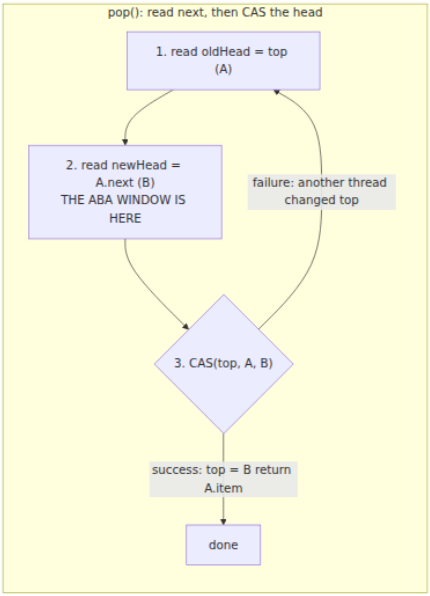
`java.util.concurrent` offers a rich set of lock-free structures while equivalent C++ code generally lives in specialized libraries.

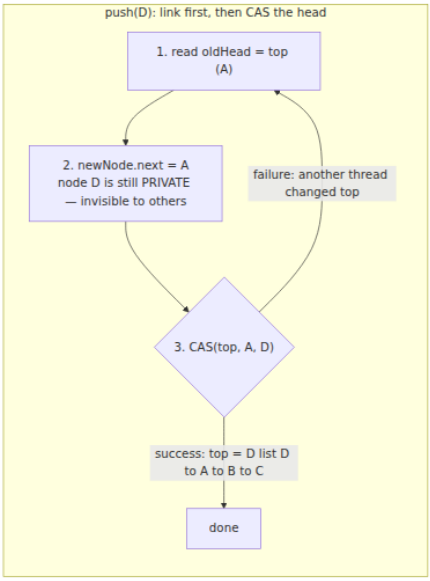
The classic structures

Treiber stack

The simplest lock-free structure (R. K. Treiber, 1986): a singly-linked list where push and pop both CAS the head pointer.

```
public class TreiberStack<E> {
    private static final class Node<E> {
        final E item; Node<E> next;
        Node(E item) { this.item = item; }
    }
    private final AtomicReference<Node<E>> top = new AtomicReference<>(
);

    public void push(E item) {
        Node<E> newHead = new Node<>(item);
        Node<E> oldHead;
        do {
            oldHead = top.get();
            newHead.next = oldHead;
        } while (!top.compareAndSet(oldHead, newHead));
    }

    public E pop() {
        Node<E> oldHead, newHead;
        do {
            oldHead = top.get();
            if (oldHead == null) return null;
            newHead = oldHead.next;
        } while (!top.compareAndSet(oldHead, newHead));
        return oldHead.item;
    }
}
```





Note where the danger sits. In `push`, the new node is private until the CAS publishes it, so the window between reading `top` and swapping is harmless — a losing CAS simply retries. In `pop`, the thread reads `A.next` *before* the CAS and then acts on that stale reading, which is precisely the window the ABA trace above exploits. Asymmetries like this are typical: in lock-free code, the correctness argument usually turns on exactly which values were read before the commit point and whether anything could have invalidated them.

This is correct in Java because the GC handles reclamation. The same code in C++ requires a full ABA and reclamation strategy. Its practical limitation is that **every operation contends on one pointer**, so it scales poorly — it is a demonstration piece, not a high-throughput structure. Real implementations add an *elimination array*, where a push and a pop that collide can cancel each other out and both leave without touching `top` at all — a neat inversion of the usual goal, since it turns contention into an opportunity rather than a cost.

Michael-Scott queue

The standard lock-free MPMC (multi-producer, multi-consumer) FIFO queue (Michael and Scott, 1996), and the basis of

`java.util.concurrent.ConcurrentLinkedQueue`. It uses separate head and tail pointers with a sentinel node, so producers and consumers contend on different cache lines rather than a single hot pointer.

Its defining trick is **helping**, and it is the idea worth taking away. Enqueue is logically two steps — link the new node to the current tail, then advance the tail pointer — and a thread can be preempted between them, leaving the queue in an intermediate state with the tail lagging. Rather than wait, any thread that observes a lagging tail **completes the other thread's operation** before proceeding with its own. This is how the algorithm achieves lock-freedom: a stalled thread never blocks the structure, because whoever arrives next finishes its work. Helping is a general technique in non-blocking algorithms and the reason they can tolerate arbitrary preemption.

The caveat: linked-node queues allocate per element and scatter nodes across memory, so every traversal is a potential cache miss. For high-throughput pipelines a bounded array-based queue usually wins on raw numbers despite being algorithmically less interesting.

Bounded ring buffers and the LMAX Disruptor

The highest-throughput designs in practice are usually **bounded ring buffers** over a pre-allocated array, and the canonical example is the LMAX Disruptor. Its lesson is that *mechanical sympathy matters as much as the algorithm*:

- **Pre-allocated array of fixed power-of-two size.** No per-operation allocation, no GC pressure, and the modulo becomes a bitmask. Entries are contiguous, so traversal is prefetcher-friendly (Volume 1, Chapter 3).
- **Sequence numbers rather than pointers.** Producers and consumers publish monotonically increasing sequences; a consumer may read up to the published producer sequence. No CAS on a shared head pointer in the single-producer configuration — just a store.
- **Single-writer principle.** If exactly one thread writes a given sequence counter, it needs no CAS at all, only an ordered store. Eliminating contention beats optimizing it.
- **Cache-line padding everywhere.** Producer and consumer sequence counters are padded so they occupy separate cache lines. Without

padding, the producer's writes invalidate the line the consumer is reading on every single publish — false sharing (Volume 1, Chapter 8) — and throughput falls by an order of magnitude. **This padding, not the algorithm, is often the difference between good and terrible numbers.**

The Disruptor's reported throughput advantage over `ArrayBlockingQueue` came substantially from these memory-layout decisions rather than from lock-freedom as such. That is the durable lesson: on modern hardware, data layout frequently dominates algorithmic cleverness.

Counters: the lesson that pays for the chapter

If you take one practical thing from this chapter, take this.

A shared counter incremented by many threads is the single most common scalability disaster in backend services — metrics, request counts, bytes transferred, cache hits. The naive implementations are:

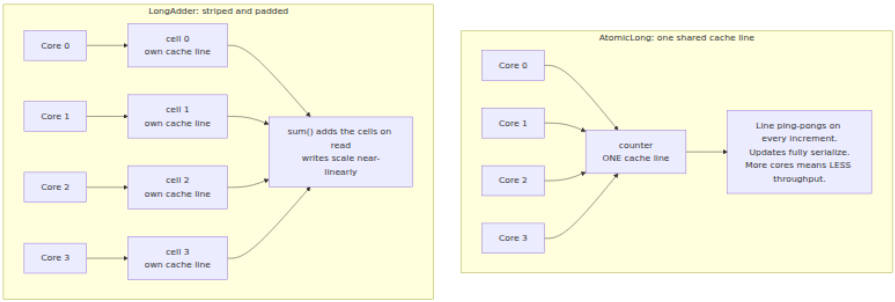
```
// (1) BROKEN under concurrency: ++ is load, add, store — lost updates
long count;  count++;

// (2) CORRECT but a scalability disaster
AtomicLong count;  count.incrementAndGet();
```

Version (2) is correct. It is also, under high concurrency, catastrophically slow — and the reason is physical, not algorithmic. The counter lives in one cache line. Every increment from every core requires that line in Modified state, so it must be invalidated in every other core's cache and transferred. With N cores incrementing, the line ping-pongs continuously; each transfer costs on the order of 100 nanoseconds on a multi-socket machine, and the operations serialize completely. Adding cores makes it *worse*, not better — the retrograde region of the USL curve from Chapter 1, observed in the wild.

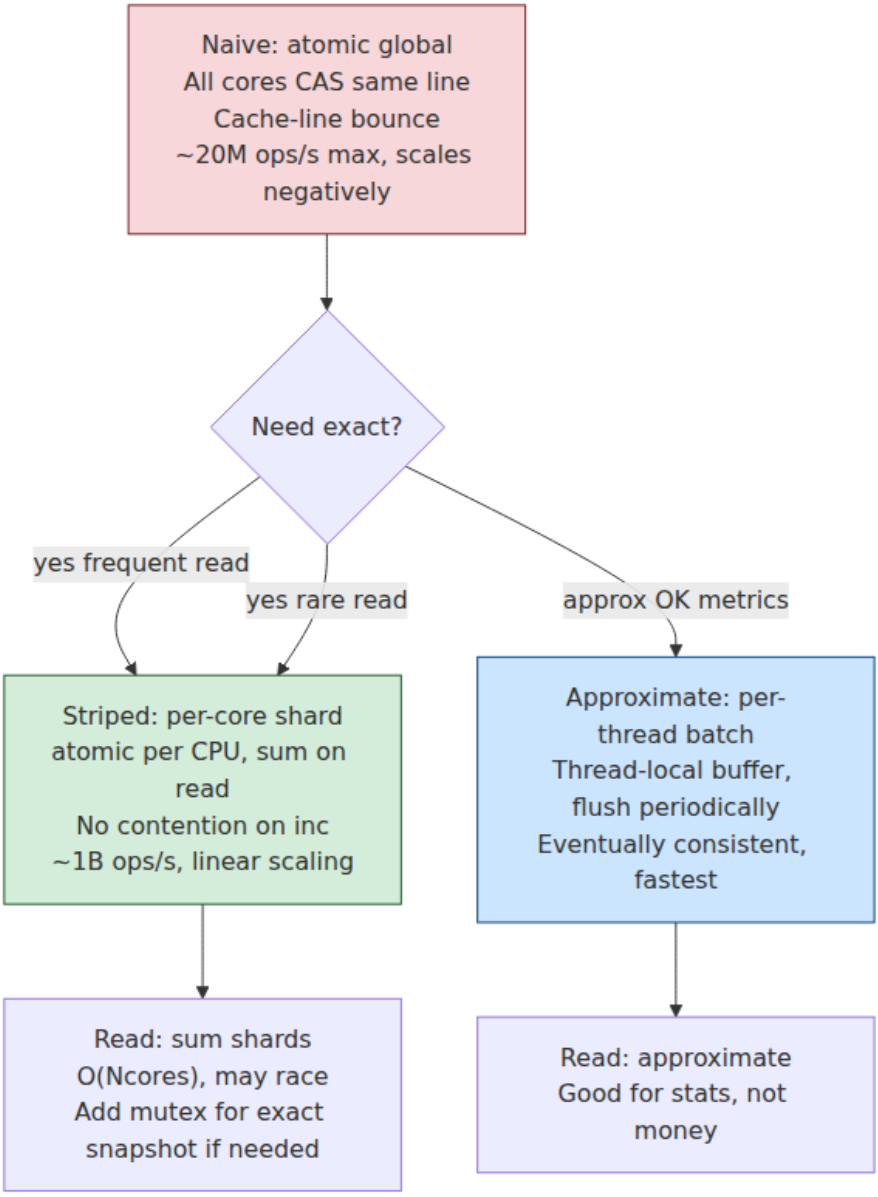
The fix is to stop sharing the cache line: give each thread (or each core, or each of N stripes) its own padded counter, and sum them only when someone reads the total.

```
// (3) The right answer: striped, padded, contention-free
LongAdder count;  count.increment();      // ... count.sum() to read
```

`LongAdder` maintains an array of `Cell` objects, each padded with `@Contended` to occupy its own cache line, and routes each thread to a cell by a thread-local probe. Under contention it dynamically grows the cell array. Writes become effectively contention-free and scale nearly linearly with cores; `sum()` walks the cells and is therefore approximate if concurrent updates are in flight — which is exactly the right trade for metrics, where you need a fast writer and an occasional reader.



The same pattern appears throughout systems programming under different names: per-CPU counters in the Linux kernel, sharded counters in distributed databases, `prometheus` client libraries batching per-thread. The principle generalizes past counters: **when a write-heavy shared value becomes hot, partition it and aggregate on read.** Note the trade you are making — you exchange a strongly-consistent value readable at any instant for an eventually-consistent one that is exact only at rest. For metrics that is free; for a value that gates a decision, such as a quota, it is not, and you need the exact primitive or a different design.


Naive: atomic global
All cores CAS same line
Cache-line bounce
~20M ops/s max, scales negatively
Need exact?
yes frequent read
yes rare read
approx OK metrics
Striped: per-core shard
atomic per CPU, sum on read
No contention on inc
~1B ops/s, linear scaling
Approximate: per-thread batch
Thread-local buffer, flush periodically
Eventually consistent, fastest
Read: sum shards
O(Ncores), may race
Add mutex for exact snapshot if needed
Read: approximate
Good for stats, not money

When to use lock-free, and when not to

Reach for lock-free when:

- Contention is very high on a small, hot structure and you have measured a mutex as the bottleneck.
- Tail latency dominates your SLO and lock-holder preemption spikes are visible at p99.9.
- Blocking is prohibited: signal handlers, interrupt contexts, hard real-time deadlines.
- A well-tested library implementation already exists — which is the case for the overwhelming majority of real needs.

Prefer a mutex when:

- Contention is low. An uncontended mutex is one atomic operation (Chapter 2); you are optimizing nothing.
- The critical section is complex or touches multiple objects. Composing lock-free operations into a larger atomic operation is not generally possible, and attempting it is where correctness usually dies.
- The code must be maintained by a team. A mutex is legible to everyone; a hand-rolled lock-free structure is legible to its author, on a good day.
- You have not measured. This is the common case, and a mutex is the correct default.

The honest summary: **the expected value of writing your own lock-free data structure is negative for almost all backend work.** These algorithms have a track record of published, peer-reviewed versions containing bugs found years later. Correctness depends on memory-model details that vary by architecture, the failure modes are rare and catastrophic, and testing requires specialized tools — jcstress, `loom` in Rust, TSan, and model checkers (Chapter 10) — because ordinary tests will not exercise the interleavings that break. Use `java.util.concurrent`, `crossbeam`, `folly`, or your platform's equivalent, and spend your cleverness on the parts of the system that are actually yours.

The distributed-systems lens

CAS is **optimistic concurrency control**, and once you see that, the same pattern is visible at every layer of a distributed system.

The read-compute-CAS-retry loop from earlier in this chapter is structurally identical to an optimistic distributed transaction: read a value with its version, compute a new value locally, attempt to commit conditionally on the version being unchanged, and retry the whole transaction if it moved. The correspondences are direct:

In-process	Distributed
`compare_exchange(ptr, old, new)`	Conditional write on version — etcd `mod_revision`, DynamoDB condition expressions
ABA problem	Stale write from a partitioned or paused client
Version-tagged pointer	ETag, `If-Match`, fencing token, row version
CAS retry loop	Optimistic transaction retry (Volume 5, Chapter 6)
Hazard pointer	Lease or pin held on a resource
Epoch-based reclamation	Grace period before dropping an old replica or schema version
Contended atomic counter	Hot key or hot partition
`LongAdder` striping	Sharded counters, per-replica aggregation, CRDT counters

The ABA correspondence is the most instructive. ABA is what happens when identity is confused with state — the pointer is the same, so the CAS assumes nothing changed. The fix is to add a version that increases monotonically, so that "changed and changed back" is distinguishable from "unchanged." That is *exactly* the fencing-token argument from Chapter 2 and Kleppmann's Redlock critique: a paused node holding a stale lease is the distributed ABA, and a monotonic token attached to every write is the same version-tagging fix. Conditional writes with ETags, MVCC row versions, and etcd's revision numbers are all the same idea, and they exist for the same reason.

The optimistic/pessimistic calculus also transfers whole. CAS retry loops win when conflicts are rare, because the common path is one atomic operation with no blocking; they lose badly when conflicts are frequent, because wasted work grows with contention. Optimistic distributed transactions have exactly this profile, which is why databases offer both optimistic and pessimistic modes and why the right choice depends on your conflict rate rather than on principle. This is also why STM failed to displace locks (Chapter 1) — the same calculus, decided by the same measurement.

Finally, `LongAdder` is the in-process form of a CRDT counter. Both replace a single contended value with per-participant values summed on read; both trade instantaneous exactness for contention-freedom; both are exact only once updates quiesce. Volume 6 develops CRDTs properly, but the intuition is already in your hands: if a value is hot and writes dominate, stop sharing it.

Where to go next. Throughput and latency numbers for these structures under contention, and the jcstress/loom/TSan methodology for testing them, are in Chapter 10; the false-sharing physics that makes `LongAdder`'s padding matter is Volume 1, Chapter 8.

Key takeaways

- The **progress hierarchy** — blocking, obstruction-free, lock-free, wait-free — is precise. Lock-free means *some* thread progresses system-wide; wait-free means *every* thread progresses in bounded steps.
- **Lock-free is a progress guarantee, not a performance one.** It can be slower than a mutex. What it buys is immunity to lock-holder preemption, which is a tail-latency and no-blocking-allowed property.
- **CAS is the universal primitive** (consensus number ∞) and the CAS retry loop is the fundamental idiom. On ARM and POWER it is built from LL/SC, which may fail spuriously and so must always sit in a loop — hence `compare_exchange_weak` inside loops.
- The **ABA problem**: CAS tests whether a value is unchanged, not whether the structure is unchanged. Fixed by version-tagged pointers, LL/SC, deferred reclamation, or garbage collection.
- **Safe memory reclamation is the hard part** in non-GC languages. Hazard pointers give bounded memory at the cost of per-read

publication; epoch-based reclamation gives cheap reads but unbounded memory if a thread stalls. GC languages get this for free, which is a real argument for Java and Go in concurrency-heavy systems.

- **Classic structures**: the Treiber stack is simple but contends on one pointer; the Michael-Scott queue is the standard lock-free MPMC FIFO and introduces *helping*; bounded ring buffers like the Disruptor usually win on throughput, largely through pre-allocation, the single-writer principle, and **cache-line padding**.
- **A contended `AtomicLong` is a scalability disaster** because one cache line ping-pongs between cores. Use `LongAdder` or per-CPU striping: partition the hot value and aggregate on read, accepting an approximate instantaneous total.
- **Do not write your own lock-free structures.** Published algorithms have shipped with bugs found years later. Use library implementations and reserve the technique for measured, narrow needs.
- **CAS is optimistic concurrency control**, and ABA's version-tag fix is the same idea as fencing tokens, ETags, and MVCC versions one layer up.

Further reading

- Herlihy, M., "Wait-Free Synchronization," *ACM TOPLAS* 13(1), 1991 — consensus numbers and why CAS is universal. https://dl.acm.org/doi/10.1145/114005.102808
- Herlihy, M. and Shavit, N., *The Art of Multiprocessor Programming*, 2nd ed. (Morgan Kaufmann, 2020) — the standard text; progress conditions, the Treiber stack, elimination, and the M-S queue.
- Michael, M. and Scott, M., "Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms," *PODC*, 1996 — the Michael-Scott queue. https://www.cs.rochester.edu/~scott/papers/1996_PODC_queues.pdf
- Treiber, R. K., *Systems Programming: Coping with Parallelism*, IBM Research Report RJ5118, 1986 — the original lock-free stack.
- Michael, M., "Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects," *IEEE TPDS* 15(6), 2004 — the standard reclamation scheme.

- Fraser, K., *Practical Lock-Freedom* (PhD thesis, Cambridge, 2004) — epoch-based reclamation.
- Thompson, M. et al., *The LMAX Architecture* and the Disruptor technical paper — mechanical sympathy, the single-writer principle, and cache-line padding. https://martinfowler.com/articles/lmax.html
- Lea, D., and the `java.util.concurrent` API documentation for `LongAdder`, `AtomicReference`, and `ConcurrentLinkedQueue` — read the `LongAdder` class javadoc in particular; it states the contention argument concisely.
- Preshing, J., "An Introduction to Lock-Free Programming" — https://preshing.com/20120612/an-introduction-to-lock-free-programming/
- Volume 1, Chapters 4 and 8 — cache coherence and false sharing — the physics behind the counter lesson and the Disruptor's padding.
- Chapter 3 — Memory Models and Happens-Before — mandatory prerequisite for writing any of this correctly.
- Chapter 10 — Testing and Debugging Concurrent Systems — jcstress, TSan, and model checking.
- Volume 5, Chapter 6 — Concurrency Control — MVCC and optimistic transactions, the distributed twin of the CAS loop.

Chapter 5 — Deadlock, Livelock, and Starvation

What this chapter covers. Chapters 1 through 4 were mostly about *safety* — making sure concurrent code never computes a wrong answer. This chapter is about *liveness* — making sure it computes an answer at all. The two failure classes are fundamentally different: a safety violation produces a wrong result you might catch in a test, while a liveness violation produces silence — a process that is still running, consuming memory, holding connections, passing health checks, and doing nothing. We treat deadlock rigorously: the four Coffman conditions that are jointly necessary for it, wait-for graphs and cycle detection as the formal tool, and the four strategic responses — prevention, avoidance, detection-and-recovery, and deliberate ignorance — compared honestly, including where each is actually used in production systems. We then build a taxonomy of real deadlocks that goes well beyond the textbook two-mutex example:

lock-ordering deadlocks smeared across module boundaries by callbacks, the thread-pool submit-and-wait deadlock that has taken down more services than any other entry in this chapter, bounded-queue resource deadlocks, and single-thread self-deadlock. Livelock and starvation get the same treatment — definition, mechanism, production form, and fix — because they fail the same liveness property by different routes. Finally, we cover how you actually find these failures in a running system, and what happens to all of this reasoning when the wait-for cycle spans services that share no memory and no global view.

Learning goals — after this chapter you should be able to:

- Distinguish safety from liveness failures, and explain why liveness failures systematically escape testing.
- State the four Coffman conditions, explain why all four are necessary, and map each prevention technique to the condition it breaks.
- Draw the wait-for graph for a stuck system and find the cycle.
- Apply lock ordering as the primary in-process prevention technique, and write a correct tryLock-with-backoff fallback when no order can be imposed.
- Explain why the Banker's algorithm is taught everywhere and deployed almost nowhere, and why database engines instead detect deadlocks and kill a victim.
- Recognize the thread-pool submit-and-wait deadlock and its async cousin on sight, in code review, before it ships.
- Define livelock precisely, explain why synchronized retry produces it, and justify exponential backoff with jitter as the fix.
- Reason about fairness: what unfair locks buy in throughput, what they cost in starvation, and when to pay for fairness.
- Interpret a `jstack` deadlock report, Go's runtime deadlock panic, and the concept behind lockdep-style order checkers.
- Explain why distributed systems mostly replace deadlock detection with timeouts, retries, and idempotency, and what that trade accepts.

Liveness failures are a different class of bug

A safety property says "nothing bad ever happens": no lost update, no torn read, no two threads in the critical section. A liveness property says "something good eventually happens": every request eventually gets a

response, every thread that wants the lock eventually gets it. The distinction is due to Lamport, and it matters operationally, not just formally, because the two classes fail differently in every way that affects an on-call engineer.

A safety failure leaves evidence. The corrupted row, the double-charged card, the impossible counter value — there is an artifact, and you can work backward from it. A liveness failure leaves an absence. The process is alive by every cheap signal: the PID exists, the liveness probe answers (it runs on a thread that is not the stuck one), CPU may even be busy (livelock burns CPU by definition). What is missing is progress, and progress is much harder to probe than existence. This is why mature services monitor *throughput and queue depth per pool*, not just process health — a topic Volume 11 returns to.

Liveness failures also escape testing systematically, and for a structural reason. A deadlock requires a particular interleaving: thread 1 must acquire A and be preempted at exactly the point where thread 2 has acquired B. Under light test load, the window is nanoseconds wide and the schedule rarely lands in it. Under production load — more threads, more preemption, GC pauses stretching hold times — the window is hit daily. The bug was present all along; the schedule that expresses it was not. Chapter 10 covers the tools that hunt schedules deliberately; this chapter gives you the theory to design the bug out instead.

One more framing point. Chapter 2 stated the three requirements of correct mutual exclusion: mutual exclusion, progress, bounded waiting. This chapter is what the second and third requirements look like when violated at system scale. Deadlock and livelock violate progress. Starvation violates bounded waiting. The chapter is organized around exactly that split.

Deadlock, rigorously

The Coffman conditions

Coffman, Elphick, and Shoshani's 1971 survey distilled deadlock into four conditions that must *all* hold simultaneously. This is the single most useful theoretical result in the chapter, because "all four are necessary"

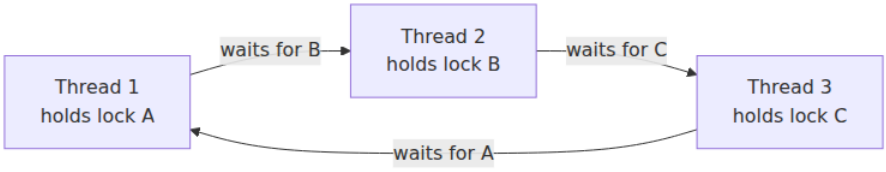
means "break any one and deadlock is impossible" — every prevention technique ever devised is an attack on one of the four.

1. **Mutual exclusion.** The resources involved cannot be shared; holding one excludes others. Mutexes by definition; also database row locks, a connection from a bounded pool, a slot in a bounded queue.
2. **Hold and wait.** A thread holds at least one resource while waiting to acquire another. The dangerous pattern is not holding locks, and not waiting — it is doing both at once.
3. **No preemption.** Resources cannot be forcibly taken from their holder; they are released only voluntarily. True of mutexes in every mainstream runtime — there is no safe way to rip a lock out of a thread's hands mid-critical-section, because the invariants inside are broken.
4. **Circular wait.** A cycle of threads exists in which each waits for a resource held by the next.

Note what is *not* on the list: nothing about two threads, nothing about two locks, nothing about mutexes specifically. Any resource that is exclusively held, waited for while holding, and not preemptible qualifies — which is why the taxonomy later in this chapter includes thread-pool workers and queue slots, and why Volume 5's database transactions deadlock on row locks in exactly the same formal structure.

Wait-for graphs

The formal tool for reasoning about condition 4 is the **wait-for graph**: one node per thread, and an edge from T1 to T2 when T1 is blocked waiting for a resource T2 currently holds. The theorem is clean: with single-instance resources (a mutex is a resource with exactly one instance), **the system is deadlocked if and only if the wait-for graph contains a cycle**. Deadlock detection is therefore cycle detection, which is linear-time graph traversal — cheap enough that database engines run it routinely, as we will see.

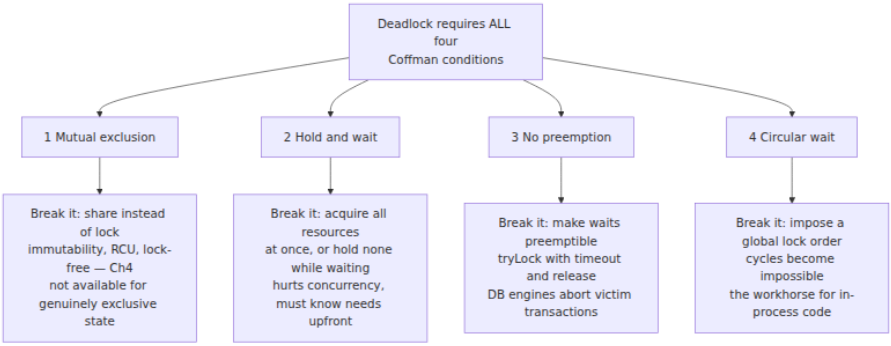


The three-party cycle above is worth staring at, because it defeats the intuition trained on two-thread examples. No pair of these threads is in conflict — T1 and T3 never touch a common lock-pair in conflicting order between just the two of them. The deadlock only exists as a property of the whole graph, which is why code review of any single module cannot find it, and why the ordering discipline described below must be *global*.

The generalization to resources with multiple interchangeable instances — a pool of 20 connections, a semaphore with N permits — is the **resource-allocation graph**, with nodes for both threads and resources, assignment edges (resource to holder) and request edges (thread to resource). There the theorem weakens: a cycle is necessary for deadlock but not sufficient, because a waiting thread might be satisfied by an instance held outside the cycle. Detection then requires a reduction-style algorithm rather than plain cycle finding. The practical consequence: deadlocks involving pools are *harder to detect and harder to see* than mutex deadlocks, which is one reason the pool-based entries in our taxonomy have such long production careers.

Four strategies, honestly compared

The literature offers four responses to deadlock. All four are used in production software today — but in very different niches, and pretending otherwise is how textbooks mislead.



Prevention: break a condition

Breaking mutual exclusion means not using exclusive resources at all: immutable data, thread confinement, the lock-free structures of Chapter 4, RCU from Chapter 2. Where it applies it is the best answer — you

cannot deadlock on locks you do not hold — but some state is irreducibly exclusive, so this is a strategy for shrinking the problem, not eliminating it.

Breaking hold-and-wait means acquiring everything you need in one atomic step, or releasing what you hold before waiting. Havender described this discipline for OS/360 in 1968. It works when resource needs are known up front, and you will see it below as the "acquire both locks or neither" structure of the ordered transfer. Its cost is pessimism: you hold resources for the whole operation even if you would only have needed them briefly, and layered software often *cannot* know its full resource needs up front — the callee's locks are the callee's business.

Breaking no-preemption means making waits abortable: acquire with `tryLock` and a timeout, and on failure release everything you hold and retry. Note the honest framing — nobody preempts the *holder*; the *waiter* preempts itself. This is the fallback when ordering cannot be imposed, shown in code below, and it is also the philosophical basis of detection-and-recovery: aborting a database transaction is preemption of every lock it holds, made safe by rollback.

Breaking circular wait — lock ordering — is the workhorse. Impose a total order on all locks; require that any thread holding lock L acquires only locks greater than L. Then every edge in the wait-for graph points "upward" in the order, and a cycle would require an edge pointing downward — impossible. This is Chapter 2's "establish and document a lock ordering" rule, now with its proof.

The canonical demonstration is the account transfer. The naive version deadlocks: transfer(a, b) and a concurrent transfer(b, a) each take their first lock and wait forever for the second. The fix orders acquisition by an intrinsic property of the accounts:

```
final class Account {
    private final long id;            // unique, orderable — this is
the lock order
    private long balanceCents;

    Account(long id, long initialCents) { this.id = id; this.balanceCen
ts = initialCents; }
    long id() { return id; }

    // Callers must hold this account's monitor.
    void depositLocked(long cents)  { balanceCents += cents; }
    void withdrawLocked(long cents) {
        if (balanceCents < cents) throw new IllegalStateException("insu
fficient funds");
        balanceCents -= cents;
    }
}

static void transfer(Account from, Account to, long cents) {
    if (from.id() == to.id()) throw new IllegalArgumentException("same
account");
    // Always lock the lower id first, regardless of transfer
direction.
    Account first  = from.id() < to.id() ? from : to;
    Account second = from.id() < to.id() ? to   : from;
    synchronized (first) {
        synchronized (second) {
            from.withdrawLocked(cents);
            to.depositLocked(cents);
        }
    }
}
```

Every thread now locks accounts in ascending id order, so no cycle can form no matter how transfers are directed or interleaved. When objects lack a natural key, `System.identityHashCode` plus a single global tie-breaker lock for the rare collision is the standard Java idiom (Goetz et al. give the full version). Three practical notes. First, the order must be *documented* — in a comment at the lock's declaration at minimum, because the next engineer cannot obey an order they cannot see. Second, the ordering must be *stable*: ordering by a mutable field is a deadlock deferred. Third, ordering has a real cost in API design — it forbids "acquire as you go" and forces call sites to know all locks up front, which is precisely what layered software makes hard. That difficulty is the subject of the taxonomy below.

When no order can be imposed — the locks are acquired in an order dictated by external events, or they belong to different subsystems that cannot know about each other — the fallback is **tryLock with backoff**, which breaks no-preemption instead:

```
static void transferWithBackoff(ReentrantLock fromLock, ReentrantLock
toLock,
                                Runnable moveMoney) throws InterruptedE
xception {
    long backoffNanos = TimeUnit.MICROSECONDS.toNanos(50);
    while (true) {
        if (fromLock.tryLock()) {
            try {
                if (toLock.tryLock()) {
                    try { moveMoney.run(); return; }
                    finally { toLock.unlock(); }
                }
            } finally { fromLock.unlock(); }
        }
        // Failed to get both: hold NOTHING while waiting, and
randomize the retry
        // delay so two symmetric contenders cannot collide forever
(livelock).
        long jittered = ThreadLocalRandom.current().nextLong(backoffNan
os);
        TimeUnit.NANOSECONDS.sleep(1 + jittered);
        backoffNanos = Math.min(backoffNanos * 2,
TimeUnit.MILLISECONDS.toNanos(10));
    }
}
```

The two load-bearing details are in the comment: on failure the thread releases *everything* before sleeping (no hold-and-wait during the retry), and the sleep is *randomized*. Deterministic backoff would have two symmetric threads acquire, collide, release, sleep the same interval, and collide again indefinitely — which is livelock, and we will return to exactly this failure shape.

Avoidance: the Banker's algorithm, and why you will never use it

Dijkstra's Banker's algorithm — named for a banker deciding which loan requests to grant — sits between prevention and detection: grant a resource request only if the resulting state is *safe*, meaning there exists some order in which all threads can run to completion even if each demands its declared maximum. On each request, the algorithm

simulates: could the remaining pool satisfy some thread's worst case? Retire it, reclaim its resources, repeat. If every thread can be retired, the state is safe and the request is granted; otherwise the requester waits even though resources are free right now.

It is a genuinely beautiful algorithm, and it is almost never used in general-purpose software, for reasons worth being precise about. It requires every thread to **declare its maximum resource needs in advance** — unknowable in layered software where a method call may take locks its caller has never heard of. It requires a **fixed population** of threads and resources, while servers create both dynamically. It requires **central mediation of every acquisition**, putting a global bookkeeping step on what Chapter 2 worked hard to make a single CAS. And it is conservative: it blocks requests that would in fact have completed fine, paying real concurrency for hypothetical safety. Where the preconditions do hold — closed embedded systems, some real-time schedulers, admission control against a fixed pool — the idea survives. As advice for the readers of this book: know it, cite it, do not wait for a chance to deploy it.

Detection and recovery: where deadlock is handled for real

The strategy that *is* deployed at scale inverts the problem: let deadlocks happen, detect the cycle, and kill a participant. This requires the ability to preempt safely — to take resources back from a victim without corrupting state — and that is exactly what database transactions provide. Rollback restores the pre-transaction state, releases every lock, and leaves the world consistent. The transaction machinery of Volume 5, Chapter 5 is, among other things, a licence to preempt.

So database engines are the one place most backend engineers encounter formal deadlock detection running in production. InnoDB maintains a wait-for graph over transactions blocked on row locks and checks for cycles when a wait begins; on finding one it rolls back a victim — preferring the transaction that has modified fewer rows, i.e. the cheaper rollback — which fails with `ER_LOCK_DEADLOCK`. (InnoDB even lets you disable detection with `innodb_deadlock_detect` and fall back to plain lock-wait timeouts, a documented trade for very high-contention workloads where graph maintenance itself becomes a cost.) PostgreSQL waits `deadlock_timeout` — one second by default — before bothering to

run detection on the theory that most waits resolve themselves, then aborts a waiting transaction with SQLSTATE `40P01` if it finds a cycle.

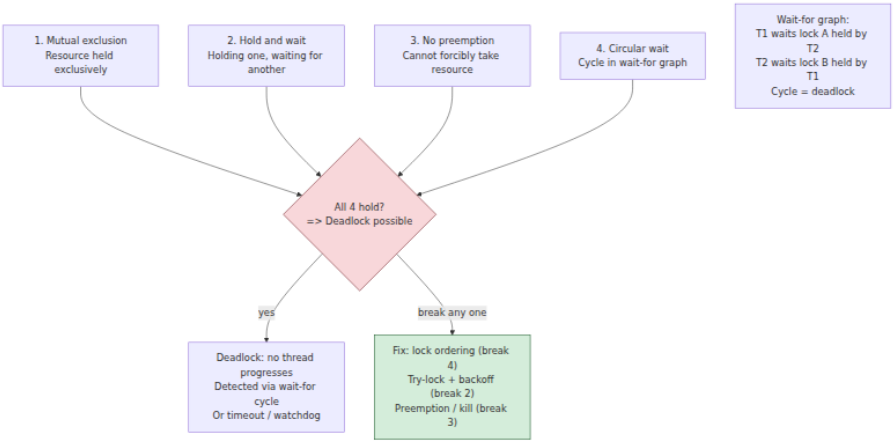
Two consequences for the application engineer. First, **victim selection is policy, not justice**: the engine kills whichever transaction is cheapest to roll back, which may be yours, repeatedly — a transaction that is always the cheapest victim can be starved by this mechanism, deadlock recovery producing starvation. Second, and this is the part teams get wrong constantly: a deadlock error from the database is a *retryable* error by design. The engine broke the cycle precisely so that the survivors and the retried victim can proceed. Application code must catch the deadlock error class and retry the whole transaction — with backoff and jitter, and only if the transaction is safe to re-execute, which is an idempotency obligation the application owns. An application that surfaces `40P01` to the end user has implemented half of detection-and-recovery. Reducing deadlock *frequency* remains worthwhile — and the technique is the same lock ordering as above: touch rows and tables in a consistent order across your transactions, exactly the transfer example wearing SQL clothing. Volume 5, Chapter 6 shows how MVCC removes many read-write conflicts entirely, shrinking the lock footprint that deadlocks feed on.

The ostrich strategy, stated without embarrassment

The fourth strategy is to do nothing. This is not always negligence; it is an expected-cost judgment: probability of the deadlock times cost per occurrence, against the engineering cost of the fix. Operating systems make this choice deliberately — Linux does not run deadlock avoidance over application mutexes, because the cost would be universal and the benefit rare. A stateless service replica that might deadlock once a quarter, gets killed by its health-check-driven restart, and loses nothing that a retry does not recover, may be entirely rationally left alone.

The honest version of the strategy has three preconditions people skip: the probability estimate must be real rather than hopeful (a deadlock whose window widens with load will find you at the worst time); the *blast radius* must be bounded (a deadlocked replica behind a load balancer is an annoyance; a deadlocked singleton holding leadership is an outage — and note the restart is doing informal detection-and-recovery, so something must detect stuckness, typically a liveness probe that actually

exercises the stuck path); and the decision must be written down, because an undocumented ostrich is indistinguishable from ignorance.



A taxonomy of real deadlocks

The two-mutex example is the fruit fly of deadlock: ideal for study, rarely what actually bites. Production deadlocks come in recurring shapes, and recognizing the shape in code review is worth more than any amount of theory.

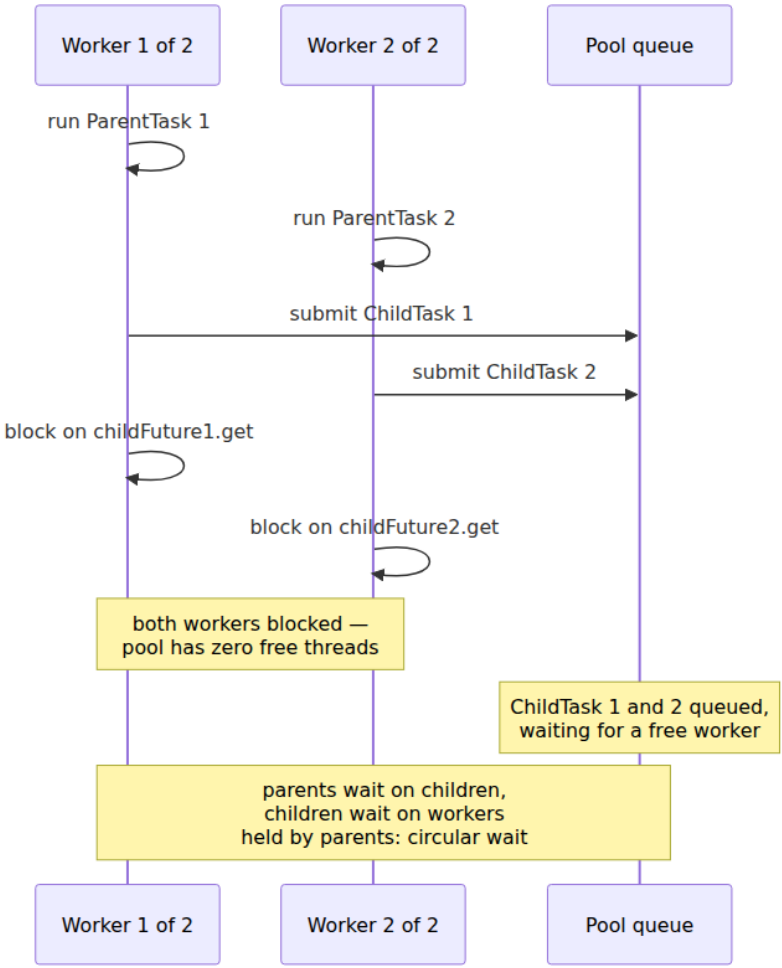
Lock-ordering deadlock across module boundaries

Within one module, lock ordering is a discipline problem. Across modules, it is a *visibility* problem: each module's locking is correct in isolation, and the cycle only exists in composition. The classic vector is the callback. Module A holds its lock while invoking a listener; the listener, living in module B, takes B's lock. Elsewhere, a thread in B holds B's lock and calls into A's public API, which takes A's lock. Each module locks consistently *by its own lights*; the wait-for cycle crosses the boundary where neither can see it. This is Chapter 2's rule — **never call unknown code while holding a lock** — with the mechanism spelled out: a callback invoked under a lock silently makes your lock part of every caller's lock order, without their knowledge or consent. The fix is mechanical: snapshot the listener list under the lock, release, then invoke. More generally, treat "which locks may be held when this function is called" as part of an API's contract; the observers, GUI toolkits, and cache-invalidation hooks of the world are where these deadlocks breed.

Thread-pool submit-and-wait: the classic production outage

This one deserves its diagram, because it involves no mutexes at all and therefore evades every lock-oriented review habit. The resource is a **worker thread** in a bounded pool.

A task running *on* the pool submits a subtask *to the same pool* and blocks waiting for its result. Under light load there are free workers and the subtask runs; the code works for months. Then a traffic spike fills the pool with parent tasks — every worker is occupied by a parent blocked waiting for a child, and every child is in the queue waiting for a worker. Check the Coffman conditions: workers are exclusively held (1), parents hold a worker while waiting for another (2), nothing preempts a blocked worker (3), and parents wait on children who wait on the workers the parents hold (4). The pool is deadlocked at exactly its configured size, forever.



The insidious property is load dependence: the deadlock requires the pool to be saturated with parents, so it appears only at peak — the worst possible time — and vanishes when you restart the service and load drops, destroying the evidence. Fixes, in rough order of preference: **never block a pool worker on work scheduled on the same bounded pool** — restructure as a continuation (`thenCompose` and friends) so the parent releases its worker instead of holding it; run subtasks on a *separate* pool, sizing the dependency DAG of pools such that waits only point "downward" (this is lock ordering again, with pools as the locks); or use a pool designed for the pattern — Java's

`ForkJoinPool` exists substantially because its work-stealing and `ManagedBlocker` machinery let a blocked worker be compensated for, which ordinary fixed pools do not.

The **async variant** is the same graph with different nouns, and it is rampant. On an event loop (Chapter 6), code that *synchronously blocks* the loop's thread waiting for a future that can only be completed *by* that loop — `future.get()` in a Netty handler, `.Result` on a .NET UI context, `runBlocking` inside a coroutine on the same single-threaded dispatcher (Chapter 8) — deadlocks with a pool of size one, immediately and deterministically. The single-threaded case is at least easy to reproduce; the small-pool case, like the thread pool above, waits for saturation.

Resource deadlock on bounded queues

Two services, each with a bounded request queue, each calling the other. A's queue is full of requests that need responses from B; B's queue is full of requests that need responses from A; each service's workers are all blocked producing into the other's full queue. No locks anywhere — the exclusively-held resource is a *queue slot* — but all four Coffman conditions hold, with "queue slot" substituted for "mutex." The same shape appears entirely in-process with two stages of a pipeline connected by bounded channels in both directions.

Bounded queues are not the villain here — unbounded queues merely replace visible deadlock with invisible memory exhaustion. The bound is doing its job: surfacing the fact that the *topology* contains a cycle. The real fixes are cycle-aware: break the request cycle in the service graph, make one direction non-blocking (drop or shed rather than wait when the queue is full), or bound the *wait* rather than the queue. This is the doorstep of backpressure design, which Volume 10 treats properly; note for now that "everything blocks politely" composes into deadlock when the blocking relation has a cycle.

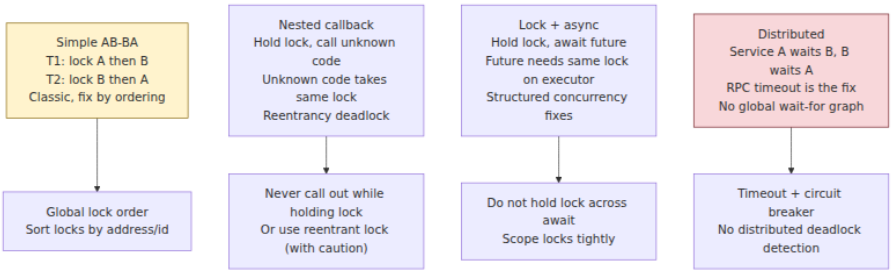
Self-deadlock: one thread, one lock

The minimal deadlock needs one thread. A thread holding a non-reentrant mutex calls — usually via some indirection that obscures it — a function that acquires the same mutex. The wait-for graph is a self-loop: the thread waits for itself. `pthread_mutex_t` in its default mode will deadlock this way (`PTHREAD_MUTEX_ERRORCHECK` turns it into an `EDEADLK` error instead, which is strictly kinder); a Go `sync.Mutex` will too, and Go's

philosophy explicitly rejects reentrancy. Java's `synchronized` and `ReentrantLock` are reentrant and silently absorb the re-acquisition — which prevents this deadlock at the cost Chapter 2 discussed: reentrancy usually papers over a layering confusion about which functions assume the lock is held. Either way the underlying bug is the same, and the `_locked`-suffix convention from Chapter 2 is the cure. The re-acquisition is almost never written in one visible function; it arrives through a callback, a signal handler, or an override.

Distributed deadlock

The fifth family — wait-for cycles spanning processes and machines — changes the problem qualitatively enough that it gets the distributed-systems lens section to itself below.

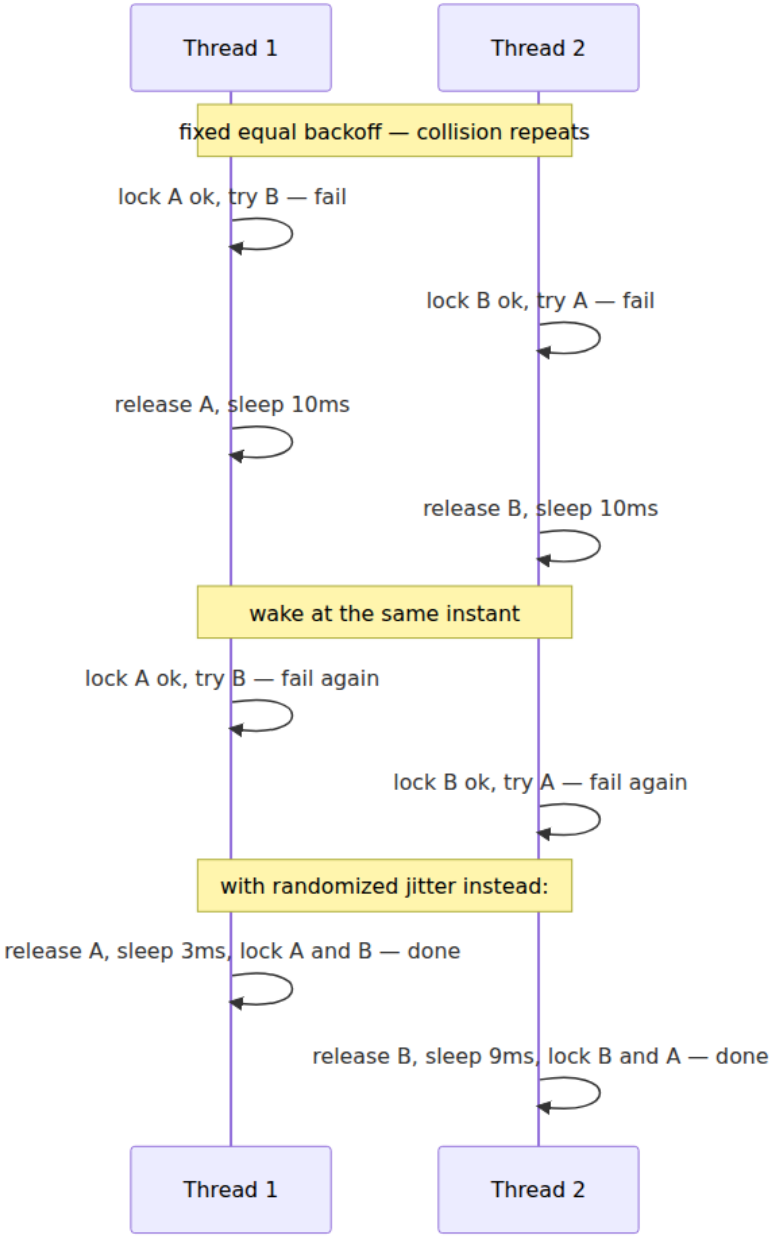


Livelock

Deadlock is silent stillness; **livelock** is furious motion without progress. The threads are not blocked — they run, change state, consume CPU — but the system as a whole gets nowhere, because each participant's reaction to the others perpetually re-creates the conflict. The canonical image is two people meeting in a corridor, each stepping aside in the same direction, repeatedly and forever. Formally the progress property is violated just as surely as in deadlock; operationally livelock is *worse to diagnose*, because every cheap signal reads healthy — CPU busy, threads runnable, no thread parked for a suspicious duration. Nothing looks stuck except the throughput graph.

The mechanism that manufactures livelock is **symmetric retry with correlated timing**. Take the tryLock loop from earlier and remove the jitter: two threads attempt their lock pairs in opposite orders, each gets its first lock, each fails its second, each releases and sleeps *the same*

fixed interval, and each wakes at the same instant to collide again. The collision is not bad luck; the protocol's determinism guarantees it recurs. The same shape appears wherever contenders share a retry policy: optimistic-concurrency retries against a hot row, CAS loops under extreme contention (Chapter 4's note on livelock in lock-free algorithms — lock-freedom guarantees *some* thread progresses, but obstruction-free designs can livelock), and, at the largest scale, fleets of clients retrying a struggling service on identical schedules.



The fix is to **break the symmetry**, and the standard tool is **randomized exponential backoff**: on the Nth consecutive failure, wait a *random*

duration drawn from a window that grows (typically doubles) with N. The randomness decorrelates the contenders so collisions stop repeating; the exponential growth adapts the retry rate to the observed level of contention. This is old, battle-proven engineering: classic Ethernet's CSMA/CD resolved collisions on a shared coaxial medium with truncated binary exponential backoff — after a collision each station waits a random number of slot times drawn from a window that doubles per attempt — and it worked well enough to carry the protocol from Metcalfe and Boggs's 1976 paper into decades of deployment. The same analysis reappears in Marc Brooker's widely-cited AWS write-up on backoff and jitter for distributed retries, whose simulations show "full jitter" — sleep a uniform random duration between zero and the exponentially-growing cap — beating both plain exponential backoff and half-jittered variants on total work and time-to-completion under contention. An alternative symmetry-breaker is hierarchy: give one contender priority (by id, by role) so conflicts always resolve the same direction. That trades livelock risk for starvation risk, which is our next topic.

Starvation and fairness

Starvation is the third liveness failure: the system as a whole makes progress, but some particular thread never does — bounded waiting, the third requirement from Chapter 2, is violated. It is subtler than deadlock and livelock because the aggregate metrics look fine; throughput is healthy, and the victim's requests are quietly rotting in a corner of the latency distribution.

Unfair locks, barging, and why the defaults are unfair

The purest source of starvation is the lock itself. An **unfair** lock hands itself to whichever thread grabs it first at release time — and the thread most likely to win is the one already running on a warm core, quite possibly the previous holder back for more. A newly arriving thread can **barge** past a queue of parked waiters: they must be woken, scheduled, and migrated to a warm cache before they can even attempt the acquisition (the futex slow path of Chapter 2), while the barger is already executing with the lock's cache line resident. A **fair** (FIFO) lock grants strictly in arrival order, which eliminates starvation and costs real throughput: every handoff to a parked thread pays wakeup latency during

which the lock sits idle, and under contention the convoy of forced handoffs can cost an order of magnitude in throughput versus barging.

This is why the defaults across the industry are unfair. Java's `ReentrantLock` documents the choice explicitly: the default is non-fair; `new ReentrantLock(true)` buys FIFO fairness at a documented throughput cost; and even a fair lock's untimed `tryLock()` is documented to barge past waiters — an escape hatch that exists precisely because barging is cheap. Pthread mutexes make no fairness promise at all, and futex wakeups do not guarantee FIFO order. The engineering judgment underneath: starvation under an unfair lock is *probabilistic* — statistically, everyone gets in eventually under normal load — and the pathological case (one thread repeatedly losing for seconds) is rare enough that paying the fairness tax on every operation is a bad trade. Reach for a fair lock when you have *observed* starvation or when tail latency of the slowest waiter matters more than aggregate throughput; measure both before and after.

Where else starvation lives

Reader-writer locks are the classic policy trap, covered in Chapter 2: reader-preferring policies starve writers under a steady reader stream; writer-preferring policies can starve readers. The starvation is not a bug in the lock — it is the documented consequence of a policy choice, and you must know which policy your platform's default is.

Priority scheduling starves structurally: a strict-priority scheduler runs a lower-priority thread only when no higher-priority thread is runnable, so sustained high-priority load means *never*. Real-time schedulers accept this by design (Volume 2, Chapters 1–2); general-purpose schedulers refuse it — Linux's CFS and its EEVDF successor allocate weighted fairness, so low `nice` values dilute rather than eliminate a thread's share. Related but distinct is **priority inversion** — a low-priority thread *holding a lock* a high-priority thread needs, with medium-priority work preempting the holder — treated in Chapter 2 with the Mars Pathfinder story and the priority-inheritance fix; recall that its backend analogue, background jobs holding resources request handlers need, is alive and well in systems with no explicit priorities at all.

And note the earlier example from the database world: deadlock **victim selection** is a starvation mechanism when the same cheap transaction is repeatedly chosen for rollback. Fairness questions follow deadlock

recovery around; any policy that resolves contention by consistently sacrificing the same party converts a liveness mechanism into a targeted denial of progress.

Seeing it in production

Theory tells you deadlock is a cycle; operations requires you to find it at 3 a.m. The tools are better than most engineers realize.

JVM: thread dumps. The JVM ships a deadlock detector. `jstack <pid>` (or `kill -3`, or `ThreadMXBean.findDeadlockedThreads`) walks the monitor wait-for graph and reports cycles explicitly — including, with `jstack -l`, `java.util.concurrent` locks, not just `synchronized` monitors:

```
Found one Java-level deadlock:
=============================
"transfer-2":
  waiting to lock monitor 0x00007f8a1c004e00 (object
0x000000076ab3c8f0, an Account),
  which is held by "transfer-1"
"transfer-1":
  waiting to lock monitor 0x00007f8a1c006190 (object
0x000000076ab3c940, an Account),
  which is held by "transfer-2"

Java stack information for the threads listed above:
...
Found 1 deadlock.
```

That is the wait-for cycle of this chapter, printed by the runtime, with stack traces attached. Every JVM engineer should have triggered and read one on purpose before reading one under duress. Note what it does *not* find: pool deadlocks and queue deadlocks, where the waiting is on futures and queue slots the JVM does not model as locks. For those, the diagnostic is a full thread dump read by a human: every pool worker parked in `future.get` or `queue.put`, none runnable — a pattern you now know by name.

Native code. `pstack <pid>` or `gdb -p <pid>` plus `thread apply all bt` gives the same raw material: all threads blocked in `__lll_lock_wait` (glibc's futex wait), and cross-referencing who holds what from the mutex owner fields reconstructs the cycle by hand.

Go. The runtime panics with `fatal error: all goroutines are asleep - deadlock!` plus all goroutine stacks — but only when *every* goroutine is blocked. One background ticker goroutine keeps the process "live" and mutes the detector, so partial deadlocks — the common kind in a server that always has an HTTP listener parked in `accept` — produce no panic. There the tools are `net/http/pprof`'s goroutine profile (thousands of goroutines parked at the same `chan send` / `mutex.Lock` line is the smoking gun) and the mutex/block profiles for contention and wait attribution.

Lock-order checkers. The most interesting tool class does not wait for the deadlock. The Linux kernel's **lockdep** records, at runtime, the order in which lock *classes* are acquired, building the "held while acquiring" relation across all executions — and complains the first time any pair of classes is ever seen in both orders, *even though no deadlock occurred on that run*. That is the crucial property: it converts a probabilistic schedule-dependent failure into a deterministic one — any single execution of both code paths, in any interleaving, exposes the inversion. It validates the ordering discipline rather than hunting the unlucky schedule. The idea transfers: ThreadSanitizer reports lock-order inversions in user code, and an afternoon's work wrapping your project's lock type with a debug-build order-tracking layer pays for itself the first time it fires in CI. Chapter 10 covers these tools, and schedule-exploration testing generally, in depth.

The distributed-systems lens

Everything so far assumed a wait-for graph *somebody can see* — a runtime, an engine, a debugger with all threads in one address space. The defining feature of distributed deadlock is that **no such observer exists**. Service A's threads wait on RPCs to B, B's transactions wait on row locks held by C's requests, C waits on A — each system sees only its outgoing edges, and the cycle exists only in the composition. Local detectors are structurally blind to it: every participant looks "slow, waiting on a dependency," which is indistinguishable from ordinary congestion. The cross-service bounded-queue deadlock from the taxonomy is the two-node case; real service graphs offer much longer cycles, often through a shared database that neither service team thinks of as part of "their" call graph.

The theory exists. Chandy, Misra, and Haas's 1983 **edge-chasing** algorithm detects distributed cycles without assembling a global graph: a blocked process sends a *probe* stamped with its identity along its wait-for

edges; each blocked recipient forwards the probe along its own waiting edges; if your own probe comes back to you, you are on a cycle, and (in the standard formulation) the detecting initiator aborts as the victim. Knapp's 1987 survey catalogues this and the other families — and also catalogues the failure modes, chiefly *phantom deadlocks*: with no global snapshot, a probe can traverse edges that no longer coexist, detecting a cycle that never existed at any single instant and aborting an innocent victim. Edge-chasing is real in tightly-coupled distributed databases that own both the locks and the messaging. It is essentially absent from general microservice architectures, because the preconditions fail: waits happen in a dozen runtimes and protocols with no common notion of a wait-for edge, and no channel exists to chase edges across team and vendor boundaries.

What the industry does instead is honest and unglamorous: **timeouts, retries, and idempotency** (Volume 6, Chapter 9). A timeout is crude preemption — it breaks Coffman condition 3 without ever identifying a cycle. It cannot distinguish deadlock from slowness, so it will sometimes abort work that would have finished (the phantom-victim problem again, accepted rather than solved), which is exactly why the retry must exist, and why the retried operation must be idempotent for the combination to be safe. Read that stack as this chapter's strategy table transplanted: timeout = preemption, retry = recovery, idempotency = the rollback-equivalent that makes preemption safe, jittered backoff = the livelock guard. Nothing in the distributed toolkit is new; it is detection-and-recovery with the detection replaced by a deadline, because a deadline is the only detector that needs no global view.

Two distributed pathologies deserve naming. **Two-phase commit blocking** (Volume 5, Chapter 10): a participant that has voted yes must hold its locks until the coordinator's verdict; if the coordinator dies at that moment, participants hold locks — blocking an arbitrary set of unrelated transactions — until it recovers. Not a cycle, but the same operational signature as deadlock (locks held indefinitely by a party that cannot proceed), and the reason 2PC is called a blocking protocol and consensus-based commit exists.

And **metastable failure / retry storms** are livelock's distributed cousin. A fleet of clients retrying an overloaded service adds retry load on top of organic load; the extra load slows the service further, causing more timeouts, causing more retries. Past a tipping point the system enters a

state that is self-sustaining *even after the original trigger is gone* — everything running hot, useful throughput near zero, exactly livelock's signature at datacenter scale. Bronson, Aghayev, Charapko, and Zhu's HotOS 2021 paper named this class "metastable failures" and observed that the sustaining feedback loop is usually a well-intentioned mechanism — retries, failovers, cache-miss storms. The mitigations are the livelock fixes writ large: jittered exponential backoff, retry budgets that cap amplification, circuit breakers that convert retry pressure into fast failure, and load shedding to break the feedback loop (Volume 11).

Key takeaways

- **Liveness failures give no answer rather than a wrong answer.** They leave no artifact, pass health checks, and escape testing because they require a specific schedule; monitor progress and queue depth, not just process existence.
- **Deadlock requires all four Coffman conditions** — mutual exclusion, hold-and-wait, no preemption, circular wait. Every countermeasure breaks exactly one; know which one yours breaks.
- **Deadlock is a cycle in the wait-for graph.** With single-instance resources the equivalence is exact; with pools, cycles are necessary but not sufficient, which makes pool deadlocks harder to see.
- **Lock ordering is the workhorse of prevention**: a global acquisition order makes cycles impossible. It must be total, stable, and written down — and its enemy is the callback invoked under a lock, which drafts your lock into orders you never agreed to.
- **tryLock-with-backoff is the fallback** when ordering is infeasible: release everything on failure, and randomize the backoff, or you will trade deadlock for livelock.
- **The Banker's algorithm is instructive, not deployable** in general software: it needs declared maximum demands, a fixed population, and centrally mediated acquisition. **Detection-and-recovery is what actually ships** — in database engines, whose transactions make preemption safe via rollback. Treat database deadlock errors as retryable; that retry is your half of the protocol.
- **The thread-pool submit-and-wait deadlock** needs no mutexes: parents hold all the workers and wait for children who need a worker. It fires only at saturation. Never block a bounded pool's

worker on work scheduled on the same pool; the event-loop `future.get()` is the same bug with pool size one.

- **Livelock is motion without progress**, manufactured by symmetric retries with correlated timing; the fix, from Ethernet to cloud SDKs, is randomized exponential backoff or an explicit symmetry-breaking hierarchy.
- **Fairness costs throughput** — barging beats FIFO handoff because wakeups are expensive — which is why locks default unfair and why `ReentrantLock(true)` is opt-in. Starvation also arrives via RW-lock policy, strict priorities, and repeated deadlock-victim selection.
- **Runtimes will show you the cycle**: `jstack` prints Java-level deadlocks, Go panics when all goroutines sleep (and only then), and lockdep-style order checkers find inversions before any deadlock occurs — the rare tool that makes a probabilistic bug deterministic.
- **Distributed systems replace detection with deadlines**: no global wait-for graph exists, so timeouts (crude preemption) + retries (recovery) + idempotency (safe re-execution) + jitter (livelock guard) is the pragmatic stack; edge-chasing lives on inside distributed databases. 2PC's blocking window and metastable retry storms are the distributed relatives of deadlock and livelock respectively.

Further reading

- Coffman, E. G., Elphick, M., and Shoshani, A., "System Deadlocks," *ACM Computing Surveys* 3(2), 1971 — the four conditions and the original survey of strategies.
- Havender, J. W., "Avoiding deadlock in multitasking systems," *IBM Systems Journal* 7(2), 1968 — ordered and all-at-once acquisition as OS/360 operating discipline.
- Dijkstra, E. W., "Cooperating Sequential Processes" (EWD 123, 1965) — the Banker's algorithm and the "deadly embrace"; available in the E. W. Dijkstra Archive, University of Texas. https://www.cs.utexas.edu/users/EWD/
- Chandy, K. M., Misra, J., and Haas, L. M., "Distributed Deadlock Detection," *ACM Transactions on Computer Systems* 1(2), 1983 — the edge-chasing probe algorithm.

- Knapp, E., "Deadlock Detection in Distributed Databases," *ACM Computing Surveys* 19(4), 1987 — the survey, including phantom deadlocks.
- Goetz, B. et al., *Java Concurrency in Practice* (Addison-Wesley, 2006), Chapter 10 — lock-ordering deadlocks, open calls, and the tie-breaking idiom for ordering by identity hash.
- Herlihy, M. and Shavit, N., *The Art of Multiprocessor Programming*, 2nd ed. (Morgan Kaufmann, 2020) — progress conditions: wait-freedom, lock-freedom, obstruction-freedom, and starvation.
- Metcalfe, R. M. and Boggs, D. R., "Ethernet: Distributed Packet Switching for Local Computer Networks," *CACM* 19(7), 1976 — the origin of exponential backoff as a collision-resolution discipline.
- Brooker, M., "Exponential Backoff and Jitter," AWS Architecture Blog, 2015 — the simulation-backed case for full jitter. https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
- Bronson, N., Aghayev, A., Charapko, A., and Zhu, T., "Metastable Failures in Distributed Systems," *HotOS* 2021 — retry storms and sustaining feedback loops as a failure class.
- Linux kernel documentation, "Runtime locking correctness validator" — the lockdep design. https://www.kernel.org/doc/html/latest/locking/lockdep-design.html
- MySQL 8.0 Reference Manual, "InnoDB Deadlock Detection" — wait-for graph, victim choice, and the `innodb_deadlock_detect` trade-off. https://dev.mysql.com/doc/refman/8.0/en/innodb-deadlock-detection.html
- PostgreSQL documentation, "Deadlocks" and `deadlock_timeout` — detection policy and the ordering advice for applications. https://www.postgresql.org/docs/current/explicit-locking.html
- Java Platform API, `java.util.concurrent.locks.ReentrantLock` — the fairness contract, the throughput caveat, and barging `tryLock()`.
- Volume 5, Chapters 5–6 — transactions and MVCC: why databases can preempt safely, and how MVCC shrinks the lock footprint.

- Volume 6, Chapters 8–9 — coordination, and the timeout/retry/idempotency stack this chapter leans on.

Chapter 6 — Asynchronous I/O and Event Loops

What this chapter covers. Chapter 1 mapped the models of concurrency and observed that for I/O-bound workloads — which is to say, for most backend services — the event loop is one of the two dominant answers, the other being lightweight threads (Chapter 8). This chapter opens up the event loop and examines every layer of it: why thread-per-connection stops scaling and what exactly runs out; what a non-blocking file descriptor actually is and what `EAGAIN` means; the thirty-year evolution of the multiplexing syscalls from `select` through `poll` to `epoll` and `kqueue`, including the level-triggered/edge-triggered distinction that is responsible for a whole genre of production bugs; `io_uring` and the completion-based model, which is not an incremental improvement on `epoll` but a different paradigm — you submit operations rather than register interest; and the anatomy of a real event loop, with its timers, its deferred callbacks, its thread pool for the work that cannot be made non-blocking, and its one inviolable rule. We then do three case studies mechanically — nginx, Node.js, Redis — and treat two problems every event-loop server eventually meets: write-side backpressure and the thundering herd on shared accept. The distributed-systems lens closes the loop: the reactor you build inside one process is the same object that sits at the heart of every proxy and sidecar in your fleet, and event-loop lag is queueing delay in exactly the sense of Little's Law from Chapter 1.

This chapter assumes you know what a file descriptor and a syscall are (Volume 2, Chapter 5), and how a TCP socket's buffers behave (Volume 3; Volume 2, Chapter 10 for the kernel side). Chapter 8 builds on this one: coroutines and async/await are, underneath, a compiler-assisted way to write programs against exactly the machinery described here.

Learning goals — after this chapter you should be able to:

- Explain quantitatively why thread-per-connection hits a wall — which resources are exhausted, in what order — and what readiness-based multiplexing changes.
- Describe `O_NONBLOCK` and the `EAGAIN` / `EWOULDBLOCK` contract precisely, and explain why every fd owned by an event loop must be non-blocking.
- Compare `select`, `poll`, and `epoll` in terms of what crosses the user/kernel boundary per call, and explain why epoll's persistent interest list changes the complexity class.
- State the level-triggered and edge-triggered contracts exactly; write a correct edge-triggered drain loop; and explain both failure modes of getting it wrong — the lost-wakeup hang and the hot-fd starvation of every other connection.
- Explain io_uring's submission/completion ring architecture, how a completion model differs fundamentally from a readiness model, and why some fleets restrict io_uring anyway.
- Draw the phases of a production event loop, state the "never block the loop" rule and its consequences, and explain why file I/O gets offloaded to a thread pool even in a "fully async" runtime.
- Implement socket-level backpressure correctly: bounded write queues, `EPOLLOUT` watched only while the queue is non-empty, and a policy for slow clients.
- Use event-loop lag as the primary health metric of an async service, and connect it to queueing delay and fleet-wide tail latency.

Why thread-per-connection hits a wall

The natural first architecture for a network server is one thread (or, earlier, one process) per connection: accept, hand the socket to a thread, and let that thread issue ordinary blocking `read` and `write` calls. The code reads exactly like its own control flow — sequential, imperative, obvious — which is a genuine and underrated virtue. Through the 1990s this was the dominant design, and for servers handling hundreds of concurrent connections it remains a perfectly defensible one today.

The trouble begins when concurrency rises into the thousands, and it was named by Dan Kegel in his widely circulated "C10K problem" page, first

written in 1999: web servers of the day fell over far below the ten thousand concurrent connections that the hardware — even 1999 hardware — should have handled comfortably. The bottleneck was not bandwidth or CPU cycles spent on useful work. It was the per-connection cost of the threading model itself, and it comes in three parts.

Memory. Each thread needs a stack. On Linux the default is 8 MiB of virtual address space per thread; physical pages are committed lazily, so the real cost is typically tens of kilobytes of touched stack plus the kernel's per-thread structures — but "tens of KiB times ten thousand" is already hundreds of megabytes of memory doing nothing except remembering where ten thousand mostly-idle conversations left off. And it is the *worst-touched* page count that sticks: a single deep call chain (a logging library, a TLS handshake) permanently commits stack pages that idle connections never give back. Compare the alternative: an event-driven server represents an idle connection as a small heap object — a few hundred bytes of state machine — instead of a stack.

Scheduling and context switches. A blocking read that completes wakes a thread; the kernel must schedule it, switch to it, and later switch away. Volume 2, Chapters 1 and 2 costed this out: a context switch runs on the order of a microsecond directly, and its indirect cost — the cache and TLB pollution afterward — is often larger. A server with ten thousand threads processing small messages spends a startling fraction of its cycles switching rather than working, and the scheduler's own bookkeeping over enormous run queues adds to it. Worse, the switches are *involuntary* concurrency: threads are preempted mid-operation, which is what forces all the locking of Chapter 2 onto every shared structure the connections touch.

The cliff is bimodal, not gradual. While every thread fits in memory and the run queue is short, thread-per-connection performs beautifully. As it saturates, throughput does not plateau — it degrades, because switch overhead and memory pressure grow with concurrency while useful work does not. This is the USL's β term (Chapter 1) with an operating-system referent.

The insight behind the alternative is that ten thousand connections do not represent ten thousand things *happening* — at any instant almost all of them are idle, waiting for bytes. What we need is not ten thousand call stacks; it is one loop that can efficiently ask the kernel, "of these ten thousand file descriptors, which have something for me *right now*?" —

and a way to structure the program so that each answer is handled without blocking. That question is **readiness-based I/O multiplexing**, and the structure is the **event loop**. The price, stated honestly up front: control flow is inverted. Your program is no longer a sequence of operations that happens to wait; it is a pile of callbacks (or, with Chapter 8's machinery, suspended coroutines) that the loop invokes when the kernel says the world has changed.

Non-blocking file descriptors: the contract

Everything downstream depends on one flag and one errno, so we state the contract precisely.

By default a socket fd is **blocking**: `read` on an empty socket buffer sleeps the calling thread until data arrives; `write` to a full send buffer sleeps until space frees; `accept` on an empty accept queue sleeps until a connection completes. Setting `O_NONBLOCK` — via `fcntl(fd, F_SETFL, ...)`, or at creation with `SOCK_NONBLOCK` / `accept4` — changes exactly one thing: **any operation that would have slept instead returns -1 immediately with `errno` set to `EAGAIN`** (or `EWOULDBLOCK`; on Linux they are the same value, but portable code checks both).

`EAGAIN` is not an error. It is the kernel saying "not now — come back when I tell you." A non-blocking fd plus a multiplexer is a complete protocol: try the operation; on `EAGAIN`, register interest and go do something else; when the multiplexer reports readiness, try again. Two consequences follow immediately:

- **Every fd an event loop owns must be non-blocking, without exception.** Readiness notification is a hint, not a guarantee — POSIX permits spurious readiness (a checksummed-bad packet can be discarded after `select` said readable; `man 2 select` documents this), and with multiple consumers another thread may drain the socket first. A blocking `read` after a stale readiness report hangs the entire loop. The rule is the moral twin of Chapter 2's mandatory `while` around a condition wait: readiness, like a wakeup, may be spurious, so the operation itself must be safe to attempt and fail.
- **Short operations are normal.** A non-blocking `write` of 64 KiB may accept 11 KiB and return 11; the remaining bytes are your problem. Handling that correctly is the backpressure section later in this chapter.

One scope note that surprises people: `O_NONBLOCK` does approximately nothing for **regular files**. A read from a file on disk never returns `EAGAIN`; it blocks in the kernel for as long as the I/O takes, flag or no flag. Readiness semantics simply do not apply — a file is always "ready" in the sense the interface can express, even when the bytes are milliseconds of seek away. Hold that thought; it explains a structural feature of every event-loop runtime, and it is one of the problems io_uring exists to solve.

The multiplexers: select, poll, epoll, kqueue

select and poll: stateless, O(n) per call

`select(2)` is the original (4.2BSD, 1983). You pass three bitmaps of fds — readable, writable, exceptional — and the kernel returns with the bitmaps overwritten to show which are ready. Its limits are structural. The bitmap is a fixed-size array: `FD_SETSIZE` is 1024 on Linux glibc, and it is a limit on the *numeric value* of the fd, not the count — one long-lived server that leaks its way past fd 1023 is undefined behavior territory. The sets are destroyed by each call, so you rebuild them every iteration. And the cost is O(n) three times over: userspace builds bitmaps over all n fds, the kernel scans all n to poll their state, and userspace scans the results — every iteration, even if one fd of the ten thousand is ready.

`poll(2)` fixes the interface but not the algorithm: an array of `struct pollfd` replaces the bitmaps, so there is no `FD_SETSIZE` ceiling and no per-call destruction of your interest set. But the array still crosses into the kernel on every call, the kernel still walks all n entries, and you still scan n `revents` fields to find the k ready ones. With ten thousand mostly-idle connections, both syscalls do work proportional to the ten thousand to discover the dozen. This per-call O(n) is precisely the C10K bottleneck on the multiplexing side.

epoll: a persistent interest list in the kernel

`epoll` (Linux 2.5.44 era, stabilized in 2.6) restructures the problem by making the interest set a **kernel object with state**, so you stop retransmitting it:

- `epoll_create1(0)` creates the epoll instance — itself an fd, so instances compose and can be monitored.

- `epoll_ctl(epfd, EPOLL_CTL_ADD | MOD | DEL, fd, &ev)` mutates the persistent interest list: which fds, which event mask (`EPOLLIN`, `EPOLLOUT`, `EPOLLET`, …), and a 64-bit `epoll_data_t` cookie of your choosing — typically a pointer to your per-connection state — returned to you with each event.
- `epoll_wait(epfd, events, maxevents, timeout)` blocks until at least one event is available and returns *only the ready ones*.

The mechanism is the inversion that matters: when you `EPOLL_CTL_ADD` an fd, epoll hooks that file's kernel wait queue. When data arrives — in interrupt/softirq context, as the network stack delivers into the socket buffer (Volume 2, Chapter 10) — the callback pushes the fd onto the epoll instance's **ready list**. `epoll_wait` merely drains that list. Nobody scans the interest set; readiness is pushed to the ready list at the moment it occurs, and retrieval costs O(ready), not O(interested). Registration is paid once per fd lifetime instead of once per loop iteration. This is the difference between "ask ten thousand sockets how they are doing" and "read the list of sockets that raised their hand," and it is why epoll's cost is flat in the number of idle connections.

`kqueue` (FreeBSD 2000, inherited by macOS and the other BSDs) is the same idea with a more general and arguably cleaner interface: a single `kevent()` call both submits changes and retrieves events, and the "filter" abstraction covers not just sockets but vnodes, signals, timers, and process events. Windows' answer, IOCP, belongs to the next section because it is not a readiness model at all. Portable loop libraries — libuv, libevent — exist precisely to paper over this per-OS divergence.

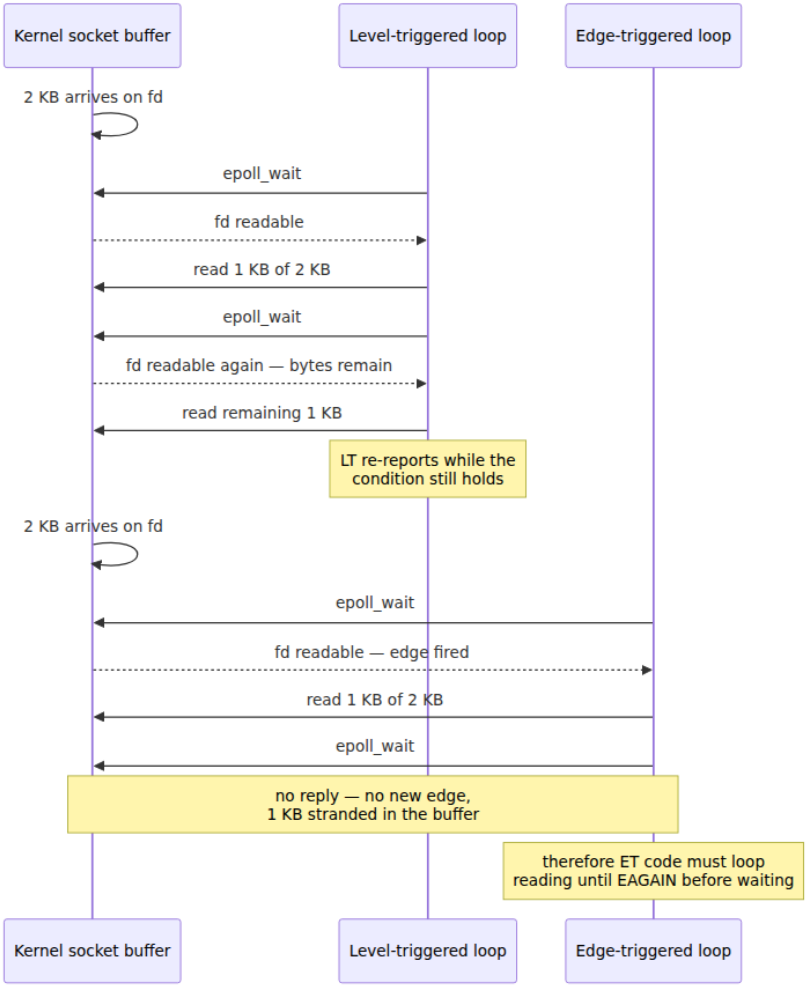
Level-triggered versus edge-triggered, precisely

Epoll offers two notification contracts, and the difference is the sharpest correctness edge in this chapter.

Level-triggered (LT, the default). `epoll_wait` reports an fd whenever its condition *currently holds* — readable while any bytes remain unread, writable while buffer space remains. Read half the data and call `epoll_wait` again, and the fd is reported again. LT is forgiving: leftover work re-announces itself every iteration.

Edge-triggered (ET, `EPOLLET`). `epoll_wait` reports an fd only on a *transition* — when new data arrives on a socket, not while data merely sits there. Read half the data and call `epoll_wait` again, and you get

nothing: no new edge has occurred. The remaining bytes wait in the buffer, silently, forever — or until the peer happens to send more, which for many protocols it will not do, because it is waiting for your reply to the request whose tail you never read. This is the classic ET hang: a distributed deadlock manufactured from one missing loop, and it appears under load and in integration, rarely at the desk.



The ET contract is therefore: **on every event, drain the fd — loop the operation until it returns `EAGAIN` — before returning to `epoll_wait`.** (This is also why ET plus a blocking fd is doubly wrong: the drain loop's

final read would hang.) But the fix creates the opposite hazard. Draining means doing *unbounded* work per event: a peer that streams data as fast as you read — a fast producer on a fat pipe — keeps the drain loop spinning on one fd while every other connection in the loop starves. One hot connection becomes a livelock-flavored starvation of thousands (Chapter 5's vocabulary applies exactly). Production ET code therefore bounds the work — read at most N buffers per event, and if not yet at `EAGAIN`, put the fd on an application-level ready list to resume after one full pass over the other ready fds. At which point you have reimplemented, in userspace, the fairness that LT gives you for free. This is why the honest guidance is: **use LT until you have a measured reason not to**; ET saves wakeups and syscalls for high-throughput streaming workloads, and nginx uses it, but it moves two correctness obligations (drain-to-EAGAIN, fairness) from the kernel into your code.

Why ET exists at all: LT has a multi-consumer problem. If several threads wait on one epoll fd in LT mode, a single readable socket can wake more than one of them, and both race into the same `read`. `EPOLLONESHOT` is the targeted fix — after delivering one event the fd is disarmed, delivering nothing further until a thread explicitly re-arms it with `EPOLL_CTL_MOD` when its processing is done. That makes "exactly one thread owns this fd's event at a time" a kernel guarantee, at the cost of one extra syscall per event. Multi-threaded epoll designs essentially require it; single-threaded loops never need it.

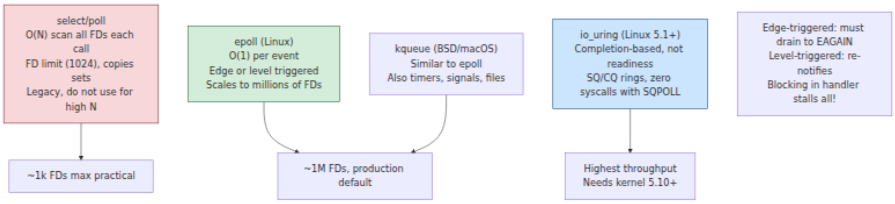
A minimal, correct epoll echo server

The following is complete and compilable, and it exhibits the three disciplines just derived: every fd non-blocking, accept drained to `EAGAIN`, reads drained to `EAGAIN` (connections are registered ET to make the drain obligation visible).

```c
#include <errno.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <unistd.h>

#define MAX_EVENTS 64

int main(void) {
    int listen_fd = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
    int one = 1;
    setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &one, sizeof one);

    struct sockaddr_in addr = {0};
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port        = htons(9000);
    if (bind(listen_fd, (struct sockaddr *)&addr, sizeof addr) < 0 ||
        listen(listen_fd, SOMAXCONN) < 0) {
        perror("bind/listen");
        exit(1);
    }

    int epfd = epoll_create1(0);
    struct epoll_event ev = { .events = EPOLLIN, .data = { .fd = listen
_fd } };
    epoll_ctl(epfd, EPOLL_CTL_ADD, listen_fd, &ev);

    struct epoll_event events[MAX_EVENTS];
    for (;;) {
        int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
        if (n < 0) {
            if (errno == EINTR) continue;
            perror("epoll_wait");
            exit(1);
        }

        for (int i = 0; i < n; i++) {
            int fd = events[i].data.fd;

            if (fd == listen_fd) {
                /* Drain the accept queue to EAGAIN: several
connections
                 * may sit behind a single readiness report. */
                for (;;) {
                    int conn = accept4(listen_fd, NULL, NULL, SOCK_NONB
LOCK);
```

```
                    if (conn < 0) {
                        if (errno == EAGAIN || errno == EWOULDBLOCK) br
eak;
                        if (errno == EINTR) continue;
                        perror("accept4");
                        break;
                    }
                    struct epoll_event cev = {
                        .events = EPOLLIN | EPOLLET,
                        .data   = { .fd = conn }
                    };
                    epoll_ctl(epfd, EPOLL_CTL_ADD, conn, &cev);
                }
                continue;
            }

            /* Connection fd, registered edge-triggered:
             * read until EAGAIN or the edge is lost. */
            for (;;) {
                char buf[4096];
                ssize_t r = read(fd, buf, sizeof buf);
                if (r > 0) {
                    /* Echo back. write may accept fewer bytes than
                     * offered; a real server must queue the remainder
                     * and watch EPOLLOUT — see the backpressure
                     * section. This demo drops what will not fit. */
                    ssize_t off = 0;
                    while (off < r) {
                        ssize_t w = write(fd, buf + off, (size_t)(r - o
ff));
                        if (w > 0) { off += w; continue; }
                        if (w < 0 && errno == EINTR) continue;
                        break;                      /* EAGAIN */
                    }
                } else if (r == 0) {                /* peer closed */
                    close(fd);   /* close also removes fd from epoll */
                    break;
                } else if (errno == EAGAIN || errno == EWOULDBLOCK) {
                    break;                          /* drained */
                } else if (errno == EINTR) {
                    continue;
                } else {
                    close(fd);
                    break;
                }
            }
        }
    }
}
```

Note what per-connection state this server carries: none beyond the fd itself, because echo is stateless between events. A real protocol handler attaches a state machine — parse position, partial frame, write queue — via `epoll_data_t`'s pointer form, and that heap object *is* the connection, in the same sense that a stack was the connection in thread-per-connection.



Completion-based I/O: io_uring, IOCP, and the proactor

Everything so far is a **readiness** model: the kernel tells you an operation *would now succeed*, and you then perform it yourself. The alternative is a **completion** model: you tell the kernel to *perform the operation* — here is the fd, the buffer, the length — and it notifies you when the operation has finished, results in hand. Design-pattern literature names the two **Reactor** and **Proactor** (Schmidt et al., *Pattern-Oriented Software Architecture* vol. 2); Windows has been completion-based for three decades via I/O Completion Ports, where threads block on `GetQueuedCompletionStatus` and pull finished operations off a queue — IOCP is the design Windows-native servers and the .NET runtime are built on. Linux's historical attempts at completion I/O (POSIX AIO, the old `io_submit` interface) were limited and little-loved; the modern answer is `io_uring`.

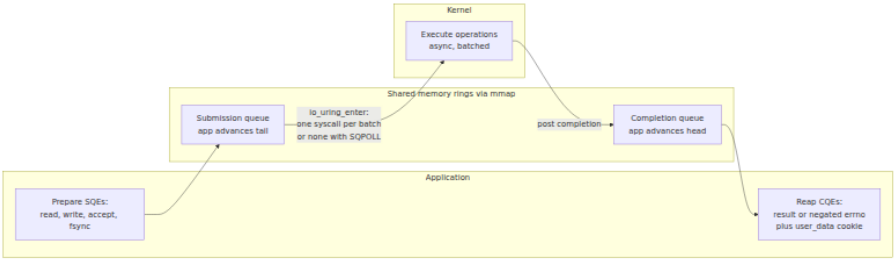
io_uring: two rings of shared memory

`io_uring`, merged in Linux 5.1 (2019, Jens Axboe), is built around two circular buffers **mapped into shared memory** between the application and the kernel:

- The **submission queue (SQ)** holds submission queue entries (SQEs). An SQE describes one operation: an opcode (`READ`, `WRITE`, `ACCEPT`, `RECV`, `SEND`, `FSYNC`, `TIMEOUT`, `OPENAT`, and by now dozens more), an fd, buffer, length, offset, flags, and a `user_data` cookie returned with the completion.

- The **completion queue (CQ)** holds completion queue entries (CQEs): the `user_data` cookie plus a result field carrying the return value or negated errno — exactly what the equivalent syscall would have returned.

The application writes SQEs and advances the SQ tail with an ordinary atomic store; the kernel consumes them and posts CQEs; the application reaps CQEs by advancing the CQ head. Because the rings are shared memory, **the syscall boundary decouples from the operation count**: one `io_uring_enter` call submits an entire batch of SQEs and can simultaneously wait for completions. With `IORING_SETUP_SQPOLL`, a kernel-side polling thread watches the SQ and submission requires *no syscall at all* while the thread is awake; reaping completions is likewise just reading shared memory. Against the backdrop of Volume 2, Chapter 5 — syscalls costing hundreds of nanoseconds, more in the post-Spectre-mitigation world — amortizing or eliminating per-operation syscalls is a large fraction of io_uring's performance story. The rest comes from pre-registration: `io_uring_register` lets you register buffers and fd sets once, so the kernel skips per-operation pinning and fd-table lookups.



The programming model, via the `liburing` helper library:

```c
struct io_uring ring;
io_uring_queue_init(256, &ring, 0);

/* Submit: describe the operation, do not perform it. */
struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
io_uring_prep_read(sqe, conn_fd, buf, sizeof buf, 0);
io_uring_sqe_set_data(sqe, conn_state);        /* our cookie */
io_uring_submit(&ring);                        /* batch boundary */

/* ... later, in the loop ... */
struct io_uring_cqe *cqe;
io_uring_wait_cqe(&ring, &cqe);
struct conn *c = io_uring_cqe_get_data(cqe);
int res = cqe->res;            /* bytes read, 0 on EOF, or -errno */
io_uring_cqe_seen(&ring, cqe);
/* the data is already in buf — no read call to make */
```

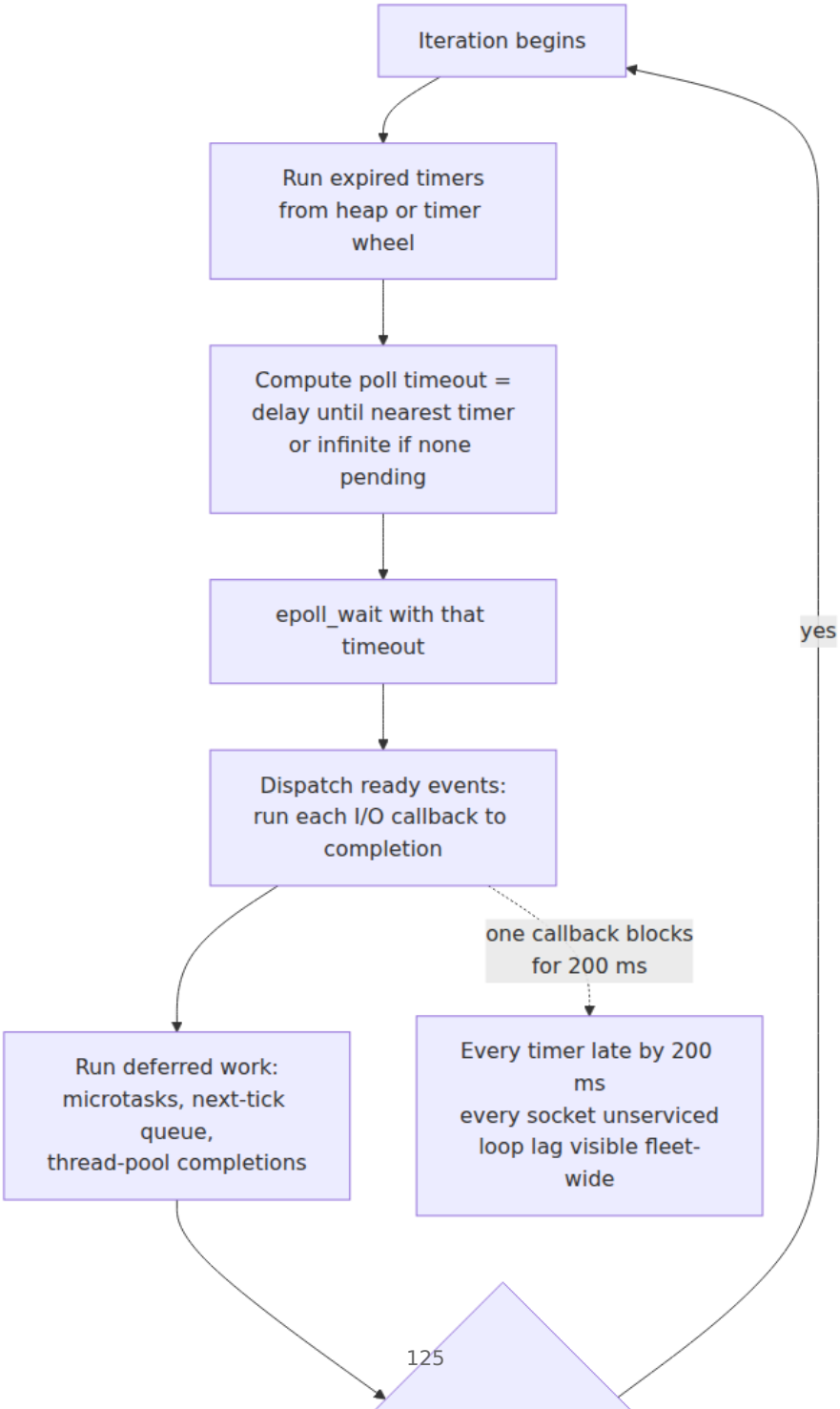
Contrast this with the epoll server above and the paradigm difference is visible in the code shape. With epoll you register *interest* and the I/O calls stay in your program; readiness is advice. With io_uring you submit *operations*; when the CQE arrives, the read has already happened — the bytes are in your buffer. Three consequences follow. First, the drain-to-EAGAIN discipline and the LT/ET distinction simply do not exist here; in their place is management of in-flight operations and their buffer lifetimes (a buffer lent to the kernel must stay valid until its CQE — a new class of use-after-free if you get it wrong). Second, **regular files work**: since you are asking for completion rather than readiness, the "a file is always ready" problem dissolves, and disk I/O joins network I/O in the same uniform loop — the gap that forced event-loop runtimes to bolt on thread pools, as the next section shows. Third, operations can be chained (`IOSQE_IO_LINK`) so that, e.g., a write completes before a linked fsync starts, pushing small state machines into the kernel.

The caveats, stated plainly. io_uring's implementation is large, complex, and it has been a prolific source of kernel vulnerabilities — Google reported in 2023 that io_uring bugs accounted for a majority of recent exploit submissions to their kernel bug-bounty programs, and consequently restricted or disabled it on ChromeOS, on Android for apps, and on much of their production fleet. Docker's default seccomp profile blocks the io_uring syscalls, and many managed/multi-tenant environments do likewise. None of this means io_uring is wrong for your dedicated database host — high-performance storage engines and runtimes adopt it where the wins are real — but check what your

platform actually permits before designing around it, and expect the security posture to keep evolving. Meanwhile epoll remains the default substrate of nearly every mainstream event-loop runtime, so the readiness model is the one you will operate for years yet.

Anatomy of a production event loop

A multiplexer is not yet an event loop. A production loop — libuv, libevent, Netty's `EventLoop`, Tokio's reactor+executor, Envoy's `Dispatcher` — wraps the poll call in a repeating structure with a small number of universal parts.


Iteration begins
Run expired timers from heap or timer wheel
Compute poll timeout = delay until nearest timer or infinite if none pending
epoll_wait with that timeout
Dispatch ready events: run each I/O callback to completion
one callback blocks for 200 ms
Run deferred work: microtasks, next-tick queue, thread-pool completions
Every timer late by 200 ms
every socket unserviced
loop lag visible fleet-wide
yes

Timers. Every loop needs "call me in 30 s" — connect timeouts, keepalives, retries. The poll timeout is simply the delay until the earliest timer, so timers cost nothing while the loop sleeps. Two data structures dominate: a **binary min-heap** (O(log n) insert/expire, exact ordering — libuv, most runtimes) and the **hashed/hierarchical timer wheel** (buckets per tick; O(1) insert and cancel at the price of coarser granularity — the classic Varghese–Lauck design, used by the kernel itself and by Netty and Kafka). Wheels win when timers are numerous and usually *cancelled* — which describes I/O timeouts exactly: millions armed, almost none fire.

Deferred callbacks and microtasks. Every mature loop grows a "run this after the current callback, before polling again" queue — libuv's check/idle handles, JavaScript's microtask queue and `process.nextTick`, Netty's task queue. Two subtleties matter operationally. Ordering: these queues run *before* the loop returns to polling, so they express "finish this logical operation atomically with respect to I/O." Starvation: precisely because they run before polling, a callback that endlessly schedules more deferred work starves the poll — in Node, a recursive `process.nextTick` freezes all I/O in a way a recursive `setImmediate` does not, because `setImmediate` yields to the poll phase each iteration.

The rule: never block the loop. Everything in this design amortizes one thread across thousands of connections; therefore that thread's time is the shared resource, and any callback that occupies it — a synchronous file read, a blocking DNS lookup, a call into a library that takes a contended lock (Chapter 2), a 300 ms JSON parse of a pathological payload, an accidental synchronous `write` to a blocking log fd — stops *every* connection, not just its own. This is the trade Chapter 1 described when you chose this model: you gave up preemption. The scheduler cannot rescue you from a hogging callback the way it rescues other threads from a hogging thread, because scheduling is now cooperative and the unit of yielding is the callback. Latency SLOs on an event-loop service are therefore SLOs on the *longest callback*, and the p99.9 of "callback duration" is a metric worth having.

The escape hatch: the thread pool. Work that is CPU-bound or unavoidably blocking gets handed to a pool of worker threads, with completion marshalled back to the loop as an event (typically by writing to a pipe or `eventfd` that the loop polls, so the loop wakes through the same mechanism as any I/O). libuv is the canonical example, and the

reason it has a pool is the fact flagged earlier: **regular-file I/O has no useful readiness semantics on Linux** — epoll will not even accept a regular-file fd (`epoll_ctl` returns `EPERM`). So every `fs.*` operation in Node, plus `getaddrinfo` DNS lookups and some crypto, executes as a blocking call on a libuv pool thread (default 4, `UV_THREADPOOL_SIZE` to raise it), and only the *completion* flows through the loop. The "async" file API is threads in a trench coat — not a criticism, but a fact with operational teeth: four pool threads shared by file I/O and DNS means four slow disk reads make DNS resolution mysteriously slow. The same pattern under different names: Netty's `blockingTaskExecutor` conventions, Tokio's `spawn_blocking`, Go's runtime quietly parking a thread per blocking file syscall. io_uring is the first Linux interface that could retire this hack, and libuv has experimental io_uring support for exactly this reason.

Case studies, done mechanically

nginx: N loops, share nothing

nginx's architecture is a **master process** (configuration, binding sockets, supervising) plus N **worker processes**, each running one single-threaded event loop over epoll (kqueue on BSD), with `worker_processes auto` defaulting N to the core count. Each worker handles thousands of connections; a connection lives on one worker for its lifetime. Why one worker per core works: each loop saturates at most one core, so N cores want N loops; more than N adds context switching without adding capacity; fewer leaves cores idle. Because workers are separate *processes* sharing almost nothing, there are no locks on the request path and a worker crash takes only its own connections — Chapter 1's "share nothing" answer, applied within one machine. The workers share only the listening sockets, which is where the thundering-herd section below picks up the story; with `listen ... reuseport`, each worker instead gets its own listening socket and the kernel distributes connections among them. nginx uses edge-triggered epoll and carries the drain-and-fairness obligations that entails; its origin is explicitly C10K-era — Igor Sysoev began it in 2002 in substantial part to solve exactly the problem Kegel described.

Node.js: one loop, phases, and two kinds of task

Node is V8 (execution) plus libuv (the loop) plus bindings. The libuv loop proceeds in **phases** per iteration — approximately: timers (`setTimeout`/`setInterval` whose deadline has passed), pending callbacks, poll (the epoll_wait, plus running I/O callbacks), check (`setImmediate`), close callbacks. Distinct from all of these are the **microtask queues** — `process.nextTick` and resolved-promise jobs — which drain after the currently running callback completes, before the loop proceeds; this is the macrotask/microtask distinction, and it means an `await` chain continues with minimal latency but also that promise-heavy CPU work can crowd the loop just like any other callback. Everything from the previous section applies verbatim: one thread runs all JavaScript; blocking it blocks every request; file I/O and DNS ride the threadpool. The canonical self-inflicted outage:

```
const { createServer } = require('node:http');
const { monitorEventLoopDelay } = require('node:perf_hooks');

const h = monitorEventLoopDelay({ resolution: 10 });
h.enable();

createServer((req, res) => {
  if (req.url === '/report') {
    // Synchronous CPU work: nothing else runs until this returns.
    const until = Date.now() + 2000;
    while (Date.now() < until) { /* render a big report, badly */ }
    res.end('report\n');
  } else {
    res.end('ok\n');            // normally sub-millisecond
  }
}).listen(8080);

setInterval(() => {
  console.log(`loop delay p99 = ${(h.percentile(99) / 1e6).toFixed(1)}
ms`);
  h.reset();
}, 5000);
```

While `/report` spins, every concurrent `/ok` request — already accepted, bytes already in the socket buffer — waits the full two seconds, and `monitorEventLoopDelay` records it. One request paid for the work; every request paid the latency. The remedies are the standard three: chunk the work and yield (`setImmediate`), move it to `worker_threads`, or move it out of the process.

Redis: the event loop as a concurrency-control mechanism

Redis runs its own small loop (`ae.c`, epoll/kqueue/select behind one interface) and executes **every command on a single thread**. This is a correctness decision as much as a performance one: commands are naturally serialized, so `INCR`, `LPUSH` + `LTRIM`, Lua scripts — each runs atomically with respect to all others *with zero locks*, none of Chapter 2's machinery, because mutual exclusion is enforced by there being only one executor. The event loop here *is* the concurrency control. The costs are the model's usual ones: a slow command (`KEYS` on a big keyspace, a huge `SMEMBERS`, a long Lua script) stalls every client — the single-loop HOL problem with a database attached — and one instance uses one core, which is why Redis scales by running more instances (Cluster), not more threads.

Be precise about what later versions changed, because it is commonly misstated. Redis has long run a few background threads (`bio`) for slow housekeeping — fsync, closing files, and lazy free (`UNLINK`) since Redis 4. Redis 6 added **I/O threads** (`io-threads N`), which parallelize only the syscall-and-serialization edge: writing replies to sockets, and optionally reading and parsing requests (`io-threads-do-reads yes`). **Command execution remains single-threaded.** The atomicity story is untouched; what scales is the part that was measured to dominate — moving bytes to and from thousands of sockets.

Backpressure: the write side is where servers die

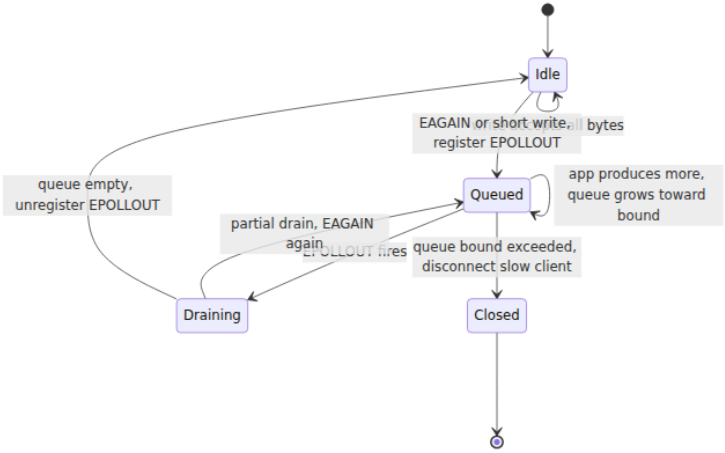
Reads are self-limiting — you read at your own pace, TCP's receive window pushes back on the sender (Volume 3). Writes are where event-loop servers grow their classic memory bug.

The kernel buffers your writes: a non-blocking `write` copies into the socket's send buffer (size governed by `SO_SNDBUF` and autotuning) and returns; TCP drains it at whatever rate the network and *the receiver* allow. Write to a slow client — a phone on bad radio, a stalled consumer, a victim of your own 10 Gbps enthusiasm — and the buffer fills. Then `write` returns a **short count**, and eventually `EAGAIN`. Both mean the same thing: *the kernel will not take more; the remainder is your problem.*

The correct protocol is a small state machine per connection:

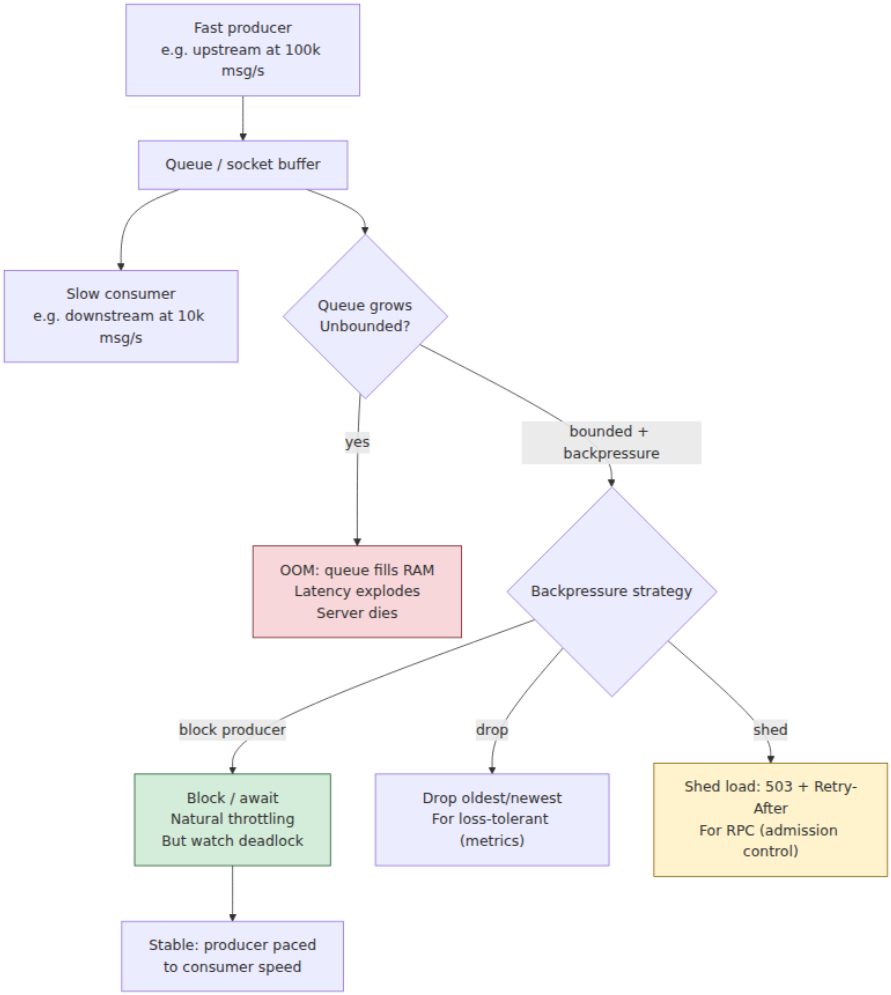
1. Try the write directly. If everything is accepted, done — this is the common case, and you never touch epoll's write machinery.
2. On a short write or `EAGAIN`, queue the remaining bytes in userspace **and only now add `EPOLLOUT` to the fd's interest set**.
3. When `EPOLLOUT` fires, drain the queue into the socket until empty or `EAGAIN` again.
4. When the queue empties, **remove `EPOLLOUT` from the interest set**.

Step 4 is not an optimization nicety, it is required for a level-triggered loop to function: a mostly-empty send buffer means the socket is *almost always writable*, so a permanently registered LT `EPOLLOUT` fires on every single `epoll_wait` — a busy loop burning a core to learn nothing. Watch writability only while you have something queued.



The remaining question is the queue's bound, and here is the classic bug: **the unbounded write queue.** The application produces data at its own rate — a pub/sub fanout, a log tail, a big query result — while the socket drains at the client's rate. Any gap accumulates in your heap. One stalled client on a high-volume feed grows its queue without limit; a few hundred of them OOM the process, taking down every healthy client with them. The signature in the postmortem is memory growth proportional to the *slowest* consumers, not to load. The defenses are policy, and they must be chosen, not defaulted: **bound the queue** and then either disconnect the slow client (Redis's `client-output-buffer-limit` does exactly this for pub/sub clients), drop or coalesce intermediate updates (fine for market

data ticks or metrics, where the latest value supersedes), or **propagate the stall backward** — stop reading from whatever source feeds this connection, so the pressure transmits upstream link by link. Propagation is what `stream.pipe` / `pipeline` implement in Node (a `write` returning `false` pauses the readable side until `'drain'`) and what TCP itself does with its windows. The same three options — disconnect, drop, or propagate — reappear at fleet scale in messaging systems, and Volume 10, Chapter 7 treats backpressure there; the socket-level version here is the atom from which those designs are built.



The thundering herd, and sharing accept

Run N workers for N cores and one question remains: who accepts new connections? Let all N poll the same listening fd and you meet the **thundering herd**: a single incoming connection makes the fd readable, all N workers wake, all N call `accept`, one wins, and N−1 burned a wakeup, a scheduling round, and a cache refill for an `EAGAIN`. At high accept rates this is real work — and it is the same shape as Chapter 2's futex-wake storms and Chapter 5's contention parades. (For processes blocked in plain `accept` the kernel has done wake-one for decades; the herd survives in the epoll formulation, because *readiness* notification historically meant notifying everyone interested.)

Three mitigations, in historical order:

- **Userspace serialization.** nginx's classic `accept_mutex`: workers take turns holding the right to register the listen fd. Works, adds latency and its own small contention.
- `EPOLLEXCLUSIVE` (Linux 4.5): register the listen fd with this flag in every worker's epoll set, and the kernel wakes only one (or at least fewer — the contract is "one or more") of the waiters per event. Simple, effective, keeps one shared accept queue, so load balances naturally to whichever worker is free.
- `SO_REUSEPORT` (Linux 3.9): each worker binds its *own* listening socket to the same address and port; the kernel hashes each incoming connection's 4-tuple to pick exactly one socket. No shared fd, no herd at all, and the accept path itself scales across cores. Two costs worth knowing. First, balance: the hash distributes *connections*, not load — long-lived heavy connections can pile onto one worker; nginx added `reuseport` and Envoy supports the same for these wins and with these caveats. Second, drain: when a worker dies or you remove a socket during a reload, connections already queued on that socket's accept queue but not yet accepted are reset — a small but real error blip on every restart, which schedulers and runtimes mitigate with careful drain sequencing (Volume 11's deployment chapters touch the operational side).

The distributed-systems lens

The event loop is the atom of your data plane. Every proxy, gateway, and sidecar in a modern fleet is a reactor of exactly this chapter's shape. Envoy's threading model is the cleanest statement: a main thread for config and control (the xDS machinery, Volume 3's service mesh discussion), plus N worker threads, each running one non-blocking event-loop dispatcher; every connection is pinned to one worker for its lifetime, so the request path takes no locks and cross-thread interaction happens by posting callbacks to another loop's queue. That is nginx's architecture with threads instead of processes, and it is HAProxy's, and it is the shape of most load balancers you will ever operate. Understanding this chapter *is* understanding what the p99 of your mesh does when someone's Lua/Wasm filter blocks, or why one worker at 100% CPU manifests as tail latency on a fraction of connections rather than uniform slowdown.

Readiness versus completion recurs as polling versus push. The epoll/io_uring distinction — "tell me when I could act" versus "act and tell me when it is done" — is the same design axis as consumer polling versus broker push in messaging (Volume 10), and short-poll versus long-poll versus server push in APIs (Volume 8). The trade-offs transfer with surprising fidelity: readiness/polling keeps the consumer in control of pacing and buffer ownership and makes backpressure trivial (you simply do not poll); completion/push minimizes latency and per-event overhead but forces an explicit story for in-flight limits and buffer lifetime — exactly the discipline io_uring demands with its submitted-buffer ownership rules.

Head-of-line blocking is scale-free. One slow callback delaying every connection in its loop is the same phenomenon as one slow request delaying every response behind it on an HTTP/1.1 pipelined connection, or one lost TCP segment stalling every multiplexed HTTP/2 stream above it (Volume 3) — a single serialized resource with mixed traffic. The mitigations rhyme at every scale: bound the unit of work (chunked callbacks / frame limits), isolate classes of traffic (separate loops or pools / separate connections), or move scheduling below the point of loss (QUIC's per-stream delivery).

Loop lag is queueing delay, and it is THE health metric. Chapter 1 gave us Little's Law and the utilization-delay curve; an event loop is a single-server queue in exactly that formalism. Event-loop delay — the gap

between when a timer or event *should* have run and when it did — is the wait time W of work sitting in the ready queue, and it inflates every response time on the process, uniformly, before any of your application code runs. That is why mature async services export it directly (Node's `monitorEventLoopDelay`; comparable dispatcher-latency stats in Envoy and Netty-based stacks) and alert on it rather than on CPU: a loop can be badly lagged at modest CPU (blocked on the "async" disk write it wasn't supposed to make) and healthy at high CPU (many small callbacks, none long). And because callers stack timeouts and retries on top (Volume 11), a lagged loop in one tier does not stay local: it becomes retry amplification upstream and tail latency two hops away. Fleet-wide p99 investigations end at somebody's blocked event loop often enough that loop-lag dashboards should be the first tab, not the last.

Key takeaways

- Thread-per-connection fails at high concurrency because per-connection cost — stack memory, context switches, scheduler load — grows with connections rather than with useful work. C10K's answer: represent idle connections as small state objects and ask the kernel which fds are ready *now*.
- `O_NONBLOCK` turns "would sleep" into `EAGAIN`, and `EAGAIN` is a protocol, not an error. Every fd in an event loop must be non-blocking, because readiness reports can be stale or spurious — the moral twin of the mandatory `while` around a condition wait.
- `select` and `poll` retransmit and rescan the whole interest set every call — O(n) per iteration. epoll keeps the interest list in the kernel and returns only the ready list; registration is paid once, retrieval costs O(ready). kqueue is the BSD equivalent.
- **Level-triggered** reports while the condition holds; **edge-triggered** reports only transitions. ET therefore requires draining every event to `EAGAIN` — miss it and data strands forever — and bounding the drain, or one hot fd starves the loop. Use LT until measurement says otherwise; use `EPOLLONESHOT` when multiple threads share an epoll set.
- io_uring is a **completion** model: submit operations through a shared-memory ring, reap results from another, batching or eliminating syscalls, and covering regular files uniformly. It is a different paradigm from epoll's readiness, with different obligations (in-flight

ops, buffer lifetime) — and enough kernel-security history that several major fleets restrict it; verify your platform allows it before designing around it.

- A production loop = poll + timers (heap or timer wheel) + deferred queues + a thread pool for blocking work. libuv's pool exists because regular-file I/O has no readiness semantics on Linux — "async" file APIs are threads underneath.
- **Never block the loop.** One stalled callback stalls every connection; your latency SLO is an SLO on your longest callback. nginx, Node, and Redis are all one-loop-per-core designs, and Redis's single executor is *why* its commands are atomic without locks.
- Writes to slow clients fill the send buffer, then short-write, then `EAGAIN`. Queue the remainder, watch `EPOLLOUT` *only while the queue is non-empty*, and **bound the queue** with an explicit policy — disconnect, drop, or propagate. Unbounded write queues are the classic event-loop OOM.
- Shared accept wakes herds; serialize it, use `EPOLLEXCLUSIVE`, or give each worker its own socket with `SO_REUSEPORT` — knowing reuseport balances connections, not load, and resets queued connections on worker death.
- Event-loop lag is queueing delay in Little's Law terms and the single best health metric of an async service; it inflates every request on the process and amplifies through retries into fleet-wide tail latency.

Further reading

- Kegel, D., "The C10K problem" — the page that named the era and catalogued the strategies. http://www.kegel.com/c10k.html
- `man 7 epoll` — the authoritative description of LT/ET semantics, `EPOLLEXCLUSIVE`, `EPOLLONESHOT`, and the documented pitfalls; also `man 2 select`, `man 2 poll`, `man 2 accept4`, `man 7 socket`.
- Lemon, J., "Kqueue: A generic and scalable event notification facility," *USENIX Annual Technical Conference (FREENIX Track)*, 2001 — the kqueue design paper.
- Axboe, J., "Efficient IO with io_uring" — the original design document; and the `liburing` repository and man pages (`io_uring_setup(2)`, `io_uring_enter(2)`, `io_uring_register(2)`). https://kernel.dk/io_uring.pdf

- Google Security Blog, "Learning to navigate the risks of io_uring" and related kCTF disclosures (2023) — the security posture behind fleet restrictions on io_uring.
- Schmidt, D., Stal, M., Rohnert, H., Buschmann, F., *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects* (Wiley, 2000) — Reactor and Proactor, defined.
- Varghese, G. and Lauck, T., "Hashed and Hierarchical Timing Wheels: Data Structures for the Efficient Implementation of a Timer Facility," *SOSP*, 1987 — the timer-wheel paper.
- libuv documentation, "Design overview" — the loop phases and the thread pool, from the source. https://docs.libuv.org/en/v1.x/design.html
- Node.js documentation, "The Node.js Event Loop" guide and `perf_hooks.monitorEventLoopDelay` — phases, microtasks, and measuring loop delay.
- nginx documentation, "Inside NGINX: How We Designed for Performance & Scale" and the `listen ... reuseport` directive documentation.
- Redis documentation and `redis.conf` comments on `io-threads`, `io-threads-do-reads`, and `client-output-buffer-limit` — what is and is not threaded, and output-buffer policy.
- Envoy documentation, "Threading model" — one non-blocking dispatcher per worker, connections pinned to workers.
- Volume 2, Chapter 5 — System Calls — why batching and avoiding syscalls matters; Volume 2, Chapter 10 — the network stack that delivers into the buffers epoll watches.
- Volume 10, Chapter 7 — backpressure in messaging systems: the fleet-scale version of this chapter's write-queue problem.

Chapter 7 — The Actor Model, CSP, and Channels

What this chapter covers. Chapter 1's taxonomy split the world into models that share mutable state and models that refuse to; Chapters 2 through 5 paid the bill for sharing. This chapter develops the refusal

seriously. Message passing is not one idea but two, older and more different from each other than the folklore suggests. The **actor model** (Hewitt, 1973) gives every concurrent entity an identity and an asynchronous mailbox, and — in its Erlang/OTP realization — the strongest fault-tolerance story in mainstream software. **CSP** (Hoare, 1978) makes the processes anonymous and the *channels* named, with a synchronous rendezvous at every communication, and is the direct ancestor of Go's goroutines and channels. We treat both rigorously — the actor axioms, supervision trees and why "let it crash" actually works, Go channel semantics down to the happens-before edges and the nil-channel idiom, the standard channel patterns — and then catalogue the failure modes honestly: unbounded mailboxes, goroutine leaks, channel deadlocks, and the race conditions message passing conspicuously does not eliminate. The chapter closes by comparing the two models head to head and connecting both to distributed systems, where message passing is not a design choice but the physics of the situation.

Learning goals — after this chapter you should be able to:

- State the three actor axioms precisely and explain what asynchronous sends and location transparency buy — and what they cost.
- Explain how Erlang/OTP turns the actor model into a reliability discipline: process isolation, links versus monitors, supervision trees, restart strategies, and the actual argument behind "let it crash."
- State classical CSP's core commitments — anonymous processes, named channels, synchronous rendezvous — and trace their lineage into occam and Go.
- Give the precise semantics of Go channels: unbuffered rendezvous and its happens-before edge, the buffered-channel capacity rule, `select`'s pseudo-random choice, and the exact behavior of nil and closed channels.
- Build and recognize the standard channel patterns: pipeline, fan-out/fan-in, worker pool, cancellation via done channel or `context`, and semaphore-via-buffered-channel.
- Enumerate the failure modes message passing does *not* remove — memory exhaustion, leaks, deadlock, ordering surprises, logical races — and know the mitigation for each.

- Choose between actors, CSP, and shared memory from workload characteristics rather than fashion.

Message passing: the other half of the map

Recall the split from Chapter 1's taxonomy. Shared-memory models let threads mutate common data structures and then spend three chapters' worth of machinery — mutexes, memory barriers, atomics — making that safe. Message-passing models take the other branch: concurrent entities own their state privately, and the *only* way information moves between them is by sending messages. There is no shared mutable state, so there is nothing to race on. The data-race hazard from Chapter 1 is not mitigated; it is eliminated by construction.

The Go proverb states the design stance in one line:

> Do not communicate by sharing memory; instead, share memory by communicating.

Unpack it, because it is denser than it looks. "Communicate by sharing memory" is the shared-state pattern: threads coordinate by writing flags and structures that other threads read, with synchronization bolted on to make the writes visible and atomic. "Share memory by communicating" inverts the ownership: at any moment, exactly one goroutine owns a piece of data, and when another goroutine needs it, the *data itself* (or a reference, with ownership conventionally transferred) travels through a channel. Synchronization is not an annotation on the data; it is the communication. The transfer of the value and the transfer of the right to touch it are the same event.

Two consequences follow immediately, and they frame everything in this chapter. First, correctness reasoning becomes local: an actor or a CSP process is *sequential code*. You can read its receive loop top to bottom and reason about it the way you reason about a single-threaded program, because within one entity there is no interleaving. The global concurrency is confined to the message layer, where it is visible and explicit. Second — and this is the honest part — the hazards do not vanish; they migrate. Races on memory become races on *message ordering*. Deadlocks on locks become deadlocks on cyclic channel waits. Unbounded queues replace unbounded lock convoys. We will meet each migration in its own section.

The two message-passing traditions differ on three axes, and holding the axes in mind makes both models easier to learn:

Axis	Actor model	Classical CSP
What has a name	The process (actor identity, PID)	The channel
Send semantics	Asynchronous, fire-and-forget, mailbox buffers	Synchronous rendezvous, sender and receiver meet
Natural habitat	Distributed by design (location transparency)	In-process coordination

The actor model

Origins and axioms

The actor model was introduced by Carl Hewitt, Peter Bishop, and Richard Steiger in a 1973 IJCAI paper, *A Universal Modular ACTOR Formalism for Artificial Intelligence*, and given its rigorous semantics by Gul Agha's 1986 monograph *Actors: A Model of Concurrent Computation in Distributed Systems*. It predates nearly everything else in this volume; it was conceived when "many communicating machines" was a research vision, which is exactly why it fits that world so well now that the vision is the default deployment topology.

An **actor** is the unification of three things:

- **State** — private, encapsulated, touchable by no one else. There is no operation, in the model, by which one actor reads another's state.
- **Behavior** — the code that runs when a message is received; conceptually a function from (state, message) to a new state and a set of effects.
- **Mailbox** — a queue of messages sent to this actor and not yet processed. The actor processes messages one at a time; while it is handling one message, others simply wait.

The model's semantics fit in one sentence, traditionally stated as three axioms. **In response to a message it receives, an actor may:**

1. **send** a finite number of messages to other actors (whose addresses it knows);

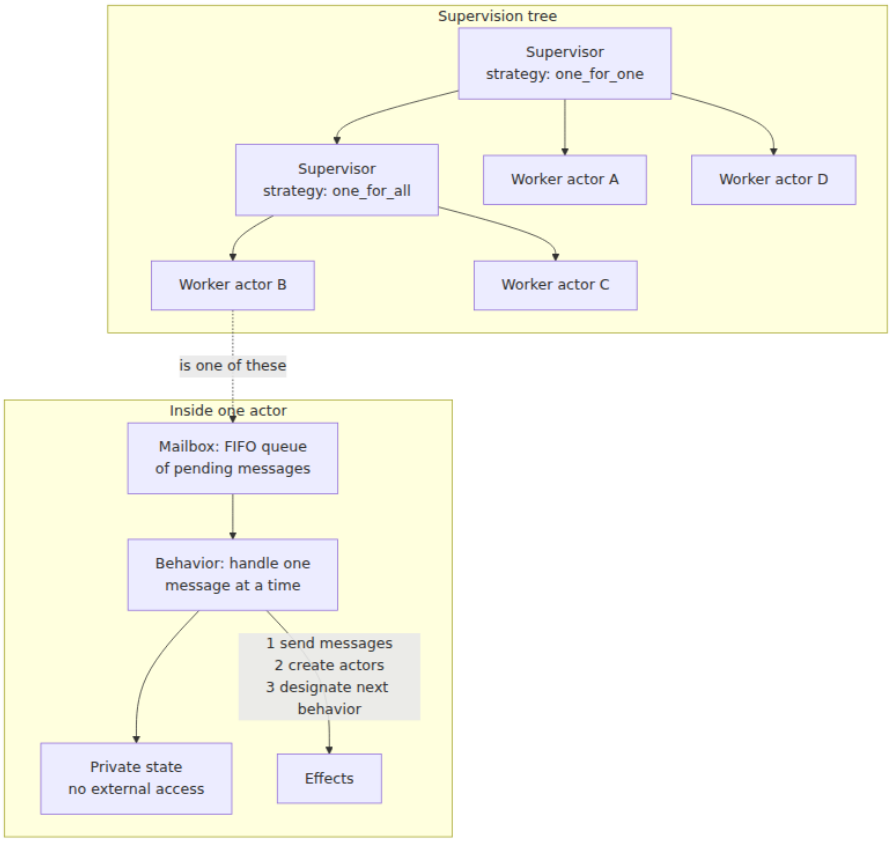
2. **create** a finite number of new actors;
3. **designate the behavior** to be used for the next message it receives.

Axiom 3 is the one people skate past, and it is where the state lives. "Designate the behavior for the next message" is how an actor changes state without mutation: a counter actor holding 4, upon receiving `increment`, designates "the counter behavior holding 5" as its successor. In Erlang this is literally a tail-recursive call with a new argument; in object-flavored actor systems it is assignment to fields that no other thread can see. Either way, the axiom guarantees the property that makes actors easy to reason about: **state transitions are serialized by the mailbox**. One message, one transition, no interleaving. An actor is a tiny single-threaded server, and the whole system is a society of them.

Two further commitments complete the model:

Sends are asynchronous and fire-and-forget. `send` deposits a message and returns. There is no acknowledgment, no return value, no blocking. If you want a reply, you include your own address in the message and the recipient sends one back — request/response is a *pattern built on top of* one-way sends, not a primitive. This is a deliberate mirror of physical reality: messages between machines are one-way datagrams at the bottom, and a model whose primitive matches the substrate distributes without translation.

Addresses are location-transparent. You send to an actor's address; whether that actor lives on your core, another core, or another continent is not visible in the sending code. This is the model's boldest promise and — as the distributed-systems lens will argue — its most dangerous one. Hold the skepticism; the mechanics first.



Erlang/OTP: the model taken seriously

Erlang is the deepest industrial realization of the actor model, and studying it repays the effort even if you never deploy a BEAM system, because OTP is a twenty-plus-year distillation of what actor systems need in production. Erlang was developed at Ericsson in the late 1980s for telephone switches — systems that must run for years, be upgraded without stopping, and keep most calls up when parts of the software fail. Joe Armstrong's 2003 PhD thesis, *Making Reliable Distributed Systems in the Presence of Software Errors*, is the definitive statement of the design and the best single document in the actor literature; its title is the thesis: the errors are coming, and reliability must be built in their presence, not on the assumption of their absence.

Processes, not threads. Erlang's actors are called processes, and the name is earned: each has its **own heap**, its own stack, and its own garbage collector. There is no shared mutable memory between processes at the language level — message sends copy the data (with a carve-out for large binaries, which are reference-counted and shared immutably). The consequences are structural. A process's GC pause pauses only that process. A process's crash cannot corrupt another's state, because it cannot *reach* another's state — the isolation that an OS gives processes at megabyte cost, BEAM gives at a few hundred words of initial footprint. Processes are preemptively scheduled by the VM's own schedulers (one per core), so running hundreds of thousands or millions of them is routine, and one process looping forever cannot starve the rest. This is the property list that "lightweight process" ought to mean, and most runtimes that borrow the phrase deliver only part of it.

Links and monitors: failure as an event. Erlang's second foundational move is to make failure *observable* rather than exceptional. Two primitives:

- A **link** is bidirectional: if either linked process dies abnormally, an exit signal propagates to the other, killing it too by default. A process that sets `trap_exit` instead receives the exit signal as an ordinary message `{'EXIT', Pid, Reason}` in its mailbox and can react programmatically.
- A **monitor** is unidirectional and non-lethal: the watcher receives a `{'DOWN', Ref, process, Pid, Reason}` message when the watched process dies, and nothing propagates back.

Links express "we live and die together" — the natural relation among the pieces of one logical task. Monitors express "I want to know" — the natural relation for a request/response caller awaiting a server that might die mid-request. Everything above is built from these two.

Supervision trees. A **supervisor** is a process whose only job is to start child processes, link to them, trap exits, and *restart children according to a declared strategy* when they die. Supervisors supervise workers and other supervisors, so a system forms a tree in which every process has a parent responsible for its failure. The classic OTP restart strategies:

- **one_for_one** — a dead child is restarted alone; siblings are untouched. For children that are independent.

- **one_for_all** — one child's death causes the supervisor to terminate and restart *all* children. For children whose states are interdependent enough that a survivor holding references to a dead sibling is itself corrupt.
- **rest_for_one** — the dead child and every child started *after* it are restarted, in order. For children with a startup dependency chain: each child depends on those started before it.

Each supervisor also declares a **restart intensity**: at most N restarts within T seconds. If a child exceeds that — it is crashing in a loop, so restarting is not fixing it — the supervisor gives up and terminates itself, escalating the failure to *its* supervisor, which applies its own strategy. Failures thus climb the tree until some ancestor's restart genuinely clears the fault, or the root is reached and the node restarts. Recovery is not a code path someone remembered to write; it is a property of the tree's shape, declared in data.

Why "let it crash" actually works. The philosophy is routinely quoted and rarely argued, so here is the argument. When a process encounters a state it was not written to handle — a pattern match fails, an invariant is violated, a dependency returns garbage — there are two options. Defensive programming tries to handle the anomaly in place: catch it, log it, guess a corrective action, continue. But by hypothesis the process is now in a state *its author did not anticipate*; code written from inside that state is guessing, and a wrong guess continues execution with corrupted state, converting a loud, local, immediate failure into a quiet, spreading, delayed one. The Erlang position: the process should die at the first sign of anomaly, and its supervisor should restart it **from its initial, known-good state**. A restart is a transition from an *unknown* state to a *known* one — that is the entire trick, and it is the same reason "turn it off and on again" genuinely works for most equipment.

Three preconditions make the trick sound, and each maps to an Erlang design decision. The failing unit must be *small*, so a crash loses one request's worth of work, not the node's — hence ultra-lightweight processes. The crash must be *contained*, unable to corrupt survivors — hence heap isolation and no shared mutable state. And restart must be *cheap and lead to a good state* — hence supervisors that respawn a process in microseconds from declared initial arguments. Remove any leg and the stool falls: crashing a 4 GB shared-heap JVM to clear one request's bad state is not "let it crash," it is an outage. Armstrong's thesis

adds an empirical observation worth internalizing: a large fraction of production failures are *transient* — triggered by a rare interleaving, a peculiar input, a momentary resource state — and for transient faults a clean retry from a fresh state is not a workaround but a genuine cure. Chapter 5's deadlock discussion and Volume 11's metastable-failure material both echo this: the cheapest path out of a bad state is often not repair but rebirth.

Behaviours. OTP factors the actor patterns that recur — a server processing request/response (`gen_server`), a finite state machine (`gen_statem`), event handling, supervision itself — into **behaviours**: library modules that own the generic machinery (the receive loop, timeouts, system messages, debug tracing, code-change hooks) and call your module back for the domain logic. A `gen_server` implementation supplies `init/1`, `handle_call/3` for synchronous requests, `handle_cast/2` for asynchronous ones, and the behaviour does the rest. The division matters beyond convenience: the hard, subtle code — the code that must interact correctly with supervision, shutdown, and upgrades — is written once, by experts, and *your* code is a set of pure-ish callbacks that are trivial to test. Here is the essence of what a gen_server is, stripped to a hand-rolled receive loop with the same shape:

```erlang
-module(counter).
-export([start_link/0, increment/1, value/1]).

%% --- API (runs in the caller's process) ---

start_link() ->
    {ok, spawn_link(fun() -> loop(0) end)}.

increment(Pid) ->
    Pid ! {increment},                    % async: fire-and-forget
cast
    ok.

value(Pid) ->
    Ref = make_ref(),
    Pid ! {value, self(), Ref},           % sync: a call is two sends
    receive
        {reply, Ref, N} -> N
    after 5000 ->
        exit(timeout)
    end.

%% --- Server loop (runs in the counter's process) ---

loop(N) ->
    receive
        {increment} ->
            loop(N + 1);                  % axiom 3: next behavior
holds N+1
        {value, From, Ref} ->
            From ! {reply, Ref, N},       % axiom 1: send a message
            loop(N)
    end.
```

Note the anatomy: the "synchronous call" is two asynchronous sends plus a unique reference to match the reply, with a timeout because in this model *the server might be dead* — and the API functions hide the protocol from callers. That is `gen_server:call/2` in miniature.

Hot code loading, briefly: BEAM can hold two versions of a module simultaneously, and a process switches to the new version at its next fully-qualified call — which is why OTP loops make their recursive call as `?MODULE:loop(State)` and why behaviours define a `code_change` callback to migrate state across versions. Telecom systems upgrade without dropping calls this way. It is a niche capability, but it completes the picture: OTP treats *software change itself* as an event the running system must survive, exactly as it treats crashes.

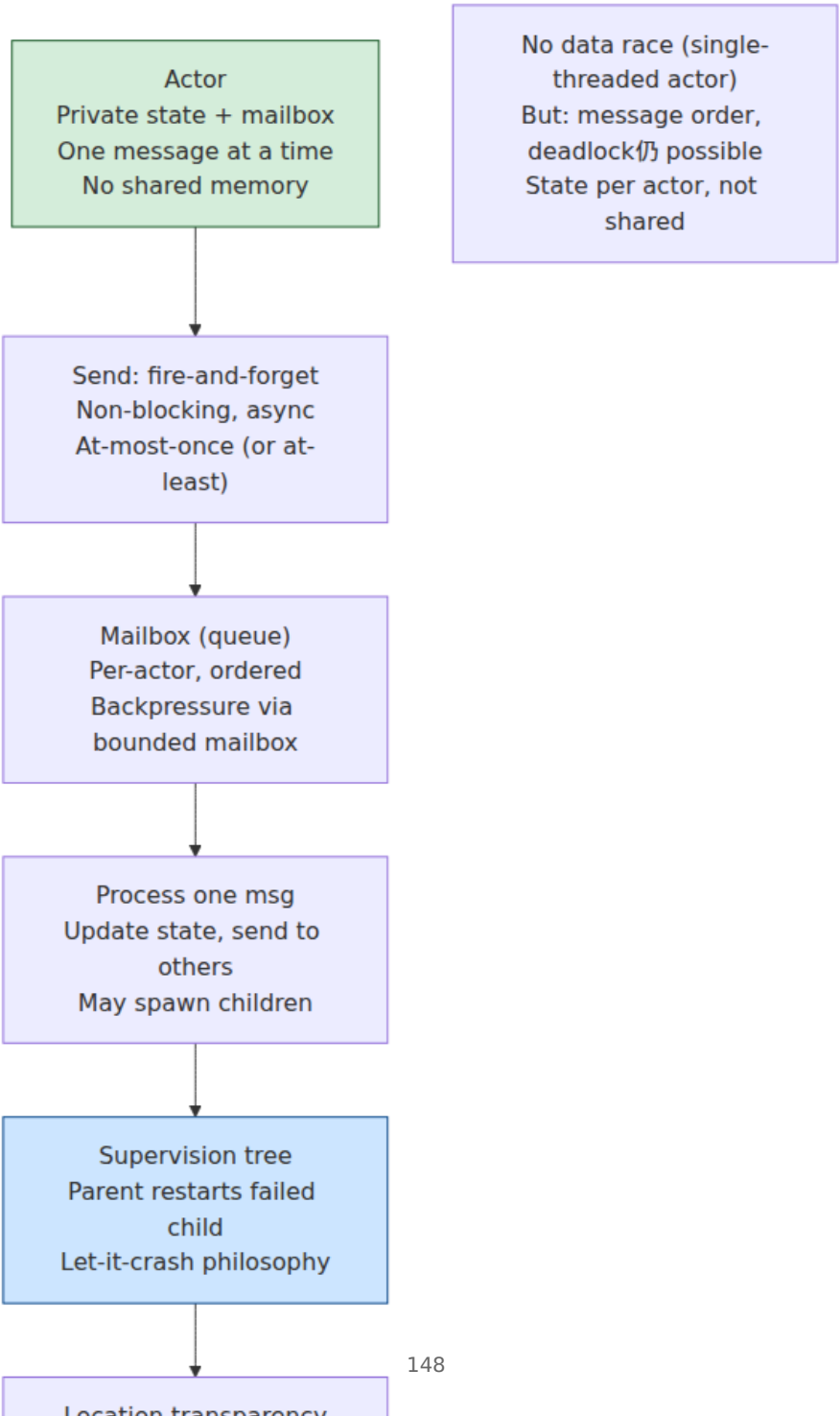
The reliability record, stated honestly. The number that follows Erlang everywhere is "nine nines": Armstrong's thesis reports that Ericsson's AXD301 ATM switch achieved 99.9999999% availability in a trial deployment carrying live traffic in a British telecom network. Treat the figure as what it is — a reported result for one system, over a specific measurement window, achieved by redundant hardware *and* OTP design together, not a property of the language — and it stops being marketing. What generalizes is not the number but the mechanism: systems built as supervision trees of small isolated restartable processes degrade gracefully and recover automatically from the transient faults that dominate production failure, and multiple decades of systems on this platform (telecom switches, RabbitMQ, WhatsApp's messaging servers, ejabberd) constitute real evidence that the mechanism holds up under load.

Akka and actors on shared-memory runtimes

Akka (JVM; and its .NET port Akka.NET, plus post-license-change forks such as Apache Pekko) demonstrates both that the actor model transplants and what is lost in transit. The model is recognizable: actors with mailboxes, `tell` for fire-and-forget, parent supervision, cluster support with location-transparent `ActorRef`s. Akka Typed improved on a long-standing weakness by giving `ActorRef[T]` a message-type parameter, so sending an actor a message it cannot handle is a compile error rather than a dead letter. Actors are multiplexed onto a small thread pool by **dispatchers**, so the lightweight-process economics broadly hold.

The caveat is the runtime underneath. The JVM is a shared-memory machine, and Akka's isolation is a *convention*, not a guarantee. Messages are passed by reference, not copied. Send a mutable object and keep a reference to it, and two actors now share mutable state — every hazard of Chapters 2 and 3 walks back in through a door the model claims not to have, and it is worse than plain shared-memory code because *nobody is locking*, since the programming model promised locks were unnecessary. The same accident happens by closing over the actor's own mutable state in a `Future` callback or a message lambda: the closure executes on some pool thread concurrently with the actor processing its next message. The discipline is well known — messages must be immutable, never close over `this` or mutable fields, use `pipeTo`-style patterns to re-enter the actor — but it is a discipline, enforced by review rather than by the runtime.

Erlang enforces it with per-process heaps and immutable terms; that enforcement, more than any single feature, is the gap between the two. There is no free path to actor safety on a shared-memory runtime: you buy it with copying, with immutability, or with vigilance.


Actor
Private state + mailbox
One message at a time
No shared memory
No data race (single-threaded actor)
But: message order, deadlock仍 possible
State per actor, not shared
Send: fire-and-forget
Non-blocking, async
At-most-once (or at-least)
Mailbox (queue)
Per-actor, ordered
Backpressure via bounded mailbox
Process one msg
Update state, send to others
May spawn children
Supervision tree
Parent restarts failed child
Let-it-crash philosophy
Location transparency

CSP: Communicating Sequential Processes

Hoare, 1978

Tony Hoare's 1978 CACM paper *Communicating Sequential Processes* proposed, with unusual directness, that input, output, and parallel composition be treated as *primitive* programming constructs — as fundamental as assignment and iteration — rather than as library calls layered over shared memory. The paper's model, refined in Hoare's 1985 book into a full process algebra with a formal theory of refinement and deadlock analysis, makes three commitments that jointly distinguish it from actors:

- **Processes are anonymous.** No PIDs, no addresses. A process cannot be sent to; it can only communicate through channels it holds.
- **Channels are the named entities.** Communication topology is a graph of channels wired between processes, fixed by whoever composed them — visible from outside, in the plumbing, rather than buried inside processes as knowledge of each other's addresses.
- **Communication is a synchronous rendezvous.** A send and its matching receive are *one event*. The sender waits until a receiver is ready, and vice versa; then the value passes, and both proceed. There is no buffer in the primitive, hence no mailbox, no queue, and no question of what happens when the queue is full — the "queue" is the blocked sender itself.

The rendezvous is the deepest difference from actors, and its consequence is **backpressure by default**. An actor-model sender outrunning its receiver grows a mailbox; a CSP sender outrunning its receiver *stops*. Flow control is not a feature to add but a property of the primitive. The cost is coupling in time: sender and receiver must both be alive and ready at the same moment, which is exactly the coupling a mailbox exists to remove. Neither choice dominates; they trade availability of the sender against boundedness of the system, a trade we re-litigate at network scale in the lens section.

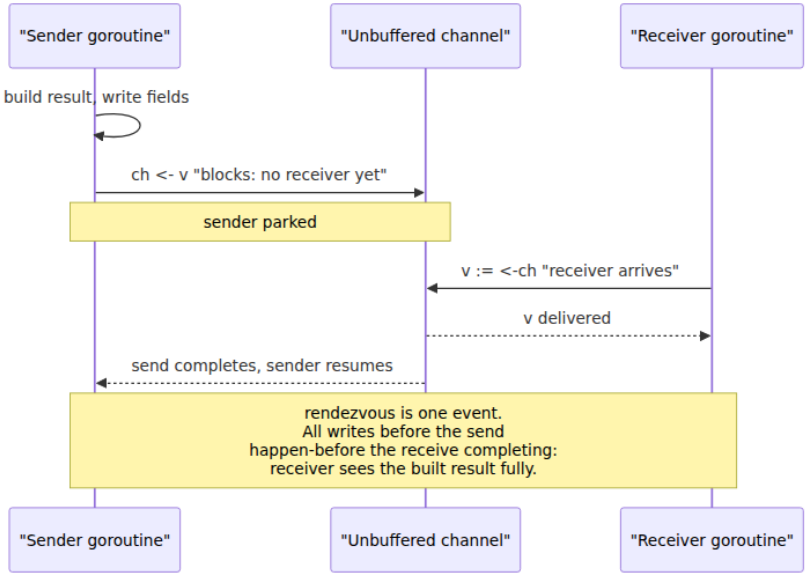
Hoare's model also contributed **alternation** — a construct that waits on several possible communications at once and takes whichever becomes ready — which is the direct ancestor of Go's `select`, and the piece that makes rendezvous programming practical: a process can serve multiple channels without committing to a blocking order.

CSP escaped the paper twice. In the 1980s it became **occam**, the language of the INMOS transputer, where channel rendezvous was implemented in hardware between processors — CSP as an instruction set. And through a lineage of languages Rob Pike worked on at Bell Labs (Newsqueak, Alef, Limbo), it became **Go**, where the channel is a first-class, typed, garbage-collected value — and, in a significant departure from the classical model, channels can carry *channels*, so the communication topology can be rewired at runtime by sending someone the channel to reply on. The `Ref` in our Erlang example and the reply-channel-in-the-message pattern in Go are the same idea wearing different clothes.

Go channels, precisely

Go's channel semantics are specified tightly, and a senior engineer should know them at the level of the specification and *The Go Memory Model*, because the corner cases — nil, closed, buffered-capacity — are exactly where production bugs live.

Unbuffered channels are a rendezvous. `ch := make(chan T)` has capacity zero. A send blocks until a receiver is ready; a receive blocks until a sender is ready; the handoff is their joint event. This is classical CSP, and it carries a memory-model guarantee that ties directly to Chapter 3: **a send on a channel happens-before the completion of the corresponding receive** — everything the sender did before the send is visible to the receiver after the receive. For unbuffered channels the guarantee is even symmetric: the receive happens-before the completion of the send, so the *sender*, once its send returns, also sees everything the receiver did before its receive began. The channel is not merely a data conduit; it is a synchronization edge, which is why transferring ownership of a pointer through a channel is safe: the happens-before edge publishes the pointee's state along with the pointer.



Buffered channels decouple, up to capacity. `make(chan T, C)` holds up to C elements; sends block only when the buffer is full, receives only when it is empty. The memory-model rule generalizes with a precision worth quoting: **the k-th receive on a channel with capacity C happens-before the (k+C)-th send completes.** Set C = 0 and you recover the rendezvous. The rule is exactly what makes a buffered channel a correct counting semaphore: with capacity 3, the 4th send cannot complete until the 1st receive has — at most 3 senders are ever "inside." A buffered channel is therefore two tools in one: a FIFO queue for values and a bounded permit-counter for control, and the semaphore pattern below uses only the second half.

`select` chooses among ready cases uniformly pseudo-randomly. A `select` blocks until at least one case can proceed; if *multiple* are ready, the specification says one is chosen "via a uniform pseudo-random selection." The randomness is a deliberate fairness device: if `select` favored, say, the first listed case, a busy channel in that position would starve the others forever, and — worse — the starvation would be invisible in tests and deterministic in exactly the way that makes bugs reproduce never. Randomization converts "case B is never served" into "case B is served about half the time," at the cost of a guarantee people persistently assume they have: **`select` provides no priority**. The

common attempt to prioritize cancellation by listing `<-ctx.Done()` first does nothing; the idiomatic fix is a nested re-check (`select` on the done channel alone, with `default`, before or after the main select) when priority genuinely matters.

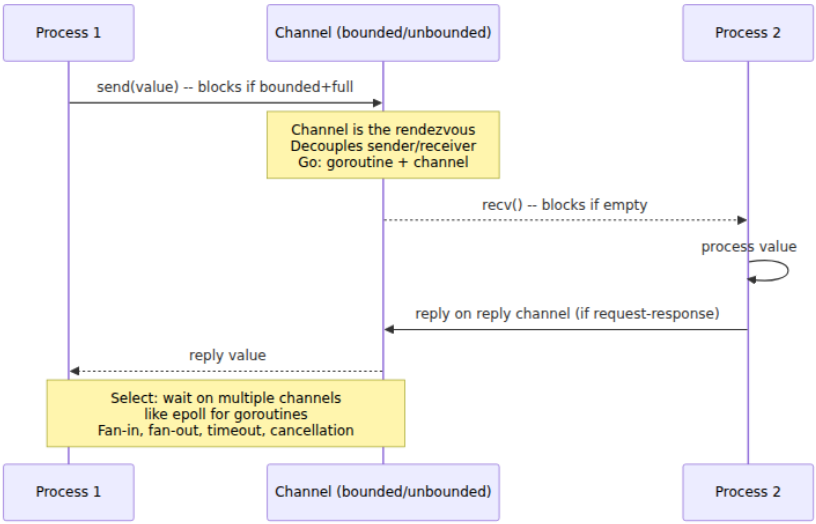
Nil and closed channels have exact, asymmetric semantics. Learn this table; every line is a production incident somewhere:

Operation	Nil channel	Open channel	Closed channel
Send `ch <- v`	Blocks forever	Blocks until receiver/space	**Panics**
Receive `<- ch`	Blocks forever	Blocks until value/close	Returns zero value immediately, `ok = false`
`close(ch)`	Panics	Closes it	Panics (double close)

The design is coherent once you see the intent. Close is a *broadcast of completion* from sender to receivers: after close, every pending and future receive completes — draining any buffered values first, then yielding zero values with `ok == false` — which is why `for v := range ch` terminates on close and why a done-channel (below) works: closing it releases *every* waiter at once, the only broadcast primitive channels have. From that intent the rules follow: only senders know when sending is finished, so **only the sender side may close**; a send on a closed channel is a protocol violation with no sane meaning, so it panics loudly rather than silently discarding data. And the nil channel's block-forever behavior, which looks like a footgun, is a load-bearing idiom: **a nil channel in a `select` disables that case**, since a case that can never proceed is simply never chosen. Setting a channel variable to nil after it closes is the standard way to take a finished input out of a multi-source merge loop without restructuring the select — we use it in the fan-in below.

Directional channel types round out the toolkit: `chan<- T` is send-only, `<-chan T` receive-only, and a bidirectional channel converts implicitly to either. The conversion is one-way — you cannot recover a bidirectional channel from a directional one — so function signatures like `func producer(out chan<- int)` and `func consumer(in <-chan int)` are compiler-checked statements of protocol role: the producer *cannot*

receive from its own output or, usefully, be closed by the wrong side, because `close` is disallowed on a receive-only channel. Cheap, local, static protocol enforcement; use it everywhere.

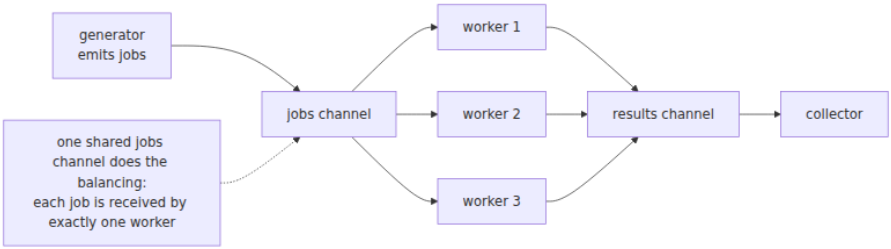


Channel patterns

A small vocabulary of shapes covers most real Go concurrency. The canonical exposition is the Go blog's *Go Concurrency Patterns: Pipelines and cancellation*; here is the distillation, with the reasoning attached.

Pipeline. Stages connected by channels: each stage receives from an inbound channel, transforms, sends outbound, and closes its outbound channel when its inbound is drained — the close propagating downstream as each `range` loop ends. Unbuffered links give a pipeline lockstep flow control; a small buffer between stages smooths bursty stages without unbounding anything.

Fan-out, fan-in. Fan-out: N goroutines receive from *one shared channel*, and the channel's own semantics distribute work — each value is received exactly once, by exactly one of them. No dispatcher, no index arithmetic, no work-stealing machinery; the channel is the load balancer, and slow workers naturally take fewer items. Fan-in: one goroutine (or a `WaitGroup`-closed merge) multiplexes several channels back into one. This is also the honest **worker pool**: fan-out over a jobs channel, fan-in of results, bounded concurrency equal to the number of workers.



Cancellation: the done channel and `context`. A pipeline must be able to stop early — the consumer errored, the request timed out — and here the CSP coupling bites: a producer blocked on a send to an abandoned channel blocks *forever*, leaking the goroutine and everything it holds. The remedy is a second channel carrying no data, only the event of its own closing: every send and receive is wrapped in a `select` that also watches `done`, so closing `done` unblocks every participant at once. `context.Context` is this pattern institutionalized — `ctx.Done()` returns exactly such a channel, adds deadline propagation and a cancellation *tree* mirroring the call tree — and Chapter 8 treats it properly as the backbone of structured concurrency. The rule to carry out of this chapter is narrower and absolute: **any goroutine that sends or receives on a channel whose other side might abandon it must select on a done channel too.** The following program is the whole pattern vocabulary in one runnable piece:

```go
package main

import (
    "context"
    "fmt"
    "sync"
    "time"
)

// generate emits the integers [1, n] until cancelled.
func generate(ctx context.Context, n int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out) // close propagates completion downstream
        for i := 1; i <= n; i++ {
            select {
            case out <- i:
            case <-ctx.Done(): // never send without a cancellation
escape
                return
            }
        }
    }()
    return out
}

// square is one pipeline stage; run several for fan-out.
func square(ctx context.Context, in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for v := range in { // range ends when 'in' is closed and
drained
            time.Sleep(10 * time.Millisecond) // simulate real work
            select {
            case out <- v * v:
            case <-ctx.Done():
                return
            }
        }
    }()
    return out
}

// merge fans multiple channels back into one (fan-in).
func merge(ctx context.Context, ins ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    wg.Add(len(ins))
    for _, in := range ins {
```

```
        go func(in <-chan int) {
            defer wg.Done()
            for v := range in {
                select {
                case out <- v:
                case <-ctx.Done():
                    return
                }
            }
        }(in)
    }
    go func() {
        wg.Wait() // close 'out' only after every input is drained
        close(out)
    }()
    return out
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 200*time.M
illisecond)
    defer cancel() // releases the whole pipeline even on early return

    nums := generate(ctx, 100)

    // Fan out: three workers share one inbound channel.
    c1, c2, c3 := square(ctx, nums), square(ctx, nums), square(ctx, num
s)

    sum := 0
    for v := range merge(ctx, c1, c2, c3) {
        sum += v
    }
    fmt.Println("partial sum before timeout:", sum)
}
```

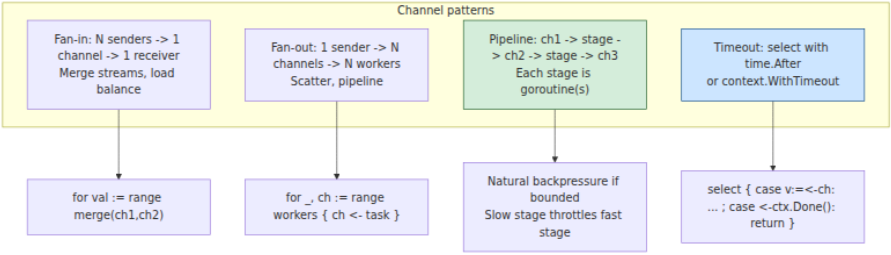
Every send in that program sits inside a `select` with `ctx.Done()`; delete any one of those selects and you have manufactured a goroutine leak that fires only when cancellation races a full channel — precisely the kind of bug Chapter 10's leak detection (`goleak`, blocked-goroutine profiles) exists to catch.

Semaphore via buffered channel. Bounding concurrency without a worker pool: a buffered channel of capacity N where acquiring a permit is a send and releasing is a receive. The capacity-C happens-before rule is what makes this correct, not just plausible:

```
sem := make(chan struct{}, 8) // at most 8 concurrent fetches

var wg sync.WaitGroup
for _, u := range urls {
    wg.Add(1)
    go func(u string) {
        defer wg.Done()
        sem <- struct{}{}        // acquire: blocks when 8 are in
flight
        defer func() { <-sem }() // release
        fetch(u)
    }(u)
}
wg.Wait()
```

(`golang.org/x/sync/semaphore` offers a weighted, context-aware version; the channel form is the idiom to recognize in review.)

or-done and errgroup, conceptually. The *or-done* wrapper takes a channel and a done channel and returns a channel that closes when either does — it factors the select-on-done boilerplate out of consumer loops so they can `range` plainly. `errgroup.Group` (in `golang.org/x/sync`) packages the dominant fan-out termination policy: run N goroutines, wait for all, return the first error, and — via `errgroup.WithContext` — cancel the shared context the moment any member fails, so siblings stop doing doomed work. It is structured concurrency in miniature, and Chapter 8 generalizes it.

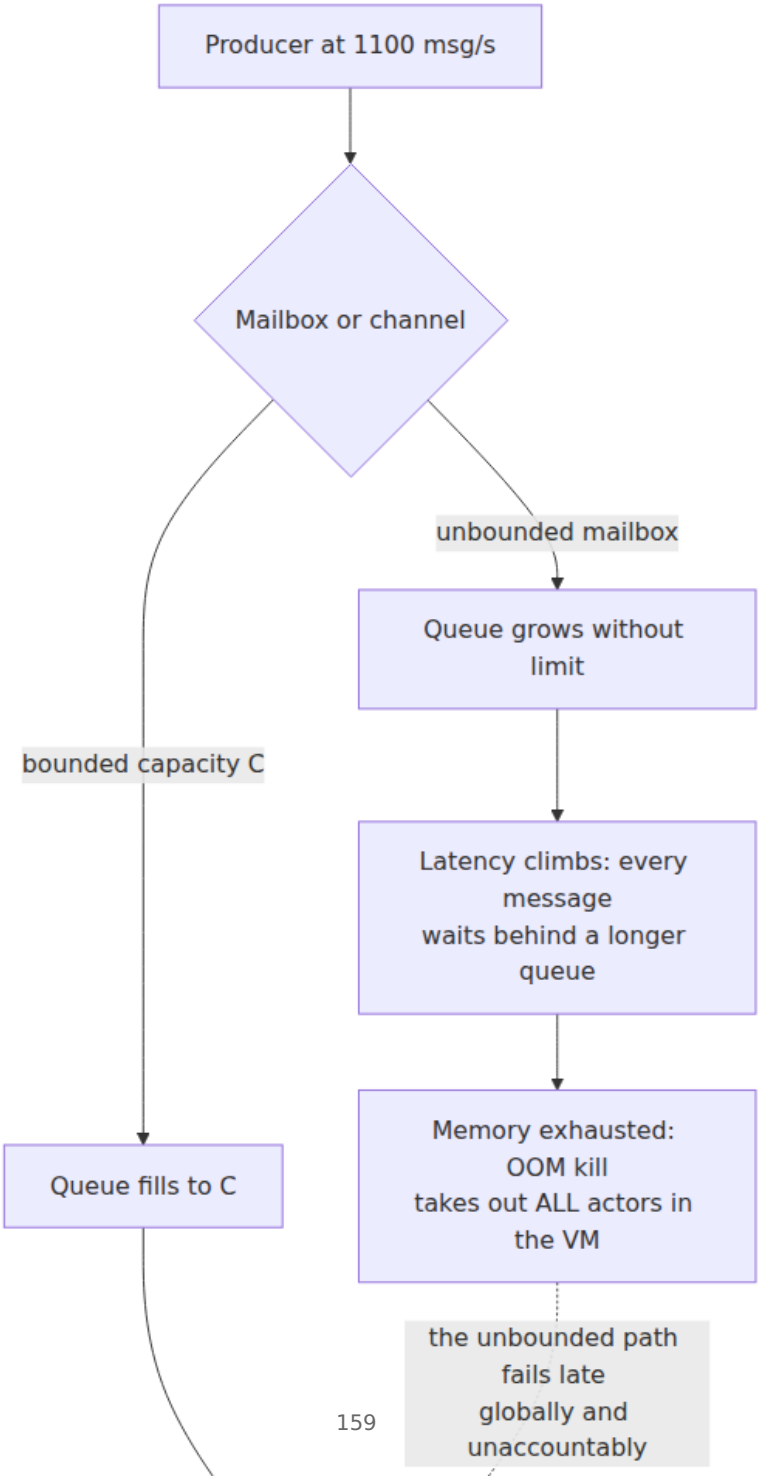


What message passing does not fix

Both models are routinely oversold. Here is the honest ledger; each entry names the migration of a Chapter 1 hazard, not a new species.

Unbounded mailboxes are unbounded memory. The actor model's asynchronous send has a hidden operand: the mailbox that absorbs the

send is a queue, and in Erlang and Akka it is *unbounded by default*. A producer that outruns its consumer by even a few percent grows that queue without limit — and the failure is doubly vicious because it is slow (minutes or hours of creep before the OOM) and self-worsening (in Erlang, a `receive` with selective pattern matching scans the mailbox, so a long mailbox makes the consumer *slower*, which grows the mailbox faster). The remedies are the standard flow-control ones: bound the queue and choose a policy for the full case — block the sender (backpressure, the CSP default recovered), shed load, or drop with a metric — or move to credit/pull-based schemes where consumers grant send permission. This is the in-process shadow of Volume 10, Chapter 7's backpressure story, and the design pressure is identical.


Producer at 1100 msg/s
Mailbox or channel
unbounded mailbox
Queue grows without limit
bounded capacity C
Latency climbs: every message waits behind a longer queue
Memory exhausted: OOM kill takes out ALL actors in the VM
Queue fills to C
the unbounded path fails late globally and unaccountably

Go's buffered channels are always bounded, so Go fails the *other* way:

Goroutine leaks from blocked communication. A goroutine parked on a channel operation that will never complete — sending to a channel nobody will drain, receiving from one nobody will close — leaks its stack, its captured references, and its slot in every resource it holds, forever; the runtime cannot collect a blocked goroutine, because blocked-forever is not distinguishable from blocked-so-far. The classic instances: a pipeline abandoned mid-stream by a consumer that returned early (the cancellation section above); the losing goroutines of a "first response wins" race sending into an unbuffered channel that the winner's receiver has left (fix: buffer of size N, or a done channel); timeouts wrapped around channel receives that abandon the sender. At a few kilobytes each, leaked goroutines can bleed a server for weeks before anyone looks at `runtime.NumGoroutine()`. Chapter 10 covers detection — `goleak` in tests, the goroutine profile's "blocked for N minutes" annotations in production.

Deadlock did not stay behind with the locks. Chapter 5's Coffman conditions apply to any exclusively-held, waited-for resource, and "the other party's readiness to communicate" qualifies. Two goroutines each blocked sending to the other; a gen_server that calls another gen_server which, handling that call, calls back into the first (caller is blocked in its own receive, so the reply never comes — a classic OTP deadline for `gen_server:call` timeouts); a pipeline stage that consumes its own downstream. Cyclic waits are cyclic waits. Go's runtime detects the *global* case — every goroutine asleep — and panics helpfully (`fatal error: all goroutines are asleep - deadlock!`), but a partial deadlock among some goroutines while others serve traffic detects nothing. The mitigations are Chapter 5's, translated: acyclic communication topology (pipelines are DAGs by construction — one reason the pattern is so durable), timeouts on synchronous calls (OTP's default 5-second call timeout is this, institutionalized), and `select` with done channels as the escape hatch.

Ordering is guaranteed less than you assume. What you get: a Go channel is a FIFO queue — values are received in the order sent. Erlang guarantees that messages between one sender and one receiver arrive in send order — **per-pair FIFO**. What you do not get, in either model: any ordering across *pairs*. If A sends to C and then, afterward, B sends to C, C may still receive B's message first — nothing ordered the two senders. If

A sends to B and then to C, nothing constrains which lands first. Two goroutines receiving from one channel may be scheduled such that the later value is *processed* first even though it was received second. Any protocol that needs cross-source ordering must build it — sequence numbers, a single serializing actor, an explicit barrier — and the "must build it" clause is Volume 6's causality material (Lamport clocks, vector clocks) making its first appearance at millimeter scale.

Race conditions survive in full. Chapter 1 was emphatic that data races and race conditions are different bugs, and message passing eliminates only the former. Every check-then-act split across two messages is still a race: read an actor's value with one message, decide, write with another — and another client's write lands between them; the fix is the same as ever, make the compound operation *one message* (compare-and-set as a message type — the actor's serial mailbox then gives you atomicity for free, which is genuinely one of the model's best gifts). Two services double-reserving inventory via polite non-overlapping messages race exactly as two threads do. TOCTOU, lost updates, ordering-dependent outcomes: all present, now spread across mailboxes where no ThreadSanitizer can see them. Message passing moves the interleaving problem up a level of abstraction — which is a real improvement, because protocol-level interleavings are coarser and more enumerable than instruction-level ones — but "fewer, more visible races" is the honest claim, not "no races."

Actors and CSP, head to head

Dimension	Actors (Erlang/OTP, Akka)	CSP (Go channels)
Named entity	Process (identity, address)	Channel (process is anonymous)
Send semantics	Asynchronous, mailbox-buffered	Synchronous rendezvous (or bounded buffer)
Backpressure	Opt-in; unbounded mailbox is the default trap	Default; blocking send *is* flow control
Topology	Implicit, inside actors' address knowledge	Explicit, in the channel wiring

Dimension	Actors (Erlang/OTP, Akka)	CSP (Go channels)
Selective wait	Selective receive on message patterns	`select` over channel cases
Failure handling	Links, monitors, supervision trees — first-class	None; explicit `error` values, `errgroup`, panics kill the process
Distribution	Native; the same send crosses machines	In-process; crossing machines means changing tools
Enforcement of isolation	Erlang: by the runtime. Akka: by convention	By convention (sharing via closures/pointers is possible)

Two rows deserve emphasis. **Distribution** is the actor model's structural advantage: because sends were *always* asynchronous, failable-in-principle, and addressed to an opaque identity, the same primitive extends across a network — Erlang's `Pid ! Msg` is the same expression whether `Pid` is local or on another node, and OTP's distribution layer has worked this way for decades. A Go channel, by contrast, is a memory object; there is no "remote channel," and moving a CSP design across machines means re-plumbing onto gRPC streams or a message broker — at which point you have rebuilt asynchronous addressed messaging, i.e., actors. **Supervision** is the other asymmetry: OTP has a complete, declarative failure-recovery architecture, while Go offers `if err != nil`, `recover` at goroutine top-level (an unrecovered panic in any goroutine kills the whole process — there is no blast-radius containment), and conventions like errgroup. That is not an oversight so much as a difference in ambition — Go builds servers whose orchestrator (systemd, Kubernetes) is the supervisor — but within the process, the Erlang story is simply richer.

Choosing, from the workload — extending Chapter 1's table rather than replacing it. Reach for **actors** when the domain decomposes into many long-lived stateful identities — sessions, devices, accounts, game entities, connection handlers — because per-entity serial mailboxes give you race-free state machines, and supervision gives each entity an independent failure domain; and reach for them decisively when the system must span nodes or survive partial failure as a design requirement. Reach for **CSP channels** when the problem is *dataflow* —

pipelines, fan-out/fan-in, bounded work distribution, in-process coordination — where explicit topology and default backpressure are exactly the properties you want, and process identity is noise. And reach past both, back to **shared memory**, when the data is genuinely shared, large, and hot — a big in-process cache or index that every request reads — because copying it through messages or serializing all access through one owner's mailbox makes the single owner a USL-style serialization point (Chapter 1's α term) that a sharded RWMutex or a lock-free structure would not be. The models compose in one program, and well-built Go services do compose them: channels for lifecycle and work distribution, a mutex around the hot map, an actor-shaped "owner goroutine" for the piece of state with the nastiest invariants.

The distributed-systems lens

Every chapter in this volume ends by walking its topic across the network. This chapter barely has to walk: **a distributed system is a message-passing concurrent system whether you like it or not**. There is no shared memory between nodes; there are only one-way, failable, reorderable messages between named endpoints. That is the actor model's substrate, described exactly. A service endpoint is an actor mailbox — an addressed queue draining into sequential-ish handlers. A message queue is a channel with persistence and weaker delivery guarantees (Volume 10, Chapter 1 makes this correspondence precise). The request/response-with-timeout you write against any RPC framework is `gen_server:call` — two one-way sends, a correlation ID, a deadline — reinvented with worse defaults. When Chapter 1 said the models that eliminate shared state "extend across the network essentially unchanged," this is the mechanism: they were built on the network's actual primitive, so there is nothing to translate.

Erlang, seen this way, is a preview that arrived decades early. A supervision tree restarting crashed processes from known-good initial state, with escalation when restarts exceed a rate limit, is a Kubernetes controller restarting crashed pods from a declared spec, with `CrashLoopBackOff` as the restart-intensity policy (Volume 12). "Let it crash" is the philosophy behind every orchestrator's decision to kill and reschedule rather than heal in place. Microservices-with-an-orchestrator is a supervision tree whose processes got heavier and whose supervisor moved out of process; the reliability argument — small isolated

restartable units beat defensive in-place recovery — transferred intact, because it never depended on Erlang, only on isolation, cheap restart, and known-good initial state.

But the network also sharpens every honest caveat from this chapter, and this is where **location transparency** must be read critically. The promise — local and remote sends look identical — is genuinely valuable for topology flexibility, and genuinely dangerous as a semantic claim, for the reasons catalogued in the *fallacies of distributed computing* (the list compiled at Sun by Peter Deutsch and colleagues, with Gosling's addition: the network is reliable, latency is zero, bandwidth is infinite, the network is secure, topology doesn't change, there is one administrator, transport cost is zero, the network is homogeneous). A local send costs nanoseconds and fails only if the process is dead; a remote send costs a network round trip's worth of tail latency and can fail *silently* — and, crucially, **indistinguishably**: a timeout does not tell you whether the message was lost before processing, lost after, or processed slowly (Volume 6, Chapter 1). Erlang is more honest than its imitators here — monitors deliver explicit `DOWN` / `noconnection` signals, and the literature never claimed remote sends were reliable, only that they were *addressable* the same way — but any abstraction that makes the remote look local invites code that treats it as local, and that code is wrong in the tails. Retrying the ambiguous timeout means possible duplication; hence exactly-once delivery is unachievable as a transport guarantee, and the real contract is at-least-once plus **idempotent handlers** — deduplication by message ID, natural idempotency, or version preconditions (Volume 6, Chapter 9). The actor axioms quietly assumed reliable delivery for the local case; across the network, that assumption is the first casualty, and your protocol design inherits the obligation.

Finally, the flow-control story completes across the network unchanged. An overloaded service with an unbounded accept queue is an actor with an unbounded mailbox, and it dies the same death — slow latency creep, then collapse — at bigger scale; every production-grade RPC stack's answer (bounded queues, load shedding, deadline propagation, HTTP/2 flow-control windows, broker consumer credits) is one of the bounded-mailbox policies from the flowchart above, applied between machines. Backpressure, like happens-before in Chapter 3, is one concept you learn once and bill twice.

Key takeaways

- **Message passing eliminates data races by construction, not race conditions.** State is private; interleaving moves to the message layer, where races are coarser and more visible but fully alive. Check-then-act across two messages is still check-then-act.
- **The actor axioms are: on receiving a message, an actor may send messages, create actors, and designate its next behavior.** The third axiom serializes state transitions through the mailbox — an actor is a small single-threaded server, which is why its internals need no synchronization.
- **Erlang/OTP is the model taken seriously**: per-process heaps enforce isolation, links and monitors make failure an observable event, and supervision trees with declared restart strategies (one_for_one, one_for_all, rest_for_one, plus restart intensity and escalation) make recovery a property of system shape.
- **"Let it crash" works because a restart is a transition from unknown state to known-good state** — and it is sound only when failing units are small, isolated, and cheap to restart. On shared-memory runtimes (Akka), isolation is a convention: immutable messages or nothing.
- **CSP names the channels, not the processes, and communicates by synchronous rendezvous** — so backpressure is the default, not a feature. Actors buy sender availability with unbounded queues; CSP buys boundedness with temporal coupling.
- **Know the Go channel rules cold**: send happens-before receive completes; the k-th receive happens-before the (k+C)-th send on capacity C (which is why a buffered channel is a correct semaphore); `select` picks among ready cases uniformly at random (no priority); send on closed panics; receive on closed yields zero immediately; nil blocks forever — and a nil channel in `select` deliberately disables a case.
- **Every channel send or receive whose counterparty might abandon it must select on a done/context channel.** This one rule prevents the goroutine-leak class; Chapter 10 shows how to detect the survivors.
- **The failure modes are migrations, not novelties**: unbounded mailboxes are memory exhaustion (bound the queue, pick a full-

queue policy — Vol 10 Ch7); cyclic channel or call waits are deadlock (Chapter 5's conditions apply verbatim); ordering is FIFO per-channel or per-sender-pair only — nothing is ordered across sources.

- **Actors distribute; channels don't.** Asynchronous addressed sends are the network's native primitive, so actor designs cross machines unchanged, while CSP designs get re-plumbed onto RPC or brokers. But location transparency is leaky — remote sends have latency, ambiguous failure, and duplication, so idempotency is mandatory (Vol 6 Ch9) — and supervision trees anticipated orchestrator restart policies (Vol 12) by decades.

Further reading

- Hewitt, C., Bishop, P., and Steiger, R., "A Universal Modular ACTOR Formalism for Artificial Intelligence," *IJCAI*, 1973 — the origin of the actor model.
- Agha, G., *Actors: A Model of Concurrent Computation in Distributed Systems* (MIT Press, 1986) — the rigorous semantics; the standard theoretical reference.
- Hoare, C. A. R., "Communicating Sequential Processes," *Communications of the ACM* 21(8), 1978 — the paper. https://dl.acm.org/doi/10.1145/359576.359585 The 1985 book of the same name develops the full process algebra and is freely available at http://www.usingcsp.com/.
- Armstrong, J., *Making Reliable Distributed Systems in the Presence of Software Errors* (PhD thesis, KTH, 2003) — supervision, "let it crash," and the AXD301 story from the source; the single best document on building reliable actor systems.
- Armstrong, J., *Programming Erlang: Software for a Concurrent World*, 2nd ed. (Pragmatic Bookshelf, 2013) — the practical companion to the thesis.
- Erlang/OTP documentation, *OTP Design Principles* — supervision trees, behaviours, and gen_server as actually specified. https://www.erlang.org/doc/system/design_principles.html
- The Go Programming Language Specification — channel types, send/receive/close semantics, and `select`'s uniform pseudo-random choice. https://go.dev/ref/spec

- *The Go Memory Model* — the happens-before guarantees for unbuffered and buffered channels stated precisely. https://go.dev/ref/mem
- "Go Concurrency Patterns: Pipelines and cancellation" (Go blog, 2014) — the canonical treatment of pipeline, fan-out/fan-in, and done-channel cancellation. https://go.dev/blog/pipelines
- Cox-Buday, K., *Concurrency in Go* (O'Reilly, 2017) — or-done, fan-in, and the pattern vocabulary developed at book length.
- Deutsch, P. et al., "The Eight Fallacies of Distributed Computing" — the checklist against which every location-transparency claim should be tested.
- Vernon, V., *Reactive Messaging Patterns with the Actor Model* (Addison-Wesley, 2015) — actor patterns on the JVM, including the discipline needed on a shared-memory runtime.
- Chapter 5 — Deadlock — the Coffman conditions that channel cycles satisfy; Chapter 8 — Coroutines and Structured Concurrency — where `context` and errgroup get their full treatment; Chapter 10 — Testing Concurrency — goroutine-leak detection.
- Volume 10, Chapter 1 — messaging systems as durable channels; Volume 10, Chapter 7 — backpressure across services; Volume 6, Chapter 9 — delivery guarantees and idempotency.

Chapter 8 — Coroutines and Structured Concurrency

What this chapter covers. Chapter 6 showed how an event loop turns a thread's blocking waits into a multiplexed stream of readiness events, and paid the price in inverted, callback-shaped control flow. This chapter covers the two ideas that fixed that. The first is the *coroutine* — a task that can suspend mid-execution and resume later, cheap enough to have a million of. We treat the mechanics precisely: stackful versus stackless suspension, what an `async/await` compiler actually generates, why the "function coloring" problem exists and how Go, Java, Rust, and Kotlin each answer it, and how M:N runtime schedulers place these tasks onto OS threads. The second idea is the discipline that makes a million tasks manageable: **structured concurrency**, the rule that every task is owned

by a lexical scope that cannot exit until the task completes, that errors propagate to the owner, and that cancelling the owner cancels everything it owns. The claim — made explicitly by its originators — is that unrestricted task spawning is to concurrency what `goto` was to sequential control flow, and that abolishing it changes what you can rely on everywhere else. We compare the real implementations (Trio, Kotlin, Java, Go's conventional version), work through cancellation and error propagation carefully, and close with the distributed-systems lens: a request fanning out across services is a distributed task tree, and deadline propagation, cross-wire cancellation, and orphaned work are this chapter's ideas wearing network protocols.

Learning goals — after this chapter you should be able to:

- Explain the difference between stackful and stackless coroutines at the level of mechanism — what is saved at suspension, where it lives, and where suspension is permitted — and state the cost/capability trade-off each makes.
- Sketch the state machine an `async/await` compiler generates from a suspending function.
- State the function-coloring problem and classify the mainstream ecosystems by how they answer it: dissolve it (Go, Java virtual threads), accept it (Rust, JavaScript, Python), or hide it (Kotlin).
- Describe M:N scheduling and work stealing conceptually, and explain why purely cooperative scheduling needs a preemption story — including what Go actually did about it.
- Make the structured-concurrency argument from first principles: why `go` / `Thread.start` /detached tasks break local reasoning, and what the scope-owns-tasks invariant restores.
- Use nurseries, `coroutineScope`, `errgroup` + `context`, or `StructuredTaskScope` to run concurrent subtasks with correct error propagation and cancellation.
- Implement cooperative cancellation correctly: checkpoints, shielded cleanup, and deadlines that compose as the minimum over ancestors.
- Extend the task-tree model across service boundaries: deadline propagation over RPC, client-disconnect cancellation, and the orphaned-work problem.

Coroutine mechanics: two ways to suspend

A coroutine is a function that can *suspend* — return control to a scheduler while remembering where it was — and later *resume* from exactly that point. Every async runtime in production is built on one of two implementations of "remember where it was," and almost every visible property of an ecosystem's concurrency — its syntax, its per-task memory cost, its FFI story, its library compatibility rules — follows from which one it chose.

Stackful: a real stack per task

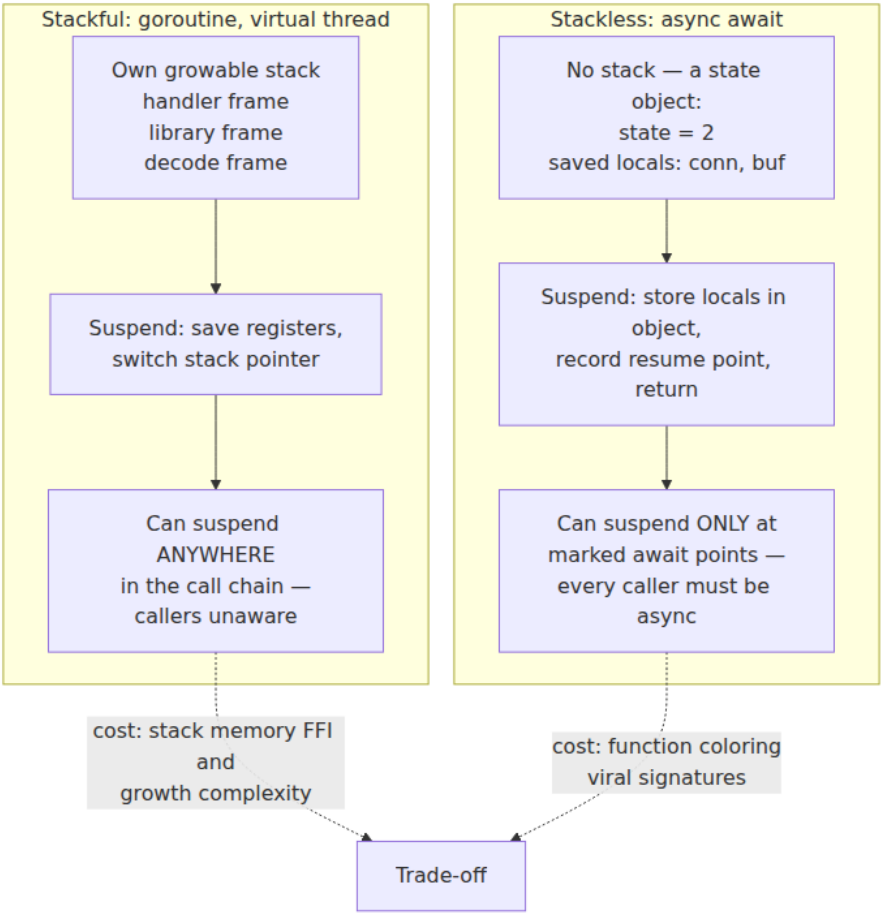
A **stackful** coroutine owns a genuine call stack, allocated by the runtime rather than the OS. Goroutines, Java virtual threads (Project Loom), and Lua coroutines work this way. To suspend, the runtime does what an OS context switch does, minus the kernel: it saves the registers (stack pointer, program counter, callee-saved registers) into the task's control block and switches the stack pointer to another task's stack. Because the entire call chain lives on the coroutine's own stack, suspension can happen *anywhere* — ten frames deep inside a library that has never heard of concurrency. A goroutine blocked on a channel receive inside `json.Decode` inside your handler suspends the whole chain as a unit; nothing in the chain needed to be marked, transformed, or recompiled.

The costs are the stack itself and the machinery to manage it. A fixed large stack per task (the classic 1 MB thread default) would cap you at thousands of tasks, so stackful runtimes make stacks small and growable: Go starts each goroutine at 2 KB and grows the stack by copying it to a larger allocation when a function prologue detects insufficient space — which is why Go pointers into stacks must be precisely tracked by the runtime, and why calling into C via cgo requires switching to a separate C-sized stack that foreign code can treat conventionally. Java virtual threads take a different route to the same end: a virtual thread's frames are ordinary JVM frames that the JVM *unmounts* — copies from the carrier thread's stack into a heap object — at a blocking point, and remounts later, possibly onto a different carrier. Both designs mean per-task memory is proportional to actual call depth, typically a few kilobytes, but both pay ongoing complexity at the boundary with code that assumes conventional stacks: FFI, stack-walking profilers, and (in Loom's case, historically) frames pinned by `synchronized` blocks or native calls that could not be unmounted.

Stackless: the function becomes a state machine

A **stackless** coroutine has no stack of its own. Instead, the *compiler* transforms the function into an object — a state machine — whose fields are the local variables that must survive a suspension, plus an integer recording which suspension point comes next. C++20 coroutines, Rust `async fn`, Python `async def`, JavaScript `async function`, and Kotlin `suspend fun` all work this way. Suspension is only possible at syntactically marked points (`await`, `.await`, `co_await`, or a call to another suspending function), because those are the only places the compiler has arranged to save state. An ordinary function called from a coroutine cannot suspend; if a suspension is needed five frames down, all five frames must be async, each one a state machine holding the one below it as a field — the "stack" is a linked (or in Rust, flattened and inlined) chain of heap or future objects rather than a contiguous memory region.

The payoff is footprint and transparency. A stackless task's memory is exactly the live variables across its suspension points — often well under a hundred bytes — computed at compile time, with no guessing at stack sizes and no growth machinery. Rust leans on this hard: an `async fn` compiles to an anonymous type implementing `Future`, sized precisely, allocated wherever you put it, and the runtime need not exist until you poll it. The cost is that suspension is a property of the function's *type*, visible in every signature from the leaf I/O call up to `main` — which is the door through which the function-coloring problem enters.



The comparison in one table:

Property	**Stackful**	**Stackless**
Where can it suspend	Anywhere in the call chain	Only at marked points in async functions
Per-task memory	KBs, proportional to call depth	Bytes to hundreds of bytes, exact at compile time
Existing blocking libraries	Work unchanged	Must be wrapped or rewritten

Property	Stackful	Stackless
Function signatures	Unchanged	Async-ness is viral through the type system
FFI / native interop	Complicated by movable, growable stacks	Trivial — no special stacks exist
Examples	Go, Java virtual threads, Lua	Rust, C++20, Python, JS, Kotlin, C#

What the compiler generates for `async/await`

`async/await` is syntactic sugar over the state-machine transform — equivalently, over continuation-passing style with the continuations reified as an object. It is worth seeing the desugaring once, concretely, because it demystifies most async "gotchas." Take a small Rust function:

```
async fn fetch_user(id: u64) -> Result<User, Error> {
    let conn = pool.acquire().await?;      // suspension point 1
    let row = conn.query_one(id).await?;   // suspension point 2
    Ok(User::from(row))
}
```

The compiler generates, conceptually, an enum with one variant per region between suspension points, holding exactly the locals live in that region, and a `poll` method that is one big state switch:

```rust
// Conceptual — what rustc generates in spirit, simplified.
enum FetchUser {
    Start { id: u64 },
    AwaitingAcquire { id: u64, fut: AcquireFuture },
    AwaitingQuery { fut: QueryFuture },        // conn moved into fut;
id no longer live
    Done,
}

impl Future for FetchUser {
    type Output = Result<User, Error>;
    fn poll(mut self: Pin<&mut Self>, cx: &mut Context) -> Poll<Self::O
utput> {
        loop {
            match &mut *self {
                FetchUser::Start { id } => {
                    let fut = pool.acquire();
                    *self = FetchUser::AwaitingAcquire { id: *id, fut }
;
                }
                FetchUser::AwaitingAcquire { id, fut } => {
                    match Pin::new(fut).poll(cx) {
                        Poll::Pending => return Poll::Pending,   //
suspend: state saved
                        Poll::Ready(Err(e)) => { *self = FetchUser::Don
e; return Poll::Ready(Err(e.into())); }
                        Poll::Ready(Ok(conn)) => {
                            let fut = conn.into_query_one(*id);
                            *self = FetchUser::AwaitingQuery { fut };
                        }
                    }
                }
                FetchUser::AwaitingQuery { fut } => {
                    match Pin::new(fut).poll(cx) {
                        Poll::Pending => return Poll::Pending,
                        Poll::Ready(Err(e)) => { *self = FetchUser::Don
e; return Poll::Ready(Err(e.into())); }
                        Poll::Ready(Ok(row)) => { *self = FetchUser::Do
ne; return Poll::Ready(Ok(User::from(row))); }
                    }
                }
                FetchUser::Done => panic!("polled after completion"),
            }
        }
    }
}
```

Every mainstream stackless implementation is a variation on this shape. Kotlin's compiler adds a hidden `Continuation` parameter to every

`suspend fun` and compiles the body into a state machine keyed by a label field; C# and JavaScript generate an equivalent `MoveNext`-style method; Python generators (`yield`-based, which is what `async def` compiles down to) keep the frame object alive between `send()` calls, which is the interpreter's version of the same trick. Three practical consequences fall out of the transform. First, `Pending`/suspension is just a *return* — the "blocked" task consumes no thread; it is a small object waiting for someone to call `poll` (or `resume`) again. Second, in poll-based Rust, a future does nothing until polled — async functions are lazy, and forgetting to `.await` a call is a no-op bug the compiler warns about. Third, the size of a task is the size of its worst suspension point: holding a large buffer as a local across an `await` embeds that buffer in every instance of the state machine.

The function-coloring problem

In 2015, Bob Nystrom's essay *What Color Is Your Function?* gave the stackless cost its lasting name. Imagine every function is either red (async) or blue (sync), with rules: red can call blue, but blue cannot call red (or can only call it in a degraded, fire-and-forget way); and you can only *await* red from within red. The colors are viral — making one leaf function red forces redness up the entire call chain — and they bifurcate the ecosystem: every combinator, every interface, every library exists twice or works for only one color. This is not an aesthetic complaint. It shows up as concrete engineering costs: `Iterator` versus `Stream`, sync and async versions of database drivers, trait methods that could not be async (Rust stabilized async fn in traits only in late 2023, and object-safety wrinkles remained), and the perennial hazard of calling a blocking function from async context and stalling an executor thread (Chapter 6's cardinal sin, now hidden behind an innocent-looking signature).

The interesting part is that the industry did not converge on one answer. There are three:

Dissolve the problem — make blocking cheap. Go's position from the start, and Java's position since Project Loom: there is only one color, and it looks blocking. A goroutine or virtual thread that "blocks" on I/O merely suspends a cheap stackful task; the runtime intercepts the blocking operation (Go through its scheduler-integrated netpoller; the JDK by retrofitting its blocking I/O and `java.util.concurrent` internals to unmount virtual threads) and the OS thread moves on. JEP 444 made

virtual threads final in Java 21 (September 2023), and its explicit pitch was Nystrom's argument inverted: keep the thread-per-request style, the debugger, the stack traces, and the entire existing synchronous library ecosystem, and make the thread cheap instead of making the code async. One vintage-specific caveat: in the initial releases, a virtual thread blocking inside a `synchronized` block *pinned* its carrier thread; JEP 491 (JDK 24, March 2025) removed that limitation for the common cases, leaving certain native-frame situations as the remaining pinning source. If you run Java 21–23, monitor pinning (`-Djdk.tracePinnedThreads`) is still an operational concern.

Accept the colors. Rust, JavaScript, and Python keep the distinction explicit and ask the programmer to manage it. Rust's justification is the strongest: it refuses both a runtime and garbage collection, so movable growable stacks are off the table, and in exchange the type system makes async cost-transparent — a task's memory is a `size_of` you can print. JavaScript never had threads to block, so async was the only game. Python has both worlds (threads and `asyncio`) and suffers the bifurcation most visibly: the sync and async halves of its ecosystem are separate library universes.

Hide the colors. Kotlin's `suspend` is technically a color — `suspend fun` compiles to CPS and cannot be called from ordinary functions — but the language works to make it fade: the call syntax is identical (no `await` keyword; calling a suspend function from a suspend function just works), the compiler makes suspension points visible in the gutter rather than the code, and `withContext(Dispatchers.IO)` gives a sanctioned way to wrap blocking calls. The color remains in signatures and in interop, but day-to-day it imposes little of JavaScript's ceremony.

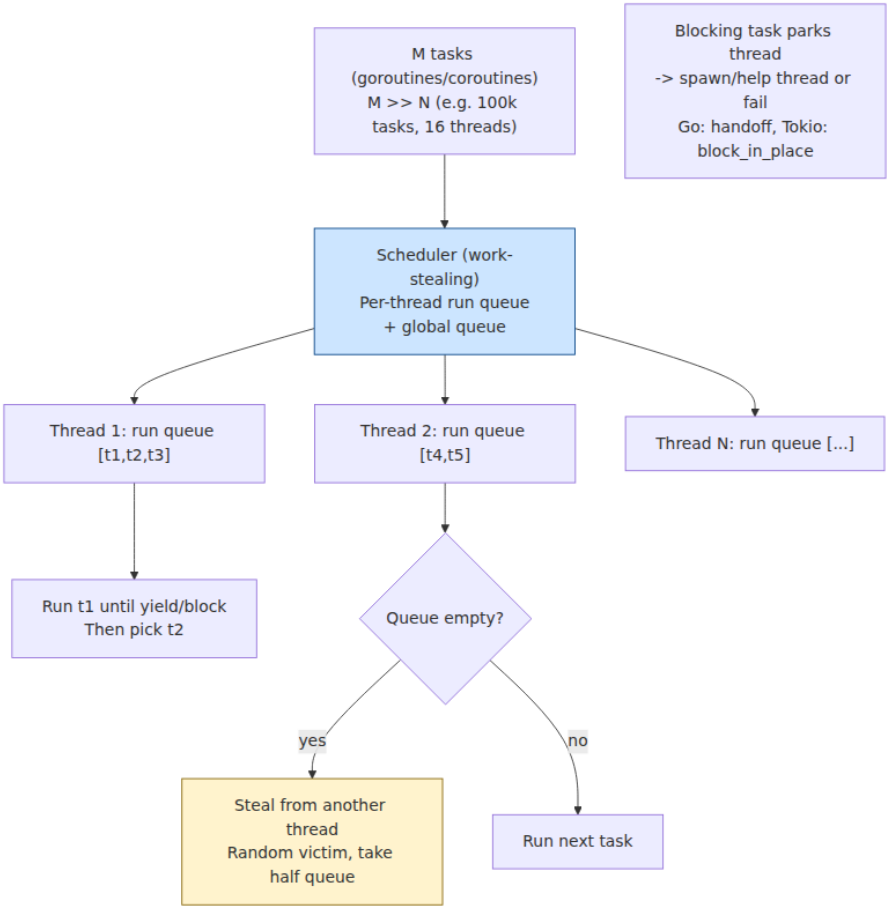
There is no free lunch among the three. Dissolving the color costs runtime complexity and a per-task footprint measured in kilobytes rather than bytes; accepting it costs ecosystem bifurcation; hiding it costs a compiler transform whose seams show at interop boundaries. What you should *not* do is treat the coloring debate as settled folklore in either direction — it is a genuine trade-off, and the right side of it depends on whether your constraint is per-task memory (proxies and routers holding a million idle connections favor stackless) or library compatibility and developer throughput (typical service backends favor the Go/Loom answer).

Runtime schedulers: putting M tasks on N threads

Whichever suspension mechanism a runtime uses, it needs a scheduler: something that maps M runnable tasks onto N OS threads, where M may be six orders of magnitude larger than N. The dominant design is **M:N scheduling with work stealing**. Each OS worker thread keeps a local run queue of tasks; a worker that empties its own queue *steals* a batch from a randomly chosen victim's queue. Local queues keep the hot path free of global contention and preserve cache locality (a task tends to resume on the core that last ran it); stealing repairs imbalance without central coordination. Go's scheduler is the canonical production example — its G-M-P model (goroutines, machine threads, and per-thread processor contexts holding the run queues) is dissected in Volume 13, Chapter 2, and we will not duplicate that here. Tokio, Rust's dominant async runtime, implements the same shape for stackless tasks: a multi-threaded work-stealing executor in which a `poll` returning `Pending` parks the task and frees the worker, and wakers re-enqueue tasks when their I/O completes. The JDK schedules virtual threads on a work-stealing `ForkJoinPool` of carrier threads.

The structural hazard of all these schedulers is that they are **cooperative**: a task yields the worker only at suspension points. A task that computes for 500 ms without suspending — parsing a huge JSON document, a tight numeric loop, an accidental `O(n²)` — holds its worker hostage, and if a handful of such tasks land together they starve every peer on the runtime (the event-loop blocking problem of Chapter 6, reborn at M:N scale). Runtimes answer with varying force. Tokio answers with *budgets*: a task that keeps polling ready resources is forced Pending after a fixed budget of operations, giving the scheduler a chance to run others — but a genuine compute loop that never touches a resource evades this, which is why Tokio provides `spawn_blocking` and `block_in_place` for CPU-bound work. Go's answer evolved instructively. Before Go 1.14, preemption happened only at function-call sites (piggybacking on the stack-growth check in function prologues), so a call-free loop was unpreemptible — the background `sysmon` thread would *mark* any goroutine running longer than about 10 ms for preemption, but the mark took effect only at the next call. Go 1.14 (February 2020) added **asynchronous preemption**: sysmon sends the hogging thread a signal (SIGURG on Unix), whose handler suspends the goroutine at the next safe point regardless of calls. The general lesson survives the details: a

cooperative runtime either grows a preemption mechanism, or it must give you a blocking-pool escape hatch and the discipline to use it — and usually both.



Structured concurrency: the go statement considered harmful

Everything so far makes tasks *cheap*. Cheap tasks make a new problem acute: what governs their *lifetimes*? The primitive every runtime hands you — `go func(...)`, `Thread.start()`, `asyncio.create_task`, `tokio::spawn`, `GlobalScope.launch` — takes a function and starts it

running, detached, with no further relationship to the code that spawned it. Call this *unstructured spawn*.

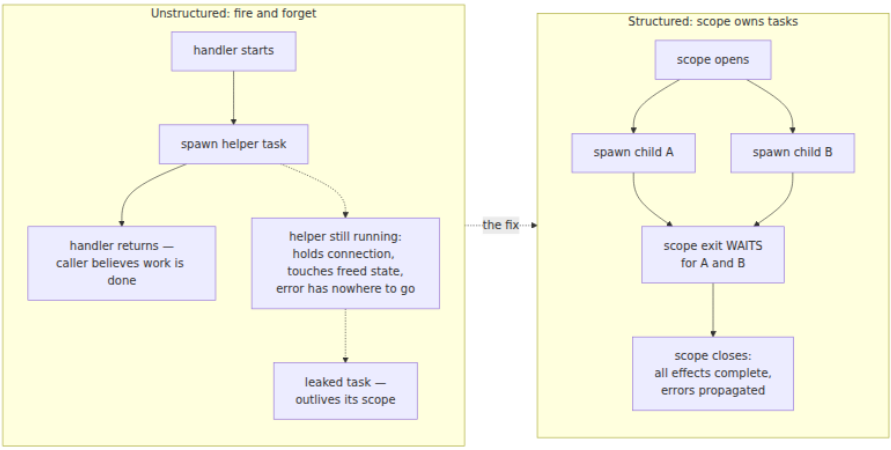
The critique of unstructured spawn has two named sources. Martin Sústrik — of ZeroMQ, and author of the C coroutine libraries libmill and libdill — coined the term **structured concurrency** around 2016 to describe libdill's discipline: coroutine lifetimes must nest, a coroutine never outlives its parent, and cancellation is a first-class, ordinary operation. Nathaniel J. Smith's 2018 essay *Notes on Structured Concurrency, or: Go Statement Considered Harmful* made the argument general and gave it its edge. Smith's observation: every spawn primitive is a directed jump — control flow splits, and one branch leaves the scope entirely — which is exactly the property that made `goto` destructive. Dijkstra's case against `goto` was not aesthetic; it was that `goto` destroys the correspondence between the program text and its execution, so you cannot reason locally. If control can arrive from anywhere and leave to anywhere, then no function boundary means anything: you cannot look at `f(); g();` and know that `f` finished before `g` began. Structured programming won because restricting control flow to nested blocks — sequence, selection, iteration, call/return — made *black-box abstraction* possible: a function's effects are over when it returns.

Unstructured spawn breaks that abstraction in precisely the same way, and the damage is not hypothetical. Four failure classes recur in every codebase that spawns freely:

1. **Tasks outlive their scope.** A handler spawns a helper and returns; the helper is still running, holding a database connection from a pool that will be exhausted by lunchtime, or touching request state that the framework has already recycled. The function returned, but its effects are not over — the black box leaks.
2. **Errors vanish.** The spawned task throws. Into what? There is no caller on its (logical) stack. Runtimes log it, invoke a global "unhandled exception" hook, or — as with a raw Go goroutine panic — take down the process. The one thing that cannot happen is the natural thing: propagation to the code responsible for the work. Result: background failures that page no one and surface weeks later as data gaps.
3. **Cancellation has no path.** The client disconnected; the request's work should stop. Which tasks belong to this request? Nobody

recorded that. Unstructured systems reconstruct the answer with ad-hoc registries and flags, and miss some.

4. **Leaks accumulate.** Each of the above, times uptime. Task leaks are the async era's memory leaks — invisible in code review, cumulative in production, diagnosed at 3 a.m. via a task-dump showing forty thousand goroutines blocked on channels nobody will ever write to (Chapter 10 covers the tooling).

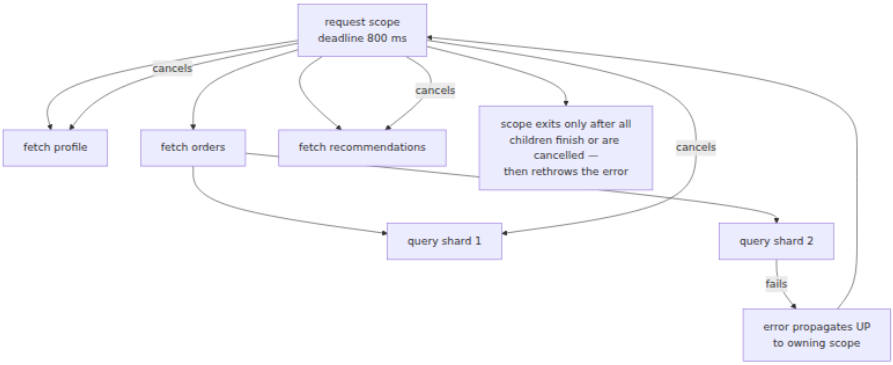


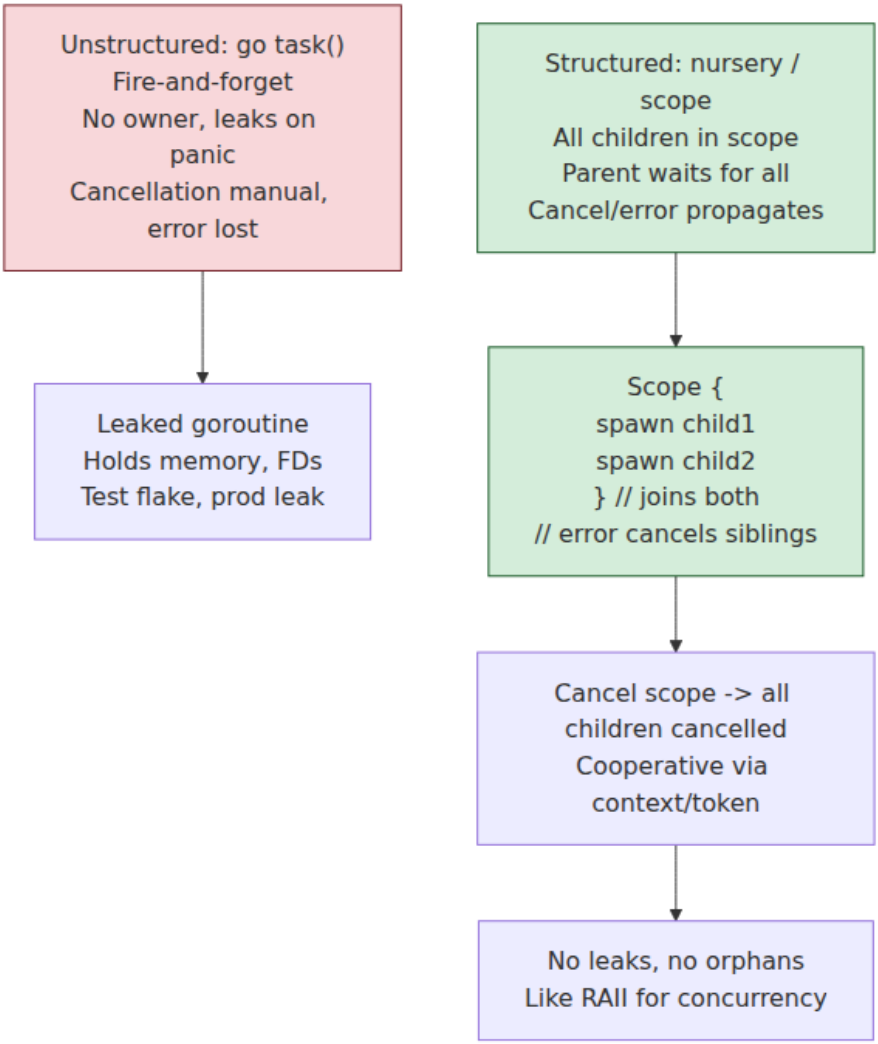
The fix: scopes own tasks

Structured concurrency is one rule with three corollaries. **The rule:** concurrency may only be introduced inside a *scope* — a syntactic block — and every task spawned in the scope is owned by it; the scope cannot be exited until every owned task has completed. **The corollaries:** (a) a child task's error propagates to the scope, where the surrounding code can catch it with ordinary `try` / `catch` or error returns; (b) cancelling the scope cancels all its children, transitively; (c) since scopes nest lexically and tasks are confined to scopes, the tasks of a program form a **tree** at runtime, mirroring the block structure of the source.

That tree is the core invariant, and each edge of it carries three obligations downward and one upward: the parent awaits the child (completion flows up), the parent's cancellation reaches the child (cancellation flows down), the parent's deadline bounds the child (time flows down), and the child's failure reaches the parent (errors flow up). Restore the tree and the four failure classes close by construction: nothing outlives its scope, every error has a propagation path, every task

has a cancellation path from the root, and a leak would require escaping the tree — which the API no longer offers. Just as removing `goto` made "what can control flow do here?" locally answerable, removing unstructured spawn makes "what is running right now, on whose behalf, and until when?" answerable by reading the enclosing scopes. And, exactly as with `goto`, the point is what the restriction *enables*: because the runtime knows the tree, it can give you accurate cross-task stack traces (Kotlin and Trio print the logical parent chain, not the scheduler's call stack), meaningful task dumps, and resource cleanup tied to scope exit.





Realizations: four ecosystems, one tree

Trio nurseries — the pattern at its purest

Nathaniel Smith's Python library **Trio** is where the pattern was first realized as the *only* way to spawn, and it remains the reference implementation of the idea. Trio has no detached spawn. The sole way to start a task is inside a nursery:

```python
import trio
import httpx

async def fetch_all(urls: list[str]) -> dict[str, str]:
    results: dict[str, str] = {}

    async def fetch(client: httpx.AsyncClient, url: str) -> None:
        resp = await client.get(url)
        resp.raise_for_status()
        results[url] = resp.text

    async with httpx.AsyncClient() as client:
        async with trio.open_nursery() as nursery:
            for url in urls:
                nursery.start_soon(fetch, client, url)
        # The nursery block does not exit until every fetch has
finished.
        # If any fetch raises, the nursery cancels the others, waits
for
        # them to actually stop, and re-raises here — an ordinary
exception
        # in ordinary control flow.
    return results

async def main() -> None:
    with trio.move_on_after(2.0):          # cancel scope: a deadline
        data = await fetch_all(["https://example.com/a", "https://
example.com/b"])
        print({k: len(v) for k, v in data.items()})

trio.run(main)
```

Two details deserve attention. First, `fetch_all` is a black box again: when it returns, all its concurrency is over — the caller cannot tell, and need not care, that it was internally concurrent. Second, `move_on_after` shows Trio's other contribution, the **cancel scope**: cancellation and timeouts are properties of *scopes*, not of individual tasks or of call sites, and they apply to everything the scope dynamically contains. Trio's design directly shaped the `asyncio.TaskGroup` added to the standard library in Python 3.11, which brings the nursery-with-error-propagation shape (though not Trio's full cancel-scope semantics) to stock `asyncio`.

Kotlin — structured concurrency as the default

Kotlin coroutines bake the tree into the runtime as the **Job hierarchy**: every coroutine has a `Job`; launching from within a `CoroutineScope`

makes the new job a child of the scope's job; failure of a child cancels the parent, which cancels the siblings; a parent job does not complete until its children do. The scope function `coroutineScope { }` gives fail-fast semantics:

```
import kotlinx.coroutines.*

data class Dashboard(val profile: Profile, val orders: List<Order>)

suspend fun loadDashboard(userId: String): Dashboard = coroutineScope {
    val profile = async { profileService.fetch(userId) }   // child 1
    val orders  = async { orderService.list(userId) }      // child 2
    // If orderService.list throws, the coroutineScope cancels the
    // profile child, waits for it to finish cancelling, and rethrows
    // the original exception to our caller. No leak, no lost error.
    Dashboard(profile.await(), orders.await())
}

suspend fun loadDashboardWithTimeout(userId: String): Dashboard =
    withTimeout(800) {           // deadline applies to the whole
subtree
        loadDashboard(userId)
    }
```

`supervisorScope { }` (and `SupervisorJob`) inverts the error policy: a child's failure is delivered to that child's handler but does *not* cancel siblings — the collect-all/supervision policy discussed below. Kotlin also demonstrates the ecosystem's learning curve: the escape hatch `GlobalScope.launch` — an unstructured spawn — was so reliably a bug (leaked work, swallowed errors) that the library marked it `@DelicateCoroutinesApi` to force an explicit opt-in.

Java — StructuredTaskScope

Loom's second act applies the same discipline to virtual threads. Fair warning on vintage: this API has had a long preview run — incubating as JEP 428 (Java 19) and JEP 437 (20), preview as JEP 453 (21), re-previewed in 22–24, and re-previewed again with a substantially redesigned API in JEP 505 (Java 25, September 2025), which replaced the `new StructuredTaskScope.ShutdownOnFailure()` constructor idiom with a static `StructuredTaskScope.open(...)` accepting pluggable `Joiner` policies. As of this writing (early 2026) it had not yet been finalized; check the current JEP status before committing to the exact API shape. The JEP 453-era form, which conveys the semantics:

```
Response handle(String userId) throws ExecutionException,
InterruptedException {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        Subtask<Profile> profile = scope.fork(() -> profileService.fetc
h(userId));
        Subtask<List<Order>> orders = scope.fork(() -> orderService.lis
t(userId));

        scope.join()            // block (cheaply — we're on a virtual
thread)
             .throwIfFailed();  // first failure cancels the other and
rethrows here

        return new Response(profile.get(), orders.get());
    }   // close() guarantees no subtask outlives this block
}
```

The load-bearing line is the last one: `close()` (via try-with-resources) *cannot return* until every forked subtask has terminated, even on the exceptional path. The lexical block and the task lifetimes coincide, enforced by the API rather than by convention.

Go — errgroup and context: the conventional version

Go is the instructive partial case: the language's spawn primitive is maximally unstructured — the `go` statement is fire-and-forget by design, with panics in a goroutine killing the process and no built-in way to wait or propagate errors — and the community answer is a pair of conventions so entrenched they are effectively language features. **`context.Context`** carries the cancellation-tree state: `context.WithCancel(parent)` and `context.WithTimeout(parent, d)` derive a child context whose `Done()` channel closes when it is cancelled *or when any ancestor is* — cancellation flows down the tree, and a derived deadline can only tighten, never loosen, the parent's. The `select`-on-`ctx.Done()` idiom (Chapter 7's channel machinery) is the cooperative checkpoint. **`golang.org/x/sync/errgroup`** supplies the scope: a `Group` waits for all its goroutines, captures the first error, and — via `errgroup.WithContext` — cancels the shared context on first failure, giving fail-fast sibling cancellation:

```
package dashboard

import (
    "context"
    "time"

    "golang.org/x/sync/errgroup"
)

func Load(ctx context.Context, userID string) (*Dashboard, error) {
    // Deadline for the whole subtree; min() with ancestors is
automatic.
    ctx, cancel := context.WithTimeout(ctx, 800*time.Millisecond)
    defer cancel() // release the timer and the subtree, on every path

    g, ctx := errgroup.WithContext(ctx)

    var profile *Profile
    var orders []Order

    g.Go(func() error {
        p, err := profileSvc.Fetch(ctx, userID) // honors ctx
internally
        if err != nil {
            return err // first error cancels ctx → siblings see Done()
        }
        profile = p
        return nil
    })
    g.Go(func() error {
        o, err := orderSvc.List(ctx, userID)
        if err != nil {
            return err
        }
        orders = o
        return nil
    })

    if err := g.Wait(); err != nil { // the scope: waits for ALL
goroutines
        return nil, err
    }
    return &Dashboard{Profile: profile, Orders: orders}, nil
}
```

This is real structured concurrency in effect — scope, propagation, cancellation, deadline composition — but it is *conventional*, not enforced. Nothing stops a bare `go` inside the closure from escaping the group; nothing makes a library accept a `Context`; a function that drops `ctx` on

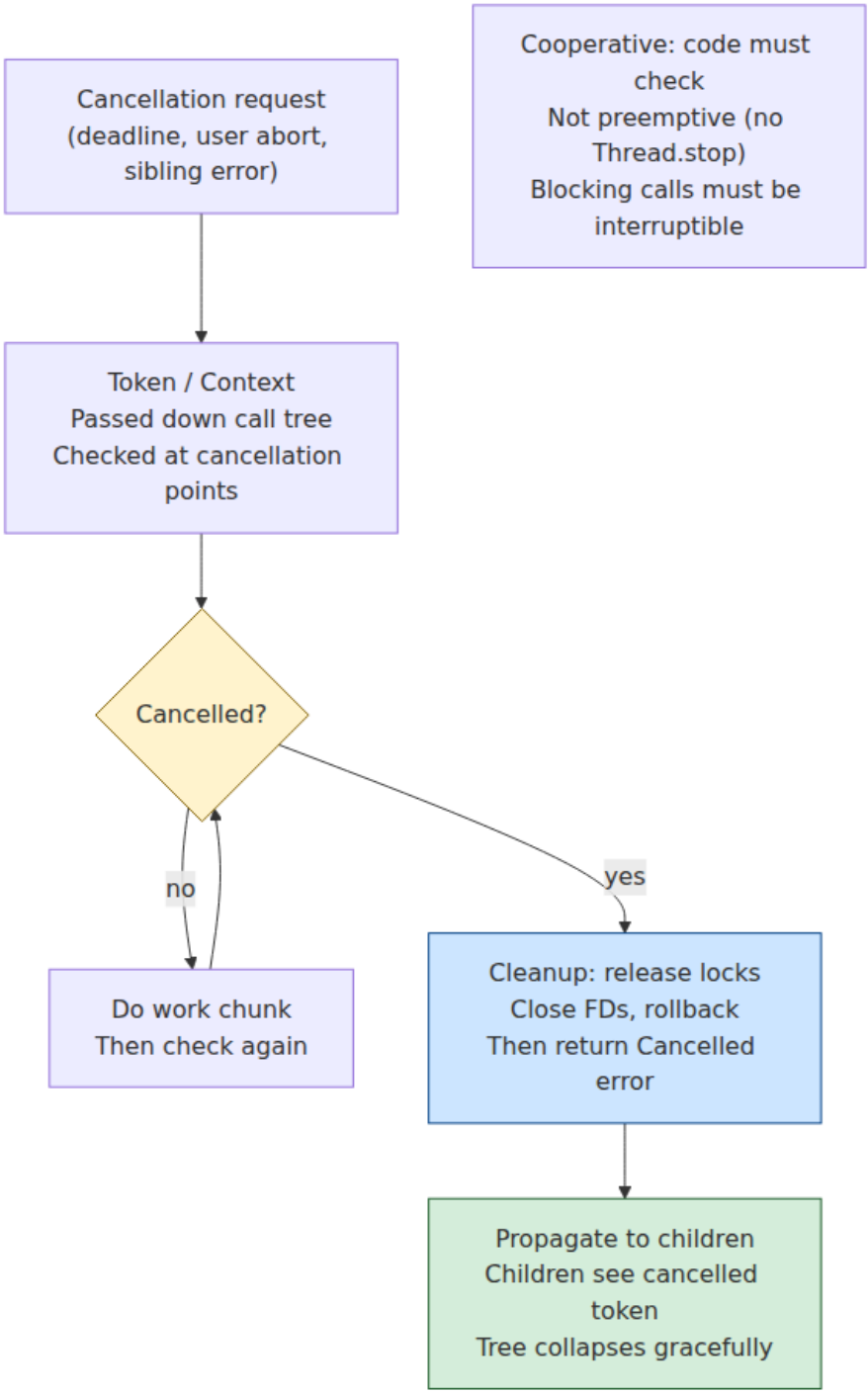
the floor silently severs the cancellation tree below it. The context-plumbing discipline — first parameter, always passed, never stored in a struct — is the tax Go pays for keeping the language small. It mostly works, and its failures are exactly the four classes above, one forgotten plumb at a time.

Cancellation done properly

Cancellation is where structured concurrency earns its complexity budget, and the first principle is negative: **forced termination is unsound**. Java learned this publicly. `Thread.stop()` — which asynchronously flung a `ThreadDeath` error into a thread at an arbitrary instruction — was deprecated in JDK 1.2 (1998) with an unusually direct explanation: a stopped thread releases its monitors *while the data they protect is mid-update*, leaving every invariant those locks guarded (Chapter 2) silently broken for every future observer. The method spent two decades deprecated and was finally defanged in JDK 20 (2023), where it throws `UnsupportedOperationException`. The same argument condemns killing tasks at arbitrary points in any runtime: a task cancelled between "debit account A" and "credit account B," or while holding a mutex, or halfway through writing a file, has corrupted state that outlives it.

The sound alternative is **cooperative cancellation**: cancellation is a *request*, delivered as state the task can observe, and the task exits at a **checkpoint** — a point where its invariants hold. The ecosystems differ only in surface. In Go, the checkpoint is explicit: `select` on `ctx.Done()`, or `if ctx.Err() != nil` in loops, and every well-behaved blocking API takes a `Context` and returns early when it fires. In Kotlin, every suspension point is implicitly a checkpoint: cancellation surfaces as a `CancellationException` thrown from the suspend call, which unwinds normally — running `finally` blocks — and which you must not swallow (catching and discarding it un-cancels the task, a classic bug). Trio is the same, with `Cancelled` raised at checkpoints. Java virtual threads reuse interruption: cancellation of a `StructuredTaskScope` interrupts its subtasks, and blocking JDK calls throw `InterruptedException`. The shared consequence: a compute loop with no checkpoints is uncancellable — the same code that starves a cooperative scheduler also ignores cancellation, one bug in two costumes — so long CPU-bound loops should poll for cancellation explicitly.

Two refinements complete the model. First, **shielded cleanup**: cleanup code often must perform cancellable operations (send a rollback, close a connection gracefully) *after* cancellation has been delivered — and would be instantly re-cancelled without protection. Runtimes provide a scoped shield: Kotlin's `withContext(NonCancellable) { ... }` inside a `finally`, Trio's `CancelScope(shield=True)`, Go's convention of `context.WithoutCancel(ctx)` (added in Go 1.21) or a fresh short-deadline context for cleanup RPCs. Shields must be small and bounded — an unbounded shielded region is an uncancellable task with extra steps. Second, **deadline composition**: in a tree, the effective deadline of any task is the *minimum over its ancestors*, and correct APIs enforce this automatically — `context.WithTimeout` derived from an already-tighter parent keeps the parent's deadline; Kotlin's nested `withTimeout` and Trio's nested cancel scopes fire whichever expires first. This is what makes timeouts *composable*: a library can impose its own internal timeout without ever extending the caller's budget.


Cancellation request (deadline, user abort, sibling error)
Cooperative: code must check
Not preemptive (no Thread.stop)
Blocking calls must be interruptible
Token / Context
Passed down call tree
Checked at cancellation points
Cancelled?
no
yes
Do work chunk
Then check again
Cleanup: release locks
Close FDs, rollback
Then return Cancelled error
Propagate to children
Children see cancelled token
Tree collapses gracefully

Errors across task boundaries

A scope with N children needs a policy for the moment child 3 fails while 1, 2, and 4 are still running. There are two defensible policies, and good APIs let you choose per scope:

- **Fail-fast:** the first failure cancels the remaining siblings, the scope waits for them to terminate, and the original error propagates to the caller. This is the right default for *co-dependent* work — a fan-out gather where a missing part makes the whole useless. It is what `coroutineScope`, Trio nurseries, `errgroup.WithContext`, and `ShutdownOnFailure` (JEP 505: `Joiner.allSuccessfulOrThrow`-style policies) all implement. Note the subtlety that fail-fast is still *wait-then-propagate*: siblings are cancelled and **awaited**, not abandoned — otherwise the scope would itself be a leak factory on the error path.
- **Supervision / collect-all:** children are independent; one failure should not kill the others. The scope runs everything to completion (or per-child restart, in the Erlang tradition — Chapter 7) and reports the outcomes together. Kotlin's `supervisorScope`, Trio patterns that catch per-task, Go groups that collect into an error slice rather than cancelling, and Java joiners that gather all subtask results implement this. Python 3.11's `ExceptionGroup` (and `except*`) exists precisely because a collect-all scope can complete with *several* failures that all deserve to propagate.

Panics and their kin deserve a policy too, because they do not respect task boundaries on their own. A panic in a bare goroutine kills the entire process — `errgroup` (since 2023-era releases) propagates a child's panic to the `Wait` caller rather than letting it escape unrelated; Kotlin delivers a child's unexpected exception through the job hierarchy to the scope (or the `CoroutineExceptionHandler` at the root); a Rust task panic is captured by the runtime and surfaces as a `JoinError` where the task is awaited. The structured rule generalizes: *any* abnormal outcome of a child — error, panic, cancellation — must become an ordinary value or exception at the scope, because the scope is the only place with enough context to decide what it means.

Practical guidance

Bound concurrency inside scopes. A scope makes it trivially easy to spawn one task per item of an unbounded collection — one per URL, per row, per queue message — which is an unbounded-resource bug with pleasant syntax (and a USL lesson from Chapter 1 besides). Bound it at the scope: `errgroup`'s `SetLimit(n)`, a semaphore acquired inside each child (Chapter 2's primitive; Chapter 9's bulkhead pattern), Kotlin's `Semaphore` or a fixed dispatcher, Trio's `CapacityLimiter`. The scope guarantees the tasks end; the limiter guarantees how many exist at once. You need both.

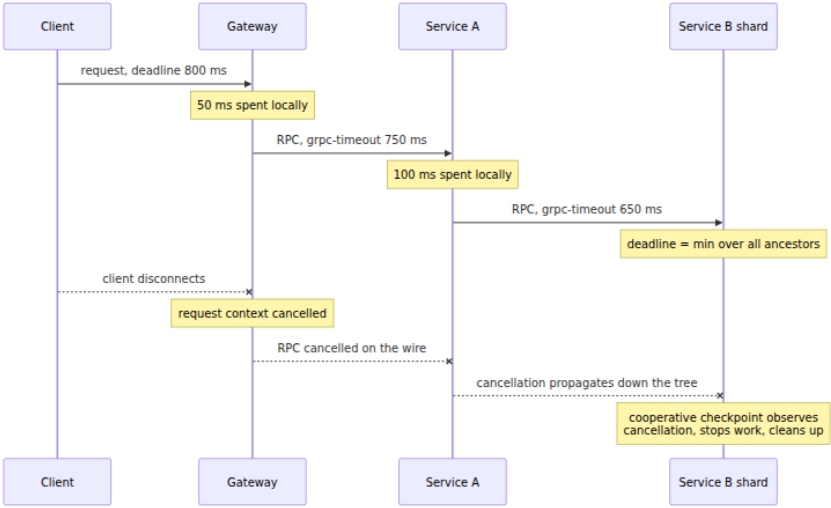
Treat fire-and-forget as a code smell with a named owner. Genuine background work — metrics flushing, cache refresh, connection reaping — does exist, but "background" should mean *owned by a longer-lived scope*, not *owned by nobody*: an application-lifetime scope created in `main`, cancelled and drained on shutdown. The difference is visible exactly when you need it — at shutdown and in task dumps.

Drain on shutdown. Graceful shutdown is structured concurrency applied at the service level: stop admitting new work (close the listener, fail readiness probes), cancel or await the request-scope subtree with a deadline, then run shielded cleanup (flush buffers, checkpoint offsets, close connections), then exit. A process whose tasks all live in one tree rooted in `main` can implement this in a dozen lines; a process full of detached tasks cannot implement it at all — it can only `kill -9` itself politely. Chapter 9 develops the pattern; Volume 11 covers its operational side (SIGTERM handling, termination grace periods, load-balancer drain).

The distributed-systems lens: the tree crosses the wire

Now widen the frame. A user request enters an edge gateway, which calls three services, one of which fans out to four shards and a downstream vendor API. Squint, and this is exactly the task tree of this chapter — except the edges are RPCs and the nodes are processes on different machines. Every structured-concurrency concern reappears, and the in-process discipline is what makes the distributed version implementable.

Deadline propagation is timeout composition over RPC. The client gives the gateway 800 ms. The gateway spends 50 ms and calls service A — which should be told it has *at most* 750 ms, not be left to its configured default of 30 s that lets it labor uselessly for a caller long gone. gRPC builds this in: the client's deadline travels in the `grpc-timeout` request header, each server sees the remaining budget in its handler context, and Go's gRPC integration surfaces it as — precisely — a `context.Context` with the deadline set, so `context.WithTimeout` composes across the wire exactly as it does across function calls: the effective deadline at any node is the minimum over its ancestors, network hops included. (Volume 8, Chapter 3 covers the mechanics; budget your hops so downstream services get useful slices, and subtract expected network time.)



Cancellation crosses the wire — if the tree is intact. When the client disconnects, the gateway's server framework cancels the request context; because the outbound RPCs were made *with that context*, gRPC signals cancellation to service A (HTTP/2 `RST_STREAM`), whose handler context fires, cancelling its calls to the shards. One disconnect drains an entire subtree of work across four machines — but only if every process kept its internal tree connected: one service that launched its downstream call from a fresh `context.Background()` severs the chain, and everything below it becomes orphaned work, burning capacity for a response nobody will read. Under load this is not waste but amplification: timeouts fire, callers retry, and the retries spawn *duplicate* subtrees while the orphans

still run — the retry-storm ingredient of metastable failure (Volume 11), and the reason retried work must be idempotent (Volume 6, Chapter 9): in a distributed tree you cannot guarantee an orphan died before its replacement started.

Sagas are supervision with compensation. A workflow that debits one service and credits another cannot "cancel" the debit once committed — cancellation in the distributed tree has a horizon, past which undo becomes *compensation*: explicitly programmed inverse actions, run in a supervised sequence. The saga pattern (Volume 5, Chapter 10) is recognizably this chapter's structure at workflow timescale — a parent scope that owns steps, observes a child's failure, and runs a policy (compensate completed siblings) instead of a cancel — and workflow engines like Temporal are, in essence, durable structured-concurrency runtimes whose scopes survive process restarts.

Deploys are shutdown at fleet scale. Rolling deploys terminate instances mid-tree constantly. The drain sequence above — stop admitting, cancel with deadline, shielded cleanup — is exactly what a well-behaved pod does between SIGTERM and SIGKILL, and it works only if the instance's work is a drainable tree (Volumes 11 and 12). The through-line of the whole lens: **request-scoped work should form one tree from the edge to the leaves, in-process scopes linked by deadline-and-cancellation-propagating RPCs.** Every break in that tree is somewhere latency hides, capacity leaks, and shutdown hangs.

Key takeaways

- **Coroutines suspend one of two ways.** Stackful coroutines (Go, Java virtual threads) own a growable stack and can suspend anywhere in the call chain, at the cost of KB-scale footprint and FFI/stack-management complexity. Stackless coroutines (Rust, C++, Python, JS, Kotlin) are compiler-generated state machines that suspend only at marked points, with byte-scale footprint at the cost of viral async signatures.
- **`async/await` is sugar over a state machine**: locals live across suspension become object fields, suspension is a return, resumption is a dispatch on a saved state index. Task size is determined by the worst suspension point.

- **Function coloring is a real trade-off with three answers**: dissolve it by making blocking cheap (Go; Java 21's virtual threads, JEP 444), accept it for cost transparency (Rust, JS, Python), or hide it behind a compiler transform (Kotlin).
- **M:N work-stealing schedulers are cooperative**, so compute-heavy tasks starve peers and ignore cancellation alike; runtimes answer with preemption (Go 1.14's signal-based async preemption), poll budgets (Tokio), and blocking pools — and you answer with checkpoints in long loops.
- **Unstructured spawn is the concurrency `goto`** (Sústrik; N. J. Smith): it breaks black-box abstraction, so tasks outlive scopes, errors vanish, cancellation has no path, and leaks accumulate.
- **Structured concurrency restores a tree**: every task is owned by a lexical scope that waits for it; errors flow up; cancellation and deadlines flow down. Trio nurseries, Kotlin `coroutineScope`, Java `StructuredTaskScope` (still preview as of early 2026), and Go's `errgroup` + `context` conventions all realize it, with decreasing degrees of enforcement.
- **Cancellation must be cooperative.** Forced termination (`Thread.stop`) breaks lock-guarded invariants; sound cancellation is a request observed at checkpoints, with shielded, bounded cleanup and deadlines that compose as the minimum over ancestors.
- **Choose an error policy per scope**: fail-fast (cancel-and-*await* siblings, propagate first error) for co-dependent work; supervision/collect-all for independent work — and route panics through the scope like any other outcome.
- **Bound concurrency within scopes, name an owner for background work, and drain the tree on shutdown** — graceful shutdown is structured concurrency at the service level.
- **A request across services is a distributed task tree.** Propagate deadlines over RPC (`grpc-timeout`), let cancellation cross the wire, keep the tree connected inside every process, and treat orphaned work as the capacity leak and retry-amplifier it is.

Further reading

- Smith, N. J., *Notes on Structured Concurrency, or: Go Statement Considered Harmful* (2018) — the essay that made the argument

general. https://vorpus.org/blog/notes-on-structured-concurrency-or-go-statement-considered-harmful/

- Sústrik, M., *Structured Concurrency* (2016) and the libdill documentation — the origin of the term and the earliest deliberate implementation. http://libdill.org/structured-concurrency.html
- Nystrom, B., *What Color Is Your Function?* (2015) — the canonical statement of the coloring problem. https://journal.stuffwithstuff.com/2015/02/01/what-color-is-your-function/
- Trio documentation, *Tasks and Cancellation* — nurseries and cancel scopes from their source. https://trio.readthedocs.io/en/stable/reference-core.html
- Elizarov, R., *Structured Concurrency* (2018) — the Kotlin team's account of adopting the model and deprecating unstructured launch. https://elizarov.medium.com/structured-concurrency-722d765aa952
- Kotlin coroutines guide, *Coroutine Context and Jobs*, *Exception Handling* — the Job hierarchy and supervisor semantics. https://kotlinlang.org/docs/coroutines-guide.html
- JEP 444: *Virtual Threads* (final, Java 21); JEP 453 / JEP 505: *Structured Concurrency* (preview line); JEP 491: *Synchronize Virtual Threads without Pinning* (JDK 24). https://openjdk.org/jeps/444, https://openjdk.org/jeps/505
- Go blog: *Go Concurrency Patterns: Context* (2014) — the context-plumbing discipline from its authors. https://go.dev/blog/context; `errgroup` package docs: https://pkg.go.dev/golang.org/x/sync/errgroup
- Clements, A., *Proposal: Non-cooperative goroutine preemption* (Go proposal 24543) — the design behind Go 1.14's signal-based preemption. https://go.googlesource.com/proposal/+/master/design/24543-non-cooperative-preemption.md
- Tokio documentation, *Tutorial* and the `tokio::task` module — a work-stealing stackless executor in production form. https://tokio.rs/tokio/tutorial
- gRPC documentation, *Deadlines* — deadline propagation across services. https://grpc.io/docs/guides/deadlines/
- Volume 4, Chapter 6 — Async I/O and Event Loops — the substrate coroutines schedule onto.

- Volume 13, Chapter 2 — The Go Runtime — the G-M-P scheduler in full depth.

Chapter 9 — Concurrency Patterns for Backend Services

What this chapter covers. Chapters 1 through 8 built the machinery: models and the scaling laws that govern them, locks and their cost model, memory visibility, lock-free structures, liveness failures, asynchronous I/O, actors and channels, and structured concurrency with cancellation. This chapter is the synthesis — the dozen or so patterns that working backend engineers actually deploy, each grounded in that machinery rather than presented as folklore. We size worker pools quantitatively instead of by superstition, using Little's Law and the USL from Chapter 1. We insist — with reasons — that every queue be bounded and every blocking call carry a timeout. We work through producer–consumer, pipelines, fan-out/fan-in with partial results, request coalescing, in-process rate limiting and load shedding, bulkheads, graceful shutdown, background work, concurrency-safe caching, state confinement, and idempotency. We close with an anti-patterns catalog — the recurring shapes of production incidents — and with the observation that ties this volume to the rest of the suite: every pattern here has a fleet-scale twin, and fleet pathologies are usually process pathologies amplified.

None of these patterns is novel. That is precisely their value: they are the survivors of several decades of production selection pressure. What is usually missing from their presentation is the *why* underneath — why bounded, why jittered, why shed early, why drain before exit. The why is Chapters 1 through 8, and this chapter cites back into them relentlessly.

Learning goals — after this chapter you should be able to:

- Size a worker pool from measured arrival rate, service time, and wait/compute ratio, and explain why the bounded queue in front of it is where latency hides.
- Choose a rejection policy — block, drop, or caller-runs — and state the backpressure property of each.
- Build producer–consumer stages and pipelines, and predict pipeline throughput from the slowest stage.

- Implement fan-out/fan-in with concurrency limits, per-call timeouts, and partial results, and say when hedged requests are worth their cost.
- Deduplicate concurrent identical work with singleflight and defend a cache against stampedes.
- Apply token-bucket rate limiting and front-door load shedding, and explain why rejecting cheap beats failing expensive.
- Isolate dependencies with bulkheads so one slow downstream cannot consume every thread.
- Implement graceful shutdown in the correct order: stop accepting, drain with a deadline, cancel stragglers, flush, exit.
- Recognize the anti-patterns — unbounded anything, retries without jitter, blocking in async context, sleep-based coordination — before they page you.

Worker pools, done quantitatively

The worker pool is the oldest pattern in this chapter and the one most often deployed with the least thought. A pool has three parameters — worker count, queue capacity, and rejection policy — and all three have quantitative answers.

Sizing the workers

Start with the workload class, because the two classes have different answers.

CPU-bound work wants roughly one worker per core. More workers than cores adds context-switch and cache-pollution overhead (Volume 2, Chapters 1 and 2) without adding compute; by the USL (Chapter 1), the coherency term β guarantees that throughput eventually *falls* as you add threads. `N = cores` or `cores + 1` is the standard answer, the `+1` covering the occasional page fault or minor stall. In a container, "cores" means the effective CPU quota, not the host's core count — a pool sized to 64 hardware threads inside a 2-CPU cgroup is a throttling machine (Volume 2, Chapter 2).

I/O-bound work spends most of its wall-clock time waiting, so each core can serve many workers. The classic starting heuristic, from Goetz's *Java Concurrency in Practice*, is:

```
N_threads ≈ N_cores × target_utilization × (1 + wait_time /
compute_time)
```

A handler that computes for 2 ms and waits 48 ms on downstream calls has a wait/compute ratio of 24, so 8 cores at full utilization support on the order of 8 × 25 = 200 threads. This is a starting point, not an answer — it assumes the waiting consumes no CPU and that nothing else bounds you (memory per thread, downstream connection limits, the USL peak).

Then refine with **Little's Law** (Chapter 1): `L = λW`. If the service must sustain λ = 2,000 requests/s at W = 50 ms mean residence time, there will be L = 100 requests in flight at steady state, and the pool must hold at least that many workers or the excess waits in the queue — which lengthens W, which by the same law grows L further. Little's Law tells you the concurrency the load *demands*; the USL tells you the concurrency the system can *profitably supply*. If the demanded number exceeds the USL peak measured for your workload, no pool size fixes it — you need to reduce the serial fraction, shed load, or scale out.

Two structural rules complete the picture. First, **separate pools per workload class**. Mixing 2 ms CPU-bound jobs and 5 s report generations in one pool means the reports occupy workers for seconds at a time and the cheap jobs queue behind them — a head-of-line blocking problem you created yourself. Give each class its own pool sized for its own wait/compute profile. This is the bulkhead pattern, developed fully below, applied to workload classes rather than dependencies. Second, **never detach the pool from observability**: export queue depth, active workers, task latency, and rejection count. Queue depth is the single most predictive signal of impending latency trouble, for the reason the next section makes precise.

The queue must be bounded

An unbounded queue in front of a pool is one of the most reliably harmful defaults in backend engineering, and it *is* a default: Java's `Executors.newFixedThreadPool` uses an unbounded `LinkedBlockingQueue`, and an unbuffered-channel-fed goroutine spawner has the same effect if you spawn per item.

The argument is short. If arrival rate exceeds service rate, the difference accumulates somewhere. An unbounded queue accumulates it in memory: the queue grows without limit, and the process eventually dies

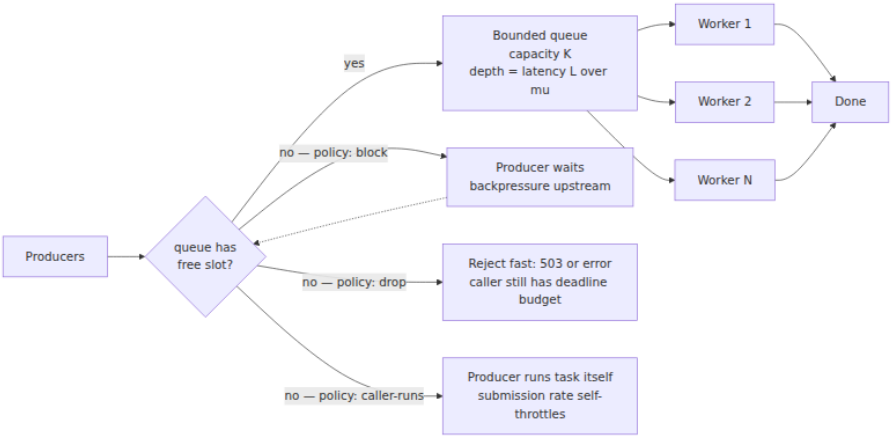
of OOM — an **unbounded queue is deferred OOM**. Worse, it dies *late* and *uselessly*: by Little's Law, a queue of length L in front of a server draining at rate μ imposes waiting time L/μ on every new arrival, so long before the OOM, every queued request is waiting tens of seconds — far past its client's timeout. The server is faithfully processing work whose requesters have already given up, which is negative work: it consumes capacity and produces nothing. **The queue is where latency hides.** Throughput metrics look fine — the pool is running flat out — while residence time W = L/μ climbs linearly with queue depth. If you remember one sentence from this chapter: a long queue is not a buffer, it is a promise of latency you have already broken.

A bounded queue converts this silent failure into an explicit, immediate decision: the queue is full, an arrival cannot be accepted — now what? That decision is the **rejection policy**, and there are three honest answers:

- **Block the submitter.** The producer waits until space frees. This is backpressure in its purest form: the slowdown propagates upstream to whoever is generating work, which is correct for internal producers (a file reader feeding a parser pool *should* slow to the parser's pace). It is wrong at the edge of a request-serving system, where "the submitter" is an accepted connection and blocking it just moves the unbounded queue into the kernel's socket buffers.
- **Drop — reject with an error.** Fail the submission immediately (`RejectedExecutionException`, HTTP 503, a full-channel `select` default case). This is load shedding at the pool boundary: the caller learns *now*, while its own deadline still has budget to try elsewhere or degrade. For work that must not be lost, "drop" means "divert to durable storage," which is the async-handoff pattern below.
- **Caller-runs.** The submitting thread executes the task itself (Java's `CallerRunsPolicy`). This has an elegant emergent property: while the caller is busy running the task, it is not submitting new ones, so the submission rate automatically throttles to what the system can absorb. It is a gentle backpressure valve — but note it runs the task on a thread that was not sized for it, so it is unsuitable when the submitter is an event loop (Chapter 6: never block the loop).

Java's `ThreadPoolExecutor` ships all of these (`AbortPolicy`, `DiscardPolicy`, `DiscardOldestPolicy`, `CallerRunsPolicy`); in Go you build them from channel operations — a blocking send blocks, a `select` with

`default` drops, and caller-runs is a `select` whose default branch invokes the function inline.



How big should the bound be? Small enough that the latency it implies is one you are willing to serve: a queue of K items in front of a pool draining μ items/s adds up to K/μ of waiting. If your latency budget for queueing is 100 ms and the pool drains 1,000 items/s, the queue bound is about 100 — not 10,000. Sizing the queue from the latency budget rather than from a vague desire for slack is the discipline that makes the rest of this chapter's timeout arithmetic work.

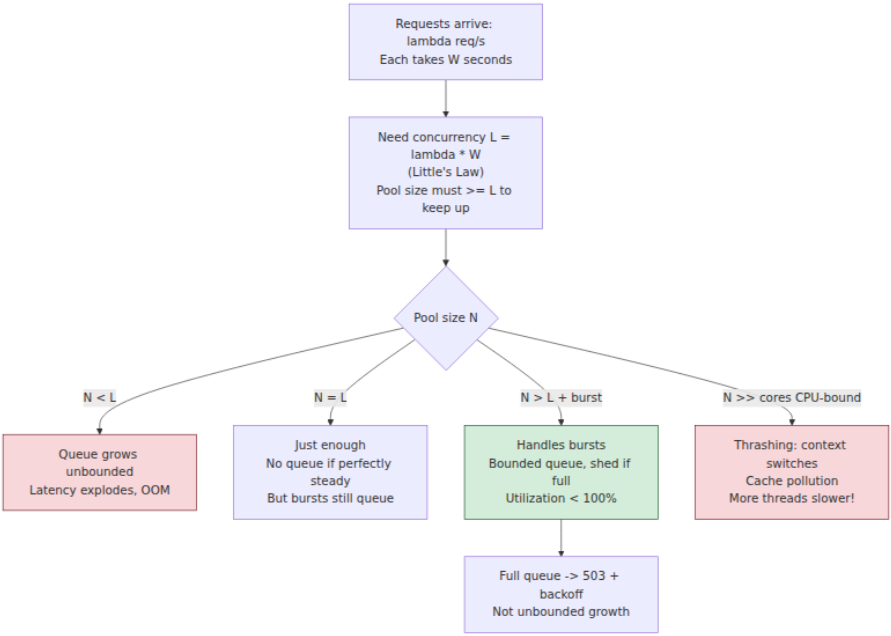
A bounded pool with graceful shutdown, in Go

The following is a complete, honest worker pool: bounded queue, explicit rejection, context cancellation, and a drain-then-exit shutdown (the shutdown ordering is treated in its own section below).

```
package pool

import (
    "context"
    "errors"
    "sync"
)

var ErrQueueFull = errors.New("pool: queue full")
var ErrShutdown = errors.New("pool: shutting down")

type Task func(ctx context.Context)

type Pool struct {
    queue  chan Task
    ctx    context.Context // cancelled to abort stragglers
    cancel context.CancelFunc
    wg     sync.WaitGroup

    mu     sync.Mutex
    closed bool
}

func New(workers, queueCap int) *Pool {
    ctx, cancel := context.WithCancel(context.Background())
    p := &Pool{
        queue:  make(chan Task, queueCap),
        ctx:    ctx,
        cancel: cancel,
    }
    p.wg.Add(workers)
    for i := 0; i < workers; i++ {
        go func() {
            defer p.wg.Done()
            for task := range p.queue { // exits when queue is closed
and drained
                task(p.ctx) // task must honor ctx cancellation
            }
        }()
    }
    return p
}

// Submit rejects immediately when the queue is full: load shedding
// at the pool boundary, while the caller's deadline still has budget.
func (p *Pool) Submit(t Task) error {
    p.mu.Lock()
    defer p.mu.Unlock()
    if p.closed {
        return ErrShutdown
```

```
    }
    select {
    case p.queue <- t:
        return nil
    default:
        return ErrQueueFull
    }
}

// Shutdown stops intake, drains queued and in-flight work until
// gracePeriod expires, then cancels the context to abort stragglers.
func (p *Pool) Shutdown(grace context.Context) {
    p.mu.Lock()
    if !p.closed {
        p.closed = true
        close(p.queue) // no more intake; workers drain what remains
    }
    p.mu.Unlock()

    done := make(chan struct{})
    go func() { p.wg.Wait(); close(done) }()

    select {
    case <-done: // drained cleanly
    case <-grace.Done(): // out of patience: cancel in-flight tasks
        p.cancel()
        <-done // workers observe cancellation and exit
    }
}
```

The Java equivalent is `ExecutorService` with an explicit `new ThreadPoolExecutor(n, n, 0, MILLISECONDS, new ArrayBlockingQueue<>(cap), policy)`, and shutdown via `shutdown()` → `awaitTermination(grace)` → `shutdownNow()` — the same three beats: stop intake, drain with a deadline, cancel stragglers.



Producer–consumer: the fundamental pattern

Strip any concurrency architecture to its skeleton and you find producer–consumer with a bounded buffer: some threads generate work, some threads consume it, and a bounded queue between them decouples their rates while limiting how far they may diverge. The worker pool is producer–consumer where consumers are homogeneous. The pipeline is producer–consumer chained. The event loop's ready queue, the actor's mailbox (Chapter 7), the socket's kernel buffer — all instances.

The classical implementation is the **blocking bounded queue**, and it is exactly the condition-variable monitor of Chapter 2: a mutex protecting the buffer, one condvar for "not empty" (consumers wait on it), one for "not full" (producers wait on it), and the mandatory `while` loop around each wait. `put` appends and signals not-empty; `take` removes and signals not-full. Every mainstream runtime ships this — `ArrayBlockingQueue`, Go's buffered channel (implemented with a lock and wait queues in the runtime, not with a condvar API, but the same monitor logically), Python's `queue.Queue`. When a colleague asks why the condvar chapter matters when nobody writes condvars anymore, the answer is:

every buffered channel and blocking queue *is* one, and its behavior under contention — who wakes, in what order, at what cost — is Chapter 2's behavior.

When throughput demands exceed what a lock-based queue delivers — remember from Chapter 2 that every `put` and `take` contends on the same mutex and bounces the same cache lines — the escalation path is the **ring buffer** of Chapter 4: a fixed-size array with head and tail indices advanced by atomic operations, single-producer/single-consumer variants requiring no atomics at all beyond memory ordering, and multi-producer variants using CAS. The LMAX Disruptor popularized this in trading systems; Go's channel and the io_uring submission/completion rings (Chapter 6) are ring buffers under the hood. The trade-off is flexibility: a ring buffer's capacity is fixed at a power of two, and blocking semantics must be layered on with parking or spinning. For nearly all backend services, the lock-based blocking queue is enough — the queue is rarely the bottleneck; the work is — but knowing the escalation path exists keeps you from inventing one.

One design rule carries over from the pool discussion: **the buffer's bound is a latency policy, not a tuning knob.** Deep buffers absorb bursts but hide sustained rate mismatch; by Little's Law the hiding shows up as residence time. Prefer a small bound plus an explicit policy at the full edge — block, drop, or shed — over a deep bound and hope.

Pipelines: chained stages, slowest stage wins

When work naturally decomposes into stages — parse, validate, enrich, persist — you can run each stage as its own pool of workers connected by bounded queues. This is **pipeline parallelism**, and it is the CSP style of Chapter 7 made concrete: stages share nothing and communicate only through channels, so each stage reasons about its own state single-threadedly.

Two laws govern pipelines. First, **throughput is the throughput of the slowest stage.** A five-stage pipeline whose stages process 10k, 8k, 2k, 9k, 12k items/s runs at 2k items/s, and the bounded queues upstream of the slow stage fill while those downstream run empty. This makes bottleneck diagnosis wonderfully mechanical: look at the queue depths; the full queue points at the bottleneck stage. (This is the Theory-of-Constraints view, and it is also exactly how you read a distributed

pipeline's consumer lag — Volume 10.) The fix is to parallelize the slow stage — give it more workers — not to enlarge its input queue, which changes nothing except latency.

Second, **latency is the sum of stage residence times**, so pipelining helps throughput but never helps, and usually hurts, single-item latency: each queue hop adds handoff cost. Pipelines earn their keep when stages have genuinely different resource profiles (a CPU-heavy stage and an I/O-heavy stage overlap beautifully) or different scaling needs; a pipeline of stages with identical profiles is usually better collapsed into one pool of workers that each do the whole job, avoiding the handoff cost entirely.

Cancellation must propagate both directions: downstream (a cancelled request's items should be dropped, not processed), and upstream (when a consumer stops consuming, producers must learn or they block forever on full queues — in Go the idiom is closing a `done` channel that every stage selects on; Chapter 8's structured concurrency gives the general discipline). The Go blog's "Pipelines and cancellation" post remains the best short treatment of the mechanics.

Fan-out/fan-in: scatter-gather with limits and timeouts

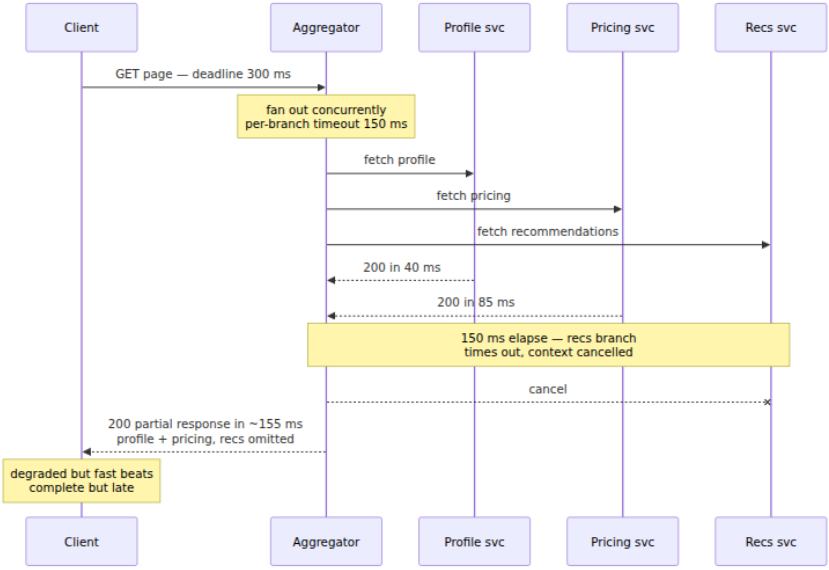
A request needs data from several sources — profile service, pricing service, recommendations. Do the calls concurrently (fan-out), collect the responses (fan-in), assemble. Total latency drops from the sum of the call latencies toward the max. Three disciplines separate the production version from the tutorial version.

Bound the fan-out. Fanning out to N sub-calls per request multiplies your in-flight concurrency by N; under load this multiplies connection counts, memory, and pressure on each downstream. Use a semaphore (Chapter 2) or a bounded worker pool to cap concurrent sub-calls — per request if N is large (fetch 500 objects, but at most 16 at a time), and globally per downstream (the bulkhead, below).

Per-call timeouts, and a plan for partial results. Fan-in's latency is the *max* of the branches, which means scatter-gather is a tail-latency amplifier: if each of 10 branches is slow 1% of the time, roughly $1 - 0.99^{10} \approx 10\%$ of requests hit at least one slow branch. Dean and Barroso's "The Tail at Scale" (CACM, 2013) is the definitive treatment of this arithmetic at fleet width — with 100 servers involved, a 1-in-100 slow

response becomes the common case. The first defense is a timeout on every branch, shorter than the request's overall deadline, plus a decision made *at design time* about what to do when a branch misses it: fail the whole request (only when the branch is essential), or return a **partial result** — the page without recommendations, the search results from 9 of 10 shards with a `partial: true` flag. Degrading to partial results is usually right and needs product agreement, which is why it must be designed rather than improvised during an incident.

Hedged requests, from the same paper, address the tail more aggressively: send the request to one replica; if no reply arrives within a threshold (say, the observed 95th-percentile latency), send a second copy to a different replica and take whichever answers first. Because only the slowest ~5% of calls hedge, the added load is a few percent, but the tail collapses — the paper reports a Google benchmark reading 1,000 keys across 100 servers in which hedging after a 10 ms delay cut 99.9th-percentile latency from 1,800 ms to 74 ms while adding roughly 2% more requests. Hedge only idempotent operations (a hedged non-idempotent write is a duplicate write — see the idempotency section), and cancel the loser once a winner returns, or you pay the duplicate work anyway.



In Go the shape is `errgroup.WithContext` plus `golang.org/x/sync/semaphore` (or `errgroup.SetLimit`) for the concurrency cap; in Java,

`CompletableFuture.allOf` with per-future `orTimeout`, or structured concurrency's scopes in recent JDKs (Chapter 8).

Request coalescing: singleflight

A cache entry for a hot key expires. Five hundred concurrent requests miss simultaneously, and all five hundred independently issue the same expensive database query. The database, sized for a cache in front of it, buckles; queries slow; more requests pile up behind the missing key. This is the **cache stampede** (also "dog-pile"), and it is a concurrency bug, not a capacity bug: the system performed five hundred units of work when one was needed.

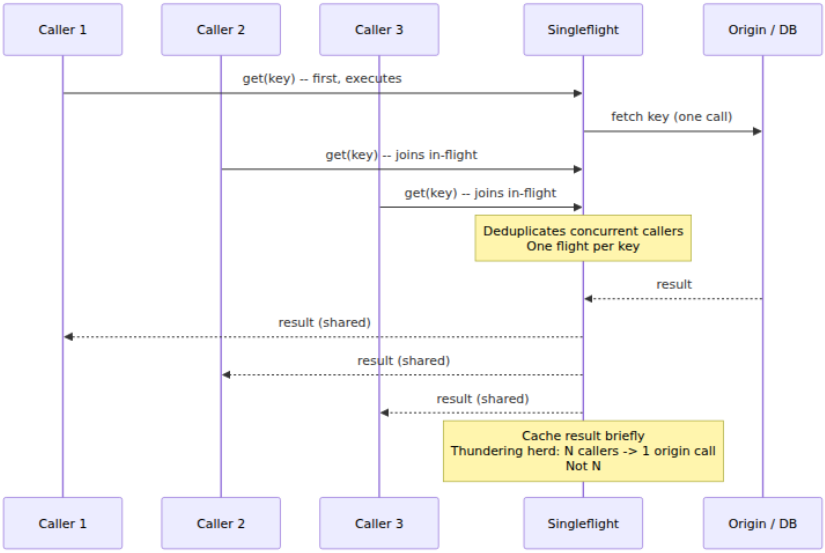
The fix is **request coalescing**: when a request for key K is already in flight, subsequent requests for K wait for that flight's result instead of launching their own. Go's `golang.org/x/sync/singleflight` is the canonical implementation — a map from key to in-flight call, a mutex, and a `WaitGroup` per call that latecomers wait on:

```
import "golang.org/x/sync/singleflight"

var group singleflight.Group

func (s *Store) GetUser(ctx context.Context, id string) (*User, error)
{
    if u, ok := s.cache.Get(id); ok {
        return u.(*User), nil
    }
    // All concurrent callers with the same key share ONE fetch;
    // the others block until it completes and receive the same result.
    v, err, shared := group.Do("user:"+id, func() (any, error) {
        u, err := s.db.FetchUser(ctx, id)
        if err != nil {
            return nil, err
        }
        s.cache.Set(id, u, s.ttlWithJitter())
        return u, nil
    })
    if err != nil {
        return nil, err
    }
    _ = shared // true when this caller received another flight's
result
    return v.(*User), nil
}
```

Java has no standard-library equivalent, but Caffeine's `LoadingCache` provides the same guarantee per key — concurrent `get`s for a missing key block on a single load — and a `ConcurrentHashMap<K, CompletableFuture<V>>` with `computeIfAbsent` builds it in a dozen lines.

Two sharp edges. First, **error propagation is amplified**: one failed flight delivers its error to every coalesced waiter, so a single transient failure becomes five hundred failures. Decide deliberately whether waiters should retry (singleflight's `Forget` drops the key so the next caller starts fresh). Second, **a slow flight holds everyone**: coalesced callers inherit the flight's latency, so the underlying call still needs its own timeout. Note also that the in-flight call typically should *not* be cancelled just because one waiter's context is — others still want the result; detach the fetch's context deliberately.

Coalescing composes with two other stampede defenses covered in the caching section below — TTL jitter and early refresh — and its distributed twin is the cache lock/lease (Volume 6, Chapter 8; Volume 7, Chapter 3 treats caching strategy end to end).



Rate limiting and admission control in-process

A service that accepts every request accepts its own death. Two mechanisms guard the front door: rate limiting (bound the *rate* of

admitted work) and load shedding (refuse work when the system is unhealthy regardless of rate). They are different tools; you usually want both.

Token bucket and leaky bucket

The **token bucket** is the workhorse. A bucket holds up to B tokens; tokens drip in at rate r per second; each admitted request removes one (or `cost` many); a request arriving to an empty bucket is rejected or delayed. The two parameters map directly onto policy: r is the sustained rate, B is the burst allowance. Implementation is a few lines, because you do not need a background filler goroutine — refill lazily from the elapsed time on each call:

```
type TokenBucket struct {
    mu     sync.Mutex
    rate   float64 // tokens per second
    burst  float64 // bucket capacity B
    tokens float64
    last   time.Time
}

func NewTokenBucket(rate, burst float64) *TokenBucket {
    return &TokenBucket{rate: rate, burst: burst, tokens: burst, last:
time.Now()}
}

func (b *TokenBucket) Allow() bool {
    b.mu.Lock()
    defer b.mu.Unlock()
    now := time.Now()
    b.tokens = math.Min(b.burst, b.tokens+now.Sub(b.last).Seconds()*b.r
ate)
    b.last = now
    if b.tokens >= 1 {
        b.tokens--
        return true
    }
    return false
}
```

Production code should use the library form — Go's `golang.org/x/time/rate.Limiter`, Guava's `RateLimiter`, resilience4j's `RateLimiter` — which add waiting-with-deadline (`Wait(ctx)`), reservation, and multi-token costs, but the mechanics are exactly the sketch above.

The **leaky bucket** is the token bucket's dual: requests enter a bounded queue that drains at a fixed rate, so the *output* is perfectly smooth regardless of input burstiness. Use it when the protected resource genuinely cannot absorb bursts (a downstream with strict pacing requirements); prefer the token bucket when short bursts are fine and you only care about the sustained rate, which is the common case. Volume 7, Chapter 9 covers rate-limiter placement at the API gateway and Volume 14, Chapter 8 the algorithm family (fixed and sliding windows, GCRA) in depth; the in-process forms here are the same algorithms an inch from the code they protect.

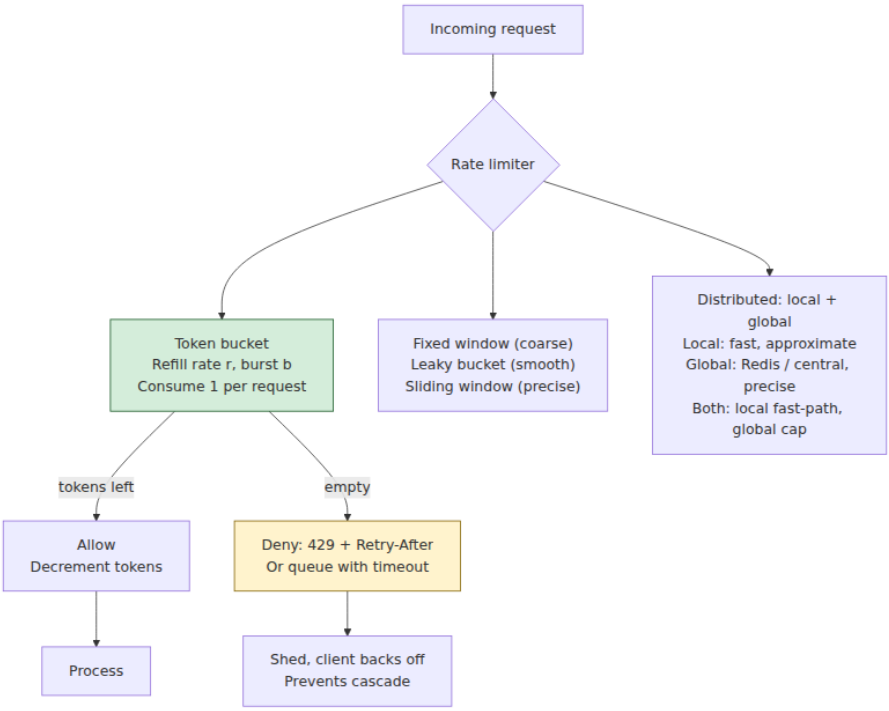
Load shedding: reject cheap, early

Rate limiting enforces a policy; **load shedding** defends survival. When the system is beyond its capacity — queue depths climbing, latency past SLO, memory pressure — the correct move is to reject some work *immediately at the front door*, before spending anything on it. Two principles govern the design:

Shed early. The worst place to fail a request is at the end: after it has consumed queue slots, worker time, and downstream calls, failing it wastes everything already spent — the negative work of the unbounded-queue discussion. A rejection at admission costs microseconds and returns a clear signal (HTTP 429/503, with `Retry-After` when you can estimate it) while the client's own deadline still has budget to back off or fail over. **Prefer rejecting cheap over failing expensive** — a shed request costs almost nothing; a timed-out request costs everything it consumed plus a retry.

Shed on a leading signal. CPU utilization is a lagging, noisy trigger. The best signals are the ones queueing theory points at: your own queue depth and in-flight count against measured capacity (Little's Law again — if sustainable in-flight is 100 and you are at 100, admission of the 101st only adds latency), and request staleness (if a request has already waited past the point where a useful response is possible — its deadline is spent — drop it unprocessed; this "expired on arrival" check is the cheapest shed of all). Prioritized shedding — drop low-priority traffic classes first, keep health checks and payments — is the natural refinement. The Google SRE book's "Handling Overload" and "Addressing Cascading Failures" chapters are the standard field guides; Volume 11 develops

shedding as a reliability practice, and Volume 11, Chapter 10 covers the pattern catalog around it.

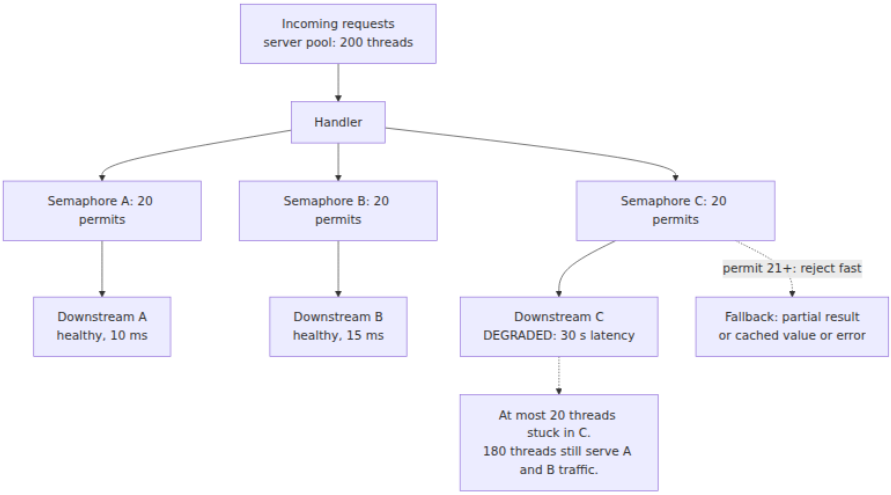


Bulkheads: isolating the blast radius

The term is naval: a ship's hull is partitioned so one flooded compartment does not sink the vessel. The software version: **partition your concurrency so one misbehaving dependency cannot consume it all.**

The failure it prevents is depressingly standard. A service calls three downstreams from one shared pool of 200 threads. Downstream C develops a 30-second latency (it is not down — down would be cheap; it is *slow*, the expensive kind of failure). Every thread that touches C now blocks for 30 s. At a modest rate of requests touching C, the shared pool drains in seconds — and now requests needing only the healthy A and B find no threads. One slow dependency has taken the whole service, which is how latency cascades between tiers (Volume 7, Chapter 11).

The bulkhead gives each dependency its own bounded concurrency: a dedicated pool per downstream, or — cheaper — a semaphore (Chapter 2) of N permits guarding calls to it. When C slows, at most N threads are stuck in C; the (N+1)th call fails fast with "bulkhead full," which is precisely the load-shedding signal, scoped to one dependency. The service degrades — C's data is missing, a partial result — instead of dying. Sizing is Little's Law yet again: expected calls/s to C times its timeout bounds worst-case in-flight; permits somewhat above the *healthy* steady state (λ × normal latency) with headroom, but far below "all the threads," give the isolation without throttling normal operation.



Choose pools over semaphores when the calls must also be *timed out from outside* (a thread pool lets you abandon a stuck call via future timeout; a semaphore-guarded synchronous call is only as bounded as its own timeout — which the next section insists exists anyway) or when the dependency work deserves different sizing and queuing. Semaphores are lighter — no extra threads, no handoff — and are the right default in async runtimes where "threads" are not the resource being protected, in-flight count is.

The **circuit breaker** is the bulkhead's temporal partner: where the bulkhead limits how much concurrency a bad dependency can consume, the breaker notices the dependency is failing (error rate or latency over a window), *opens*, and fails calls immediately without attempting them — then periodically lets a probe through (half-open) and closes again on

success. Its contributions are recovery time for the downstream (no longer hammered by doomed calls) and fast failure for callers. Netflix's Hystrix codified bulkhead-plus-breaker for a generation (now in maintenance, its documentation remains an excellent design rationale); resilience4j is the maintained successor on the JVM. The breaker's state machine, tuning, and its subtleties — per-endpoint vs per-host granularity, thundering probes on half-open — are Volume 11, Chapter 10's subject; here it suffices that breaker and bulkhead compose: the bulkhead caps the damage while the breaker is still deciding.

Timeouts everywhere

Every blocking operation in a backend service must be bounded in time. Not most — every. An unbounded blocking call is a latent permanent thread leak: the code that "cannot hang" will hang the week the switch firmware drops your TCP session without an RST, and each hung call permanently subtracts one worker (a bulkhead delays the bleed-out; only a timeout stops it). This means connect timeouts *and* read/write timeouts on every socket; query timeouts on every database call; acquisition timeouts on every pool and semaphore; `Wait(ctx)` rather than `Wait()` on every limiter. Java's historical default of infinite socket timeouts has caused more stuck fleets than any exotic failure mode; Go's `context` makes deadlines pervasive but only if you actually pass the context down and honor it.

Two structural refinements turn scattered timeouts into a system.

Deadline propagation. A request should carry one *deadline* — an absolute point in time by which the response is useful — rather than each layer inventing independent timeouts. Each downstream call gets the *remaining* budget: entered with 300 ms, spent 120 ms, the next call gets at most 180 ms. Go's `context.WithDeadline` / `WithTimeout` flows this automatically through call chains, and gRPC propagates it on the wire (`grpc-timeout` header); Chapter 8 covered the mechanics and the cancellation tree that makes a missed deadline actually *stop* the work rather than merely stop waiting for it. The alternative — fixed per-layer timeouts that ignore time already spent — produces the classic pathology of a layer faithfully working on a request whose caller gave up 200 ms ago.

Timeout hierarchies. Budgets must nest: the client's timeout should exceed the gateway's, which should exceed the service's, which should exceed the sum of its sequential downstream budgets — with margin at each level for the response to travel back. Invert the hierarchy and you get systematic orphaned work: if the service allows itself 5 s but its caller gives up at 2 s, every slow request is completed for nobody, and — worse — the caller's retry arrives while the original is still running, doubling load exactly when the system is slow (see retry storms, below). Write the budget tree down; it is a contract between teams, not an implementation detail.

Choose timeout values from measured latency distributions (e.g., a healthy p99 plus margin), not round numbers — a timeout at the p50 sheds half your good traffic; a 30 s timeout on a 50 ms call is not a bound, it is a rumor. And treat a timeout as an *ambiguous* outcome: the work may have happened. That single fact is why idempotency, treated below, is not optional.

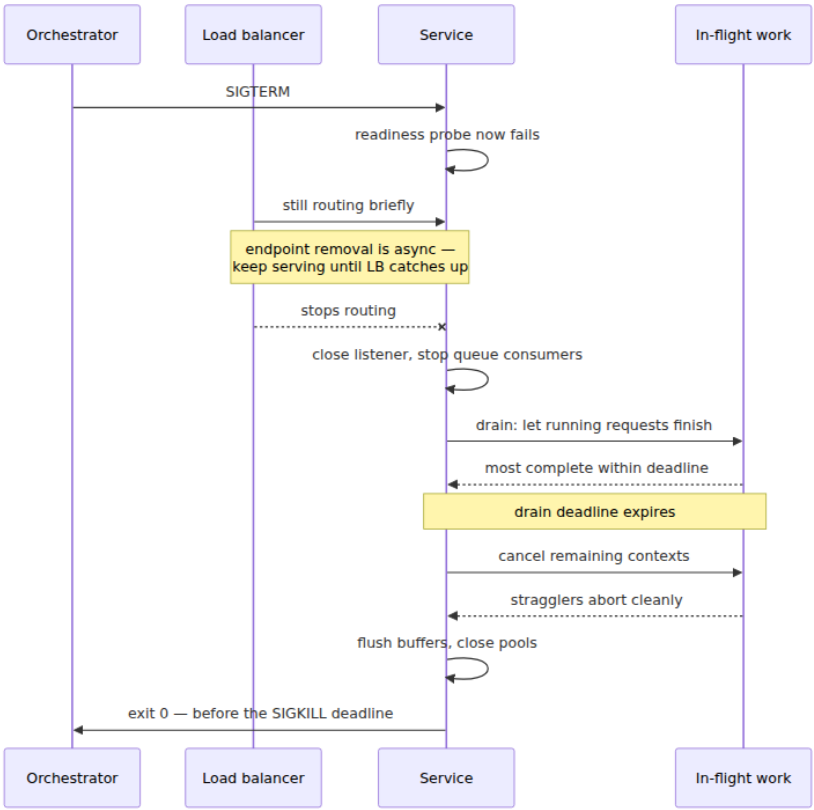
Graceful shutdown as a first-class pattern

Every process dies. Deploys, autoscaling, node drains, and spot reclamation mean a healthy fleet kills processes *constantly* — a service that drops in-flight work on exit turns every deploy into an incident. Graceful shutdown is not cleanup code; it is a pattern with a strict ordering, and the ordering is the point.

1. **Catch the signal.** Kubernetes and every orchestrator send `SIGTERM` first, then `SIGKILL` after a grace period (default 30 s). Handle `SIGTERM`; you cannot handle `SIGKILL`. Your entire shutdown must fit inside the grace period — know what it is, and set `terminationGracePeriodSeconds` to match your drain budget, not vice versa.
2. **Stop accepting new work — and tell the world first.** Flip the readiness probe to failing *before* closing the listener, and keep serving briefly while the load balancer notices and stops routing to you (Volume 12 covers probe mechanics and endpoint propagation delay; a small sleep between probe-flip and listener-close is the pragmatic standard, because endpoint removal is asynchronous and eventually consistent). Closing the listener first means the LB keeps sending requests to a closed port — visible errors during every

deploy. Also stop pulling from queues and pause background schedulers.

3. **Drain in-flight work, with a deadline.** Let running requests finish: `Server.Shutdown(ctx)` in Go, `awaitTermination` in Java. The deadline matters because drain time is bounded by your longest request, and a stuck request must not hold the process past the orchestrator's grace period — at which point `SIGKILL` drops *everything* still running, not just the straggler.
4. **Cancel stragglers.** When the drain deadline passes, cancel the remaining work's contexts (Chapter 8) so it stops cleanly — releasing locks, aborting transactions — rather than being killed mid-write.
5. **Flush and close.** Buffered telemetry, batched writes (see write-behind, below), outbound connections, the database pool. This is where the debounce/batch buffers of the next section get their contents persisted; a process that batches writes and skips this step silently loses the last batch on every deploy.
6. **Exit zero.** So the orchestrator can tell a clean stop from a crash.



The `Pool.Shutdown` in the worker-pool code above is beats 2–4 in miniature: close intake, drain with deadline, cancel. Note the interlock with retries and idempotency: even a perfect drain cancels *some* work sometimes, so clients must retry, so retried work must be safe to repeat. Graceful shutdown, retries, and idempotency are one mechanism seen from three sides.

Background work patterns

Not all concurrency serves a request. Cache refreshers, metric flushers, cleanup sweeps, and deferred processing have their own pattern vocabulary.

Periodic tasks, with jitter. The naive periodic task — `every 60s: refresh` — has a fleet bug: all N instances deployed at the same moment

tick at the same moment, and the shared dependency they refresh from receives N simultaneous requests every minute — a self-inflicted **thundering herd**, invisible at N=1 and proportional to fleet size. The fix costs one line: randomize each instance's phase (initial delay uniform in [0, period)) and, better, jitter each interval (e.g., uniform in [0.5, 1.5] × period), so the fleet's ticks decorrelate and stay decorrelated even after synchronized restarts. The same jitter principle reappears in retry backoff below, and for the same reason: synchronized fleets are load spikes.

Debounce and batching. Many operations have a large per-invocation cost and a small per-item cost — a database round trip, an fsync, an HTTP call to a metrics API. Batching amortizes: buffer items and flush when either the batch reaches size K or an age limit T expires, whichever comes first. The size bound caps memory and downstream request size; the time bound caps the latency an item spends waiting for peers — flushing on size-or-time is the whole trick, and both bounds are mandatory (size alone starves trickles forever; time alone lets bursts build huge batches). This is **write-behind** when the buffered items are writes: acknowledge fast, persist in batches, and accept the trade — a crash loses the unflushed window. That loss window is why write-behind buffers appear by name in the shutdown flush step, and why data that cannot tolerate the window needs the next pattern instead.

Async handoff to durable queues. An in-memory queue dies with the process; "graceful" shutdown narrows the window but cannot close it (kernel panics do not drain). Work that must survive — the order confirmation email, the billing event — must be handed off to storage that outlives the process *before* you acknowledge the request: a message broker or a database table consumed by workers. The in-process pattern reaches its honest limit here, and the baton passes to Volume 10 — including the outbox pattern (Volume 10, Chapter 6), which makes the "write the business row and enqueue the work" step atomic instead of a two-writes race.

Concurrency-safe caching in-process

An in-process cache is a shared mutable map read by every request thread — a concentrated dose of everything Chapters 2–4 warned about, so the pattern deserves its own treatment.

The map itself. The design space from Chapter 2 applies directly. A single mutex around a map is correct and fine at moderate rates. `ConcurrentHashMap` — striped/per-bin locking with CAS fast paths — is the JVM default answer at essentially all rates and is hard to beat. Go's `sync.Map` is more specialized than its name suggests: it maintains a read-only map reached without locking plus a dirty map behind a mutex, and its own documentation scopes it to two cases — keys written once and read many times (caches), or disjoint key sets per goroutine. For write-heavy or mixed workloads a plain `map` under `sync.RWMutex` (mind Chapter 2's RW-lock caveats) or a sharded map (Chapter 2's striping, verbatim) typically wins; benchmark with your actual read/write mix rather than assuming "concurrent map" means "faster map." Production caches also need eviction and TTLs, which is why the real answer is usually a purpose-built library — Caffeine on the JVM (which also supplies per-key coalescing loads, as noted above), Ristretto or groupcache-style libraries in Go.

The stampede, revisited and completed. Singleflight collapses concurrent misses for one key, but two more failure shapes remain. *Synchronized expiry*: entries cached at the same moment with the same TTL expire at the same moment — a deploy warms the cache, and exactly TTL later, a miss storm across many keys at once. Fix with **TTL jitter**: TTL × uniform[0.9, 1.1], so expiries decorrelate — the thundering-herd fix again, applied to cache metadata. *Hot-key expiry cost*: for a very hot key, even one coalesced refresh means every request briefly waits on a miss. Fix with **early refresh**: when a read finds an entry past a soft threshold (say 80% of TTL), trigger one background refresh (guarded by singleflight so it is exactly one) while still serving the stale-but-valid value. Requests never wait on the refresh at all. The full defense is the stack: singleflight + TTL jitter + early refresh, and Volume 7, Chapter 3 places it in the broader caching architecture.

State machines under concurrency: confine or lock

Much backend state is a state machine — an order moving through `created → paid → shipped`, a connection through a protocol handshake, a saga through its steps. Concurrent access to a state machine is where lock-based code gets genuinely hard, because the invariants span *transitions*, not just fields: check-then-act sequences must be atomic, callbacks fire mid-transition, and the lock must cover exactly the right

extent (Chapter 2's "never call unknown code under a lock" bites hardest here).

The alternative is Chapter 7's move: **confine the state to one goroutine/actor and serialize commands to it.** The state machine's data is owned by a single goroutine; everyone else sends commands over a channel (with replies over per-command response channels); the owner processes commands one at a time. There is no lock because there is no sharing — transitions are atomic by construction, the owner can safely fire callbacks and do I/O mid-command without holding anything, and the command channel gives ordering, a natural audit log, and a bounded mailbox for backpressure. This is the actor pattern stripped to one actor.

When does each win? **Locking** wins when operations are short, simple, and latency-critical — an uncontended mutex is tens of cycles (Chapter 2), while a channel round trip to another goroutine costs two handoffs and possibly two context switches, easily microseconds. It also wins when many independent instances exist (a million sessions do not want a million goroutines — or rather, they can have them in Go, but a striped lock over a session map is simpler). **Confinement** wins when transitions are complex or multi-step, when transitions involve I/O or callbacks, when ordering of operations matters, or when the lock discipline has already failed audit once. A useful default: guard *data* with locks; give *processes* (things with lifecycle and ordered transitions) an owner. And confinement composes with everything above: the owner goroutine is a worker pool of size one with a bounded queue, and every rule about bounds, rejection, and drain applies to its mailbox.

Idempotency as a concurrency property

Timeouts are ambiguous (the work may have happened), shutdowns cancel mid-flight (the client will retry), hedges duplicate requests, and at-least-once queues redeliver. Every pattern in this chapter therefore manufactures the same phenomenon: **the same logical operation arriving more than once, possibly concurrently.** A system built from these patterns must make re-execution safe — idempotency is not a distributed-systems nicety bolted on later; it is a local consequence of retries plus ambiguity.

Some operations are naturally idempotent (set field to X; delete row). The rest are made idempotent with a **dedupe key**: the client attaches a

unique idempotency key to the operation; the server records the key with the outcome and returns the recorded outcome for any repeat. Note the in-process concurrency content of that sentence: two requests bearing the same key can arrive *simultaneously*, so "check whether the key exists, then execute" is a textbook check-then-act race (Chapter 2). The check-and-claim must be atomic — `putIfAbsent` on a concurrent map, an atomic insert with a unique constraint — and the second arrival must then either wait for the first's result or receive "in progress." If that structure sounds familiar, it should: an idempotency-key store is singleflight with a persistent result, and both are the same shape as `computeIfAbsent`. Deduplication windows, key design, and exactly-once semantics across process boundaries are Volume 6, Chapter 9's subject; the point here is that you cannot implement even the local half correctly without this volume's tools.

Anti-patterns: a field catalog

Each of these recurs in postmortems because each works fine in testing and fails under production load — usually by violating a bound this chapter insisted on.

- **Unbounded anything.** Unbounded queues (deferred OOM plus hidden latency, as above), unbounded goroutine/thread spawn (accept loop spawns per connection with no cap; a slow downstream makes each one long-lived; memory and scheduler collapse), unbounded retries, unbounded fan-out, unbounded caches without eviction. The audit question for any code review: *what bounds this?* If the answer is "traffic is never that high," the bound is your users' patience.
- **Per-request thread spawn.** The pre-pool design: create a thread per request, destroy it after. Thread creation costs a syscall and stacks cost memory (Volume 2); under a spike you create thousands of threads precisely when you can least afford to, and the scheduler thrashes. Pools exist to amortize creation and — more importantly — to *bound* concurrency. (Goroutines and virtual threads make creation cheap but do not repeal the bound: a million goroutines blocked on one slow downstream still hold a million requests' memory and a million downstream intents. Cheap threads change the constant, not the pattern.)

- **Retry without jitter or budget.** A dependency blips for 2 s; every caller times out and retries in lockstep; the dependency recovers into a synchronized wave of double load and goes down again — the **retry storm**, a self-sustaining oscillation that is the systems-scale cousin of Chapter 5's livelock: everyone is busy, nobody progresses. The fixes are mechanical: exponential backoff *with jitter* (decorrelate the fleet — the same fix as periodic-task jitter), a retry *budget* (e.g., retries may add at most 10% extra load, enforced with a token bucket over retries), retry only retryable errors, and never retry on top of a layer that also retries — multiplied across three layers, three retries each become 27 attempts. And respect the timeout hierarchy: a retry issued after the caller's own deadline has expired is pure waste.
- **Blocking in async context.** A synchronous database driver call, an uninstrumented DNS lookup, or a `synchronized` block on an event-loop thread stalls every connection multiplexed onto that thread — the cardinal sin of Chapter 6. Symptom: p99 latency spikes across *all* endpoints when one endpoint's dependency slows. Fix: bounded worker pool for the blocking work (with all of this chapter's rules), or an actually-async driver.
- **Shared mutable singletons.** The lazily-initialized global — a config object, a client, a `SimpleDateFormat` (famously not thread-safe), a shared `Random` — reached from every request thread. Unsafe publication (Chapter 3), check-then-act races on initialization, and hidden contention on what looks like a stateless helper. Fix: immutable after construction, eager or `once`-guarded init (`sync.Once`, class-init idiom), or per-thread/request instances.
- **Sleep-based coordination.** `sleep(100ms)` after starting a dependency "so it's ready"; polling a flag with sleeps instead of a condvar/channel; a test that sleeps and asserts. Every such sleep encodes a timing assumption that load, CI, or a slow VM will falsify — the flaky test is the honest version of the pattern telling you it is broken (Chapter 10). Fix: wait on the actual condition — readiness check, channel receive, `CountDownLatch` — with a deadline.
- **Ignoring cancellation.** Accepting a `context.Context` and never checking it; catching `InterruptedException`, swallowing it, and looping. The deadline-propagation machinery is end-to-end: one layer that ignores cancellation converts every timeout above it into orphaned running work below it, and makes graceful shutdown's "cancel stragglers" step a no-op. Check cancellation at loop

boundaries and before expensive steps; pass the context all the way down.

The distributed-systems lens

The claim made at the start — that these are load-bearing patterns — has a second half: **every pattern in this chapter has a fleet-scale twin**, because the constraints that produced it (bounded resources, rate mismatch, partial failure, tail latency) do not care about process boundaries.

In-process pattern	Fleet-scale twin	Where
Worker pool with bounded queue	Autoscaled consumer group on a partitioned topic	Volume 10
Bounded queue + rejection policy	Broker with retention limits and producer backpressure	Volume 10
Pipeline stages with queues	Stream-processing topology; stage lag = full queue	Volume 10
Singleflight	Distributed lock/lease or cache lock on the shared cache	Volume 6 Ch. 8; Volume 7 Ch. 3
Token bucket / load shedding	API-gateway rate limiting and admission control	Volume 7 Ch. 9
Bulkhead + circuit breaker	Per-dependency quotas; mesh outlier ejection	Volume 11 Ch. 10
Deadline propagation	Request deadlines on the wire, e.g. gRPC	Volume 8
Graceful shutdown	Rolling deploys and connection draining	Volume 12
Idempotency keys	Exactly-once-effect processing over at-least-once delivery	Volume 6 Ch. 9

The correspondences are not analogies; they are the same mathematics re-hosted. Little's Law does not know whether the queue is an `ArrayBlockingQueue` or a Kafka partition — consumer lag *is* queue depth, and lag × drain rate *is* the latency hiding in it. A retry storm between services is the in-process retry storm with network hops added. A fleet's

synchronized cron jobs are the unjittered ticker at N instances. A stampede onto Redis is a stampede onto a `HashMap`, at scale.

Which yields the meta-point, and it is the reason this chapter sits in the concurrency volume rather than the system-design one: **get the single-process version right first, because fleet pathologies are usually process pathologies amplified.** A service that ships unbounded queues, unjittered retries, and missing deadlines does not become a well-behaved distributed system when you run 50 replicas of it — it becomes 50 correlated copies of the same failure, synchronized by the shared dependencies between them. The distributed patterns of Volumes 6–12 assume disciplined processes underneath; this chapter is that discipline.

Key takeaways

- **Size pools from measurement, not vibes.** CPU-bound: ≈ cores (the container quota, not the host). I/O-bound: cores × (1 + wait/compute) as a start, then Little's Law ($L = \lambda W$) for demanded concurrency and the USL for supplied. Separate pools per workload class.
- **Bound every queue.** An unbounded queue is deferred OOM plus hidden latency — the queue is where latency hides ($W = L/\mu$). Size the bound from the latency budget, and choose the rejection policy deliberately: block (backpressure), drop (shed early), or caller-runs (self-throttling).
- **Producer–consumer with a bounded buffer is the fundamental pattern** — the blocking queue is Chapter 2's monitor; the ring buffer is Chapter 4's escalation. Pipelines chain it, and pipeline throughput is the slowest stage — read the full queue to find the bottleneck.
- **Fan-out amplifies the tail.** Cap the fan-out, put a timeout on every branch, design partial results with product intent, and hedge only idempotent calls (The Tail at Scale).
- **Coalesce duplicate work.** Singleflight turns N concurrent identical misses into one call; with TTL jitter and early refresh it makes cache stampedes a solved problem.
- **Guard the front door.** Token buckets bound admitted rate; load shedding rejects cheap and early — before the request consumes anything — instead of failing expensive at the end.

- **Bulkhead every dependency.** A semaphore or pool per downstream caps how many threads a slow dependency can take; circuit breakers add fast failure and recovery time (Volume 11, Chapter 10).
- **Every blocking call carries a timeout**, deadlines propagate with remaining budget, and budgets nest: client > gateway > service > downstream. A timeout is an ambiguous outcome — which is why idempotency is mandatory, not optional.
- **Shutdown is ordered:** fail readiness, keep serving until the LB notices, stop intake, drain with a deadline, cancel stragglers, flush, exit 0 — inside the orchestrator's grace period.
- **Jitter everything periodic** — tickers, TTLs, retry backoff — because synchronized fleets are load spikes. Batch on size *or* time, never just one.
- **Confine state machines to an owner; lock plain data.** Commands over a bounded channel buy atomic transitions and ordering; locks buy latency. Know which you are buying.
- **The anti-patterns are bound violations:** unbounded anything, retry without jitter/budget, blocking the event loop, shared mutable singletons, per-request spawn, sleep-based coordination, ignored cancellation. Ask of every mechanism: *what bounds this?*
- **Fleet pathologies are process pathologies amplified.** Every pattern here has a distributed twin governed by the same math; get the process right first.

Further reading

- Dean, J. and Barroso, L. A., "The Tail at Scale," *Communications of the ACM* 56(2), 2013 — tail-latency amplification under fan-out, hedged and tied requests. https://cacm.acm.org/research/the-tail-at-scale/
- Goetz, B. et al., *Java Concurrency in Practice* (Addison-Wesley, 2006) — Chapter 6 (task execution), Chapter 8 (thread-pool sizing, the wait/compute formula, saturation policies).
- Little, J. D. C., "A Proof for the Queuing Formula L = λW," *Operations Research* 9(3), 1961 — the identity behind every sizing argument in this chapter (see Chapter 1 of this volume).
- Ajmani, S., "Go Concurrency Patterns: Pipelines and cancellation," The Go Blog, 2014. https://go.dev/blog/pipelines

- "Go Concurrency Patterns: Context," The Go Blog, 2014 — deadline propagation as an API discipline. https://go.dev/blog/context
- `golang.org/x/sync/singleflight` — package documentation; the canonical coalescing implementation. https://pkg.go.dev/golang.org/x/sync/singleflight
- `golang.org/x/time/rate` — the production token bucket for Go. https://pkg.go.dev/golang.org/x/time/rate
- Netflix Hystrix wiki, "How it Works" — bulkheads, circuit breaking, and fallbacks; in maintenance mode but the design rationale remains a reference. https://github.com/Netflix/Hystrix/wiki/How-it-Works
- resilience4j documentation — the maintained JVM implementations of bulkhead, rate limiter, circuit breaker, and time limiter. https://resilience4j.readme.io/
- Beyer, B. et al. (eds.), *Site Reliability Engineering* (O'Reilly, 2016) — Chapter 21 "Handling Overload" and Chapter 22 "Addressing Cascading Failures." https://sre.google/sre-book/handling-overload/
- Brooker, M., "Timeouts, retries, and backoff with jitter," *Amazon Builders' Library* — retry budgets and jitter, quantitatively. https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/
- Thompson, M. et al., "Disruptor: High performance alternative to bounded queues" (LMAX technical paper, 2011) — the ring-buffer design referenced in the producer–consumer section.
- Chapter 1 of this volume — Little's Law and the USL; Chapter 2 — condition variables, semaphores, striping; Chapter 5 — livelock and starvation; Chapter 7 — actors and CSP; Chapter 8 — structured concurrency and cancellation; Chapter 10 — testing what this chapter builds.

Chapter 10 — Testing and Debugging Concurrent Systems

What this chapter covers. Every chapter in this volume has ended with some version of the same warning: this class of bug will not be found by ordinary testing. This closing chapter is about what *does* find them. The honest starting point is a negative result — the interleaving space of even

a small concurrent program is astronomically large, a conventional test suite explores a sliver of it, and the sliver it explores is biased toward the boring schedules. Everything that works in this domain works by attacking that fact directly: race detectors that extract evidence from a single observed execution rather than waiting for a failing one, schedule-exploration tools that take control of the scheduler and enumerate interleavings, stress harnesses that widen the sliver, observability that lets production itself act as the test fleet, and — most important — designs that shrink the space that needs exploring in the first place.

We build a taxonomy of concurrency defects, then treat dynamic race detection in depth: the vector-clock machinery underneath ThreadSanitizer and the Go race detector, what these tools guarantee, what they cost, and what they can never find. We then cover schedule exploration — stress, perturbation, and true model checking with Lincheck, loom, and their ancestor CHESS — and deterministic simulation in the FoundationDB style. The second half is diagnostic: deadlock and leak detection, the observability a concurrent system must export before it misbehaves, and a disciplined reproduction strategy for the bug that so far exists only in an incident report. The distributed sibling of every technique here appears in Volume 6, Chapter 12 — Testing Distributed Systems; this chapter stays on one machine.

Learning goals — after this chapter you should be able to:

- Explain why the interleaving space defeats naive testing, and classify a concurrency defect as a data race, atomicity violation, ordering bug, deadlock, leak, or performance pathology.
- Describe how happens-before race detection works — vector clocks, shadow memory, TSan's instrumentation — and state its guarantees and costs honestly.
- Run the Go race detector and TSan appropriately in CI and canaries, and read their reports.
- Use jcstress for memory-model conformance and Lincheck or loom for model-checked testing of small concurrent components, including linearizability checking against a sequential specification.
- Diagnose a deadlock from a thread dump and a leak from goroutine/thread counts, and use goleak and lockdep-style ordering checks preventively.

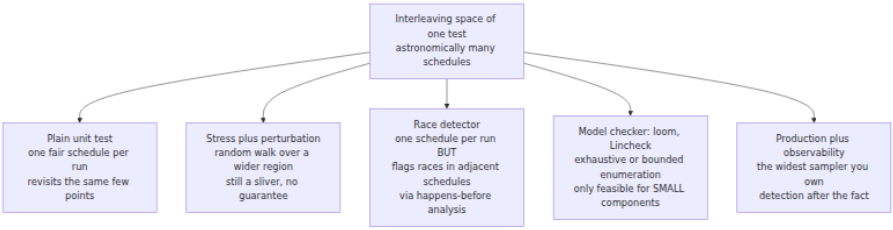
- Specify the observability a concurrent system needs in production, and execute a reproduction strategy for a suspected concurrency bug.
- Design components so that their concurrency is confined and exhaustively testable, and structure CI lanes to spend the sanitizer budget where it pays.

Why ordinary testing does not work

Recall the diagnosis from Chapter 3. Concurrency bugs are **probabilistic and load-dependent**: the bad interleaving needs two threads to arrive within a window that may be a few instructions wide, so it essentially never fires at test-suite load and fires routinely at production load. They are **erased by observation**: a log statement, a debugger, or a `-O0` build inserts enough work and enough incidental barriers to close the window, which is why the term *heisenbug* was coined for exactly this class. And they **present as impossible states** — a field that is assigned in every constructor observed as null, a counter holding a value no code path produces — so the initial investigation usually heads in the wrong direction entirely.

Add to that the arithmetic. For n threads each executing k atomic steps, the number of interleavings is the multinomial coefficient $(nk)!/(k!)^n$. Two threads of ten steps each already give 184,756 interleavings; three threads of ten steps give on the order of 10^{12}; realistic components are beyond enumeration in any direct sense. A deterministic unit test explores exactly one point in this space per run — and not a uniformly random point. Production OS schedulers are boringly fair: they run each thread for a full quantum, so context switches land at quantum boundaries, not inside your four-instruction race window. Running the test ten thousand times mostly revisits the same few schedules.

The consequence is worth stating as a principle, because it shapes everything in this chapter: **a passing concurrent test is weak evidence.** It certifies one schedule. The techniques below are all ways of buying stronger evidence — either by extracting more information per execution (race detection), by visiting more of the space per unit of effort (schedule exploration and model checking), or by shrinking the space until visiting all of it is feasible (design for testability).



A taxonomy of the prey

Different defects yield to different tools, so the first diagnostic step is knowing which species you are hunting.

Defect	Definition	Chapter	Best-fit tools
Data race	Two unordered accesses to one location, at least one a write	Ch. 3	TSan, Go `-race`
Atomicity violation	Individually-synchronized steps composing a non-atomic whole (check-then-act, read-modify-write)	Ch. 1, 2	Lincheck/loom, code review, stress
Ordering bug	Correct operations, missing happens-before edge between them (unsafe publication, missed signal)	Ch. 3	TSan, jcstress, loom
Deadlock / livelock	Circular wait on locks or messages; or perpetual mutual retreat	Ch. 5	Thread dumps, lockdep-style checks, Go runtime
Leak	Threads, goroutines, or tasks that never terminate; unbounded queues	Ch. 6, 8	goleak, thread/ goroutine metrics
Performance pathology	Contention, lock convoys, false sharing, event-loop stalls	Ch. 2, 6, 9	Contention profiles, latency histograms (Vol. 2, Ch. 11)

Two distinctions from earlier chapters must stay sharp here. First, Chapter 1's distinction between a **data race** (a memory-model concept: unordered conflicting accesses) and a **race condition** (a logic concept: correctness depending on timing). A data race is mechanically detectable, and the tools in the next section detect it with remarkable reliability. A race condition — a perfectly synchronized check-then-act on a bank balance — involves no unordered memory access at all, and no race detector will ever flag it. Second, atomicity violations and ordering bugs frequently *involve* no data race either, once every access goes through a lock or an atomic; they are bugs in the composition, not the accesses, which is why linearizability checking (below) exists as a separate discipline from race detection.

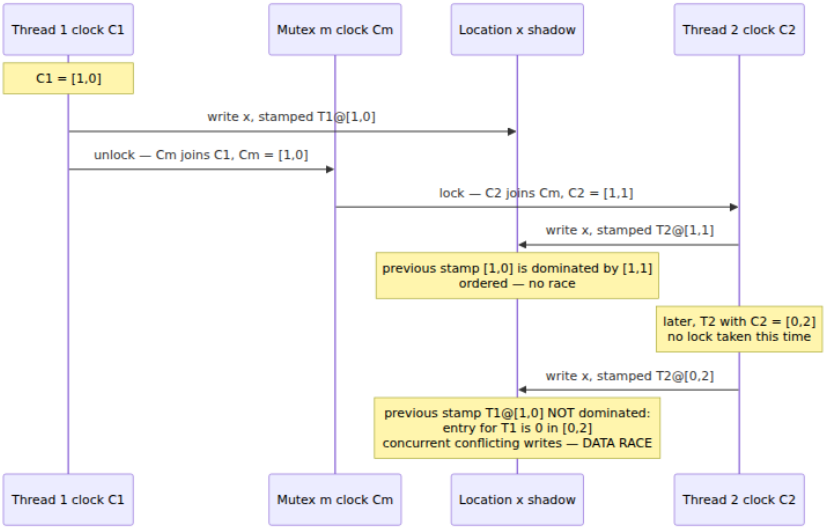
Dynamic race detection

Happens-before detection with vector clocks

The dominant approach to race detection is beautifully direct: implement Chapter 3's happens-before relation at runtime and check every memory access against it. The mechanism is the **vector clock**, the same device Volume 6, Chapter 2 uses across machines.

Each thread carries a vector clock — one logical-clock entry per thread — and increments its own entry at each synchronization operation. Each synchronization object (mutex, atomic variable, channel) also carries a clock. On a *release* operation (unlock, atomic store, channel send), the thread's clock is joined into the object's clock. On an *acquire* (lock, atomic load, channel receive), the object's clock is joined into the thread's. The result is that thread B's clock dominates thread A's entry precisely when everything A did before its release happens-before what B does after its acquire — the relation, computed incrementally.

For each memory location, the detector keeps a record of recent accesses, each stamped with the accessing thread and that thread's clock value at the time. On every new access, it checks the new access against the stored ones: if a previous *conflicting* access (write-write, write-read, or read-write) is not ordered before the current one under the recorded clocks, the two accesses are concurrent, and that is a data race — by definition, not by heuristic.



Full vector clocks on every access are expensive, and the practical detectors descend from the **FastTrack** insight (Flanagan and Freund, 2009): the vast majority of locations are accessed in an already-ordered fashion, so a single (thread, clock) *epoch* suffices to represent the last write, and the full vector is materialized only for the read-shared minority. This is what makes happens-before detection affordable enough to run on real programs.

Two properties follow from the construction and both matter operationally. First, **no false positives** for the synchronization idioms the tool understands: a report means two accesses really were unordered under the language's happens-before rules. (The caveat is idioms the tool cannot see — hand-rolled synchronization through `epoll`, memory-mapped concurrency with another process, or lock-free tricks expressed in ways the instrumentation misses — which can produce reports you must then adjudicate. The older *lockset* approach of Eraser, which flagged any location not consistently protected by one lock, produced far more false alarms and lost.) Second, and less obvious: the detector finds races **beyond the schedule that executed**. It does not require the racy accesses to collide in time — only to occur, in any order, without a happens-before edge between them. A race that would corrupt memory only in a one-in-a-million interleaving is reported on the millionth-of-the-way-there interleaving where the two accesses happened seconds apart.

This is the property that makes running a race detector over an ordinary, single-schedule test suite so much more powerful than the suite itself: one execution testifies about a whole neighborhood of adjacent schedules. The limit is equally important — it testifies only about the code paths that *ran*. A race on an error path your tests never enter is invisible. Race detection multiplies the value of coverage; it does not substitute for it.

ThreadSanitizer

ThreadSanitizer (TSan) is the production implementation of this idea for C, C++, and (via the same runtime) Go and Swift. It has two halves. The compiler half — enabled with `-fsanitize=thread` in Clang and GCC — instruments every memory access and every atomic operation with a call into the runtime. The runtime half maintains the vector clocks and, for every 8 bytes of application memory, a block of **shadow memory** recording the most recent accesses: for each, a thread identifier, an epoch, the access size and offset, and whether it was a write. In the classic v2 design this was four shadow cells per 8-byte word — a small ring of access history — with an application address mapping to its shadow by pure address arithmetic, no locks on the fast path. Each new access is compared against the stored cells under the happens-before clocks; an unordered conflict is reported with both stacks, which is why TSan reports are so unusually actionable — you get the current access's stack *and* the previous access's stack, plus where each thread was created.

The costs are significant and should be planned for, not discovered. The published figures are roughly **5–15× slowdown and 5–10× memory overhead**, and Go's documentation for the same runtime quotes a similar range (memory roughly 5–10×, execution commonly 2–20× depending on workload). Shadow memory reserves a large virtual address range up front. These numbers mean TSan is a *test and canary* configuration, not a production default — but they are also far cheaper than the alternative, which is a week of incident archaeology per escaped race.

The Go race detector

Go ships the same TSan runtime behind a single flag:

```
go test -race ./...
go build -race ./cmd/server   # a race-enabled binary for canary
deployment
```

The compiler instruments memory accesses and the runtime maps Go's synchronization — mutexes, channels, `sync/atomic`, `WaitGroup`, goroutine creation — onto acquire/release events for the vector clocks. A minimal racy test and its report:

```
package cache

import (
    "sync"
    "testing"
)

type Cache struct {
    entries map[string]string // written without synchronization — the
bug
}

func (c *Cache) Set(k, v string) { c.entries[k] = v }
func (c *Cache) Get(k string) string { return c.entries[k] }

func TestConcurrentSet(t *testing.T) {
    c := &Cache{entries: make(map[string]string)}
    var wg sync.WaitGroup
    for i := 0; i < 2; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            c.Set("k", "v")
            _ = c.Get("k")
        }()
    }
    wg.Wait()
}
```

```
$ go test -race ./cache
==================
WARNING: DATA RACE
Write at 0x00c000110270 by goroutine 8:
  runtime.mapassign_faststr()
      /usr/local/go/src/runtime/map_faststr.go:203 +0x0
  example.com/cache.(*Cache).Set()
      /home/user/cache/cache.go:12 +0x4a
  example.com/cache.TestConcurrentSet.func1()
      /home/user/cache/cache_test.go:21 +0x38

Previous write at 0x00c000110270 by goroutine 7:
  runtime.mapassign_faststr()
      /usr/local/go/src/runtime/map_faststr.go:203 +0x0
  example.com/cache.(*Cache).Set()
      /home/user/cache/cache.go:12 +0x4a
  [... trimmed ...]

Goroutine 8 (running) created at:
  example.com/cache.TestConcurrentSet()
      /home/user/cache/cache_test.go:19 +0xec
==================
--- FAIL: TestConcurrentSet (0.01s)
    testing.go:1399: race detected during execution of test
FAIL
```

Note what the report gives you: both stacks, down to the runtime map-assign both goroutines raced through, and the creation site of the racing goroutine. Note also that the test *passed functionally* — two writes of the same value to a map usually don't corrupt anything observable in one run — and the detector failed it anyway, because the accesses were unordered. That is the extract-more-evidence-per-execution property in action.

Operational guidance, in order of value: run `-race` on the **entire test suite in CI as a required lane** (the roughly 10× cost on test time is almost always affordable; if it is not, run it on every merge to main rather than every push, never less). Run a `-race` build as a **canary** — one instance of the real service, taking a slice of real traffic, sized for the overhead — because production traffic exercises paths and interleavings tests never will. Set `GORACE="halt_on_error=1"` in CI so the first report fails fast, and treat every report as a bug: because false positives are essentially absent, "it's benign" is nearly always wrong and was covered as such in Chapter 3.

Java: jcstress and the JMM-conformance approach

Java has no TSan; instrumenting the JVM's memory accesses from outside is impractical, and no happens-before race detector ships with the platform. (Research systems exist, and IntelliJ's debugger, async-profiler, and JFR form the practical diagnostic ecosystem — but there is no `-race` flag.) What Java has instead is the sharpest instrument anywhere for a *different* question: **jcstress**, the OpenJDK harness for memory-model conformance.

A jcstress test is a tiny litmus test — typically two or three methods touching two or three fields — which the harness runs across **billions of iterations**, deliberately varying thread placement, compilation, and timing, and records a **histogram of observed outcomes** checked against annotations declaring which outcomes the JMM permits:

```
@JCStressTest
@Outcome(id = {"1, 1", "1, 0", "0, 1"}, expect = ACCEPTABLE,
         desc = "Some interleaving of the stores and loads.")
@Outcome(id = "0, 0", expect = ACCEPTABLE_INTERESTING,
         desc = "Store-load reordering: legal without volatile.")
@State
public class StoreBufferTest {
    int x, y;

    @Actor void thread1(II_Result r) { x = 1; r.r1 = y; }
    @Actor void thread2(II_Result r) { y = 1; r.r2 = x; }
}
```

```
  [OK] com.example.StoreBufferTest
    (JVM args: [-server])
  Observed state   Occurrences              Expectation  Interpretation
            0, 0         2,481   ACCEPTABLE_INTERESTING  Store-load
reordering: legal...
            0, 1    98,177,270               ACCEPTABLE  Some
interleaving...
            1, 0    97,442,118               ACCEPTABLE  Some
interleaving...
            1, 1       310,655               ACCEPTABLE  Some
interleaving...
  [... trimmed ...]
```

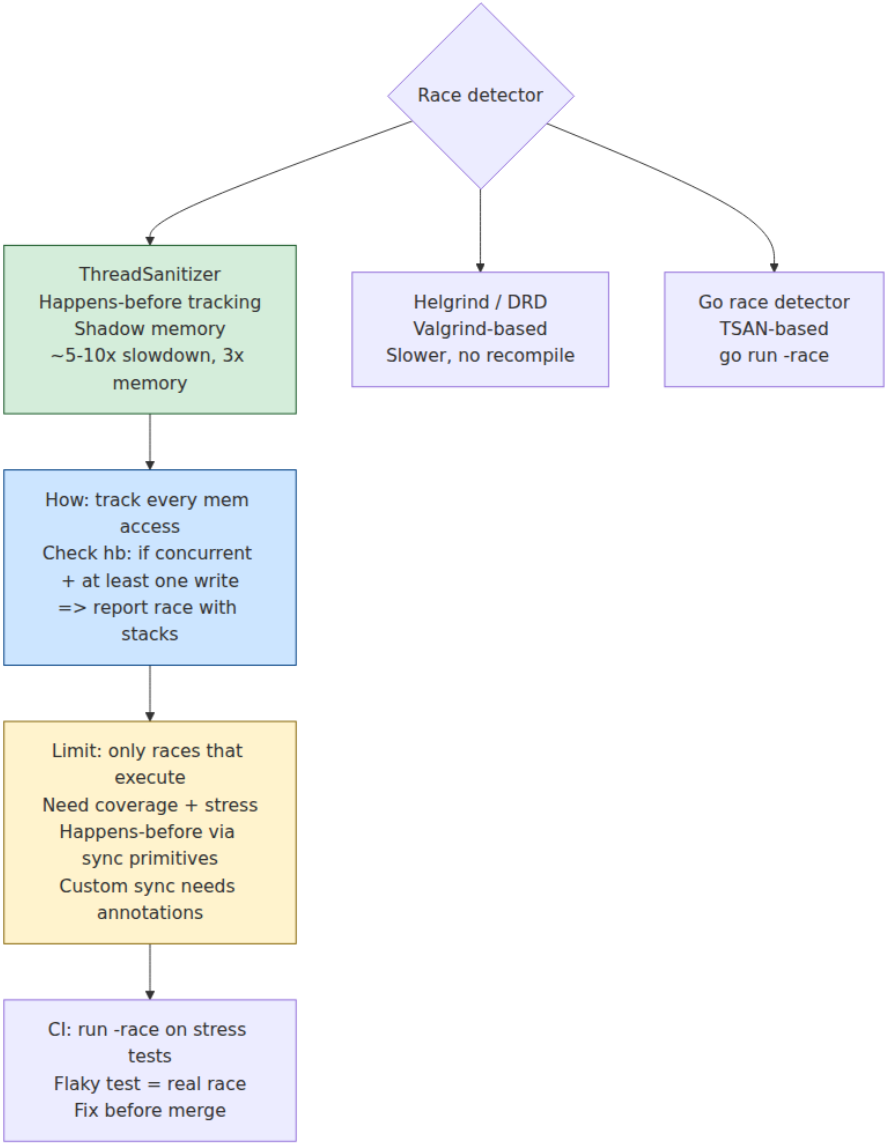
That 2,481 count is Chapter 3's store-buffer litmus test observed empirically — the outcome no interleaving explains, occurring a few thousand times in two hundred million runs. This is the only practical way

to *check* a claim about the JMM ("this idiom is safe without volatile") rather than argue about it, and it is how OpenJDK itself validates JIT and runtime changes against the model. Use it when you write anything clever enough to depend on memory-model details — and note the scope: jcstress validates small idioms, not applications. For application-level Java concurrency testing, the model-checking tools of the next section are the right instrument.

What race detectors do not find

Worth its own subsection, because over-trusting a clean TSan run is a common failure mode:

- **Race conditions without data races.** The synchronized check-then-act, the two correctly-locked updates that must be atomic together, the TOCTOU against a file system. No unordered access, no report. This is Chapter 1's distinction, load-bearing again.
- **Races on paths not executed.** The detector sees only the coverage your workload achieves. Error paths, timeouts, and shutdown sequences — precisely where concurrency bugs cluster, because they run rarely and interleave with everything — need deliberate exercise.
- **Deadlocks, leaks, and performance pathologies.** Different species, different tools, below.
- **Some synchronization the tool cannot model** — cross-process shared memory, custom futex use — where you may get noise or silence, neither trustworthy.


Race detector
ThreadSanitizer
Happens-before tracking
Shadow memory
~5-10x slowdown, 3x memory
Helgrind / DRD
Valgrind-based
Slower, no recompile
Go race detector
TSAN-based
go run -race
How: track every mem access
Check hb: if concurrent + at least one write => report race with stacks
Limit: only races that execute
Need coverage + stress
Happens-before via sync primitives
Custom sync needs annotations
CI: run -race on stress tests
Flaky test = real race
Fix before merge

Stress testing and schedule exploration

Stress alone is weaker than it feels

The obvious response to "one schedule per run" is many threads, many iterations, many runs — and every mature concurrent codebase has a stress suite. It has genuine value: it widens the sampled region, and combined with a race detector it drives coverage into contention paths. But understand why it saturates. The scheduler is fair and quantized; heavy stress mostly generates *more* samples from the *same* well-trodden distribution of schedules. Bugs that need a context switch inside a specific five-instruction window can survive years of overnight stress runs. Empirically, stress finds the shallow half of the interleaving space quickly and then flatlines.

Two escalations attack the distribution itself.

Schedule perturbation

Inject noise at synchronization points: a randomized `Thread.yield()`, `sched_yield()`, or microsecond sleep just before or after lock operations, atomic accesses, and queue operations — either by hand at suspicious points or via instrumentation. This moves context switches off quantum boundaries and into the windows where races live, and it is cheap: no new framework, works in any language, composes with `-race`. It remains probabilistic — better dice, same game.

Systematic exploration: model checking the scheduler

The rigorous escalation is to *take control of* the scheduler and enumerate schedules systematically. The ancestor is Microsoft Research's **CHESS** (Musuvathi et al., OSDI 2008), which established the key empirical fact making this tractable: most concurrency bugs are exposed by schedules with a **small number of preemptions** — context switches at *non*-voluntary points. By bounding preemptions at two or three and enumerating within that bound, CHESS turned an intractable space into an exhaustive search that found and — critically — *deterministically reproduced* long-standing heisenbugs.

Its practical descendants:

Lincheck (JetBrains, for JVM code) is the tool of choice for concurrent data structures on the JVM. You declare the operations; Lincheck

generates concurrent scenarios, runs them under either a stress strategy or a **managed model-checking strategy** (it controls all switch points, exploring interleavings under a preemption bound in the CHESS lineage), and verifies every observed outcome against the *sequential* behavior of the same class — a linearizability check, discussed below.

```
class CounterTest {
    private val c = Counter()          // the class under test

    @Operation fun inc() = c.inc()
    @Operation fun get() = c.get()

    @Test
    fun modelCheck() = ModelCheckingOptions()
        .iterations(100)               // scenarios to generate
        .invocationsPerIteration(10_000)
        .check(this::class)
}
```

When it fails, Lincheck prints the minimal failing scenario *and the interleaving trace* — which thread executed which line in which order — turning a heisenbug into a deterministic, reviewable counterexample. That reproducibility, common to all the model-checking tools and impossible for plain stress, is half their value.

loom (Rust) goes further for small tests: within a bounded test body, it explores the interleavings *exhaustively*, including the weak-memory behaviors of the C11 model that a real x86 machine would never show you — Chapter 3's argument about testing on x86, answered in software. You write the test against `loom::sync` types instead of `std::sync`, wrap it in `loom::model(|| { ... })`, and loom re-executes the closure under every meaningfully distinct schedule, using state-space reduction to prune equivalent ones. The constraint is honest and hard: state space explodes with test size, so loom tests are necessarily *tiny* — two or three threads, a handful of operations. This is not a weakness to engineer around; it is the design pressure to build concurrent components small enough to model-check, which is the theme of the design-for-testability section. Loom is how the core of `tokio` is tested.

Linearizability checking: testing against a sequential specification

Lincheck's verification step deserves independent attention, because it is the most general answer to "what does *correct* even mean for a concurrent structure?" The answer, from Chapter 4: **linearizability** — every operation appears to take effect atomically at some instant between its invocation and its response, consistent with the sequential specification. That definition is directly checkable: record a concurrent *history* (invocations and responses with their overlap), then search for a legal sequential ordering that respects real-time order and matches the sequential spec. If none exists, the history is a correctness violation, whatever the internals did.

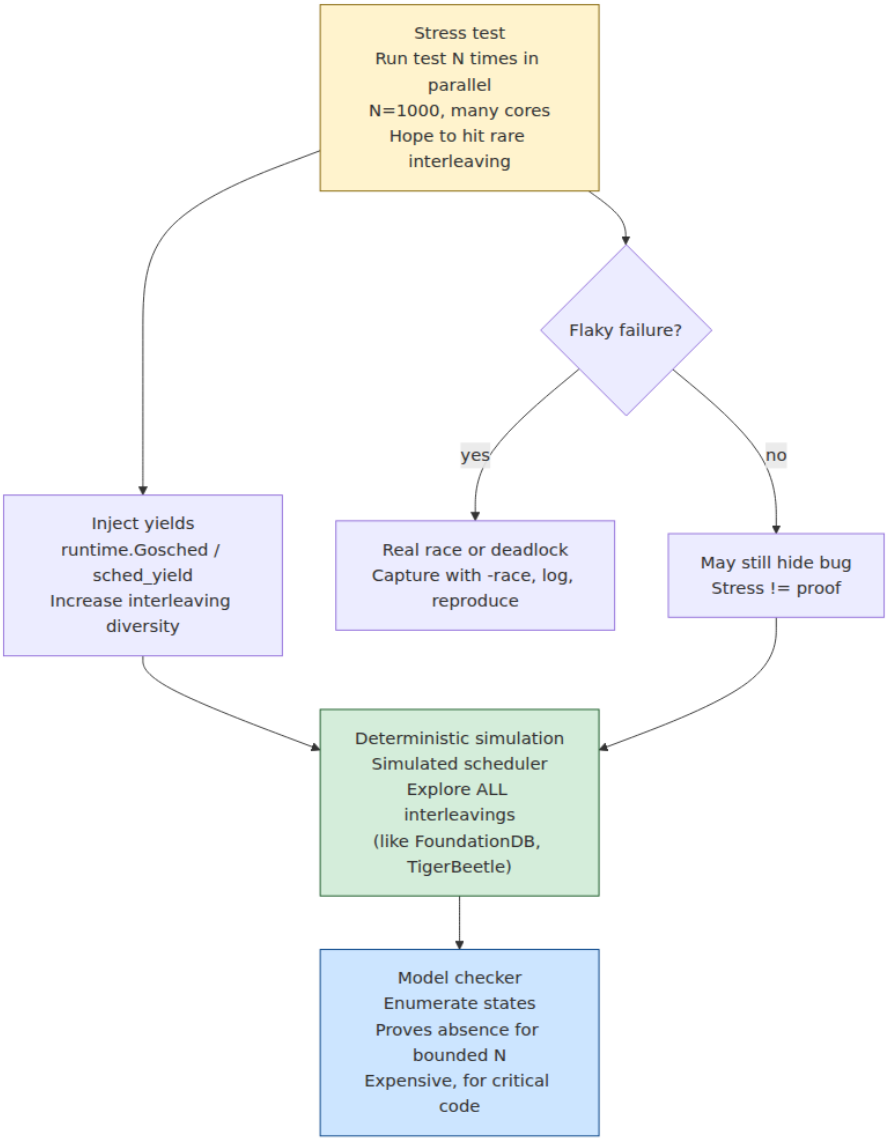
This is property-based testing lifted to concurrency: instead of asserting specific outcomes, you supply the sequential model and let the checker generate scenarios and validate all outcomes against it. The search is NP-hard in general and exponential in practice, which is why checkers work on short histories — and why the same shape reappears at the distributed layer as **Jepsen** driving real databases and **Knossos** checking the recorded histories for linearizability (Volume 6, Chapter 12, where it properly belongs).

Deterministic simulation

The most radical position: if nondeterminism is the enemy, remove it. **FoundationDB** wrote its database in Flow, a C++ dialect providing actor-style concurrency that compiles to a *single-threaded* event loop. In test mode the entire cluster — every "machine," the network, the disks, the clocks — runs inside one process, driven by a seeded PRNG; concurrency is simulated, time is virtual, and every run is exactly reproducible from its seed. On top of this the simulator injects faults — message delay and reorder, partitions, disk corruption, machine crashes, even coordinated "buggify" misbehaviors the code deliberately enables in simulation — at rates no real environment approaches. A nightly fleet runs enormous numbers of seeds; any failure is re-runnable, bisectable, and debuggable forever, because the seed *is* the bug report.

The trade is severe and explicit: you must build on the simulable substrate from day one — retrofitting is close to impossible — and the simulator only tests what it models. But it converts the worst property of concurrency bugs (irreproducibility) into a non-issue, and it is the

intellectual ancestor of the deterministic-simulation and Antithesis-style testing covered for distributed systems in Volume 6, Chapter 12.



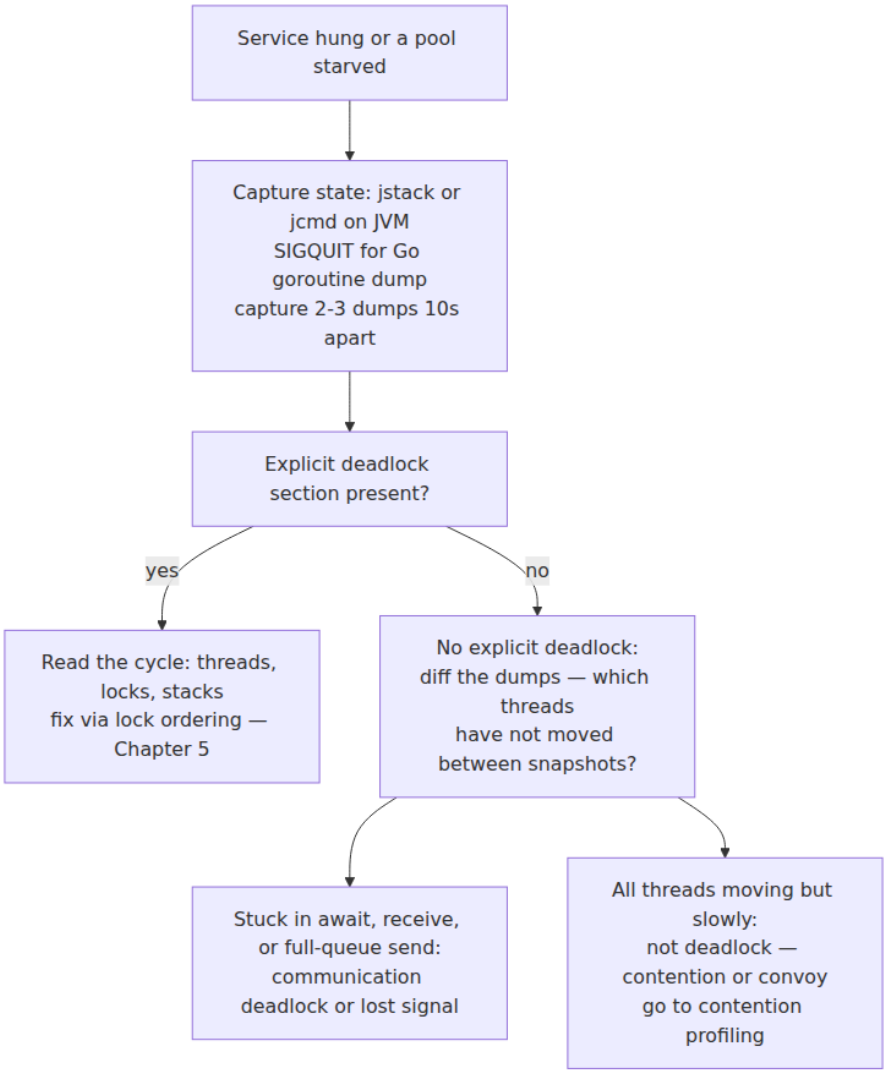
Deadlock and leak detection

Reading the deadlock out of a thread dump

Deadlocks (Chapter 5) are the friendliest concurrency bug: once entered, they persist, so a post-hoc snapshot contains the whole story. The JVM's thread-dump machinery (`jstack <pid>`, or `jcmd <pid> Thread.print`, or SIGQUIT) runs a cycle detector over the monitor and `java.util.concurrent` lock graphs and reports findings explicitly:

```
Found one Java-level deadlock:
=============================
"pool-1-thread-1":
  waiting to lock monitor 0x00007f3a1c0062c8 (object
0x000000076b8a2d10,
  a java.lang.Object), which is held by "pool-1-thread-2"
"pool-1-thread-2":
  waiting to lock monitor 0x00007f3a1c004e18 (object
0x000000076b8a2d20,
  a java.lang.Object), which is held by "pool-1-thread-1"

Java stack information for the threads listed above:
===================================================
"pool-1-thread-1":
        at com.example.transfer.Account.transferTo(Account.java:41)
        - waiting to lock <0x000000076b8a2d10> (a java.lang.Object)
        - locked <0x000000076b8a2d20> (a java.lang.Object)
        [... trimmed ...]

Found 1 deadlock.
```

Both threads, both locks, both stacks, and the cycle stated outright — from here the fix is Chapter 5's lock-ordering discipline. The detector only sees waits it can model: it will not flag a cycle that runs through a condition-variable wait, a full bounded queue, or an external resource, so a hung service with *no* reported deadlock still warrants reading the dump for clusters of threads parked in `await` or `poll` — a "communication deadlock" the cycle detector cannot see.



Go's runtime panics with `fatal error: all goroutines are asleep - deadlock!` and full stacks when *every* goroutine is blocked — invaluable, but only for total deadlock; a partial deadlock among some goroutines while others serve traffic triggers nothing. For those, SIGQUIT dumps every goroutine's stack, and the same diff-two-snapshots technique applies. Capturing multiple dumps a few seconds apart is the single most

useful habit here: a deadlocked thread is identical in every snapshot, and that invariance is what separates "stuck" from "slow" at a glance.

The *preventive* counterpart is **lock-order checking** in the style of the Linux kernel's lockdep: record the order in which lock classes are acquired, build the acquired-before graph, and report any cycle — flagging a *potential* deadlock the first time the inconsistent order executes, even though no deadlock occurred on that run. This is the same extract-evidence-per-execution move as happens-before race detection, applied to deadlocks, and libraries exist for most ecosystems (and are cheap to hand-roll for the handful of locks that matter in one service).

Leak detection

Thread, goroutine, and task leaks (Chapters 6 and 8) kill slowly — each leaked goroutine pins its stack and everything it references — and the detection story has a test half and a production half. In Go tests, **uber-go/goleak** asserts at test end that no unexpected goroutines survive:

```
import "go.uber.org/goleak"

func TestMain(m *testing.M) {

goleak.VerifyTestMain(m)         // fails the package if goroutines
leak
}

// or, per test:
func TestWorkerShutdown(t *testing.T) {
    defer goleak.VerifyNone(t)
    w := StartWorker()
    w.Stop()                          // if Stop fails to reap the
worker, this test fails
}
```

A goleak failure prints the leaked goroutine's stack — usually a goroutine parked forever on a channel nobody will ever close, which is Chapter 8's argument for structured concurrency made empirical: ownership that guarantees children terminate makes this class of test pass by construction. The production half is a metric: export thread count, goroutine count (`runtime.NumGoroutine()`), and pending-task counts, and alert on trend, not threshold. A goroutine count that climbs and never descends across a load cycle is a leak announcing itself weeks before the OOM.

Debugging in production

The observability you need before the incident

You do not get to instrument a concurrent system *after* it misbehaves; the evidence is gone and the heisenbug rules apply. The kit below is decided at design time:

- **Per-pool saturation and queue depth.** For every thread pool and worker pool: active workers, queue length, and time-in-queue. Chapter 9's argument: queue depth is the earliest warning of every downstream concurrency pathology.
- **Event-loop lag** for async runtimes — the scheduling delay of a no-op task (Chapter 6). It is the single metric that catches loop-blocking, and it must be measured continuously, not sampled during incidents.
- **Lock-contention profiles.** Go: `runtime.SetMutexProfileFraction` plus `go tool pprof http://.../debug/pprof/mutex` attributes wait time to contended acquisition sites. JVM: JFR's monitor-blocked and park events do the same with stacks. System-wide: `perf lock` (Volume 2, Chapter 11 covers the tooling in depth). Contention profiles are how you find the convoy before the latency graph does.
- **Thread and goroutine counts** as leak canaries, per the previous section.
- **Causal logging.** A log line without a request identifier is nearly useless in a concurrent service, because interleaved output from a thousand requests is noise. Propagate a request/trace ID through every context and include it in every line; then a single request's story can be reassembled in order, which is a poor-man's happens-before reconstruction. Distributed tracing generalizes this across services (Volume 11, Chapter 4).
- **Flight recording.** JFR is designed for always-on use (its steady-state overhead is targeted at roughly one percent): a continuous ring buffer of scheduling, contention, and allocation events, dumpable *after* something odd happens. Continuous profilers serve the same role elsewhere. This is the closest production gets to a time machine, and it costs almost nothing.

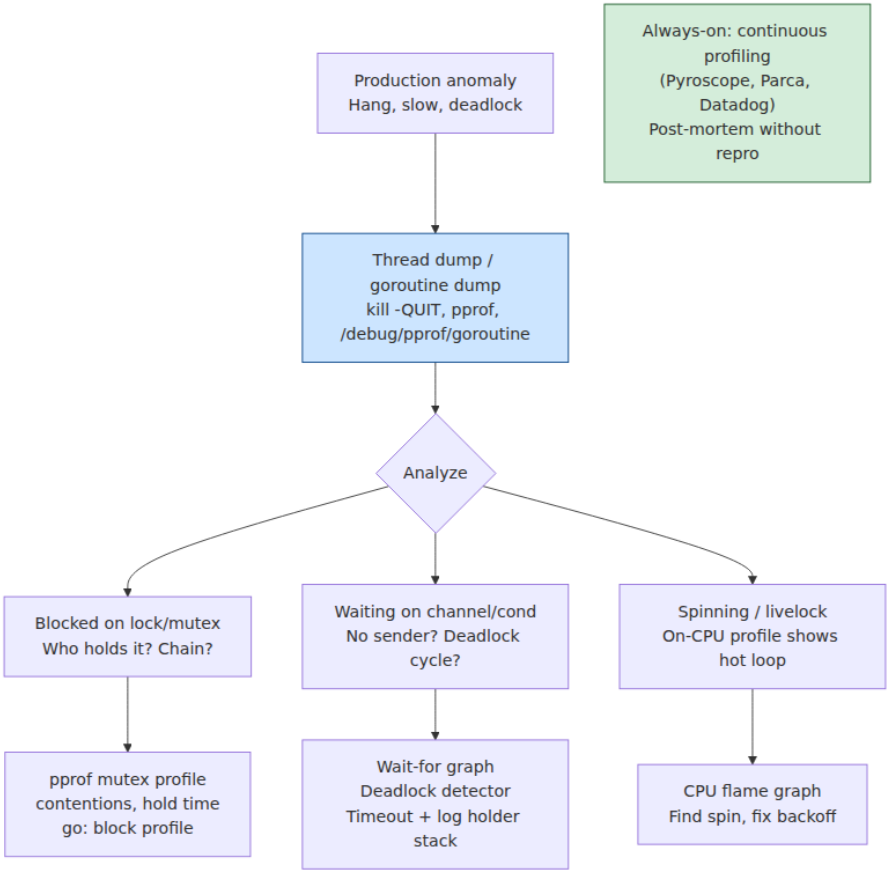
Two tools traditionally reached for deserve honest deflation. **Core dumps** capture one instant — excellent for deadlocks (the cycle is sitting there) and post-crash state, nearly useless for races, whose evidence is an

ordering of events that no snapshot contains. **Interactive debuggers** are worse than useless for races: stopping one thread reshapes every schedule, and the observer effect is maximal. Both remain fine tools for the non-concurrent majority of bugs; know which bug you have.

Reproduction strategy

When production symptoms smell like concurrency — impossible states, load correlation, disappearance under instrumentation — reproduce methodically rather than by rerunning the test suite and hoping:

1. **Increase pressure.** More concurrent load than production, on fewer cores' worth of capacity, with the suspected operations forced to overlap. You are trying to widen and repeatedly hit the window.
2. **Change the parallelism, both directions.** Pin the process to one core (`taskset -c 0`) or set `GOMAXPROCS=1` : if the bug *vanishes*, true parallelism is implicated — memory ordering or a tight data race; if it *persists*, it lives in logical interleaving and will be far easier to catch deterministically. Then raise thread counts well past core count to force preemption at unusual points. Both outcomes of every experiment are signal.
3. **Run the detector builds** against the reproduction workload — `-race` , TSan, with schedule perturbation if needed. A race report obtained this way is usually endgame.
4. **Bisect with the detector**, not with the symptom. `git bisect` where the test is "does the race detector report under the reproduction workload" converges fast precisely because the detector does not need the failure to manifest, only the racy accesses to execute.



Designing for testability

The strongest position in this chapter is the one that needs the least tooling: **arrange the code so that its concurrency is small enough to verify exhaustively, and everything else is sequential.**

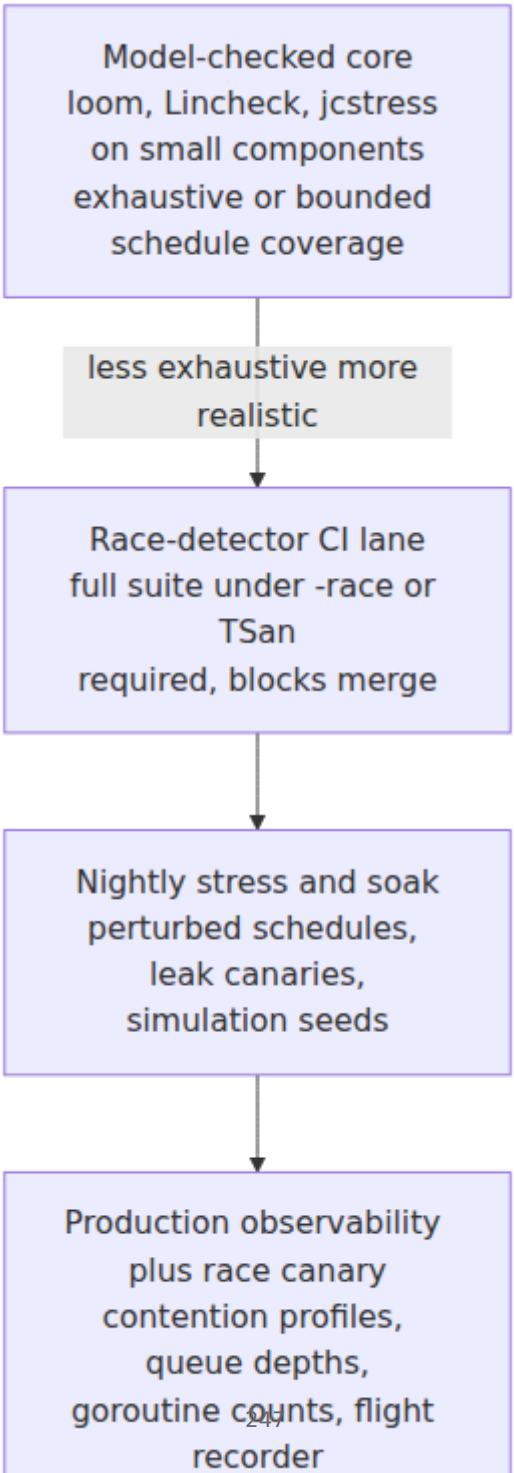
- **Confine concurrency** (Chapter 9). A service whose shared mutable state lives behind a handful of small, explicit components — a concurrent cache, a work queue, an actor mailbox — has a handful of things to test with Lincheck or loom, and a large sequential remainder testable the ordinary way. A service where any handler may touch any state has an untestable everything.

- **Inject the clock and the executor.** Code that calls `time.Now()` or spawns onto a global pool cannot be scheduled by a test. Take the clock and the executor as dependencies, and a test can use a fake clock (fire the timeout *now*) and a deterministic single-threaded executor (run the continuations in a chosen order). Every serious async runtime provides these hooks; use them.
- **Never sleep in tests.** `sleep(100ms)` before an assertion is a bet that the concurrent work finishes in time — lost on loaded CI machines (flaky) while wasting 100 ms on fast ones (slow), and *no* sleep length fixes both. Replace every sleep with an explicit synchronization point: a `CountDownLatch` or channel the worker signals (Chapter 2), a completion future the test awaits. A flaky-test dashboard dominated by sleeps is a solved problem being tolerated.
- **Make shutdown a first-class tested path.** Leaks and deadlocks cluster in teardown because it runs concurrently with everything and is tested by nothing. goleak plus an explicit start/stop/start test per component covers a disproportionate share of real incidents.

CI strategy: spending the sanitizer budget

The lanes, in priority order — the pyramid mirrors the classic testing pyramid, with schedule coverage decreasing and realism increasing as you rise:

1. **Required on every change:** unit tests, plus race-detector builds (`-race`, TSan) over the full suite. Non-negotiable and affordable; a race report blocks merge.
2. **Required for concurrent components:** their Lincheck/loom/jcstress suites — these are fast *because* the components are small, which is the design pressure working as intended.
3. **Nightly:** stress lanes with schedule perturbation under the race detector, long-running soak tests watching leak canaries, and (where you have it) simulation seeds. Nightly because the cost is hours, and because these lanes find bugs *statistically* — their value is cumulative schedule coverage, not per-run verdicts.
4. **Continuous:** a race-enabled canary in production, and the observability of the previous section, which is the widest schedule-sampler you will ever own.


Model-checked core
loom, Lincheck, jcstress
on small components
exhaustive or bounded
schedule coverage
less exhaustive more
realistic
Race-detector CI lane
full suite under -race or
TSan
required, blocks merge
Nightly stress and soak
perturbed schedules,
leak canaries,
simulation seeds
Production observability
plus race canary
contention profiles,
queue depths,
goroutine counts, flight
recorder

The distributed-systems lens

Every technique in this chapter has a distributed sibling, and Volume 6, Chapter 12 is that sibling's chapter; the mapping is worth internalizing now because the *reasoning* transfers even where the mechanisms cannot.

This chapter	Volume 6, Chapter 12
Happens-before race detection, vector clocks in TSan	Jepsen-style history checking; the same vector clocks, now tracking messages
Schedule exploration, preemption bounding	Fault injection: partitions, message delay and reorder, clock skew
Deterministic simulation of threads	FoundationDB/Antithesis-style simulation of whole clusters
Linearizability checking with Lincheck	Linearizability checking with Knossos over Jepsen histories
Goroutine leak detection	Orphaned-workflow and stuck-saga detection
Thread dump of one process	Consistent-ish snapshot of a fleet — much harder, often impossible

One asymmetry drives most of the differences: a race detector works because it can instrument *every* access to shared memory through a compiler; no such chokepoint exists across machines, where state is shared by messages traversing infrastructure you do not control. So the distributed tools shift from *instrumenting the mechanism* to *checking the history*: record every operation's invocation and response at the boundary, then validate the history against a consistency model — which is exactly the Lincheck verification step, minus the managed scheduler. And where this chapter perturbs schedules, Jepsen perturbs the network, because in a distributed system the network *is* the scheduler: partitions, delays, and reorders are its context switches.

The shared moral is the one this volume has been building toward: **you cannot test correctness into a concurrent system — at either scale.** The interleaving space (or failure space) is too large for any test regime to certify by sampling. What actually works is the pincer this chapter described: *design* the system so its correctness argument is small — confined concurrency, happens-before discipline, structured ownership,

and their distributed analogues — and then *verify aggressively* with tools that extract maximal evidence per execution. Testing validates the design's argument; it cannot substitute for the design having one.

Key takeaways

- **A passing concurrent test certifies one schedule** out of an astronomical space, sampled by a boringly fair scheduler. All effective techniques either extract more evidence per run, explore schedules systematically, or shrink the space by design.
- **Know your prey.** Data races, atomicity violations, ordering bugs, deadlocks, leaks, and contention pathologies are different species found by different tools. Race detectors find data races only — never race conditions, and never bugs on unexecuted paths.
- **Happens-before race detection is the workhorse.** Vector clocks plus shadow memory let TSan and Go's `-race` report races that never manifested as failures, with essentially no false positives for supported idioms, at roughly 5–15× CPU and 5–10× memory — a price that belongs in CI and canaries, not production defaults.
- **Java's instrument is jcstress**: billions of iterations of litmus tests, outcome histograms checked against the JMM — conformance testing for memory-model claims, not application testing.
- **Stress alone flatlines**; schedule perturbation improves the dice; **model checking changes the game** — Lincheck and loom enumerate interleavings and hand you deterministic counterexamples, but only for components small enough to enumerate. Build components that small.
- **Linearizability checking** — validating concurrent histories against a sequential spec — is the general correctness test for concurrent structures, and reappears intact as Jepsen/Knossos.
- **Deterministic simulation** (FoundationDB) removes nondeterminism entirely: seeded, replayable universes with aggressive fault injection, bought by building on a simulable substrate from day one.
- **Deadlocks are snapshot-debuggable** — jstack names the cycle; diff repeated dumps for what the detector cannot see. **Leaks are trend-debuggable** — goleak in tests, goroutine/thread counts as production canaries.

- **Production observability is decided at design time**: queue depths, event-loop lag, contention profiles, causal request IDs, flight recorders. Core dumps and debuggers are deadlock tools, not race tools.
- **Reproduce methodically**: raise pressure, vary core counts in both directions (either result is signal), then let a detector build plus bisection finish the job.
- **Design for testability**: confined concurrency, injected clocks and executors, latches instead of sleeps, tested shutdown. Then spend the sanitizer budget: race lane required, model-checked cores required, stress nightly, canary always.

Further reading

- Serebryany, K. and Iskhodzhanov, T., "ThreadSanitizer — Data Race Detection in Practice," *WBIA*, 2009 — the original TSan design and the case against lockset detection. https://research.google/pubs/threadsanitizer-data-race-detection-in-practice/
- Clang documentation, *ThreadSanitizer* — https://clang.llvm.org/docs/ThreadSanitizer.html — supported platforms, flags, and the published overhead ranges.
- Flanagan, C. and Freund, S., "FastTrack: Efficient and Precise Dynamic Race Detection," *PLDI*, 2009 — the epoch optimization that makes happens-before detection affordable.
- Vyukov, D. and Gerrand, A., "Introducing the Go Race Detector," Go blog, 2013 — https://go.dev/blog/race-detector — and the reference page https://go.dev/doc/articles/race_detector.
- Savage, S. et al., "Eraser: A Dynamic Data Race Detector for Multithreaded Programs," *ACM TOCS*, 1997 — the lockset approach; historically important, and instructive on false positives.
- The jcstress project and wiki — https://github.com/openjdk/jcstress — samples are the best tutorial on JMM litmus testing.
- Koval, N., Fedorov, A., et al., "Lincheck: A Practical Framework for Testing Concurrent Data Structures on JVM," *CAV*, 2023 — and the project docs at https://github.com/JetBrains/lincheck.
- The loom crate documentation — https://docs.rs/loom — including its honest discussion of state explosion and test-size limits.

- Musuvathi, M. et al., "Finding and Reproducing Heisenbugs in Concurrent Programs," *OSDI*, 2008 — CHESS and iterative context bounding, the foundation of managed-schedule testing.
- Wilson, W., "Testing Distributed Systems with Deterministic Simulation," Strange Loop, 2014 — the FoundationDB testing story; see also https://apple.github.io/foundationdb/testing.html.
- uber-go/goleak — https://github.com/uber-go/goleak — goroutine leak assertions for Go tests.
- Kleppmann-adjacent but essential: Jepsen analyses and the Knossos checker — https://jepsen.io/analyses — the distributed continuation, with Volume 6, Chapter 12.
- Volume 2, Chapter 11 — the profiling and tracing tools referenced throughout the production section; Volume 15, Chapter 1 — where these lanes fit an overall testing strategy.